

**TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG
ĐẠI HỌC BÁCH KHOA HÀ NỘI**

Lưu trữ và xử lý dữ liệu lớn

**Nhóm tác giả:
Trần Văn Đặng
Nguyễn Hữu Đức
Nguyễn Bình Minh
Trần Việt Trung**

Lời nói đầu

Trong thời đại số hóa hiện nay, dữ liệu đã trở thành tài nguyên quý giá nhất của thế kỷ 21. Sự bùng nổ của dữ liệu lớn (Big Data) đang thay đổi cách chúng ta sống, làm việc và suy nghĩ. Từ các mạng xã hội, thương mại điện tử, đến các thiết bị IoT và nghiên cứu khoa học, dữ liệu được tạo ra với tốc độ chưa từng có trong lịch sử.

Cuốn sách "Lưu trữ và Xử lý Dữ liệu Lớn" được biên soạn nhằm cung cấp cho sinh viên và các nhà nghiên cứu một cái nhìn toàn diện về các công nghệ và kỹ thuật xử lý dữ liệu lớn. Từ những khái niệm cơ bản về dữ liệu lớn và khoa học dữ liệu, đến các công nghệ tiên tiến như NoSQL, MapReduce, Spark, và các kiến trúc xử lý dữ liệu hiện đại.

Cuốn sách được chia thành các chương chính:

Chương 1: Tổng quan về lưu trữ và xử lý dữ liệu lớn - Giới thiệu về sự bùng nổ dữ liệu, khoa học dữ liệu và các đặc trưng 5V của dữ liệu lớn.

Chương 2: Hệ sinh thái Hadoop - Nền tảng cơ bản cho xử lý dữ liệu lớn.

Chương 3: Hệ thống Tập tin Phân tán Hadoop HDFS - Công nghệ lưu trữ dữ liệu phân tán.

Chương 4: Cơ sở dữ liệu phi quan hệ NoSQL - Từ các khái niệm cơ bản, CAP theorem đến các hệ thống NoSQL phổ biến như DynamoDB và Presto.

Chương 5: Hệ thống truyền thông điệp phân tán - Với Apache Kafka cho xử lý luồng dữ liệu.

Chương 6: MapReduce - Mô hình lập trình cho xử lý dữ liệu lớn song song và phân tán.

Chương 7: Các kỹ thuật xử lý luồng dữ liệu lớn với Spark Streaming - Từ Spark Streaming đến Structured Streaming.

Chương 8: Kiến trúc Lambda & Kappa - Các kiến trúc xử lý dữ liệu hiện đại.

Chương 9: Phân tích dữ liệu lớn với SparkML - Học máy và phân tích dữ liệu với Spark ML.

Cuốn sách này được biên soạn dựa trên kinh nghiệm giảng dạy và nghiên cứu của nhóm tác giả tại Trường Đại học Bách Khoa Hà Nội. Chúng tôi đã cố gắng trình bày các khái niệm một cách dễ hiểu, kèm theo các ví dụ thực tế và bài tập để sinh viên có thể áp dụng ngay vào thực tế.

Trong quá trình biên soạn, chúng tôi đã nhận được sự đóng góp quý báu từ nhiều thế hệ sinh viên Bách Khoa Hà Nội thông qua các khóa học, dự án nghiên cứu và phản hồi trong quá trình học tập. Những ý kiến đóng góp của các em đã giúp chúng tôi cải thiện nội dung và cách trình bày của cuốn sách.

Cuốn sách này còn nhiều thiếu sót và sẽ không thể hoàn thành được nếu không có sự trợ giúp từ các công cụ trí tuệ nhân tạo (AI) trong quá trình biên soạn, chỉnh sửa và tối ưu hóa nội dung. Chúng tôi trân trọng ghi nhận sự đóng góp của công nghệ AI trong việc nâng cao chất lượng giáo dục.

Chúng tôi rất mong nhận được sự góp ý từ bạn đọc để cuốn sách ngày càng hoàn thiện hơn. Mọi ý kiến đóng góp xin gửi về địa chỉ: Khoa Công nghệ Thông tin, Trường Đại học Bách Khoa Hà Nội, Số 1 Đại Cồ Việt, Hai Bà Trưng, Hà Nội.

Trân trọng cảm ơn!

Nhóm tác giả

Trường Đại học Bách Khoa Hà Nội

Hà Nội, 2024

Mục lục

Lời nói đầu	3
Mục lục	5
Chương 1. Tổng quan về lưu trữ và xử lý dữ liệu lớn	12
1.1. Sự bùng nổ của dữ liệu lớn	12
1.2. Khoa học dữ liệu: Kỷ nguyên mới của khám phá khoa học	13
1.2.1. Dữ liệu: Vàng đen của thế kỷ 21	14
1.2.2. Đặc điểm 5V của dữ liệu lớn	16
1.3. Công nghệ cho lưu trữ và xử lý dữ liệu lớn	17
1.3.1. Thách thức với tổ chức lưu trữ dữ liệu	17
1.3.2. Thách thức với tích hợp và xử lý dữ liệu	18
1.3.3. Thách thức với phân tích dữ liệu lớn	19
1.3.4. Thách thức trong khai thác và trực quan hoá dữ liệu lớn	20
1.4. Kết luận	22
Chương 2. Hệ sinh thái Hadoop	23
2.1. Giới thiệu Apache Hadoop	23
2.2. Hệ thống tệp tin phân tán Hadoop (HDFS)	24
2.3. Mô hình lập trình MapReduce	26
2.4. YARN – Bộ điều phối tài nguyên cụm Hadoop	26
2.5. Apache Hive	28
2.6. Apache Pig	29
2.7. Apache HBase	30
2.8. Apache Sqoop	31
2.9. Apache Kafka	32
2.10. Apache Oozie	34

2.11. Apache ZooKeeper	35
2.12. Tổng kết chương	36
Chương 3. Hệ thống Tập tin Phân tán Hadoop HDFS	39
3.1. Tổng quan về Hệ thống Tập tin Phân tán	39
3.1.1. Thách thức trong thiết kế Hệ thống Tập tin Phân tán	40
3.2. Kiến trúc và Thành phần của Hadoop HDFS	40
3.2.1. NameNode	40
3.2.2. DataNode	41
3.2.3. Secondary NameNode	41
3.3. Cơ chế Lưu trữ và Truy xuất Dữ liệu trong HDFS	42
3.3.1. Khối dữ liệu (HDFS Block)	42
3.3.2. Nhân bản (Replication)	42
3.3.3. Quy trình Đọc và Ghi tập tin	43
3.3.4. Tóm tắt đặc điểm thiết kế HDFS	44
3.4. Giao tiếp với HDFS qua dòng lệnh	44
3.5. Các tính năng nâng cao và Tối ưu hóa trong HDFS	45
3.5.1. Khả năng Mở rộng và Tính sẵn sàng cao	45
3.5.2. Tối ưu hóa hiệu năng	45
3.6. Quản lý Siêu dữ liệu Phân tán	46
3.6.1. Vấn đề của kiến trúc tập trung trong HDFS truyền thống	46
3.6.2. Các giải pháp quản lý siêu dữ liệu trong hệ thống DFS hiện đại	46
3.6.3. Tổng kết và so sánh các mô hình quản lý siêu dữ liệu	47
3.7. Các mô hình nhất quán trong Hệ thống Tập tin Phân tán	48
3.7.1. Tổng quan các mô hình nhất quán	48
3.7.2. So sánh các mô hình nhất quán	49
3.7.3. Ứng dụng và lựa chọn mô hình phù hợp	50
3.8. Caching và Lưu trữ đa tầng (Tiered Storage)	51
3.8.1. Mô hình lưu trữ phân tầng	51
3.8.2. Caching - Bộ nhớ đệm dữ liệu	51
3.8.3. Lợi ích và thách thức của Caching và Tiered Storage	52
3.8.4. Các hệ thống tiêu biểu	52

3.8.5. Tổng kết	53
3.9. Erasure Coding trong Hệ thống lưu trữ phân tán	53
3.9.1. Nguyên lý hoạt động của Erasure Coding	53
3.9.2. So sánh Erasure Coding với Replication	53
3.9.3. Ứng dụng trong các hệ thống thực tế	54
3.9.4. Thách thức trong triển khai Erasure Coding	54
3.9.5. Kết luận về Erasure Coding	55
3.10. Các kỹ thuật tối ưu hiệu năng trong Hệ thống Tập tin Phân tán	55
3.10.1. Prefetching - Tiên đoán dữ liệu	55
3.10.2. Write Coalescing - Gom ghi	55
3.10.3. Speculative Execution - Thực thi dự phòng	56
3.10.4. Data Striping - Phân mảnh dữ liệu song song	56
3.10.5. Parallelism - Truy cập song song	56
3.10.6. Tổng kết về các kỹ thuật tối ưu hiệu năng	56
Chương 4. Cơ sở dữ liệu phi quan hệ NoSQL	59
4.1. Giới thiệu về NoSQL	59
4.1.1. Các Kỹ Nguyên của Cơ Sở Dữ Liệu	59
4.1.2. Mô hình dữ liệu trước NoSQL	62
4.1.3. RDBMS: one size fits all needs	65
4.1.4. Thời kỳ sau khi NoSQL ra đời	67
4.2. Khái niệm và các trường hợp sử dụng NoSQL	70
4.2.1. Tại sao NoSQL?	70
4.2.2. SQL vs NoSQL	72
4.2.3. Các trường hợp sử dụng NoSQL	75
4.2.4. DB engines ranking according to their popularity (2019)	78
4.3. Các mô hình dữ liệu trong NoSQL	79
4.3.1. Mô hình dữ liệu quan hệ: Ôn tập	79
4.3.2. Mô hình dữ liệu Key/Value	81
4.3.3. Column family store	89
4.3.4. Mô hình dữ liệu đồ thị (Graph Data Model)	96
4.3.5. Document store	106

4.4.	Cơ sở dữ liệu phi quan hệ NoSQL - Phần 1: CAP theorem	115
4.5.	CAP theorem	115
4.5.1.	Vấn đề mở rộng RDBMS truyền thống	115
4.5.2.	Giải pháp hiện tại và thách thức	116
4.5.3.	Định lý CAP	120
4.5.4.	Các mô hình consistency	124
4.6.	Amazon DynamoDB	128
4.6.1.	Giới thiệu & Nguyên tắc thiết kế	128
4.6.2.	Kiến trúc hệ thống & Cơ chế lỗi	129
4.6.3.	Quản lý dữ liệu & Đảm bảo tính khả dụng	133
4.6.4.	Tổng kết & Đánh giá	141
4.7.	Xử lý truy vấn SQL cho dữ liệu lớn với Presto	144
4.7.1.	Giới thiệu & Động lực	144
4.7.2.	Tổng quan Presto	145
4.7.3.	Mô hình thực thi & Tối ưu hóa	155
4.7.4.	Đánh giá hiệu năng & Ứng dụng	176
Chương 5.	Hệ thống truyền thông điệp phân tán	183
5.1.	Mục tiêu học tập	183
5.2.	Giới thiệu về hệ thống xuất bản - đăng ký	183
5.2.1.	Kiến trúc và nguyên lý hoạt động	184
5.2.2.	Các mô hình triển khai	184
5.2.3.	Ứng dụng và Pattern phổ biến	185
5.2.4.	Thách thức và Giải pháp	185
5.2.5.	Giới thiệu về Apache Kafka	186
5.3.	Tổ chức và quản lý dữ liệu trong Kafka	186
5.3.1.	Các chủ đề (topic) trong Kafka	186
5.3.2.	Phân vùng (partition) của chủ đề (topic)	187
5.3.3.	Nhật ký phân vùng (partition log) và vị trí (offset)	187
5.3.4.	Bản sao phân vùng (partition replication) trong Kafka	188
5.3.5.	Thời gian tồn tại của bản ghi trong Kafka topic	189
5.4.	Sản xuất và tiêu thụ dữ liệu trong Kafka	189

5.4.1. Producer: Sản xuất dữ liệu	189
5.4.2. Consumer: Tiêu thụ dữ liệu	191
5.4.3. Cơ chế xử lý sự cố của consumer	192
5.4.4. Giới hạn về dữ liệu Consumer có thể đọc trong Kafka	192
5.4.5. Khả năng mở rộng và tốc độ của Kafka	192
5.5. Tóm tắt chương	193
5.6. Thuật ngữ quan trọng	193
5.7. Câu hỏi ôn tập	194

Chương 6. MapReduce – Mô hình lập trình cho xử lý dữ liệu lớn song song & phân tán **195**

6.1. Giới thiệu về MapReduce	195
6.2. Mô hình lập trình và kiến trúc	196
6.2.1. Mô hình lập trình MapReduce	196
6.2.2. Kiến trúc thực thi và vòng đời job	197
6.3. Tối ưu hóa và xử lý lỗi	199
6.3.1. Chịu lỗi và fault tolerance	199
6.3.2. HDFS và tính cục bộ dữ liệu	200
6.3.3. Giảm chi phí shuffle	200
6.3.4. Xử lý data skew	200
6.3.5. Speculative execution và cấu hình	201
6.4. Mẫu thiết kế và lập trình	202
6.4.1. Biến đổi và tổng hợp	202
6.4.2. Join trên dữ liệu lớn	202
6.4.3. Secondary sort	203
6.4.4. Đồ thị và thuật toán lặp	203
6.5. Giới hạn của MapReduce và bức tranh hệ sinh thái	203
6.5.1. Các chủ đề nâng cao	204
6.6. Ví dụ thực tế	204
6.7. Kết luận	207
6.8. Bài tập và câu hỏi ôn tập	207

Chương 7. Các kỹ thuật xử lý luồng dữ liệu lớn với Spark Streaming	209
7.1. Dữ liệu luồng	209
7.2. Spark Streaming	210
7.2.1. Các khái niệm và thao tác cơ bản	210
7.2.2. Caching và persistence	213
7.2.3. Biến tích lũy (accumulator) và biến quảng bá (broadcast variable)	214
7.2.4. Cơ chế chịu lỗi của Spark Streaming	217
7.3. Structured Streaming	218
7.3.1. Dữ liệu đầu vào	220
7.3.2. Biến đổi	222
7.3.3. Đầu ra	223
7.3.4. Trigger	224
7.4. Kết chương	224
 Chương 8. Kiến trúc Lambda & Kappa	 225
8.1. Kiến trúc Lambda	225
8.1.1. Khái niệm cốt lõi: Xử lý theo lô và xử lý luồng	225
8.1.2. Các thành phần chính của Kiến trúc Lambda	226
8.1.3. Mô hình nhất quán cuối cùng (Eventual Consistency Model)	228
8.1.4. Ưu điểm của kiến trúc Lambda	228
8.1.5. Thách thức và hạn chế	229
8.2. Kiến trúc Kappa	230
8.2.1. Kappa như một sự đơn giản hóa của kiến trúc Lambda	230
8.2.2. Nguyên tắc cốt lõi của kiến trúc Kappa	230
8.2.3. Các thành phần chính	230
8.2.4. Luồng dữ liệu trong kiến trúc Kappa	231
8.2.5. Xử lý lại dữ liệu trong kiến trúc Kappa	231
8.2.6. Ưu điểm của kiến trúc Kappa	232
8.2.7. Thách thức và hạn chế	232
 Chương 9. Phân tích dữ liệu lớn với SparkML	 234
9.1. Giới thiệu chung	234

9.2. Spark ML	235
9.2.1. Transformer	236
9.2.2. Estimator	238
9.2.3. Pipeline	239
9.3. Kết chương	240
Tài liệu tham khảo	241

Chương 1

Tổng quan về lưu trữ và xử lý dữ liệu lớn

1.1. Sự bùng nổ của dữ liệu lớn	12
1.2. Khoa học dữ liệu: Kỷ nguyên mới của khám phá khoa học	13
1.3. Công nghệ cho lưu trữ và xử lý dữ liệu lớn	17
1.4. Kết luận	22

1.1. Sự bùng nổ của dữ liệu lớn

Thế giới đang chứng kiến sự bùng nổ dữ liệu với tốc độ chưa từng có. Theo IDC ¹, dự kiến đến năm 2025 tổng dung lượng dữ liệu số trên Internet sẽ tăng lên 175 zettabyte, gấp 10 lần so với năm 2018. Sự bùng nổ dữ liệu lớn này được thúc đẩy bởi nhiều yếu tố, bao gồm:

Sự phát triển của Internet và các dịch vụ trực tuyến Sự phổ biến của Internet và các dịch vụ trực tuyến đã tạo ra lượng dữ liệu khổng lồ từ các hoạt động như mua sắm, mạng xã hội, tìm kiếm và truyền thông. Các nền tảng như Facebook, Twitter, Instagram và LinkedIn cung cấp dữ liệu về hành vi người dùng, xu hướng xã hội và sở thích cá nhân. Người dùng liên tục tạo ra bài viết, hình ảnh, video và các phản hồi tương tác; mỗi hành động đều góp phần mở rộng kho dữ liệu. Thương mại điện tử cũng là nguồn dữ liệu quan trọng với thông tin về sản phẩm, giá, phương thức thanh toán và vị trí giao hàng. Những dữ liệu này được dùng để cải thiện trải nghiệm, phát triển sản phẩm và tối ưu hóa tiếp thị.

Sự gia tăng của các thiết bị thông minh và IoT (Internet of Things) Các thiết bị như điện thoại thông minh, thiết bị đeo và cảm biến IoT liên tục thu thập, truyền tải dữ liệu. Chúng hiện diện từ nhà ở, phương tiện cho tới các thành phố thông minh, góp phần tạo ra lượng dữ liệu khổng lồ theo thời gian.

¹<https://www.seagate.com/files/www-content/our-story/trends/files/Seagate-WP-DataAge2025-March-2017.pdf>

Các thí nghiệm nghiên cứu sử dụng dữ liệu lớn Các thí nghiệm trong các lĩnh vực như sinh tin học, vật lý và thiên văn học tạo ra khối dữ liệu rất lớn phục vụ việc giải quyết các vấn đề khoa học phức tạp. Những dữ liệu này được dùng để xây dựng mô hình, kiểm chứng giả thuyết và thúc đẩy các khám phá mới.

Ứng dụng rộng rãi của dữ liệu lớn trong các lĩnh vực khác nhau Dữ liệu lớn được ứng dụng trong y tế, tài chính, bán lẻ, sản xuất và quản lý nhà nước. Mục tiêu là tối ưu hóa quy trình, nâng cao trải nghiệm người dùng và hỗ trợ các quyết định chiến lược ở quy mô tổ chức.

Bối cảnh dữ liệu hiện nay không chỉ đến từ các nền tảng tiêu dùng như mạng xã hội và thương mại điện tử mà còn từ các hệ thống doanh nghiệp và thiết bị biên (edge/IoT). Các báo cáo gần đây cho thấy quy mô dữ liệu toàn cầu tiếp tục tăng theo cấp số mũ với đóng góp ngày càng lớn từ cảm biến, nhật ký vận hành và tín hiệu ứng dụng. Sự chuyển dịch này làm thay đổi cả cấu trúc lẫn động học của dòng dữ liệu.

Lượng dữ liệu tăng trưởng kéo theo áp lực đáng kể lên hạ tầng lưu trữ và xử lý. Nhiều tổ chức phân tầng dữ liệu “nóng” phục vụ tác vụ thời gian thực và dữ liệu “lạnh” cho phân tích theo lô, đồng thời thiết kế đường ống dữ liệu có thể hấp thụ dòng sự kiện tốc độ cao. Nhu cầu giảm độ trễ khiến xử lý tại biên (edge) trở thành lựa chọn tự nhiên cho các kịch bản nhạy thời gian.

Cùng với quy mô, yêu cầu về quản trị dữ liệu ngày càng trở nên trọng yếu. Chất lượng, tính nhất quán và khả năng truy xuất nguồn gốc (data lineage) quyết định độ tin cậy của phân tích. Các khung tuân thủ về bảo mật và quyền riêng tư đặt ra tiêu chuẩn cho thu thập, lưu trữ và sử dụng dữ liệu một cách có trách nhiệm, đặc biệt khi kết hợp với các mô hình học máy.

Khía cạnh bền vững cũng cần được cân nhắc. Lưu trữ và xử lý khối lượng dữ liệu lớn tiêu tốn năng lượng đáng kể, làm gia tăng chi phí vận hành và dấu chân carbon. Tối ưu vòng đời dữ liệu, nén hiệu quả và loại bỏ dữ liệu kém giá trị theo chính sách rõ ràng giúp cân bằng giữa giá trị phân tích và chi phí cũng như tác động môi trường.

Tóm lại, bùng nổ dữ liệu không chỉ là tăng về lượng mà còn là thay đổi về tốc độ, đa dạng và yêu cầu quản trị. Các đặc trưng này sẽ được khái quát hóa trong mô hình 5V và phản ánh trực tiếp vào các lựa chọn kiến trúc ở các phần tiếp theo.

1.2. Khoa học dữ liệu: Kỷ nguyên mới của khám phá khoa học

Khoa học đã trải qua bốn kỷ nguyên chính, mỗi kỷ nguyên đánh dấu một bước tiến lớn trong phương pháp tiếp cận và khám phá thế giới:

Khoa học thực nghiệm (Experimental Science) Kỷ nguyên này bắt đầu từ thời kỳ tiền sử, khi con người bắt đầu quan sát và ghi chép các hiện tượng tự nhiên. Các nhà khoa học chủ yếu dựa vào giác quan và các công cụ đơn giản như kính thiên văn và kính hiển vi để thu thập dữ liệu.

Khoa học lý thuyết (Theoretical Science) Bắt đầu vào thế kỷ 17 với sự ra đời của phương pháp khoa học, các nhà khoa học xây dựng các lý thuyết để giải thích các hiện tượng tự nhiên dựa trên quan sát và dữ liệu thu thập được.

Khoa học tính toán (Computational Science) Bắt đầu vào thế kỷ 20 với sự phát triển của máy tính, các nhà khoa học sử dụng máy tính để mô phỏng các hiện tượng tự nhiên và kiểm tra các lý thuyết.

Khoa học dữ liệu (Data Science) Kỷ nguyên này bắt đầu vào thế kỷ 21 với sự bùng nổ dữ liệu. Các nhà khoa học sử dụng các kỹ thuật khoa học dữ liệu để phân tích và khai thác thông tin có giá trị từ dữ liệu lớn.

Khoa học dữ liệu (Data Science) nổi lên như một lĩnh vực liên ngành, kết hợp các kỹ thuật thống kê, học máy, phân tích dữ liệu và các lĩnh vực liên quan để khai thác thông tin có giá trị từ dữ liệu. Khoa học dữ liệu được xem là "kỷ nguyên thứ tư của khám phá khoa học," tiếp nối các kỷ nguyên khoa học thực nghiệm, khoa học lý thuyết và khoa học tính toán.

1.2.1. Dữ liệu: Vàng đen của thế kỷ 21

Trong thế kỷ 20, dầu mỏ được xem là nguồn tài nguyên quý giá nhất, thúc đẩy sự phát triển kinh tế và công nghiệp trên toàn cầu. Tuy nhiên, bước sang thế kỷ 21, vai trò này đã được thay thế bởi một nguồn tài nguyên mới, quý giá không kém: dữ liệu. Dữ liệu được ví như "vàng đen" của thời đại mới, sở hữu tiềm năng to lớn trong việc định hình tương lai của xã hội và nền kinh tế.

Dữ liệu là thông tin thô được thu thập từ nhiều nguồn khác nhau, bao gồm các hoạt động kinh doanh, giao dịch thương mại điện tử, tương tác trên mạng xã hội, sử dụng thiết bị thông minh, và nhiều nguồn khác. Dữ liệu chứa đựng vô số thông tin giá trị về hành vi, sở thích, nhu cầu của con người, từ đó giúp các doanh nghiệp, tổ chức và chính phủ đưa ra những quyết định sáng suốt và hiệu quả hơn.

Dưới đây là một số lý do khiến dữ liệu trở nên quý giá:

Dữ liệu phản ánh thực tế Dữ liệu cung cấp cho chúng ta bức tranh chân thực về thế giới xung quanh, giúp hiểu rõ hơn về hành vi con người, xu hướng thị trường và các yếu tố tác động đến mọi khía cạnh của đời sống. Thông qua dữ liệu, các nhà nghiên cứu



Hình 1.1 The end of science - Sự chuyển đổi từ khoa học truyền thống sang khoa học dữ liệu

có thể phát hiện ra các mô hình và xu hướng mà bằng cách tiếp cận truyền thống có thể bị bỏ qua.

Dữ liệu hỗ trợ ra quyết định Nhờ phân tích dữ liệu, các doanh nghiệp có thể hiểu rõ hơn về khách hàng, tối ưu hóa hoạt động kinh doanh, phát triển sản phẩm và dịch vụ phù hợp, đồng thời đưa ra chiến lược marketing hiệu quả hơn. Phân tích dữ liệu giúp dự đoán hành vi tương lai của khách hàng và điều chỉnh các chiến lược kinh doanh theo đó.

Dữ liệu thúc đẩy đổi mới Dữ liệu là nguồn nguyên liệu quan trọng cho các công nghệ tiên tiến như trí tuệ nhân tạo (AI) và học máy (machine learning). Những công nghệ này sử dụng dữ liệu để tự động hóa quy trình, nâng cao hiệu quả và tạo ra những sản phẩm, dịch vụ mới mẻ. Ví dụ, AI có thể cải thiện khả năng chẩn đoán trong y học, tối ưu hóa chuỗi cung ứng trong sản xuất, và cá nhân hóa trải nghiệm người dùng trong thương mại điện tử.

Dữ liệu mang lại lợi thế cạnh tranh Doanh nghiệp sở hữu và khai thác dữ liệu hiệu quả sẽ có lợi thế cạnh tranh đáng kể so với các đối thủ. Họ có thể đưa ra những quyết định nhanh chóng, chính xác, và đáp ứng tốt hơn nhu cầu của khách hàng. Các doanh nghiệp này cũng có thể tối ưu hóa chi phí, nâng cao hiệu suất và phát triển những mô hình kinh doanh mới dựa trên sự hiểu biết sâu sắc từ dữ liệu.

Như vậy, dữ liệu không chỉ là tài nguyên quan trọng của thế kỷ 21 mà còn là yếu tố then chốt giúp các tổ chức, doanh nghiệp và chính phủ đưa ra những quyết định thông minh, thúc đẩy đổi mới và duy trì lợi thế cạnh tranh trong môi trường kinh tế toàn cầu.

1.2.2. Đặc điểm 5V của dữ liệu lớn



Hình 1.2 Đặc điểm 5V của dữ liệu lớn: Volume, Velocity, Variety, Veracity, Value

Dữ liệu lớn (Big Data) là một thuật ngữ dùng để mô tả tập dữ liệu có kích thước khổng lồ, phức tạp và tốc độ tăng trưởng nhanh chóng, vượt quá khả năng xử lý của các hệ thống xử lý dữ liệu truyền thống. Lưu ý rằng khái niệm các hệ thống lưu trữ và xử lý dữ liệu truyền thống ở đây là các hệ thống xuất hiện và phổ biến trước 2006 (trước khi có sự ra đời của Hadoop và Mapreduce).

Để mô tả tính chất của dữ liệu lớn, người ta thường sử dụng mô hình 5V, bao gồm:

Volume (Khối lượng) Dữ liệu lớn có kích thước khổng lồ, có thể lên đến hàng petabyte (PB) hoặc exabyte (EB). Khối lượng dữ liệu lớn khiến cho việc lưu trữ và xử lý trở nên phức tạp.

Velocity (Tốc độ) Dữ liệu lớn được tạo ra và thu thập liên tục với tốc độ rất nhanh, đòi hỏi hệ thống xử lý phải có khả năng xử lý dữ liệu theo thời gian thực (real-time).

Variety (Đa dạng) Dữ liệu lớn có nhiều dạng khác nhau, bao gồm dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc. Điều này khiến cho việc xử lý và phân tích dữ liệu trở nên khó khăn hơn.

Veracity (Tính xác thực) Đảm bảo tính chính xác và tin cậy của dữ liệu là một thách thức lớn đối với dữ liệu lớn, do dữ liệu có thể bị nhiễu hoặc sai sót trong quá trình thu thập và lưu trữ.

Value (Giá trị) Mục tiêu cuối cùng của việc thu thập và xử lý dữ liệu lớn là khai thác giá

trị tiềm ẩn từ dữ liệu để phục vụ cho các mục đích khác nhau như ra quyết định, cải thiện hiệu quả hoạt động, phát triển sản phẩm mới, v.v.

1.3. Công nghệ cho lưu trữ và xử lý dữ liệu lớn

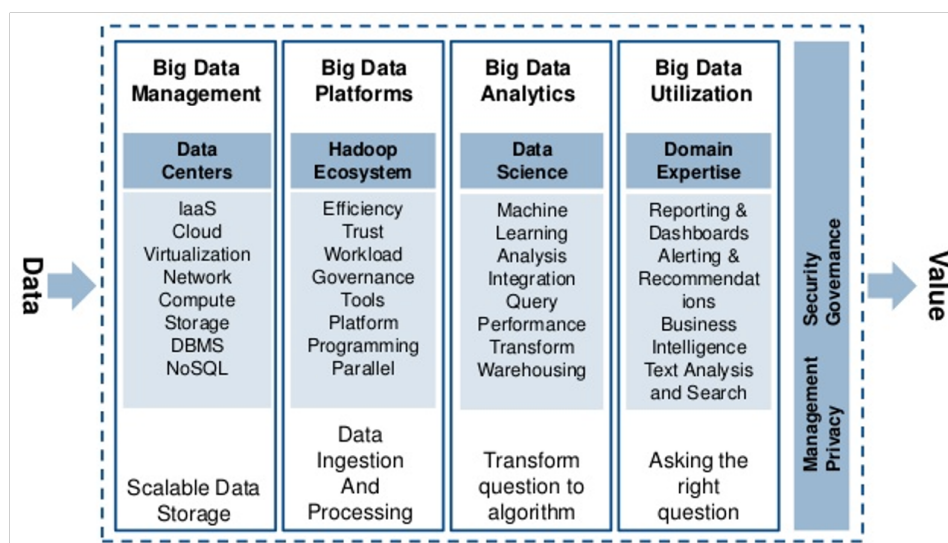
Các công nghệ cần thiết cho tổ chức lưu trữ và xử lý dữ liệu lớn có thể đưa vào kiến trúc phân tầng như sau:

Tầng lưu trữ dữ liệu Bao gồm các nền tảng, công nghệ và giải pháp phục vụ cho mục tiêu tổ chức lưu trữ dữ liệu hiệu quả và khả mở.

Tầng tích hợp và xử lý dữ liệu Bao gồm các nền tảng, công nghệ và giải pháp phục vụ cho tích hợp dữ liệu từ nhiều nguồn và thực hiện các công việc liên quan tới biến đổi và tính toán trên dữ liệu.

Tầng phân tích dữ liệu Bao gồm các nền tảng, công nghệ, và giải pháp phục vụ cho mục tiêu cài đặt các giải thuật khai phá dữ liệu dựa trên công nghệ xử lý dữ liệu được cung cấp ở tầng tích hợp và xử lý dữ liệu.

Tầng ứng dụng và khai thác dữ liệu Bao gồm các công nghệ trực quan hoá dữ liệu hiệu quả và các kiến thức lĩnh vực cho phép biểu diễn các bài toán cụ thể và diễn giải các kết quả phân tích dữ liệu.



Hình 1.3 Kiến trúc phân tầng các công nghệ cho lưu trữ và xử lý dữ liệu lớn

1.3.1. Thách thức với tổ chức lưu trữ dữ liệu

Các thách thức đối với các hệ thống tổ chức lưu trữ dữ liệu:

Khả năng mở rộng (Scalability) Hệ thống phải có khả năng mở rộng theo chiều ngang (horizontal scaling) và theo chiều dọc (vertical scaling) để đáp ứng được sự gia tăng về khối lượng dữ liệu. Mở rộng theo chiều ngang: Thêm các nút (node) mới vào hệ thống để phân tán dữ liệu và xử lý song song. Mở rộng theo chiều dọc: nâng cấp phần cứng của các nút hiện có, chẳng hạn như tăng dung lượng lưu trữ và bộ nhớ RAM.

Khả năng truy cập (Accessibility) Hệ thống phải duy trì hiệu quả truy cập dữ liệu (read/write - I/O) ngay cả khi khối lượng dữ liệu tăng lên. Điều này đòi hỏi các kỹ thuật lưu trữ phân tán hiệu quả và tối ưu hóa truy vấn dữ liệu.

Tính minh bạch (Transparency) Trong môi trường phân tán, người dùng cần có thể truy cập dữ liệu qua mạng một cách dễ dàng như thể dữ liệu được lưu trữ cục bộ. Người dùng không cần biết vị trí vật lý của dữ liệu để truy cập. Các hệ thống cần cung cấp giao diện truy cập thống nhất và ẩn đi tính phức tạp của phân tán dữ liệu.

Tính sẵn sàng (Availability) Hệ thống phải đảm bảo khả năng truy cập dữ liệu liên tục bất kể số lượng người dùng, lỗi hệ thống hoặc các tác động khác của việc phân tán. Các hệ thống cần có khả năng chịu lỗi cao (fault tolerance) để đảm bảo tính sẵn sàng ngay cả khi một số nút trong hệ thống gặp sự cố.

1.3.2. Thách thức với tích hợp và xử lý dữ liệu

Dữ liệu lớn đòi hỏi năng lực thu nạp, hòa trộn và xử lý dữ liệu có *khả năng mở rộng* để khai thác triệt để giá trị từ các nguồn đa dạng, phân tán và liên tục gia tăng. Bên cạnh hiệu năng, các yêu cầu về chất lượng, quản trị và an toàn thông tin chi phối mạnh thiết kế đường ống dữ liệu.

Thách thức đối với tích hợp dữ liệu:

Tích hợp từ nhiều nguồn dữ liệu khác nhau Dữ liệu thường đến từ các hệ thống bổ sung cho nhau (OLTP, nhật ký ứng dụng, cảm biến/IoT, dịch vụ bên thứ ba). Khó khăn xuất phát từ nguồn phân tán, giao diện kết nối không đồng nhất (API, CDC, tệp theo lô) và khác biệt về tốc độ/phát sinh sự kiện. Bài toán là thu nạp ổn định, có thứ tự và có thể mở rộng.

Đa dạng định dạng dữ liệu Định dạng trải từ văn bản, JSON, XML đến ảnh/video và nhị phân miền chuyên biệt. Hệ thống thu thập cần chuẩn hóa, chuẩn bị siêu dữ liệu và lựa chọn giữa schema-on-read hay schema-on-write phù hợp với mục tiêu phân tích.

Mô hình và lược đồ dữ liệu khác nhau Các nguồn dùng mô hình và lược đồ riêng. Cần ánh xạ sang mô hình thống nhất, quản lý tiến hóa lược đồ (schema evolution) và sử dụng đăng ký lược đồ (schema registry) để bảo đảm tương thích ngược/tiến.

Bảo mật và quyền riêng tư Dữ liệu có thể chứa thông tin nhạy cảm/PII. Cần kiểm soát truy cập, mã hóa, ẩn danh hóa/giả danh hóa và áp dụng chính sách DLP trong toàn bộ vòng đời thu thập và lưu trữ.

Ví dụ ngắn. Hệ thống thu nạp clickstream (JSON) từ Kafka kết hợp giao dịch từ cơ sở dữ liệu qua CDC; dữ liệu được chuẩn hóa vào một sơ đồ chung, xử lý sự cố trật tự/thất lạc sự kiện và bù trễ giữa các nguồn trước khi ghi vào lakehouse.

Thách thức đối với xử lý dữ liệu khả mở:

Xử lý theo lô Cần kết hợp sức mạnh xử lý phân tán để hoàn thành khối lượng lớn trong thời gian bảo đảm (SLA). Tổ chức theo cửa sổ thời gian, lập lịch hợp lý và tối ưu vị trí dữ liệu giúp giảm chi phí di chuyển dữ liệu.

Xử lý luồng dữ liệu lớn Ứng dụng thời gian thực yêu cầu độ trễ thấp, semantics đáng tin cậy (at-least-once/exactly-once) và cơ chế watermark cho dữ liệu đến trễ. Hệ thống phải xử lý backpressure và mở rộng linh hoạt theo lưu lượng.

Hạn chế của kỹ thuật truyền thống Các mô hình song song truyền thống (OpenMP, MPI) có độ dốc học tập cao, khó mở rộng linh hoạt và phức tạp về chịu lỗi; thường cần hạ tầng HPC đắt đỏ, không phù hợp cho nhiều kịch bản dữ liệu lớn định hướng sự kiện.

Ví dụ ngắn. Phát hiện gian lận trực tuyến kết hợp chấm điểm streaming trên Flink/Spark Structured Streaming với cập nhật mô hình theo lô qua đêm; pipeline bảo đảm exactly-once và kiểm soát độ trễ đầu-cuối dưới ngưỡng nghiệp vụ.

1.3.3. Thách thức với phân tích dữ liệu lớn

Phân tích dữ liệu lớn đòi hỏi các thuật toán có khả năng mở rộng để xử lý hiệu quả khối lượng dữ liệu khổng lồ, phức tạp và đa chiều. Thách thức này thể hiện rõ qua hai khía cạnh: lời nguyền số chiều và yêu cầu tính toán phân tán.

Lời nguyền số chiều (Curse of Dimensionality) là hiện tượng khi số lượng thuộc tính (features) trong dữ liệu tăng lên, số lượng mẫu cần thiết để đạt được độ chính xác mong muốn cũng tăng theo hàm mũ. Ví dụ, với bài toán phân loại hình ảnh, việc tăng độ phân giải từ 32x32 lên 64x64 pixel không chỉ tăng số chiều từ 1024 lên 4096, mà còn đòi hỏi số lượng mẫu huấn luyện tăng gấp nhiều lần để duy trì độ chính xác. Trong thực tế, số lượng mẫu huấn luyện thường bị giới hạn, dẫn đến hiệu suất của thuật toán phân loại có thể giảm khi số chiều tăng quá cao.

Để giải quyết thách thức này, chúng ta có hai hướng tiếp cận chính: giảm chiều dữ liệu và phân tán hóa tính toán.

Hướng thứ nhất tập trung vào việc giảm kích thước dữ liệu một cách thông minh:

- **Lấy mẫu con (Sub-sampling):** Chọn tập con đại diện dựa trên phân phối xác suất hoặc các ràng buộc về tính đại diện. Ví dụ, trong bài toán phân tích hành vi người dùng trên mạng xã hội, ta có thể chọn mẫu theo tầng (stratified sampling) để đảm bảo tỷ lệ các nhóm tuổi, giới tính được giữ nguyên.
- **Phân tích thành phần chính (PCA):** Tìm các trục chính chứa nhiều thông tin nhất của dữ liệu. Trong nhận dạng khuôn mặt, PCA có thể giảm hàng nghìn pixel xuống vài chục eigenfaces mà vẫn giữ được đặc trưng quan trọng để nhận dạng.
- **Trích chọn thuộc tính:** Kết hợp feature extraction (tạo thuộc tính mới) và feature selection (chọn thuộc tính quan trọng). Ví dụ, trong phân tích văn bản, ta có thể dùng word embeddings (extraction) kết hợp với mutual information (selection) để giảm số chiều từ hàng triệu từ vựng xuống vài trăm chiều có ý nghĩa nhất.

Hướng thứ hai tập trung vào phân tán hóa tính toán để xử lý dữ liệu lớn hiệu quả:

- **Phân tán k-NN:** Sử dụng MapReduce để tính song song khoảng cách giữa các điểm dữ liệu. Map phase tính khoảng cách từ điểm truy vấn đến các điểm trong phân vùng cục bộ, Reduce phase tổng hợp và chọn k láng giềng gần nhất. Kỹ thuật này cho phép mở rộng k-NN lên hàng tỷ điểm dữ liệu.
- **SVM phân tán:** Chia bài toán tối ưu SVM thành các bài toán con nhỏ hơn, giải song song trên nhiều máy rồi kết hợp kết quả. Phương pháp này đã được ứng dụng thành công trong phân loại văn bản và hình ảnh quy mô lớn.

Ví dụ thực tế. Hệ thống gợi ý sản phẩm của Amazon xử lý hàng tỷ tương tác người dùng-sản phẩm. Họ kết hợp giảm chiều bằng matrix factorization để tìm các yếu tố tiềm ẩn, với tính toán phân tán trên Spark để cập nhật mô hình theo thời gian thực. Điều này cho phép đưa ra gợi ý cá nhân hóa cho hàng trăm triệu người dùng với độ trễ thấp.

1.3.4. Thách thức trong khai thác và trực quan hoá dữ liệu lớn

Khai thác và trực quan hóa dữ liệu lớn đặt ra ba thách thức chính: chuyên môn lĩnh vực, kỹ thuật trực quan, và bảo mật quyền riêng tư. Mỗi thách thức này đòi hỏi cách tiếp cận riêng và thường đan xen, phụ thuộc lẫn nhau.

Thách thức đầu tiên là yêu cầu về chuyên môn lĩnh vực để đặt câu hỏi đúng và diễn giải kết quả chính xác. Chuyên gia dữ liệu cần kết hợp ba khía cạnh: kiến thức chuyên môn (domain expertise), kỹ năng phân tích, và hiểu biết về mục tiêu kinh doanh. Ví dụ, trong lĩnh vực y tế, việc phân tích dữ liệu bệnh nhân đòi hỏi hiểu biết sâu về các chỉ số sinh học, quy trình điều trị và tác động của các yếu tố môi trường. Không có kiến thức y khoa, một

nhà khoa học dữ liệu có thể bỏ qua các mối tương quan quan trọng hoặc rút ra kết luận sai lệch về hiệu quả điều trị.

Thách thức thứ hai liên quan đến kỹ thuật và công cụ trực quan hóa. Với dữ liệu lớn, việc tạo ra các biểu diễn trực quan vừa thông tin vừa hiệu quả về mặt tính toán là không đơn giản:

- **Tối ưu hóa hiệu năng:** Khi trực quan hóa hàng triệu điểm dữ liệu, cần áp dụng các kỹ thuật như tổng hợp (aggregation), lấy mẫu thích ứng (adaptive sampling) và phân cụm (clustering) để duy trì tính tương tác. Ví dụ, bản đồ nhiệt của Uber hiển thị nhu cầu đi lại theo thời gian thực bằng cách tự động điều chỉnh độ phân giải dựa trên mức độ phóng to.
- **Thiết kế tương tác:** Người dùng cần các công cụ để khám phá dữ liệu một cách trực quan như lọc động (dynamic filtering), khoan sâu (drill-down) và liên kết nhiều góc nhìn (linked views). Tableau và Power BI cung cấp các tính năng này, nhưng việc thiết kế dashboard hiệu quả vẫn đòi hỏi hiểu biết sâu về nguyên tắc thiết kế và tâm lý nhận thức.
- **Kể chuyện bằng dữ liệu:** Trực quan hóa không chỉ là vẽ biểu đồ mà còn là cách dẫn dắt người xem qua một câu chuyện có ý nghĩa. Điều này đòi hỏi kỹ năng chọn lọc thông tin quan trọng, sắp xếp logic và thiết kế trải nghiệm tương tác phù hợp với đối tượng người dùng.

Thách thức thứ ba là đảm bảo bảo mật và quyền riêng tư trong quá trình khai thác và trực quan hóa. Điều này đặc biệt quan trọng khi làm việc với dữ liệu nhạy cảm như thông tin y tế, tài chính hay hành vi người dùng:

- **Kiểm soát truy cập:** Xây dựng hệ thống phân quyền chi tiết (fine-grained access control) cho phép người dùng chỉ thấy dữ liệu họ được phép truy cập. Ví dụ, một bác sĩ chỉ có thể xem thông tin bệnh nhân của mình, trong khi nhà nghiên cứu chỉ thấy dữ liệu đã được ẩn danh.
- **Bảo vệ quyền riêng tư:** Áp dụng các kỹ thuật như k-anonymity, differential privacy để đảm bảo không thể nhận dạng cá nhân từ dữ liệu tổng hợp. Netflix Prize đã phải dừng vì các nhà nghiên cứu chứng minh có thể kết hợp dữ liệu đánh giá phim đã ẩn danh với thông tin công khai để nhận dạng người dùng.
- **Tuân thủ quy định:** Đảm bảo việc khai thác và trực quan hóa tuân thủ các quy định như GDPR (EU) và HIPAA (Hoa Kỳ). Điều này bao gồm ghi nhật ký truy cập, xin phép người dùng và có khả năng giải thích cách dữ liệu được sử dụng.

Ví dụ thực tế. Một bệnh viện lớn xây dựng hệ thống dashboard theo dõi hiệu quả điều trị. Họ kết hợp: (1) chuyên gia y tế để xác định các chỉ số quan trọng, (2) kỹ thuật trực quan như sparklines để theo dõi xu hướng và biểu đồ phân cấp để khám phá chi tiết, và (3) differential privacy để bảo vệ thông tin bệnh nhân khi chia sẻ báo cáo tổng hợp với các bên liên quan.

1.4. Kết luận

Chương này đã giới thiệu tổng quan về lưu trữ và xử lý dữ liệu lớn, từ bối cảnh bùng nổ dữ liệu đến các thách thức cốt lõi trong việc khai thác giá trị từ dữ liệu. Chúng ta đã thấy rằng dữ liệu lớn không chỉ là vấn đề về quy mô mà còn liên quan đến tốc độ, đa dạng, độ tin cậy và giá trị - được tổng hợp trong mô hình 5V. Kiến trúc phân tầng cho lưu trữ và xử lý dữ liệu lớn đã được đề xuất, trong đó mỗi tầng đối mặt với những thách thức riêng về khả năng mở rộng, hiệu năng và độ tin cậy.

Các thách thức này đòi hỏi cách tiếp cận mới trong thiết kế hệ thống và thuật toán. Từ việc tổ chức lưu trữ phân tán, xử lý dữ liệu khả mở, đến phân tích và trực quan hóa hiệu quả - mỗi khía cạnh cần được giải quyết một cách tổng thể, có tính đến các yêu cầu về bảo mật, quyền riêng tư và tuân thủ quy định. Đặc biệt, sự kết hợp giữa chuyên môn lĩnh vực và kỹ thuật là yếu tố then chốt để khai thác thành công giá trị từ dữ liệu lớn.

Trong các chương tiếp theo, chúng ta sẽ đi sâu vào từng tầng của kiến trúc, khám phá các giải pháp cụ thể cho việc lưu trữ, xử lý, phân tích và khai thác dữ liệu lớn. Mỗi giải pháp sẽ được phân tích chi tiết về mặt lý thuyết, đồng thời minh họa qua các ví dụ thực tế từ các hệ thống đang vận hành.

Chương 2

Hệ sinh thái Hadoop

2.1. Giới thiệu Apache Hadoop	23
2.2. Hệ thống tập tin phân tán Hadoop (HDFS)	24
2.3. Mô hình lập trình MapReduce	26
2.4. YARN – Bộ điều phối tài nguyên cụm Hadoop	26
2.5. Apache Hive	28
2.6. Apache Pig	29
2.7. Apache HBase	30
2.8. Apache Sqoop	31
2.9. Apache Kafka	32
2.10. Apache Oozie	34
2.11. Apache ZooKeeper	35
2.12. Tổng kết chương	36

2.1. Giới thiệu Apache Hadoop

Apache Hadoop là một nền tảng mã nguồn mở được thiết kế để lưu trữ và xử lý khối lượng dữ liệu lớn trên cụm máy tính. Nền tảng này cung cấp khả năng lưu trữ **scalable** (có thể mở rộng) và **reliable** (đáng tin cậy), đồng thời hỗ trợ **xử lý dữ liệu song song** trên phần cứng thông dụng.

Hadoop được thiết kế theo kiến trúc **scale-out**, tức là mở rộng bằng cách thêm nhiều máy chủ thay vì nâng cấp một máy chủ duy nhất. Kiến trúc này cho phép mở rộng cụm lên hàng chục nghìn node và vận hành hiệu quả trên phần cứng thông dụng. Hệ thống được thiết kế với khả năng chịu lỗi cao, trong đó hồng học phần cứng được xem là điều bình thường. Khi phát hiện lỗi, hệ thống sẽ tự động nhân bản dữ liệu và chuyển tác vụ sang node khác.

Hadoop được phát triển dựa trên các nghiên cứu của Google về hệ thống tập tin phân tán và mô hình MapReduce, cụ thể là các bài báo về **Google File System (GFS)** và **MapReduce**. Mục tiêu chính là giải quyết bài toán lưu trữ và xử lý dữ liệu lớn trong môi trường phân tán.

Hệ sinh thái Hadoop bao gồm bốn thành phần cốt lõi:

- **Hadoop HDFS (Hadoop Distributed File System)**: Hệ thống tập tin phân tán cung

cấp khả năng lưu trữ dữ liệu dung lượng lớn trên cụm máy tính.

- **Hadoop MapReduce**: Mô hình lập trình phân tán cho xử lý dữ liệu lớn theo kiểu batch.
- **Hadoop YARN (Yet Another Resource Negotiator)**: Hệ thống quản lý tài nguyên và lập lịch tác vụ, cho phép nhiều ứng dụng chạy đồng thời trên cùng cụm.
- **Hadoop Common**: Bộ thư viện và tiện ích chung hỗ trợ các module khác của Hadoop.

Các thành phần này hoạt động phối hợp để tạo thành nền tảng lưu trữ và xử lý dữ liệu lớn toàn diện. HDFS đảm bảo lưu trữ tin cậy, MapReduce cung cấp mô hình lập trình đơn giản cho xử lý phân tán, và YARN điều phối hiệu quả việc sử dụng tài nguyên. Sự kết hợp này giúp doanh nghiệp xử lý khối lượng dữ liệu khổng lồ với chi phí hợp lý trên hạ tầng phần cứng thông dụng.

2.2. Hệ thống tệp tin phân tán Hadoop (HDFS)

HDFS (Hadoop Distributed File System) là hệ thống quản lý tệp tin phân tán được thiết kế để lưu trữ các tệp dữ liệu lớn một cách tin cậy trên cụm máy tính. HDFS được tối ưu hóa cho các tệp có kích thước lớn (hàng trăm MB đến TB) và hỗ trợ mô hình truy cập dữ liệu **write-once, read-many** (ghi một lần, đọc nhiều lần).

Hệ thống được thiết kế để đạt **thông lượng cao** thay vì độ trễ thấp, do đó phù hợp với các ứng dụng xử lý **batch** hơn là các ứng dụng yêu cầu truy xuất ngẫu nhiên với độ trễ thấp. Dữ liệu thường được ghi vào HDFS một lần (ví dụ: nhập kho dữ liệu thô) và sau đó được đọc và phân tích nhiều lần.

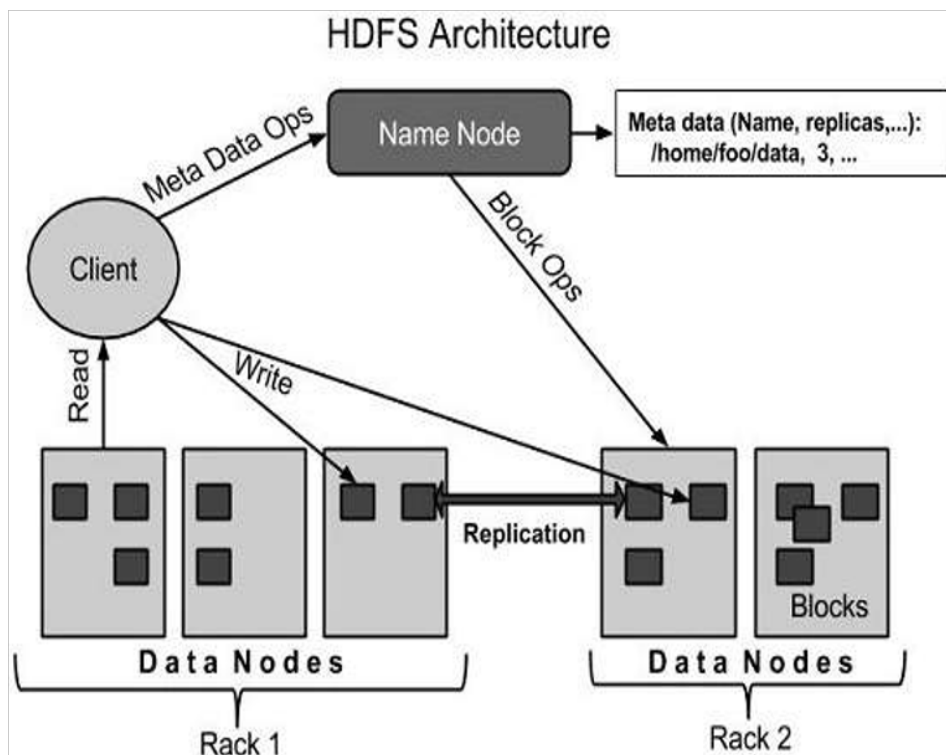
HDFS được thiết kế để chạy trên phần cứng thông dụng thay vì các thiết bị lưu trữ chuyên dụng đắt tiền. Hệ thống tận dụng nhiều máy tính rẻ tiền và chấp nhận rằng lỗi phần cứng sẽ xảy ra thường xuyên, bù lại bằng cơ chế chịu lỗi ở cấp phần mềm.

Kiến trúc HDFS tuân theo mô hình **master/slave**. Một cụm HDFS bao gồm một máy chủ trung tâm gọi là **NameNode** và nhiều máy chủ lưu trữ dữ liệu gọi là **DataNode**.

NameNode quản lý **metadata** của hệ thống tệp tin, bao gồm thông tin cấu trúc thư mục, tên tệp, và ánh xạ của từng tệp tới các khối dữ liệu. NameNode cũng điều phối việc truy cập tệp từ phía client. Các DataNode chịu trách nhiệm lưu trữ các **block** (khối dữ liệu) của tệp và xử lý các yêu cầu đọc/ghi thực tế từ client.

Mỗi tệp trong HDFS được chia thành nhiều block (mặc định mỗi block có kích thước 64 MB hoặc 128 MB). Mỗi block được **replicate** (nhân bản) thành nhiều bản sao và phân phối trên nhiều DataNode khác nhau trong cụm để đảm bảo tính tin cậy. Mặc định, mỗi block HDFS được lưu thành 3 bản sao trên ba DataNode khác nhau.

Cơ chế nhân bản dữ liệu giúp HDFS đạt độ tin cậy cao. Khi một DataNode bị hỏng, hệ thống vẫn còn các bản sao của dữ liệu trên các node khác, và NameNode sẽ tự động điều phối việc sao chép lại dữ liệu để duy trì đủ số bản sao mặc định. Ngoài ra, NameNode có thể triển khai một **Secondary NameNode** để định kỳ sao lưu và hợp nhất nhật ký edit log, hỗ trợ phục hồi nhanh metadata khi NameNode gặp sự cố.



Hình 2.1 Kiến trúc hệ thống HDFS với mô hình master/slave

HDFS được tối ưu cho **thông lượng đọc/ghi tuần tự cao**, do đó có một số hạn chế:

- Không phù hợp cho các ứng dụng yêu cầu độ trễ truy xuất dữ liệu rất thấp (ví dụ: truy vấn tương tác thời gian thực)
- Không hiệu quả cho việc lưu trữ số lượng lớn các tệp nhỏ (do metadata của tất cả tệp được giữ trong bộ nhớ NameNode)
- Không hỗ trợ sửa đổi nội dung tệp tùy ý (chỉ cho phép append ở cuối tệp)

Mặc dù có những hạn chế này, HDFS đóng vai trò nền tảng cốt lõi cho lưu trữ dữ liệu lớn. Hệ thống hy sinh một số tính năng POSIX để đổi lấy khả năng mở rộng và chịu lỗi vượt trội trong môi trường phân tán. Nhờ HDFS, các ứng dụng Hadoop có thể lưu trữ hàng petabyte dữ liệu trên cụm hàng nghìn node một cách tin cậy và hiệu quả.

2.3. Mô hình lập trình MapReduce

MapReduce là mô hình lập trình phân tán mặc định trong Hadoop, được Google đề xuất lần đầu năm 2004. MapReduce cung cấp một framework đơn giản cho phép xử lý dữ liệu lớn theo kiểu song song mà không cần quan tâm đến các chi tiết phức tạp của lập trình phân tán.

Mô hình MapReduce dựa trên hai hàm do người lập trình định nghĩa: hàm **Map** và hàm **Reduce**. Ý tưởng chính là phân chia một bài toán xử lý dữ liệu lớn thành nhiều phần nhỏ, thực hiện qua hai giai đoạn: giai đoạn Map (ánh xạ) và giai đoạn Reduce (giảm thiểu).

MapReduce nổi bật nhờ tính **đơn giản, linh hoạt và khả năng mở rộng cao**. Lập trình viên chỉ cần tập trung viết hai hàm Map và Reduce, còn lại việc phân chia công việc, phân phối tác vụ lên các node trong cụm, thu gộp kết quả trung gian, và xử lý lỗi đều do framework tự động đảm nhiệm.

Vai trò của MapReduce trong hệ sinh thái Hadoop:

MapReduce đóng vai trò là **engine xử lý dữ liệu chính** trong Hadoop 1.x, cung cấp khả năng xử lý batch hiệu quả cho các bài toán phân tích dữ liệu lớn. Trong Hadoop 2.x với YARN, MapReduce trở thành một trong nhiều framework có thể chạy trên cụm, cạnh tranh tài nguyên với Spark, Flink, và các framework khác.

MapReduce tận dụng tối đa hạ tầng Hadoop thông qua các cơ chế tối ưu hóa như **Data Locality** (đặt tác vụ gần dữ liệu), **Minimal Data Exchange** (hạn chế trao đổi dữ liệu giữa các node), và **Fault Tolerance** (chịu lỗi tự động). Nhờ những cơ chế này, MapReduce có thể chạy hiệu quả trên cụm hàng nghìn node, xử lý hàng petabyte dữ liệu theo lô.

Tích hợp với các thành phần khác:

MapReduce tích hợp chặt chẽ với HDFS để đọc/ghi dữ liệu, với YARN để quản lý tài nguyên, và với các công cụ như Hive (dịch HiveQL thành MapReduce jobs), Pig (dịch Pig Latin thành MapReduce jobs), và Sqoop (sử dụng MapReduce để import/export dữ liệu). Sự tích hợp này tạo nên một hệ sinh thái hoàn chỉnh cho xử lý dữ liệu lớn.

Tuy nhiên, MapReduce có những hạn chế như độ trễ cao cho các tác vụ lập và tương tác, dẫn đến sự ra đời của các framework mới như Spark và Flink. Dù vậy, MapReduce vẫn là nền tảng quan trọng và được sử dụng rộng rãi cho các bài toán batch processing truyền thống.

2.4. YARN – Bộ điều phối tài nguyên cụm Hadoop

Trong phiên bản Hadoop 1.x, hệ thống MapReduce vừa chịu trách nhiệm xử lý dữ liệu vừa quản lý tài nguyên và lập lịch công việc trên cụm, dẫn đến những hạn chế về hiệu năng

và tính linh hoạt. Hadoop 2.x đã giới thiệu **YARN (Yet Another Resource Negotiator)** để tách biệt chức năng quản lý tài nguyên khỏi việc xử lý dữ liệu, biến Hadoop thành một nền tảng tổng quát hơn.

YARN đóng vai trò là lớp quản lý tài nguyên tập trung của Hadoop, cho phép nhiều mô hình xử lý khác nhau (MapReduce, Spark, Flink, Tez, v.v.) cùng chạy trên một cụm Hadoop một cách hiệu quả.

Kiến trúc YARN dựa trên hai thành phần chính:

- **ResourceManager**: Quản lý tài nguyên toàn cục, chịu trách nhiệm cấp phát tài nguyên (CPU, bộ nhớ) cho các ứng dụng trên toàn cụm.
- **ApplicationMaster**: Mỗi ứng dụng có một ApplicationMaster riêng để lập lịch và phối hợp thực thi các tác vụ trên các NodeManager.
- **NodeManager**: Tác nhân quản lý tài nguyên tại mỗi node, phân bổ container và giám sát việc thực thi tác vụ.

Khi một ứng dụng được submit, ResourceManager cấp container đầu tiên để khởi chạy ApplicationMaster. ApplicationMaster sau đó yêu cầu ResourceManager cấp thêm các container trên những NodeManager thích hợp để chạy các tác vụ của ứng dụng. Toàn bộ quá trình này diễn ra tự động dưới sự điều phối của YARN.

YARN mang lại những lợi ích quan trọng cho Hadoop:

- **Đa dạng hóa workload**: Cho phép nhiều loại ứng dụng chạy đồng thời trên cùng một cụm Hadoop (MapReduce, Spark, Flink, Tez, v.v.)
- **Tăng tính linh hoạt**: Biến cụm Hadoop từ nền tảng chỉ dành cho MapReduce thành nền tảng đa năng cho mọi kiểu ứng dụng big data
- **Cải thiện khả năng mở rộng và đa người dùng**: ResourceManager có thể cấu hình các chính sách lập lịch (FIFO, Fair Scheduler, Capacity Scheduler) để đảm bảo chia sẻ tài nguyên hợp lý
- **Quản lý tài nguyên hiệu quả**: YARN điều phối việc phân bổ tài nguyên một cách công bằng và hiệu quả giữa các ứng dụng khác nhau

Tóm lại, YARN là lớp điều phối quan trọng cho phép Hadoop quản lý tài nguyên hiệu quả ở quy mô lớn, tăng thông lượng cụm và tận dụng tối đa phần cứng. Việc ra đời YARN là một bước ngoặt giúp hệ sinh thái Hadoop phát triển phong phú, vì các dự án khác (Hive, Pig, Spark, v.v.) có thể tích hợp và chạy trên chung một cụm Hadoop thay vì phải thiết lập hạ tầng riêng.

2.5. Apache Hive

Apache Hive là một hệ thống kho dữ liệu (data warehouse) xây dựng trên nền tảng Hadoop, cho phép người dùng thực hiện các truy vấn dữ liệu lớn bằng ngôn ngữ gần gũi với SQL. Hive được phát triển tại Facebook nhằm giúp các nhà phân tích dữ liệu quen thuộc với SQL có thể làm việc với dữ liệu lưu trên HDFS mà không cần viết code MapReduce phức tạp.

Hive cung cấp ngôn ngữ truy vấn cấp cao **HiveQL** (gần giống SQL) để trừu tượng hóa việc lập trình MapReduce. Khi người dùng viết các câu lệnh HiveQL (SELECT, JOIN, GROUP BY, v.v.), Hive sẽ tự động dịch chúng thành các job MapReduce (hoặc Tez, Spark) tương ứng để thực thi trên cụm Hadoop. Kết quả truy vấn được trả về dưới dạng bảng giống như khi làm việc với cơ sở dữ liệu truyền thống.

Các đặc trưng chính của Hive:

- **Phù hợp cho phân tích dữ liệu lớn:** Hive được tối ưu cho các truy vấn dạng batch trên tập dữ liệu rất lớn (hàng trăm GB, TB). Các truy vấn HiveQL thường dùng để tổng hợp dữ liệu (data summarization) hoặc thực hiện phân tích chuyên sâu như phân cụm khách hàng, dự đoán churn.
- **Thân thiện với người dùng:** Nhờ Hive, các nhà phân tích có thể truy vấn dữ liệu lớn bằng SQL quen thuộc, thay vì phải học lập trình Java MapReduce. Điều này giúp mở rộng đối tượng sử dụng Hadoop và dân chủ hóa việc phân tích dữ liệu lớn trong doanh nghiệp.
- **Schema on read:** Hive áp dụng mô hình "schema-on-read", tức là dữ liệu được lưu trên HDFS có thể ở định dạng thô (file text, Parquet, ORC...), và người dùng chỉ định schema khi đọc dữ liệu. Cách tiếp cận này rất linh hoạt khi làm việc với dữ liệu đa dạng hoặc thay đổi cấu trúc.

Vì Hive chạy trên Hadoop, độ trễ thực thi truy vấn thường tính bằng phút (do phải khởi tạo job MapReduce/Tez). Hive phù hợp cho xử lý batch hơn là truy vấn tương tác tức thì.

Tuy nhiên, trong vai trò là "cầu nối" giữa thế giới SQL và Hadoop, Hive đã trở thành một thành phần quan trọng, được sử dụng rộng rãi cho các ứng dụng như báo cáo dữ liệu lớn, phân tích kinh doanh, và tích hợp với các công cụ BI. Hive biến cụm Hadoop thành một kho dữ liệu khổng lồ có thể truy vấn bằng SQL, tận dụng sức mạnh xử lý song song của Hadoop để trả lời các câu hỏi mà dữ liệu quá lớn không thể phân tích trên hệ thống truyền thống.

2.6. Apache Pig

Apache Pig là một nền tảng xử lý dữ liệu lớn ở mức cao, nhưng thay vì cung cấp giao diện SQL như Hive, Pig cung cấp ngôn ngữ kịch bản chuyên dụng tên là **Pig Latin** để người dùng có thể biểu diễn các luồng xử lý dữ liệu một cách linh hoạt. Pig được phát triển tại Yahoo! nhằm đơn giản hóa việc viết các tác vụ ETL (Extract-Transform-Load) và thao tác dữ liệu phức tạp trên Hadoop.

Chương trình Pig Latin trông giống một chuỗi các lệnh xử lý dữ liệu (đọc dữ liệu, lọc, biến đổi, gộp nhóm, join, sắp xếp...). Ưu điểm của Pig là cho phép người dùng mô tả các bước xử lý theo dạng data flow một cách tự nhiên, dễ hiểu và dễ bảo trì hơn so với viết trực tiếp code Java MapReduce.

Các đặc điểm nổi bật của Pig Latin:

Độ dễ lập trình: Ngôn ngữ Pig Latin đơn giản và cô đọng, giúp đạt được việc thực thi song song một cách gần như tự động cho nhiều tác vụ xử lý dữ liệu thông thường. Người dùng chỉ việc viết các câu lệnh xử lý tuần tự, việc song song hóa và tối ưu hóa thực thi sẽ do Pig đảm nhận. Những thao tác phức tạp gồm nhiều bước biến đổi dữ liệu có thể được biểu diễn rõ ràng dưới dạng đồ thị dòng dữ liệu (data flow graph) trong Pig Latin.

Tối ưu hóa: Pig có khả năng tự động tối ưu hóa chuỗi lệnh mà người dùng viết. Nhờ biết trước toàn bộ luồng xử lý, Pig có thể áp dụng các kỹ thuật tối ưu nội tại (như kết hợp các bước, đẩy các bộ lọc lên sớm) trước khi chuyển thành các tác vụ MapReduce thực thi, giúp chương trình chạy hiệu quả hơn mà người dùng không cần can thiệp.

Mở rộng: Pig cho phép người dùng tự định nghĩa các hàm (UDF – User Defined Functions) bằng ngôn ngữ Java, Python... và chèn vào kịch bản Pig Latin để xử lý những logic đặc thù khi cần. Điều này mang lại tính linh hoạt cao và mở rộng khả năng của Pig cho nhiều trường hợp sử dụng.

Về cách hoạt động, Pig Engine (chạy trên client) sẽ biên dịch kịch bản Pig Latin thành một chuỗi các job MapReduce tương ứng trên Hadoop. Mỗi câu lệnh trong Pig Latin thường được biến thành một hoặc vài tác vụ MapReduce. Người dùng không cần quan tâm chi tiết từng job MapReduce, Pig sẽ lo việc đó, tương tự như HiveQL được dịch thành job MapReduce. Do đó, Pig về bản chất cũng sử dụng Hadoop MapReduce làm engine thực thi phía sau.

Pig đặc biệt hiệu quả cho các tác vụ transform và join dữ liệu. Chẳng hạn, nếu cần lấy dữ liệu từ nhiều nguồn, lọc các bản ghi theo điều kiện, tính toán tổng hợp rồi kết hợp dữ liệu lại, thì thay vì viết nhiều MapReduce phức tạp, ta có thể viết một script Pig duy nhất gồm các bước rõ ràng. Pig cũng hỗ trợ đa truy vấn (multi-query) trong một lần quét dữ liệu để tối ưu hiệu suất. Do Pig chạy trên Hadoop, nó phù hợp cho xử lý batch và có độ trễ tương

tự Hive (vài phút cho job).

Tóm lại, Apache Pig cung cấp một lớp trừu tượng linh hoạt và mạnh mẽ cho lập trình xử lý dữ liệu lớn trên Hadoop, hướng tới các kỹ sư dữ liệu muốn thao tác, biến đổi dữ liệu phức tạp mà không muốn viết code Java thuần túy. Pig và Hive đều nhằm đơn giản hóa việc sử dụng Hadoop: Hive hướng đến đối tượng dùng SQL, còn Pig hướng đến kịch bản xử lý dữ liệu tùy biến. Cả hai sẽ dịch công việc của người dùng thành các job MapReduce chạy trên cụm Hadoop.

2.7. Apache HBase

Apache HBase là một hệ quản trị cơ sở dữ liệu NoSQL phân tán được xây dựng trên Hadoop, thường được gọi là "cơ sở dữ liệu của Hadoop". HBase được thiết kế dựa theo mô hình **Bigtable** của Google, nhằm lưu trữ bảng dữ liệu lớn gồm hàng tỷ dòng và hàng triệu cột trên cụm máy, hỗ trợ khả năng truy cập ngẫu nhiên với độ trễ thấp.

Nói cách khác, nếu HDFS tương tự như hệ file, thì HBase tương tự như một database chạy trên HDFS, cho phép đọc/ghi theo kiểu bảng.

Các đặc trưng chính của HBase:

Mô hình wide-column: Dữ liệu trong HBase được tổ chức dưới dạng bảng nhưng là bảng "rộng", có thể có số lượng cột rất lớn (hàng nghìn cột), các cột được nhóm thành column family. Mỗi ô (cell) có một khóa dòng (row key), một tên cột và một giá trị; ngoài ra HBase còn lưu phiên bản (timestamp) cho mỗi ô, tức hỗ trợ lưu lịch sử phiên bản của dữ liệu. Khác với quan hệ trong SQL cố định lược đồ, các hàng trong HBase có thể sparsely populated (không nhất thiết có giá trị cho mọi cột).

Khả năng mở rộng và dung lượng cực lớn: HBase có thể scale-out trên cụm hàng trăm node để chứa các bảng dữ liệu kích thước hàng chục petabyte. Bảng HBase có thể gồm hàng tỷ hàng và mỗi hàng có thể có hàng triệu cột. Dữ liệu được tự động phân mảnh theo hàng (region) và phân tán trên các RegionServer khắp cụm. Khi dữ liệu tăng, HBase tự động chia nhỏ region và phân bổ sang node khác, đảm bảo hiệu năng.

Hiệu năng ghi/đọc thời gian thực: HBase được tối ưu cho truy cập kiểu ngẫu nhiên theo khóa dòng với tính nhất quán mạnh (strictly consistent reads/writes). Nó hỗ trợ tốc độ ghi rất cao – có thể hàng trăm ngàn thao tác chèn (INSERT) mỗi giây – nhờ cơ chế lưu dữ liệu tuần tự vào log và cấu trúc lưu trữ LSM-tree. Đọc theo khóa dòng cũng rất nhanh do dữ liệu được đánh chỉ mục theo khóa và có cache (block cache, Bloom filter) tại RegionServer. HBase phù hợp cho các ứng dụng cần truy cập dữ liệu dạng record-level (theo từng dòng) gần thời gian thực, như lưu trữ thông tin người dùng để phục vụ web, hoặc ghi nhật ký sự kiện với tốc độ cao.

Tích hợp chặt với Hadoop: HBase sử dụng HDFS làm lớp lưu trữ dưới (HFiles nằm trên HDFS) nên thừa hưởng tính tin cậy và khả năng chịu lỗi của HDFS. HBase cũng tích hợp với MapReduce – ta có thể viết job MapReduce để đọc/ghi trực tiếp dữ liệu từ HBase (ví dụ dùng HBase như nguồn hoặc đích trong MapReduce). Ngoài ra, HBase còn hỗ trợ xuất dữ liệu sang hệ thống khác (như xuất sang file HDFS, tích hợp với Hive qua Apache Phoenix để hỗ trợ SQL).

Tuy nhiên, HBase không phải là RDBMS quan hệ, do đó nó không hỗ trợ SQL hay join phức tạp như các hệ quản trị truyền thống. Để truy vấn HBase, người dùng thường phải sử dụng API ở mức lập trình (Java, REST, Thrift) hoặc các câu lệnh scan, get, put theo khóa. Không có ngôn ngữ truy vấn kiểu SQL trực tiếp trên HBase (ngoại trừ dùng thêm các layer như Apache Phoenix). Do đó, HBase phù hợp cho việc truy cập ngẫu nhiên theo khóa và khối lượng lớn (ví dụ tra cứu và cập nhật thông tin theo userID), hơn là cho phân tích ad-hoc. Nhiều tính năng phức tạp như quản lý quan hệ, join nhiều bảng, transaction ACID đa bảng đều không được HBase hỗ trợ đầy đủ như trong RDBMS.

Tóm lại, Apache HBase lấp vào chỗ trống của Hadoop khi cần một cơ sở dữ liệu NoSQL phân tán có khả năng đọc/ghi thời gian thực trên Big Data. Nếu HDFS+MapReduce/Hive phục vụ tốt cho xử lý batch và phân tích tổng hợp, thì HBase phục vụ cho các ứng dụng cần đọc/ghi ngẫu nhiên, truy vấn chi tiết từng bản ghi với độ trễ thấp. Một ví dụ triển khai HBase nổi tiếng là Facebook Messaging đã từng sử dụng HBase để lưu trữ tin nhắn cho hàng trăm triệu người dùng, khai thác ưu điểm mở rộng linh hoạt của HBase trên cụm Hadoop.

2.8. Apache Sqoop

Trong hệ sinh thái Hadoop, **Apache Sqoop** là công cụ chuyên dụng để trao đổi dữ liệu số lượng lớn giữa Hadoop và các cơ sở dữ liệu quan hệ (RDBMS). Tên "Sqoop" ghép từ SQL và Hadoop, thể hiện mục đích của nó: nhập và xuất dữ liệu giữa SQL databases và HDFS/Hive/HBase.

Sqoop cho phép import dữ liệu từ một hệ quản trị quan hệ (ví dụ MySQL, PostgreSQL, Oracle, SQL Server...) vào HDFS một cách tự động và hiệu quả, cũng như export dữ liệu ngược từ Hadoop trở lại RDBMS.

Khi import, Sqoop sẽ kết nối đến CSDL nguồn qua JDBC, thực thi truy vấn (hoặc duyệt từng bảng) và trích xuất dữ liệu theo từng luồng song song về HDFS. Sqoop thực hiện import bằng cách khởi chạy một job MapReduce chỉ gồm các Map task (hoặc các task Map độc lập) – mỗi task chịu trách nhiệm nhập một phần dữ liệu (ví dụ một phân đoạn của bảng được chỉ định bằng điều kiện WHERE hoặc theo primary key). Các bản ghi từ CSDL sẽ được các task này ghi trực tiếp xuống HDFS.

Kết quả, dữ liệu từ bảng quan hệ sẽ được lưu thành các file (thường ở định dạng text

CSV hoặc định dạng nhị phân tùy chọn như Avro, Parquet) trong một thư mục HDFS. Ta có thể chỉ định Sqoop lưu dữ liệu nhập vào thẳng một bảng trong Hive hoặc HBase tương ứng nếu muốn tích hợp với chúng (trong Sqoop 1 hỗ trợ import trực tiếp vào Hive/HBase).

Tương tự, với export, Sqoop đọc dữ liệu từ HDFS (ví dụ từ các file kết quả xử lý) và chèn trở lại vào bảng trong CSDL đích. Quá trình export cũng được thực hiện song song: dữ liệu HDFS được chia splits, mỗi task Map sẽ đẩy một phần dữ liệu vào CSDL qua các batch INSERT hoặc via một cơ chế load của CSDL. Sqoop đảm bảo các loại dữ liệu được chuyển đổi phù hợp giữa định dạng Hadoop và SQL (ví dụ NULL, delimiter, v.v.) trong quá trình nhập/xuất.

Nhờ Sqoop, việc tích hợp dữ liệu giữa hệ thống Hadoop và hệ thống giao dịch hiện có trở nên thuận tiện. Một số trường hợp sử dụng phổ biến của Sqoop gồm: nhập toàn bộ bảng giao dịch từ data warehouse vào Hadoop để phân tích chuyên sâu; xuất kết quả phân tích từ Hive/HDFS ra một bảng SQL để các ứng dụng kinh doanh sử dụng; di chuyển dữ liệu hàng loạt giữa kho dữ liệu truyền thống và "hồ dữ liệu" Hadoop. Thay vì viết các script thủ công để kéo dữ liệu qua lại (vốn dễ lỗi và chậm), Sqoop tự động hóa quá trình này và tận dụng song song để tăng tốc độ truyền dữ liệu. Chỉ với một lệnh cấu hình đơn giản, người dùng có thể nhập hàng triệu dòng dữ liệu từ Oracle vào Hadoop trong thời gian ngắn.

Tóm lại, Apache Sqoop là công cụ ETL bulk load hai chiều giữa Hadoop và thế giới quan hệ. Nó giúp Hadoop không bị cô lập mà có thể kết nối với hệ thống dữ liệu truyền thống, đóng vai trò quan trọng trong kiến trúc phân tích dữ liệu doanh nghiệp, nơi Hadoop thường xuyên phải trao đổi dữ liệu với các kho OLTP/OLAP hiện hữu.

2.9. Apache Kafka

Apache Kafka là một nền tảng streaming phân tán được thiết kế cho việc xử lý dòng dữ liệu thời gian thực với thông lượng cực cao. Về cơ bản, Kafka triển khai một hệ thống đăng ký/phát dữ liệu (publish-subscribe) phân tán, cho phép các ứng dụng producer (nguồn phát dữ liệu) gửi thông điệp vào các chủ đề (topic), và các consumer (bên tiêu thụ) đăng ký nhận thông điệp từ các chủ đề đó. Kafka ban đầu được phát triển bởi LinkedIn, sau này mở nguồn và trở thành dự án Apache được sử dụng rộng rãi bởi hàng nghìn công ty.

Kafka trong hệ sinh thái Hadoop thường được dùng để thu thập và phân phối dữ liệu streaming (dữ liệu sinh liên tục) vào hệ thống big data. Ví dụ, Kafka có thể thu thập log sự kiện người dùng từ hàng trăm máy chủ web và đẩy vào một topic "website_events"; từ đó các ứng dụng Hadoop/Spark có thể tiêu thụ dòng dữ liệu này để xử lý gần thời gian thực.

Kafka nổi bật với các đặc điểm sau:

Thông lượng và độ bền cao: Kafka được thiết kế để đạt thông lượng ghi và đọc rất

lớn – có thể xử lý hàng triệu thông điệp mỗi giây với độ trễ vài mili-giây. Dữ liệu gửi vào Kafka được ghi đệm trên đĩa theo cơ chế commit log tuần tự và được nhân bản giữa các node (broker) trong cụm, đảm bảo không mất mát thông điệp và chịu được lỗi node. Kafka cho phép lưu trữ các bản ghi theo thứ tự và duy trì chúng trong một khoảng thời gian cấu hình (ví dụ 7 ngày), nhờ đó consumer có thể đọc lại dữ liệu quá khứ hoặc khi cần có thể ”phát lại” stream dữ liệu.

Mô hình pub-sub linh hoạt, mở rộng: Các producer gửi sự kiện vào topic một cách phi trạng thái, không cần biết đến consumer. Mỗi topic có thể được chia thành nhiều phân vùng (partition), giúp tăng khả năng song song và thông lượng. Mỗi partition là một log được sắp thứ tự, và Kafka đảm bảo thứ tự trong từng partition. Các consumer group có thể đọc song song các partition (mỗi consumer trong nhóm đọc một phần partition) để tăng tốc tiêu thụ, nhưng đồng thời Kafka cũng cho phép nhiều nhóm consumer độc lập đọc cùng một dữ liệu mà không ảnh hưởng nhau (mô hình multi-subscriber). Nhờ đó, Kafka kết hợp ưu điểm của hàng đợi (queuing) và pub-sub: vừa mở rộng tiêu thụ bằng cách chia tải, vừa cho phép nhiều bên cùng nhận mọi thông điệp khi cần.

Ứng dụng rộng rãi và tích hợp tốt: Kafka thường đóng vai trò trung gian dẫn truyền dữ liệu trong kiến trúc dữ liệu hiện đại, được ví như ”đường ống dữ liệu thời gian thực”. Các trường hợp sử dụng tiêu biểu gồm: thu thập log sự kiện, số liệu thời gian thực; truyền dữ liệu từ hệ thống giao dịch vào Hadoop để phân tích (thông qua Kafka Connect tích hợp với database); xếp hàng công việc; làm message bus trong kiến trúc vi dịch vụ. Kafka tích hợp chặt với Hadoop qua các connector và với các framework stream processing (như Spark Streaming, Flink, Kafka Streams) để xây dựng ứng dụng phân tích dòng (stream analytics). Các công ty thương mại điện tử và dịch vụ tài chính sử dụng Kafka để xử lý dữ liệu giao dịch, clickstream ngay khi chúng phát sinh, phục vụ phân tích thời gian thực (ví dụ phát hiện giao dịch gian lận tức thì). Nhiều tập đoàn lớn (hơn 80

Tóm lại, Apache Kafka bổ sung cho Hadoop khả năng xử lý dữ liệu streaming quy mô lớn. Trong hệ sinh thái Hadoop, Kafka thường đảm nhận tầng thu thập và phân phối sự kiện (ingest), đóng vai trò nguồn dữ liệu đầu vào liên tục cho các hệ thống như Spark, Storm hoặc lưu vào HDFS/HBase để phân tích sau. Kafka giúp các thành phần trong hệ thống dữ liệu được ”phân tách lỏng lẻo” – producers và consumers không cần kết nối trực tiếp mà thông qua Kafka như một hàng đợi sự kiện đáng tin cậy. Điều này làm cho kiến trúc hệ thống linh hoạt và chịu lỗi hơn: nếu một thành phần tạm thời chậm hoặc gián đoạn, Kafka vẫn lưu giữ dòng dữ liệu cho đến khi thành phần đó bắt kịp, đảm bảo không mất dữ liệu.

2.10. Apache Oozie

Khi làm việc với hệ sinh thái Hadoop, ta thường cần chạy nhiều tác vụ (jobs) theo một trình tự hay lịch nhất định. **Apache Oozie** được thiết kế để giải quyết nhu cầu này: Oozie là một hệ thống quản lý và lập lịch luồng công việc (workflow scheduler) cho Hadoop. Nó cho phép ta định nghĩa chuỗi các job Hadoop (ví dụ: một job MapReduce, sau đó một job Hive, rồi một shell script, v.v.) và quản lý việc thực thi tuần tự hoặc song song các job đó một cách tự động.

Oozie Workflow được mô tả dưới dạng một đồ thị có hướng không chu trình (DAG) gồm các action (mỗi action có thể là một tác vụ Hadoop cụ thể). Các đỉnh (node) trong đồ thị là các action hoặc control node (như bắt đầu, kết thúc, rẽ nhánh quyết định), còn cạnh biểu thị luồng thực thi (và điều kiện chuyển tiếp nếu có). Ví dụ, ta có thể định nghĩa một workflow: chạy một job MapReduce A, sau đó nếu thành công thì chạy tiếp job Hive B, nếu thất bại thì gửi email thông báo và kết thúc. Oozie sẽ đảm bảo thực thi theo kịch bản đó: đợi A xong rồi quyết định chạy B hay email dựa trên kết quả A.

Oozie hỗ trợ nhiều loại action tích hợp sẵn cho các công việc Hadoop thường dùng: chạy MapReduce (bao gồm Hadoop Streaming), chạy Pig script, chạy Hive/HCatalog, chạy Sqoop, chạy DistCp (sao chép dữ liệu phân tán), cũng như chạy các chương trình Java hoặc shell tùy ý. Oozie cũng cung cấp cơ chế Coordinator cho phép lên lịch lặp lại workflow theo thời gian (ví dụ mỗi giờ, mỗi ngày) hoặc phụ thuộc vào sự sẵn sàng của dữ liệu (ví dụ khi file mới đến HDFS). Nhờ đó, ta có thể thiết lập các pipeline xử lý dữ liệu định kỳ một cách tự động. Chẳng hạn, mỗi ngày vào 1h sáng tự động khởi chạy luồng công việc gồm: nhập dữ liệu ngày mới bằng Sqoop, xử lý bằng một loạt job MapReduce/Pig, rồi kết xuất kết quả ra bảng Hive và gửi thông báo.

Vì Oozie chạy trên Hadoop, nó có thể dễ dàng quản lý các job ngay trong cluster (Oozie thường chạy như một service trên một node trong cụm). Oozie cung cấp giao diện web, API và các lệnh dòng lệnh để người dùng theo dõi trạng thái workflow, dừng, khởi động lại, v.v. Oozie cũng tích hợp với YARN để gửi các job vào hàng đợi phù hợp.

Với các hệ thống big data phức tạp, Oozie trở thành ”nhạc trưởng” điều phối nhiều thành phần khác nhau. Nó giúp tự động hóa và theo dõi quy trình xử lý dữ liệu end-to-end. Độ tin cậy của Oozie cũng cao: nếu một workflow thất bại giữa chừng, ta có thể sửa lỗi và chạy lại từ bước thất bại hoặc thiết lập retry, giúp quá trình xử lý dữ liệu không bị gián đoạn hoàn toàn. Tóm lại, Apache Oozie là công cụ không thể thiếu để quản lý luồng công việc Hadoop phức tạp, giúp tiết kiệm công sức vận hành và đảm bảo các bước xử lý diễn ra đúng trình tự, thời điểm.

2.11. Apache ZooKeeper

Trong các hệ thống phân tán lớn (như cụm Hadoop), việc các node phối hợp với nhau một cách nhất quán và tin cậy là thách thức lớn. **Apache ZooKeeper** được phát triển nhằm cung cấp các dịch vụ phối hợp phân tán thiết yếu một cách tập trung và đáng tin cậy. ZooKeeper đóng vai trò như một hệ thống quản lý cấu hình và đồng bộ cho các ứng dụng phân tán.

Cụ thể, ZooKeeper cung cấp cho các ứng dụng một kho lưu trữ dữ liệu phân tán dạng cây thư mục (giống hệ thống file), cho phép nhiều client kết nối để đọc/ghi các cấu hình hoặc trạng thái cần chia sẻ. Thông qua đó, ZooKeeper hiện thực các chức năng phổ biến sau:

Quản lý cấu hình: ZooKeeper lưu trữ và phân phát thông tin cấu hình cho các node. Ví dụ, trong cụm Hadoop, ZooKeeper có thể lưu thông tin về địa chỉ NameNode chính, giúp các DataNode hoặc client truy vấn khi cần.

Đặt tên (naming): ZooKeeper cung cấp một không gian tên chung để các dịch vụ đăng ký tên và tìm kiếm lẫn nhau.

Đồng bộ hóa phân tán (distributed synchronization): ZooKeeper hỗ trợ các cơ chế khóa, hàng rào (barrier) để các tiến trình phối hợp nhịp nhàng, tránh xung đột. Thay vì mỗi nhóm tự cài đặt thuật toán khóa phức tạp (dễ lỗi), họ có thể dùng các primitive do ZooKeeper cung cấp.

Dịch vụ nhóm (group membership): ZooKeeper theo dõi thành viên trong một nhóm node. Ví dụ, ZooKeeper có thể theo dõi danh sách các RegionServer đang hoạt động trong HBase cluster; nếu một node rời đi (do sự cố), các node khác được thông báo để bầu node chủ mới hoặc chuyển tác vụ.

Bầu chọn leader: ZooKeeper thường được dùng để thực hiện bầu chọn node leader trong các cụm phân tán. Các node tham gia sẽ thông qua ZooKeeper để đảm bảo một và chỉ một node được chọn làm leader tại một thời điểm (sử dụng cơ chế ephemeral znode). Hadoop YARN ResourceManager HA hay HBase Master HA đều sử dụng ZooKeeper để bầu leader khi cần failover.

ZooKeeper được thiết kế với tính nhất quán mạnh và độ tin cậy cao. Nó sử dụng thuật toán Paxos (hoặc phiên bản ZAB – ZooKeeper Atomic Broadcast) để đảm bảo tất cả các server ZooKeeper (chạy thành cụm nhỏ lẻ) duy trì cùng một trạng thái và cung cấp giao dịch cập nhật tuần tự. Các thay đổi trong ZooKeeper được áp dụng theo mô hình nhất quán tuần tự và các client có thể đăng ký watch để nhận thông báo khi dữ liệu thay đổi hoặc node mới xuất hiện/biến mất. Điều này rất hữu ích để xây dựng các hệ phân tán vững chắc: thay vì các node phải liên lạc peer-to-peer phức tạp, chúng có thể dựa vào ZooKeeper như một ”trọng tài chung” tin cậy để trao đổi trạng thái.

Trong Hadoop, ZooKeeper được sử dụng ở nhiều thành phần: HDFS High Availability

dùng ZooKeeper để giám sát NameNode và kích hoạt failover; YARN dùng ZooKeeper để lưu thông tin ResourceManager chủ; HBase hoàn toàn phụ thuộc ZooKeeper để theo dõi các máy chủ vùng (RegionServer) và điều phối Master; Kafka dùng ZooKeeper (trong các phiên bản cũ, trước khi chuyển sang Kafka Raft) để quản lý thông tin các broker, chủ đề, phân vùng. Ngoài ra, người quản trị cũng có thể sử dụng ZooKeeper để lưu các metadata tùy ý cho ứng dụng big data của mình.

Tóm lại, Apache ZooKeeper là dịch vụ điều phối tập trung cung cấp các chức năng thiết yếu như khóa phân tán, quản lý cấu hình, phát hiện dịch vụ và giám sát trạng thái cho các hệ thống lớn. Nó giúp giảm độ phức tạp khi phát triển hệ thống phân tán, bởi các lập trình viên có thể tận dụng các primitive tin cậy do ZooKeeper cung cấp thay vì tự phát triển từ đầu (vốn dễ mắc lỗi). ZooKeeper thường hoạt động như "người giữ nhịp" thầm lặng phía sau, đảm bảo sự nhất quán và ổn định cho toàn bộ hệ sinh thái Hadoop và các hệ thống phân tán liên quan.

2.12. Tổng kết chương

Chương 2 đã giới thiệu hệ sinh thái Hadoop – bao gồm những thành phần cốt lõi và mở rộng xung quanh Hadoop – tạo nên một nền tảng mạnh mẽ cho lưu trữ và xử lý dữ liệu lớn. Ở phần đầu, ta đã tìm hiểu hai nền tảng chính là HDFS và MapReduce: HDFS giải quyết bài toán lưu trữ dữ liệu lớn một cách tin cậy, khả năng mở rộng cao trên cụm máy rẻ tiền, trong khi MapReduce cung cấp mô hình lập trình đơn giản để xử lý song song dữ liệu khổng lồ, tận dụng hiệu quả tài nguyên của cụm. YARN ra đời tách biệt việc quản lý tài nguyên, biến cụm Hadoop thành hạ tầng đa nhiệm có thể chạy nhiều kiểu ứng dụng big data khác nhau.

Tiếp đó, chương này điểm qua các dự án mở rộng hệ sinh thái Hadoop nhằm đáp ứng những nhu cầu xử lý dữ liệu đa dạng trong thực tế. Hive cung cấp khả năng truy vấn dữ liệu lớn bằng ngôn ngữ gần SQL, phù hợp cho phân tích dữ liệu dạng kho. Pig cho phép biến đổi và xử lý dữ liệu linh hoạt qua kịch bản Pig Latin, thuận lợi cho các nhiệm vụ ETL phức tạp. HBase bổ sung kho lưu trữ NoSQL cho phép truy cập ngẫu nhiên thời gian thực trên Hadoop, hỗ trợ các ứng dụng yêu cầu đọc/ghi nhanh trên dữ liệu lớn. Sqoop giải quyết việc trao đổi dữ liệu giữa Hadoop và các hệ thống truyền thống, tích hợp Hadoop vào hệ sinh thái dữ liệu doanh nghiệp. Bên cạnh đó, các hệ thống như Kafka mang lại khả năng thu thập và xử lý dòng sự kiện real-time, Oozie giúp điều phối các luồng công việc phức tạp, và ZooKeeper đảm bảo các thành phần phân tán phối hợp với nhau chặt chẽ.

Tổng thể, hệ sinh thái Hadoop tạo thành một nền tảng lưu trữ và xử lý dữ liệu lớn toàn diện: từ thu thập dữ liệu (Kafka, Flume), lưu trữ (HDFS, HBase), tính toán batch (MapReduce, Tez, Spark) đến phân tích (Hive, Pig), quản trị luồng (Oozie, NiFi) và hỗ trợ vận hành (ZooKeeper). Sức mạnh thực sự của Hadoop không chỉ nằm ở bản thân HDFS

hay MapReduce, mà ở hệ sinh thái phong phú đáp ứng nhiều mô hình sử dụng dữ liệu khác nhau. Nhờ đó, Hadoop đã và đang được triển khai rộng rãi trong các ứng dụng thực tiễn, từ phân tích log, dữ liệu thương mại điện tử, tài chính, đến mạng xã hội và IoT, trở thành trụ cột của kỷ nguyên dữ liệu lớn.

Câu hỏi ôn tập:

Câu 1. Trình bày kiến trúc tổng thể của hệ thống Hadoop. Mô tả vai trò của HDFS, MapReduce, và YARN trong hệ thống.

Câu 2. So sánh HDFS và hệ thống tập tin truyền thống (UNIX file system) về mô hình truy cập và cấu trúc quản lý dữ liệu.

Câu 3. Trong HDFS, dữ liệu được chia nhỏ và sao lưu như thế nào để đảm bảo tính sẵn sàng và khả năng chịu lỗi? Vai trò của NameNode và DataNode trong quy trình này là gì?

Câu 4. Trình bày các đặc điểm nổi bật của mô hình MapReduce giúp nó phù hợp với môi trường xử lý dữ liệu phân tán.

Câu 5. Mô tả vai trò của Hive và Pig trong hệ sinh thái Hadoop. Khi nào nên chọn Hive thay vì Pig?

Câu 6. Phân tích quy trình thực thi một chương trình MapReduce từ lúc dữ liệu được chia input split tới khi tạo kết quả output. Mô tả các bước Map, Shuffle & Sort và Reduce với ví dụ minh họa.

Câu 7. Giả sử bạn có tập dữ liệu giao dịch bán hàng như sau (dạng CSV):

```
order_id,customer_id,product,category,amount
```

```
1001,C001,Keyboard,Electronics,25.0
```

```
1002,C002,Book,Education,10.5
```

```
1003,C001,Mouse,Electronics,15.0
```

...

Yêu cầu: Viết chương trình MapReduce (pseudo-code hoặc Java/Python) để tính tổng doanh số theo từng customer_id.

Câu 8. Một số chương trình MapReduce thực hiện không hiệu quả khi dữ liệu bị skewed (lệch phân phối). Hãy phân tích nguyên nhân và đề xuất một số kỹ thuật cải thiện hiệu suất xử lý MapReduce trong tình huống này.

Câu 9. Bạn được giao nhiệm vụ thiết kế pipeline phân tích log truy cập cho một website thương mại điện tử có lưu lượng lớn. Pipeline cần:

- Thu thập log real-time
- Lưu trữ log lâu dài

- Phân tích truy vấn hành vi người dùng

Xây dựng giải pháp tích hợp từ hệ sinh thái Hadoop (dùng Kafka, HDFS, Hive, Oozie, Spark hoặc Flink nếu cần) để giải bài toán này.

Câu 10. Cho tập dữ liệu: transactions.csv, mỗi dòng có dạng:

transaction_id,account_id,type,amount,timestamp

T001,A101,deposit,500.0,2023-01-01 10:00:00

T002,A101,withdrawal,100.0,2023-01-02 14:00:00

T003,A102,deposit,1000.0,2023-01-03 09:30:00

...

Viết một chương trình MapReduce tính số dư cuối cùng của mỗi account_id, biết rằng:

- Giao dịch "deposit" là cộng tiền
- Giao dịch "withdrawal" là trừ tiền

Gợi ý:

- Map: sinh ra cặp (account_id, +amount) hoặc (account_id, -amount)
- Reduce: tính tổng tất cả các amount theo account_id

Chương 3

Hệ thống Tập tin Phân tán Hadoop HDFS

3.1. Tổng quan về Hệ thống Tập tin Phân tán	39
3.2. Kiến trúc và Thành phần của Hadoop HDFS	40
3.3. Cơ chế Lưu trữ và Truy xuất Dữ liệu trong HDFS	42
3.4. Giao tiếp với HDFS qua dòng lệnh	44
3.5. Các tính năng nâng cao và Tối ưu hóa trong HDFS	45
3.6. Quản lý Siêu dữ liệu Phân tán	46
3.7. Các mô hình nhất quán trong Hệ thống Tập tin Phân tán	48
3.8. Caching và Lưu trữ đa tầng (Tiered Storage)	51
3.9. Erasure Coding trong Hệ thống lưu trữ phân tán	53
3.10. Các kỹ thuật tối ưu hiệu năng trong Hệ thống Tập tin Phân tán	55

Hadoop Distributed File System (HDFS) là thành phần lưu trữ cốt lõi của hệ sinh thái Hadoop, được thiết kế để quản lý các tập dữ liệu cực lớn trên các cụm máy tính phổ thông. Chương này sẽ giới thiệu chi tiết về HDFS, bắt đầu từ các khái niệm cơ bản của một hệ thống tập phân tán, sau đó đi sâu vào kiến trúc master/slave đặc trưng của HDFS. Chúng ta sẽ cùng phân tích cách HDFS chia nhỏ, lưu trữ và nhân bản dữ liệu để đảm bảo tính tin cậy và hiệu năng truy xuất thông lượng cao. Cuối cùng, chương sẽ thảo luận về các cơ chế nâng cao như khả năng mở rộng, các mô hình nhất quán và các kỹ thuật tối ưu hiệu năng đang được áp dụng trong các hệ thống dữ liệu lớn hiện nay.

3.1. Tổng quan về Hệ thống Tập tin Phân tán

Một **hệ thống tập tin phân tán (DFS)** cho phép quản lý dữ liệu trên nhiều máy tính nối mạng nhưng cung cấp cho người dùng một không gian tên (namespace) thống nhất, tạo cảm giác như đang truy cập vào một hệ thống tập cục bộ duy nhất. Mục tiêu chính của DFS là đảm bảo tính **trong suốt về vị trí (location transparency)**, **tính sẵn sàng cao (high availability)** và **khả năng chịu lỗi (fault tolerance)**. Để đạt được điều này, DFS thường sử dụng cơ chế **nhân bản dữ liệu (data replication)**, tức là lưu nhiều bản sao của cùng một dữ liệu trên các máy chủ khác nhau.

HDFS được xây dựng dựa trên kiến trúc **master/slave**, lấy cảm hứng từ Google File System (GFS). Đây là một lựa chọn thiết kế phù hợp cho các hệ thống xử lý dữ liệu lớn, nơi việc quản lý tập trung siêu dữ liệu giúp đơn giản hóa kiến trúc và tối ưu hóa cho các tác vụ đọc/ghi dữ liệu tuần tự với thông lượng cao.

3.1.1. Thách thức trong thiết kế Hệ thống Tập tin Phân tán

Khi xây dựng DFS, các nhà thiết kế phải đối mặt với nhiều thách thức, trong đó có bốn vấn đề chính:

- **Không gian tên (Namespace):** Làm thế nào để cung cấp một cây thư mục thống nhất và duy nhất cho toàn bộ hệ thống, che giấu sự phân tán vật lý của dữ liệu.
- **Tính nhất quán (Consistency):** Làm thế nào để đảm bảo rằng các client khác nhau khi truy cập cùng một tệp sẽ thấy được dữ liệu nhất quán, đặc biệt khi có các thao tác ghi đồng thời.
- **Caching:** Sử dụng bộ nhớ đệm ra sao để tăng tốc độ truy cập mà không làm ảnh hưởng đến tính nhất quán của dữ liệu.
- **Khả năng chịu lỗi (Fault Tolerance):** Hệ thống phải tự động phát hiện lỗi (hỏng hóc phần cứng, lỗi mạng) và phục hồi mà không làm gián đoạn dịch vụ hoặc gây mất dữ liệu.

HDFS đã đưa ra các giải pháp riêng cho từng thách thức này, ưu tiên sự đơn giản, khả năng mở rộng và hiệu năng cho các workload dữ liệu lớn.

3.2. Kiến trúc và Thành phần của Hadoop HDFS

HDFS có kiến trúc **master/slave** gồm ba thành phần chính: **NameNode** (master), **DataNode** (worker) và **Secondary NameNode** (trợ lý). Một cụm HDFS điển hình bao gồm một NameNode duy nhất và nhiều DataNode, chạy trên các phần cứng phổ thông.

3.2.1. NameNode

NameNode là "bộ não" của HDFS, chịu trách nhiệm quản lý toàn bộ **siêu dữ liệu (metadata)** của hệ thống tệp. Các thông tin này bao gồm:

- Cấu trúc cây thư mục (namespace).
- Các thuộc tính của tệp (quyền, thời gian tạo, kích thước).

- Ánh xạ từ mỗi tệp tới các khối dữ liệu (block) và vị trí của các khối đó trên các DataNode.

Toàn bộ metadata được lưu trong bộ nhớ RAM của NameNode để đảm bảo tốc độ truy xuất nhanh. Vì vậy, NameNode là một **điểm lỗi duy nhất (Single Point of Failure - SPOF)** trong kiến trúc HDFS truyền thống. Để dễ hình dung, có thể so sánh NameNode với ”thủ thư” của một thư viện khổng lồ. Thủ thư không giữ các cuốn sách (dữ liệu) nhưng nắm rõ vị trí của từng cuốn (ánh xạ block), danh mục (namespace) và các quy tắc mượn/trả (quyền truy cập).

3.2.2. DataNode

Các DataNode là các ”công nhân” thực hiện việc lưu trữ dữ liệu. Mỗi DataNode chịu trách nhiệm:

- Lưu trữ các khối dữ liệu (data blocks) trên ổ đĩa cục bộ.
- Phục vụ các yêu cầu đọc/ghi dữ liệu trực tiếp từ client.
- Gửi báo cáo định kỳ về trạng thái hoạt động và danh sách các khối đang lưu trữ về cho NameNode thông qua các tín hiệu **heartbeat** và **block report**.

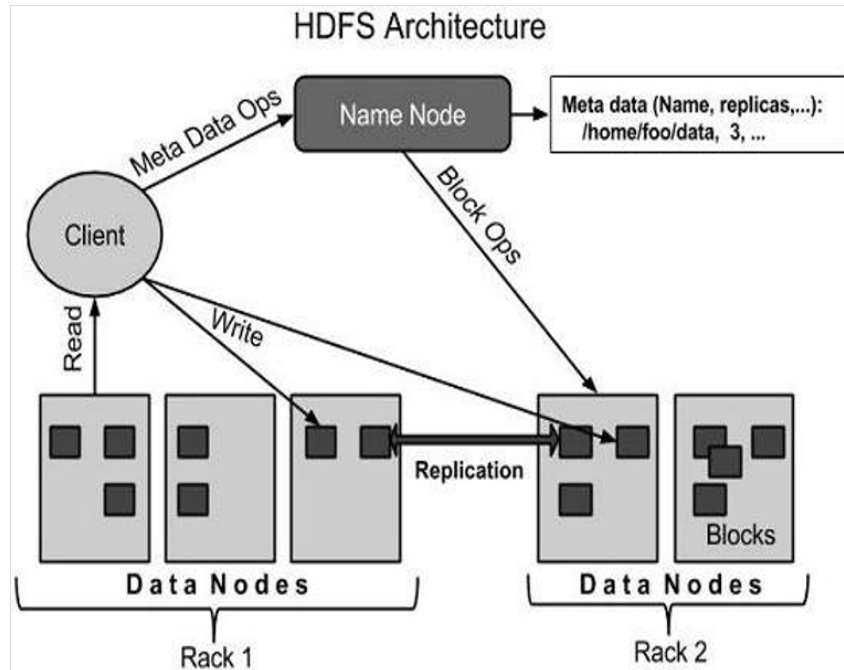
Nếu NameNode không nhận được heartbeat từ một DataNode, nó sẽ coi DataNode đó đã ”chết” và tự động khởi tạo quá trình nhân bản các khối dữ liệu từ các bản sao còn lại sang các DataNode khác. Tiếp tục với ví dụ thư viện, các DataNode chính là các kệ sách, nơi lưu trữ vật lý các cuốn sách (khối dữ liệu). Chúng chỉ làm theo chỉ dẫn của thủ thư (NameNode).

3.2.3. Secondary NameNode

Vai trò của Secondary NameNode thường bị hiểu nhầm là một NameNode dự phòng. Thực tế, nó là một tiến trình trợ giúp, có nhiệm vụ định kỳ hợp nhất (merge) các bản ghi thay đổi (EditLog) với ảnh chụp hệ thống tệp (FsImage) để tạo ra một checkpoint mới. Quá trình này giúp giảm thời gian khởi động lại của NameNode và giữ cho file EditLog không bị quá lớn. Vai trò của Secondary NameNode có thể được ví như một người ”trợ lý thủ thư”, định kỳ giúp sắp xếp lại các sổ sách (merge FsImage và EditLog) để thủ thư chính (NameNode) có thể khởi động lại nhanh chóng khi cần.

*Sơ đồ kiến trúc HDFS tổng quát: NameNode (master) quản lý metadata và không gian tên; các DataNode (slave) lưu trữ các khối dữ liệu. Khi **đọc tệp**, client gửi yêu cầu tên tệp đến NameNode (bước 1). NameNode kiểm tra metadata và trả về danh sách các khối của tệp kèm vị trí các bản sao trên các DataNode (bước 2, không hiển thị ở hình). Sau đó, client trực tiếp kết nối tới DataNode ”gần” nhất và đọc dữ liệu các khối (bước 3). NameNode chỉ tham gia ở giai đoạn điều phối ban đầu, còn trao đổi dữ liệu được thực hiện trực tiếp*

giữa client và DataNode, tránh truyền dữ liệu qua NameNode giúp loại bỏ nút thắt cổ chai (bottleneck) tại NameNode.



Hình 3.1 Sơ đồ kiến trúc hoạt động HDFS tổng quát. Client tương tác với NameNode để lấy thông tin metadata và sau đó làm việc trực tiếp với các DataNode để đọc/ghi dữ liệu.

3.3. Cơ chế Lưu trữ và Truy xuất Dữ liệu trong HDFS

3.3.1. Khối dữ liệu (HDFS Block)

HDFS chia nhỏ mỗi tệp thành các **khối (block)** có kích thước lớn, mặc định là 128MB. Kích thước khối lớn giúp tối ưu hóa cho việc truyền dữ liệu tuần tự (streaming data access), giảm thiểu chi phí tìm kiếm trên đĩa (disk seek time) và giảm số lượng metadata mà NameNode cần quản lý. Một tệp nhỏ hơn kích thước khối sẽ chỉ chiếm dung lượng bằng kích thước thực của nó.

3.3.2. Nhân bản (Replication)

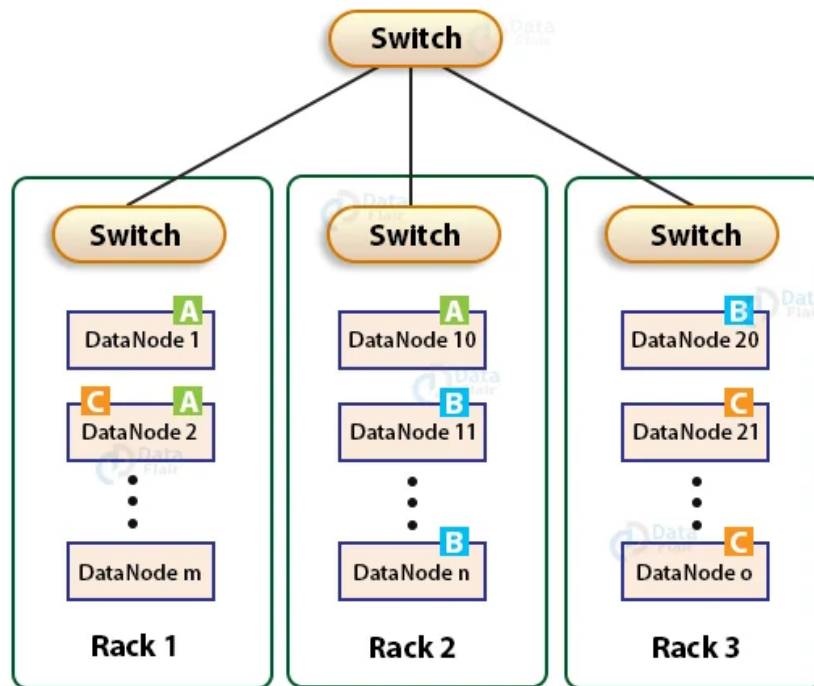
Để đảm bảo khả năng chịu lỗi, mỗi khối dữ liệu được nhân bản và lưu trữ trên nhiều DataNode khác nhau. **Hệ số nhân bản (replication factor)** mặc định là 3. HDFS sử dụng một chính sách đặt bản sao thông minh có tên là **Rack Awareness** để tăng cường độ tin cậy:

- Một bản sao được đặt trên một node trong cùng rack với client (nếu client nằm trong

cum).

- Bản sao thứ hai được đặt trên một node ở một rack khác.
- Bản sao thứ ba được đặt trên một node khác trong cùng rack với bản sao thứ hai.

Chính sách này đảm bảo rằng hệ thống vẫn có thể hoạt động ngay cả khi toàn bộ một rack gặp sự cố.



Hình 3.2 Chính sách HDFS Rack Awareness. Hai bản sao của khối dữ liệu được đặt trên cùng một tủ rack, và bản sao còn lại nằm ở một tủ rack khác.

3.3.3. Quy trình Đọc và Ghi tệp tin

Quy trình đọc dữ liệu

1. Client gửi yêu cầu đọc tệp tới NameNode.
2. NameNode kiểm tra quyền truy cập và trả về danh sách các khối của tệp cùng với địa chỉ của các DataNode chứa các bản sao của từng khối. Danh sách này được sắp xếp theo khoảng cách từ client đến DataNode.
3. Client kết nối trực tiếp đến DataNode gần nhất để đọc dữ liệu của từng khối. Nếu một DataNode gặp lỗi, client sẽ tự động thử một DataNode khác trong danh sách.

Trong suốt quá trình này, dữ liệu được truyền trực tiếp từ DataNode đến client, giúp giảm tải cho NameNode và tăng khả năng mở rộng của hệ thống.

Quy trình ghi dữ liệu

Quy trình ghi dữ liệu phức tạp hơn do cần đảm bảo tính nhất quán giữa các bản sao:

1. Client gửi yêu cầu tạo tệp tới NameNode.
2. NameNode kiểm tra và tạo một mục mới trong namespace, sau đó trả về cho client danh sách các DataNode được chọn để lưu trữ các bản sao của khối dữ liệu đầu tiên.
3. Client tổ chức các DataNode này thành một **pipeline** và bắt đầu gửi dữ liệu. Dữ liệu được đẩy từ client đến DataNode đầu tiên, sau đó được chuyển tiếp tuần tự qua các DataNode tiếp theo trong pipeline.
4. Mỗi DataNode trong pipeline sẽ gửi một gói tin xác nhận (acknowledgement - ACK) ngược lại. Client chỉ coi một khối dữ liệu đã được ghi thành công khi nhận được ACK từ tất cả các DataNode trong pipeline.
5. Sau khi ghi xong, client thông báo cho NameNode để hoàn tất việc tạo tệp.

3.3.4. Tóm tắt đặc điểm thiết kế HDFS

Các quyết định thiết kế của HDFS đều nhằm tối ưu cho các workload xử lý dữ liệu lớn, cụ thể:

- **Tối ưu cho tệp lớn và truy cập tuần tự:** Kích thước khối lớn và cơ chế streaming phù hợp với các tác vụ batch processing như MapReduce.
- **Mô hình "Write-Once-Read-Many":** HDFS đơn giản hóa vấn đề nhất quán dữ liệu bằng cách không cho phép sửa đổi nội dung tệp ở giữa. Một tệp sau khi được ghi và đóng sẽ trở thành chỉ đọc.
- **Coi lỗi phần cứng là bình thường:** HDFS được xây dựng với giả định rằng lỗi phần cứng sẽ xảy ra thường xuyên. Do đó, các cơ chế phát hiện lỗi, nhân bản và phục hồi tự động được tích hợp sâu vào kiến trúc cốt lõi.

3.4. Giao tiếp với HDFS qua dòng lệnh

Lý thuyết về HDFS sẽ trở nên hữu hình hơn khi sinh viên có thể tương tác trực tiếp với nó. HDFS cung cấp một bộ công cụ dòng lệnh mạnh mẽ, có cú pháp tương tự như các lệnh trên Linux, giúp người dùng dễ dàng quản lý tệp và thư mục.

Dưới đây là một số lệnh cơ bản và hữu ích nhất:

- **hdfs dfs -ls /path/to/file:** Liệt kê các tệp và thư mục trong đường dẫn đã cho.
- **hdfs dfs -mkdir /path/to/directory:** Tạo một thư mục mới trong đường dẫn đã cho.
- **hdfs dfs -put localfile /path/to/hdfsfile:** Sao chép một tệp từ máy local đến HDFS.
- **hdfs dfs -get /path/to/hdfsfile localfile:** Sao chép một tệp từ HDFS về máy local.
- **hdfs dfs -rm /path/to/file:** Xóa một tệp trong HDFS.
- **hdfs dfs -rm -r /path/to/directory:** Xóa một thư mục và tất cả các tệp bên trong nó trong HDFS.

Việc thực hành với các lệnh này sẽ giúp sinh viên củng cố kiến thức về không gian tên, cấu trúc thư mục và cách người dùng cuối tương tác với HDFS.

3.5. Các tính năng nâng cao và Tối ưu hóa trong HDFS

3.5.1. Khả năng Mở rộng và Tính sẵn sàng cao

Để giải quyết các hạn chế của kiến trúc NameNode đơn, các phiên bản Hadoop sau này đã giới thiệu hai tính năng quan trọng:

- **HDFS Federation:** Cho phép sử dụng nhiều NameNode độc lập trong cùng một cụm, mỗi NameNode quản lý một phần riêng của không gian tên. Điều này giúp hệ thống mở rộng theo chiều ngang về số lượng tệp và siêu dữ liệu.
- **NameNode High Availability (HA):** Sử dụng một cặp NameNode Active/Standby. NameNode Standby duy trì một bản sao đồng bộ của siêu dữ liệu và có thể tiếp quản gần như ngay lập tức nếu NameNode Active gặp sự cố, qua đó loại bỏ điểm lỗi duy nhất.

3.5.2. Tối ưu hóa hiệu năng

HDFS được tối ưu cho **thông lượng cao** thay vì độ trễ thấp. Thiết kế khối lớn (128 MB hoặc hơn) nhằm giảm thiểu thao tác đĩa và overhead quản lý, giúp việc đọc/ghi tuần tự đạt tốc độ gần với tốc độ thô của ổ đĩa. HDFS cũng tận dụng đặc tính **data locality** – thay vì truyền dữ liệu tới nơi xử lý, Hadoop cố gắng **đưa tính toán đến gần dữ liệu**. Ví dụ, các tác vụ MapReduce sẽ được lịch trình chạy trên chính DataNode chứa dữ liệu cần xử lý, nhờ đó tiết kiệm băng thông mạng và tăng tốc độ đọc dữ liệu đầu vào. Cách tiếp cận “tính toán gần dữ liệu” này là một yếu tố then chốt làm nên hiệu năng cao của hệ sinh thái Hadoop.

3.6. Quản lý Siêu dữ liệu Phân tán

Trong các hệ thống tệp phân tán (Distributed File System – DFS), siêu dữ liệu (metadata) đóng vai trò trung tâm trong việc mô tả cấu trúc và trạng thái của dữ liệu, bao gồm tên tệp, phân cấp thư mục, quyền truy cập, và vị trí vật lý của các khối dữ liệu (data blocks). Việc quản lý siêu dữ liệu hiệu quả là yếu tố then chốt để đảm bảo tính khả mở, hiệu suất và độ tin cậy của hệ thống.

3.6.1. Vấn đề của kiến trúc tập trung trong HDFS truyền thống

Hệ thống Hadoop Distributed File System (HDFS) truyền thống sử dụng kiến trúc tập trung, trong đó NameNode là thành phần duy nhất phụ trách toàn bộ siêu dữ liệu. Nó lưu trữ thông tin về hệ thống tệp, ánh xạ giữa các khối dữ liệu và các node vật lý, quyền truy cập, cũng như quản lý các thao tác như tạo/xoá/đổi tên tệp. Mặc dù mô hình này đơn giản và dễ cài đặt, nhưng nó tồn tại hai hạn chế nghiêm trọng. Thứ nhất là vấn đề điểm lỗi đơn (Single Point of Failure – SPOF). Nếu NameNode gặp sự cố, toàn bộ hệ thống sẽ không thể truy cập được bất kỳ dữ liệu nào, dù dữ liệu vẫn còn được lưu trên các DataNode. Thứ hai là hạn chế về khả năng mở rộng. Vì NameNode lưu tất cả siêu dữ liệu trong bộ nhớ chính (RAM) để tăng hiệu suất, khi khối lượng dữ liệu và số lượng tệp tăng lớn, khả năng mở rộng theo chiều ngang trở nên bất khả thi.

3.6.2. Các giải pháp quản lý siêu dữ liệu trong hệ thống DFS hiện đại

Để vượt qua các hạn chế của mô hình tập trung, các hệ thống tệp phân tán hiện đại áp dụng các kiến trúc phân tán trong quản lý siêu dữ liệu, bao gồm:

Phân mảnh không gian tên (Namespace Partitioning)

Cách tiếp cận này chia nhỏ cây không gian tên (namespace tree) thành nhiều phân vùng độc lập và phân phối chúng cho nhiều máy chủ siêu dữ liệu (Metadata Server – MDS). Ví dụ, trong hệ thống CephFS, namespace có thể được chia theo tiền tố đường dẫn (prefix-based), như /home được một MDS quản lý, trong khi /data do MDS khác phụ trách. Mỗi MDS chỉ chịu trách nhiệm xử lý các truy vấn và cập nhật trong không gian tên của mình, giúp giảm tải và tăng song song. Điểm nổi bật trong CephFS là khả năng cân bằng động (dynamic balancing). Hệ thống có thể di chuyển các phần namespace giữa các MDS khi phát hiện tình trạng mất cân bằng tải, dựa trên các chỉ số như số lượng truy cập hay khối lượng metadata xử lý.

Phân tán siêu dữ liệu bằng DHT hoặc Key-Value Store

Một số hệ thống DFS hiện đại, như Google Colossus (GFS v2), không sử dụng cấu trúc cây truyền thống mà thay thế bằng bảng ánh xạ khóa-giá trị (Key-Value Store) để lưu trữ siêu dữ liệu. Trong đó, thông tin siêu dữ liệu như ánh xạ khối tệp, quyền truy cập, thời gian sửa đổi, được lưu trong các hệ thống NoSQL như Bigtable. Điều này cho phép quản lý metadata trên quy mô rất lớn với độ trễ thấp và khả năng mở rộng tuyến tính. Cách tổ chức này giúp giảm phụ thuộc vào một máy chủ trung tâm và tăng khả năng song song trong truy vấn siêu dữ liệu, đồng thời dễ dàng mở rộng hệ thống bằng cách thêm các node lưu trữ siêu dữ liệu mới.

Cơ chế phục hồi và đảm bảo tính nhất quán

Các hệ thống DFS hiện đại chú trọng vào tính khả dụng và tính nhất quán của siêu dữ liệu thông qua nhiều cơ chế hỗ trợ.

- **Mô hình Active/Active MDS:** Trong mô hình này, nhiều MDS có thể hoạt động đồng thời và phục vụ các truy cập từ client, thay vì chỉ một MDS chính (Active/Standby) như trong HDFS. Điều này giúp tăng thông lượng hệ thống và giảm thời gian phục hồi khi một node gặp lỗi.
- **Cơ chế checkpoint và journal:** Ghi log (journal) và snapshot định kỳ (checkpoint) giúp phục hồi trạng thái metadata trong trường hợp xảy ra sự cố. HDFS sử dụng cơ chế Secondary NameNode để tạo checkpoint từ journal và metadata, giúp giảm thời gian khởi động lại.
- **Giao thức đồng thuận (Consensus Protocols):** Trong các hệ thống phân tán như Ceph hay hệ thống quản lý metadata phi tập trung, giao thức như Raft hoặc Paxos được sử dụng để đảm bảo tính nhất quán mạnh giữa các replica của metadata. Các giao thức này giúp các node đồng bộ trạng thái một cách tin cậy, ngay cả trong môi trường có lỗi mạng hoặc node chết đột ngột.

3.6.3. Tổng kết và so sánh các mô hình quản lý siêu dữ liệu

- **Tập trung (NameNode):**
 - *Ưu điểm:* Triển khai đơn giản, hiệu năng cao ở quy mô nhỏ.
 - *Nhược điểm:* SPOF, khó mở rộng.
 - *Hệ thống tiêu biểu:* HDFS cổ điển.
- **Phân mảnh namespace:**

- *Ưu điểm*: Song song, khả mở tốt, cân bằng động.
- *Nhược điểm*: Phức tạp trong điều phối.
- *Hệ thống tiêu biểu*: CephFS, Lustre.

- **KV-store/DHT:**

- *Ưu điểm*: Khả mở rất cao, dễ phân tán.
- *Nhược điểm*: Khó ánh xạ dữ liệu phức tạp.
- *Hệ thống tiêu biểu*: GFSv2, SeaweedFS.

- **Active/Active + Consensus:**

- *Ưu điểm*: Tính sẵn sàng và nhất quán cao.
- *Nhược điểm*: Chi phí đồng bộ, độ trễ.
- *Hệ thống tiêu biểu*: CephFS, Vitess.

Việc lựa chọn kiến trúc phù hợp tùy thuộc vào đặc điểm tải công việc (workload), yêu cầu về độ tin cậy, và khả năng mở rộng của hệ thống.

3.7. Các mô hình nhất quán trong Hệ thống Tập tin Phân tán

Trong các hệ thống phân tán, đảm bảo tính nhất quán của dữ liệu là một trong những thách thức cốt lõi. Khi nhiều node lưu trữ và xử lý dữ liệu cùng lúc, các truy vấn đọc có thể nhận được những giá trị khác nhau tùy theo thứ tự cập nhật và độ trễ mạng. Vì vậy, các hệ thống tập phân tán thường áp dụng những mô hình nhất quán (consistency models) khác nhau nhằm đạt sự cân bằng giữa độ chính xác, hiệu năng và khả năng mở rộng.

3.7.1. Tổng quan các mô hình nhất quán

Mô hình Strong Consistency

Mô hình nhất quán mạnh (strong consistency) đảm bảo rằng mọi truy cập dữ liệu sau một thao tác ghi (write) thành công sẽ nhận được giá trị mới nhất. Đây là mô hình chặt chẽ nhất và gần với trực giác của người dùng: nếu một tệp vừa được ghi xong, mọi người truy cập sau đó sẽ thấy ngay phiên bản mới. Mô hình này thường dựa vào các giao thức đồng bộ hóa mạnh như Paxos hoặc Raft để duy trì thứ tự cập nhật giữa các bản sao dữ liệu. **Ưu điểm**: Đơn giản cho lập trình viên, phù hợp với các ứng dụng yêu cầu logic nghiệp vụ chặt chẽ như hệ thống ngân hàng, dữ liệu giao dịch. **Nhược điểm**: Chi phí cao về hiệu năng, độ trễ cao trong môi trường mạng không ổn định.

Mô hình Eventual Consistency

Nhất quán cuối cùng (eventual consistency) là mô hình thường thấy trong các hệ thống phân tán quy mô lớn như object storage. Nó cho phép các bản sao dữ liệu ở các node khác nhau có thể không đồng nhất tại một thời điểm, nhưng đảm bảo rằng nếu không có cập nhật mới, các node sẽ dần đồng bộ và đạt cùng một giá trị. **Ưu điểm:** Hiệu năng cao, khả năng mở rộng tốt. **Nhược điểm:** Trong thời gian hội tụ, người dùng có thể thấy dữ liệu “cũ” hoặc lỗi thời. Ví dụ điển hình là Amazon S3 trong giai đoạn đầu – khi một tệp vừa được ghi, người dùng ở khu vực khác có thể vẫn đọc được phiên bản cũ trong vài giây. Gần đây, S3 đã bổ sung hỗ trợ nhất quán mạnh cho các thao tác đọc-sau-ghi (read-after-write).

Mô hình Session Semantics

Trong mô hình này, hệ thống chỉ đảm bảo nhất quán trong phạm vi một phiên làm việc (session). Một client sẽ thấy đầy đủ các thao tác ghi của chính nó trong suốt phiên, nhưng các client khác thì không đảm bảo đồng nhất ngay lập tức. Đây là mô hình phổ biến trong HDFS: một tệp chỉ được thấy đầy đủ nội dung sau khi đã hoàn tất ghi và đóng (close). **Ưu điểm:** Dễ thực hiện, hiệu quả cao trong ứng dụng ghi nhiều, đọc ít. **Nhược điểm:** Không phù hợp trong các trường hợp ghi song song bởi nhiều client.

Mô hình Immutable File

Một số hệ thống, như Kafka hoặc IPFS, áp dụng mô hình tệp bất biến – sau khi ghi xong, tệp không được chỉnh sửa. Mọi cập nhật đều dẫn đến tạo ra một phiên bản mới. Điều này giúp đơn giản hóa nhất quán vì không tồn tại ghi song song. **Ưu điểm:** Dễ tối ưu, lý tưởng cho nhật ký ghi (log), dữ liệu audit. **Nhược điểm:** Không hỗ trợ cập nhật trực tiếp, tốn dung lượng nếu ghi nhiều phiên bản.

Mô hình Transactional Semantics

Đây là mô hình cao cấp nhất, trong đó các thao tác ghi được gom thành một giao dịch (transaction) nguyên tử. Tất cả thao tác trong giao dịch hoặc cùng thành công, hoặc cùng thất bại. Các hệ thống như CephFS (ở cấu hình cao) hoặc các hệ thống cơ sở dữ liệu phân tán (như FoundationDB) có thể cung cấp mô hình này. **Ưu điểm:** An toàn và chính xác, phù hợp với nghiệp vụ phức tạp. **Nhược điểm:** Phức tạp trong thiết kế, hiệu năng thấp hơn.

3.7.2. So sánh các mô hình nhất quán

- **Strong Consistency:**

- *Mô tả chính:* Đọc luôn thấy dữ liệu mới nhất sau khi ghi.

- *Ưu điểm*: Dễ lập trình, an toàn.
- *Nhược điểm*: Hiệu năng thấp.
- *Ví dụ tiêu biểu*: Ceph (tùy cấu hình), S3 mới.

- **Eventual Consistency:**

- *Mô tả chính*: Dữ liệu hội tụ dần về cùng trạng thái.
- *Ưu điểm*: Mở rộng tốt, hiệu suất cao.
- *Nhược điểm*: Có thể thấy dữ liệu “cũ”.
- *Ví dụ tiêu biểu*: Amazon S3 (trước đây).

- **Session Semantics:**

- *Mô tả chính*: Nhất quán trong một phiên người dùng.
- *Ưu điểm*: Hiệu quả, đơn giản.
- *Nhược điểm*: Không đồng nhất khi ghi chéo.
- *Ví dụ tiêu biểu*: HDFS.

- **Immutable File Model:**

- *Mô tả chính*: Tập không sửa đổi sau khi ghi.
- *Ưu điểm*: Tối ưu lưu trữ, đơn giản hóa.
- *Nhược điểm*: Không cập nhật trực tiếp.
- *Ví dụ tiêu biểu*: Kafka, IPFS.

- **Transactional Semantics:**

- *Mô tả chính*: Ghi nhận đồng thời trong một giao dịch nguyên tử.
- *Ưu điểm*: Chính xác, an toàn nghiệp vụ.
- *Nhược điểm*: Hiệu năng thấp, phức tạp.
- *Ví dụ tiêu biểu*: CephFS (một số chế độ).

3.7.3. Ứng dụng và lựa chọn mô hình phù hợp

Không có mô hình nhất quán nào là “tối ưu” cho mọi tình huống. Trong thực tế, hệ thống lưu trữ cần lựa chọn mô hình phù hợp với đặc tính ứng dụng. Các ứng dụng ghi tập trung, ít yêu cầu cập nhật chéo như log analytics phù hợp với immutable model. Hệ thống dữ liệu tài chính hoặc các khối xử lý phân phối yêu cầu strong hoặc transactional consistency. Trong khi đó, các hệ thống lưu trữ đám mây quy mô lớn như S3 hay GCS thường ưu tiên eventual consistency để đảm bảo hiệu năng và khả năng mở rộng.

3.8. Caching và Lưu trữ đa tầng (Tiered Storage)

Trong hệ thống tệp phân tán, đặc biệt là các hệ thống xử lý dữ liệu lớn, không phải tất cả dữ liệu đều cần được truy cập với cùng tần suất hoặc độ trễ. Việc sử dụng một tầng lưu trữ duy nhất cho toàn bộ dữ liệu dễ dẫn đến lãng phí tài nguyên hoặc không đáp ứng được yêu cầu về hiệu suất. Vì vậy, các hệ thống hiện đại triển khai mô hình lưu trữ đa tầng (tiered storage), kết hợp nhiều lớp lưu trữ với đặc điểm khác nhau về tốc độ, chi phí và dung lượng. Song song với đó, kỹ thuật **caching** được áp dụng để giảm độ trễ truy cập bằng cách đưa dữ liệu nóng về gần phía compute node.

3.8.1. Mô hình lưu trữ phân tầng

Lưu trữ phân tầng là mô hình tổ chức dữ liệu theo mức độ ưu tiên truy cập. Dữ liệu được phân phối giữa các tầng vật lý khác nhau dựa trên đặc tính truy cập và tuổi thọ của dữ liệu. Ba tầng phổ biến nhất gồm:

- **Tầng nóng (Hot Tier):** Sử dụng RAM hoặc SSD, tốc độ truy xuất rất cao nhưng dung lượng và chi phí đắt đỏ. Tầng này lưu dữ liệu đang được truy cập thường xuyên, dữ liệu tạm thời trong quá trình xử lý, hoặc cache kết quả trung gian.
- **Tầng ấm (Warm Tier):** Sử dụng ổ đĩa cứng (HDD), tốc độ trung bình, chi phí thấp hơn. Thường lưu trữ dữ liệu được truy cập định kỳ như dữ liệu hệ thống, bản sao dữ liệu thời gian gần.
- **Tầng lạnh (Cold Tier):** Dựa trên object storage như Amazon S3, Google Cloud Storage, hoặc thiết bị lưu trữ dung lượng lớn với chi phí rất thấp. Lưu trữ dữ liệu lâu dài, ít khi truy cập, như dữ liệu lịch sử hoặc sao lưu (backup).

3.8.2. Caching - Bộ nhớ đệm dữ liệu

Caching là cơ chế tạm lưu dữ liệu thường xuyên truy cập vào bộ nhớ nhanh để giảm độ trễ và giảm tải I/O cho tầng lưu trữ chính. Các hệ thống DFS hiện đại triển khai caching ở nhiều cấp độ: client-side, node-local hoặc toàn hệ thống. Một hệ thống tiêu biểu là **Alluxio**, hoạt động như một tầng cache thống nhất giữa compute layer (như Spark, Presto) và backend storage (như HDFS hoặc S3). Alluxio cho phép lưu tạm dữ liệu đã truy xuất vào bộ nhớ RAM, SSD hoặc ổ đĩa cục bộ, từ đó cải thiện đáng kể tốc độ truy xuất dữ liệu trong các workflow phân tích. Trong Hadoop 3.x, caching được quản lý thông qua các **storage policy**, cho phép người quản trị chỉ định nơi lưu block cụ thể – ví dụ lưu block nhỏ và truy cập thường xuyên vào SSD, trong khi block lớn hoặc ít truy cập có thể nằm trên HDD. Hệ thống **Ceph** cũng hỗ trợ cache tiering, nơi dữ liệu được ghi vào SSD cache pool trước khi chuyển xuống

HDD pool. Điều này đảm bảo thao tác ghi nhanh trong ngắn hạn, đồng thời đảm bảo độ bền dữ liệu lâu dài.

3.8.3. Lợi ích và thách thức của Caching và Tiered Storage

Lợi ích chính của caching và lưu trữ đa tầng bao gồm:

- **Giảm độ trễ truy cập (latency):** Bằng cách đưa dữ liệu nóng về gần compute node, các hệ thống có thể xử lý nhanh hơn mà không cần truy cập backend chậm.
- **Tăng thông lượng I/O:** Dữ liệu phổ biến không bị lặp lại truy xuất từ tầng chậm, giúp giảm tắc nghẽn hệ thống lưu trữ.
- **Tối ưu chi phí:** Dữ liệu ít truy cập không chiếm dụng tài nguyên nhanh, cho phép sử dụng phần cứng phù hợp với đặc tính dữ liệu.
- **Tách compute và storage:** Caching tạo ra lớp trung gian cho phép scale compute và storage độc lập, phù hợp với mô hình elastic cloud.

Tuy nhiên, mô hình này cũng đối mặt với các thách thức:

- **Quản lý cache hiệu quả:** Xác định chính xác dữ liệu nào nên được cache, khi nào nên đẩy ra, và dùng chính sách gì (LRU, LFU, TTL...).
- **Đồng bộ dữ liệu:** Với hệ thống có ghi dữ liệu, cache phải được đồng bộ với backend để tránh lỗi nhất quán.
- **Tính phức tạp hệ thống:** Càng nhiều tầng, càng cần kiểm soát chính sách lưu trữ, giám sát và cân bằng dữ liệu động.

3.8.4. Các hệ thống tiêu biểu

- **Alluxio:**

- *Vai trò caching:* Cache dữ liệu giữa compute và storage.
- *Cấu hình phân tầng:* RAM/SSD cho hot cache, backend HDFS/S3.
- *Lợi ích cụ thể:* Tăng tốc truy xuất dữ liệu phân tích.

- **Hadoop 3.x:**

- *Vai trò caching:* Storage policy theo block.
- *Cấu hình phân tầng:* SSD, HDD, archive tiers.
- *Lợi ích cụ thể:* Quản lý dữ liệu theo nhu cầu ứng dụng.

- **Ceph:**

- *Vai trò caching:* Cache tier pool.
- *Cấu hình phân tầng:* SSD cache layer + HDD data layer.
- *Lợi ích cụ thể:* Ghi nhanh, lưu trữ bền vững.

3.8.5. Tổng kết

Caching và tiered storage là những chiến lược không thể thiếu trong hệ thống tệp phân tán quy mô lớn. Chúng cho phép tối ưu hóa tài nguyên phần cứng, cải thiện hiệu suất truy xuất và mở rộng linh hoạt theo nhu cầu của ứng dụng. Hiểu rõ nguyên lý hoạt động và triển khai đúng cách caching/lưu trữ phân tầng là kỹ năng thiết yếu trong thiết kế hệ thống lưu trữ hiện đại, đặc biệt trong các môi trường đám mây, điện toán biên và xử lý dữ liệu lớn theo thời gian thực.

3.9. Erasure Coding trong Hệ thống lưu trữ phân tán

Trong hệ thống lưu trữ phân tán, việc đảm bảo tính bền vững của dữ liệu (durability) trước nguy cơ mất mát là yêu cầu bắt buộc. Truyền thống, các hệ thống sử dụng **replication** – sao chép toàn bộ dữ liệu thành nhiều bản – để đảm bảo khả năng phục hồi khi một hoặc nhiều node bị lỗi. Tuy nhiên, replication tiêu tốn nhiều dung lượng và chi phí lưu trữ. Để giải quyết vấn đề này, **Erasure Coding (EC)** đã được phát triển như một kỹ thuật hiệu quả hơn, đặc biệt phù hợp cho hệ thống dữ liệu lớn, kho lưu trữ lâu dài hoặc môi trường đám mây.

3.9.1. Nguyên lý hoạt động của Erasure Coding

Erasure Coding là kỹ thuật mã hóa dư thừa dựa trên lý thuyết mã hóa thông tin (information theory). Dữ liệu gốc được chia thành **k khối dữ liệu (data blocks)** và sau đó tạo thêm **m khối kiểm tra (parity blocks)** thông qua các phép toán mã hóa tuyến tính. Tổng cộng có $k+m$ khối được lưu trữ trên các node khác nhau trong hệ thống. Ưu điểm chính là: chỉ cần **bất kỳ k trong số k+m khối**, hệ thống có thể khôi phục lại toàn bộ dữ liệu ban đầu. Điều này giúp tiết kiệm dung lượng so với replication trong khi vẫn đảm bảo khả năng phục hồi. Ví dụ: Trong cấu hình EC (6+3), một tệp được chia thành 6 data blocks và 3 parity blocks. Hệ thống vẫn có thể khôi phục tệp nếu tối đa 3 blocks bị mất. Dung lượng lưu trữ yêu cầu chỉ ~150% thay vì 300% như replication 3x.

3.9.2. So sánh Erasure Coding với Replication

- **Dung lượng lưu trữ:** Replication (3x) yêu cầu 300%, trong khi Erasure Coding (6+3) chỉ cần khoảng 150%.

- **Tốc độ phục hồi:** Replication nhanh hơn vì chỉ cần sao chép khối dữ liệu. EC chậm hơn do cần thực hiện tính toán giải mã.
- **Tính phức tạp hệ thống:** Replication đơn giản hơn. EC phức tạp hơn do có các bước mã hóa/giải mã.
- **Độ bền dữ liệu:** Cả hai đều tốt, nhưng EC (với nhiều parity block) có thể chịu được nhiều lỗi hơn.
- **Hiệu suất truy cập:** Replication tốt hơn cho các tệp nhỏ và truy cập thường xuyên. EC phù hợp hơn cho các tệp lớn, ít truy cập (dữ liệu lạnh).
- **Ứng dụng:** Replication là mặc định trong HDFS 2.x. EC được hỗ trợ trong HDFS 3.x và được sử dụng rộng rãi trong các hệ thống lưu trữ đám mây như Azure và S3.

Việc lựa chọn giữa replication và erasure coding phụ thuộc vào mục tiêu sử dụng: **replication** phù hợp với các tác vụ ghi nhiều, cần phục hồi nhanh, còn **erasure coding** phù hợp với kho lưu trữ dữ liệu lớn, truy cập không thường xuyên và yêu cầu tối ưu chi phí.

3.9.3. Ứng dụng trong các hệ thống thực tế

- **HDFS 3.x:** Từ phiên bản 3, HDFS hỗ trợ erasure coding gốc, cho phép người quản trị cấu hình profile EC theo từng thư mục. Việc này rất hữu ích trong các cluster có khối lượng lưu trữ lớn nhưng yêu cầu tiết kiệm tài nguyên.
- **Ceph:** Cho phép người dùng lựa chọn giữa replication hoặc EC profile khi tạo pool lưu trữ. EC pools thích hợp với kho dữ liệu lâu dài, còn replicated pools phù hợp với workload cần hiệu năng cao.
- **Azure Storage, Amazon S3 Glacier:** Các hệ thống object storage sử dụng erasure coding như một phần mặc định để tối ưu chi phí lưu trữ. Dữ liệu được phân phối toàn cầu, mã hóa và khôi phục theo mô hình EC nhằm giảm chi phí bảo trì phần cứng vật lý.

3.9.4. Thách thức trong triển khai Erasure Coding

Mặc dù mang lại nhiều lợi ích về mặt tiết kiệm dung lượng, việc triển khai EC đi kèm một số thách thức kỹ thuật:

- **Chi phí tính toán:** Việc mã hóa và giải mã sử dụng phép toán tuyến tính trên các trường hữu hạn (finite fields), đòi hỏi tài nguyên CPU, đặc biệt trong các hệ thống không có tăng tốc phần cứng (như SIMD, GPU).

- **Độ trễ khi phục hồi:** So với replication, quá trình khôi phục dữ liệu trong EC phức tạp hơn, có thể ảnh hưởng đến RTO (Recovery Time Objective).
- **Phân mảnh và cân bằng dữ liệu:** Việc phân phối khối mã hóa trên nhiều node cần được điều phối tốt để tránh mất cân bằng tải, ảnh hưởng đến throughput hệ thống.

3.9.5. Kết luận về Erasure Coding

Erasure Coding là một bước tiến quan trọng trong lĩnh vực lưu trữ phân tán, giúp tiết kiệm chi phí lưu trữ mà vẫn đảm bảo khả năng phục hồi dữ liệu cao. Với sự phát triển của phần cứng tính toán và kỹ thuật tối ưu mã hóa, EC đang trở thành lựa chọn mặc định trong nhiều hệ thống lưu trữ đám mây và dữ liệu lớn. Tuy nhiên, người triển khai cần hiểu rõ đặc tính workload để sử dụng EC một cách hiệu quả và cân bằng giữa chi phí, hiệu suất và độ tin cậy.

3.10. Các kỹ thuật tối ưu hiệu năng trong Hệ thống Tập tin Phân tán

Hệ thống tệp phân tán cần xử lý một lượng lớn yêu cầu truy cập, song song trên nhiều nút, trong khi vẫn phải đảm bảo tính nhất quán, độ bền và bảo mật. Để đạt hiệu năng tối ưu trong bối cảnh như vậy, các nhà phát triển đã đưa ra nhiều kỹ thuật cải tiến, từ tối ưu luồng truy cập đến tổ chức dữ liệu vật lý. Dưới đây là những kỹ thuật phổ biến và hiệu quả nhất.

3.10.1. Prefetching - Tiên đoán dữ liệu

Prefetching là kỹ thuật tải trước các phần dữ liệu được dự đoán sẽ cần dùng, dựa trên các mẫu truy cập trong quá khứ. Nếu người dùng thường truy cập tuần tự một chuỗi block, hệ thống có thể tải trước block kế tiếp ngay sau khi truy cập block hiện tại. Kỹ thuật này giúp **giảm độ trễ đọc** trong các workload có tính tuần tự, chẳng hạn như phân tích dữ liệu log, xử lý batch hoặc đọc media. Trong hệ thống như Alluxio hoặc Spark, prefetching được kết hợp với caching để lưu block kế tiếp vào bộ nhớ, từ đó loại bỏ I/O đĩa trong bước tiếp theo.

3.10.2. Write Coalescing - Gom ghi

Khi có nhiều truy cập ghi nhỏ xảy ra liên tục, thay vì ghi từng phần riêng lẻ (dẫn đến nhiều I/O tốn kém), hệ thống có thể gom các thao tác này lại thành một khối lớn hơn và ghi một lần duy nhất. Điều này giúp **giảm số lượng truy cập đĩa**, tăng hiệu suất, đặc biệt với thiết bị lưu trữ như HDD hoặc SSD có chi phí thiết lập I/O cao. Kỹ thuật này thường thấy trong các hệ thống log-structured hoặc streaming như Kafka, nơi dữ liệu được ghi theo batch hoặc theo segment để giảm fragmentation.

3.10.3. Speculative Execution - Thực thi dự phòng

Trong môi trường phân tán, luôn có nguy cơ một node bị chậm (straggler), làm kéo dài toàn bộ tiến trình xử lý. Kỹ thuật **speculative execution** cho phép hệ thống khởi động một bản sao dự phòng của tác vụ đang bị chậm ở node khác. Khi một trong hai bản hoàn thành, kết quả được ghi nhận và bản còn lại bị hủy. MapReduce là ví dụ kinh điển áp dụng kỹ thuật này: nếu một bản đồ (map) hoặc giảm (reduce) nào đó bị chậm bất thường, hệ thống sẽ khởi chạy lại bản đó trên node khác. Điều này giúp **giảm thời gian hoàn thành công việc (job latency)** và tăng độ tin cậy xử lý.

3.10.4. Data Striping - Phân mảnh dữ liệu song song

Data striping là kỹ thuật chia nhỏ một tệp lớn thành nhiều phần (stripe hoặc shard) và lưu trữ chúng song song trên nhiều node khác nhau. Khi truy cập, hệ thống có thể đọc hoặc ghi đồng thời các phần dữ liệu này, tăng **băng thông tổng thể (aggregate throughput)**. Kỹ thuật này là nền tảng trong các hệ thống như HDFS, GFS, hoặc Ceph, nơi mỗi block dữ liệu (thường 64MB đến 256MB) được lưu độc lập trên một node, cho phép nhiều client đọc cùng lúc mà không gây nghẽn cổ chai.

3.10.5. Parallelism - Truy cập song song

Để tận dụng toàn bộ tài nguyên của hệ thống phân tán, các thao tác I/O cần được thực hiện song song càng nhiều càng tốt. Các hệ thống DFS hiện đại cho phép:

- Đọc đồng thời nhiều block trong cùng một tệp.
- Ghi song song các shard từ các nguồn khác nhau.
- Truy cập từ nhiều client tới cùng tập dữ liệu thông qua cơ chế caching và lock-free.

Ví dụ, trong Ceph hoặc Amazon S3, một ứng dụng xử lý video có thể tải đồng thời các đoạn clip từ nhiều shard khác nhau, phân phối về các worker xử lý cục bộ.

3.10.6. Tổng kết về các kỹ thuật tối ưu hiệu năng

- **Prefetching:**
 - *Mục tiêu chính:* Giảm độ trễ đọc.
 - *Hệ thống áp dụng tiêu biểu:* Spark, Alluxio, Ceph.
- **Write Coalescing:**
 - *Mục tiêu chính:* Giảm I/O khi ghi nhỏ.

– *Hệ thống áp dụng tiêu biểu:* Kafka, HDFS, RocksDB.

- **Speculative Execution:**

– *Mục tiêu chính:* Giảm ảnh hưởng của các node chạy chậm (straggler).

– *Hệ thống áp dụng tiêu biểu:* Hadoop MapReduce, Spark.

- **Data Striping:**

– *Mục tiêu chính:* Tăng thông lượng I/O tổng thể.

– *Hệ thống áp dụng tiêu biểu:* HDFS, GFS, Ceph.

- **Parallelism:**

– *Mục tiêu chính:* Tăng khả năng mở rộng và tận dụng tài nguyên hệ thống.

– *Hệ thống áp dụng tiêu biểu:* S3, Ceph, GlusterFS.

Việc kết hợp các kỹ thuật trên cho phép hệ thống đáp ứng khối lượng công việc lớn với hiệu suất ổn định, đồng thời vẫn duy trì các yêu cầu về bảo mật, nhất quán và độ tin cậy. Tùy theo đặc điểm truy cập dữ liệu (read-heavy, write-heavy, latency-sensitive), người thiết kế hệ thống cần áp dụng kỹ thuật tối ưu phù hợp để khai thác tối đa hạ tầng phân tán.

Câu hỏi ôn tập cuối chương

1. Hãy trình bày vai trò của NameNode và DataNode trong kiến trúc HDFS. Vì sao NameNode được coi là nút cổ chai tiềm năng trong hệ thống?
2. So sánh cơ chế quản lý siêu dữ liệu tập trung trong HDFS với mô hình phân mảnh không gian tên như CephFS. Ưu và nhược điểm của từng mô hình?
3. Mô hình nhất quán “Session Semantics” phù hợp với những kiểu workload nào? Hãy phân tích ví dụ cụ thể trong HDFS.
4. Trình bày 3 lớp bảo mật trong hệ thống tập phân tán và nêu cách triển khai điển hình của mỗi lớp trong một hệ thống thực tế (ví dụ HDFS, Ceph, S3).
5. Phân biệt giữa cơ chế caching và tiered storage trong hệ thống lưu trữ phân tán. Hãy lấy ví dụ minh họa bằng một hệ thống sử dụng cả hai cơ chế.
6. Một hệ thống lưu trữ cần tiết kiệm không gian và chịu lỗi đến 2 node. Bạn sẽ chọn replication hay erasure coding? Vì sao? Trình bày cấu hình EC phù hợp và đánh giá chi phí lưu trữ.

7. Cho biết các ưu điểm và hạn chế của speculative execution trong MapReduce. Trong trường hợp nào việc kích hoạt kỹ thuật này có thể phản tác dụng?
8. Một tệp lớn 1GB được chia thành block 128MB. Trình bày cách hệ thống HDFS lưu trữ và phân phối các block này. Nếu dùng Erasure Coding 6+3, tệp sẽ được chia như thế nào về logic và phân phối vật lý?
9. Viết một đoạn mã Python sử dụng thư viện `hdfs` để đọc một tệp từ HDFS và thực hiện kiểm tra checksum từng block. Bạn có thể dùng `client.read` và `client.checksum`. Giải thích cách mã đảm bảo tính toàn vẹn dữ liệu.
10. Bạn là kỹ sư hệ thống lưu trữ trong một công ty phân tích dữ liệu lớn. Công ty bạn sử dụng Ceph để lưu log truy cập web theo thời gian thực. Hãy đề xuất chiến lược caching, phân tầng và erasure coding để tối ưu hiệu năng và chi phí lưu trữ. Phân tích ưu tiên giữa latency và throughput.

Chương 4

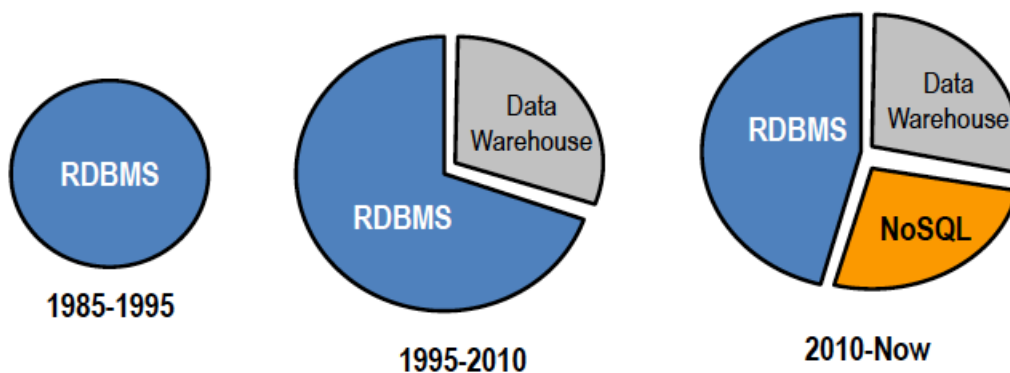
Cơ sở dữ liệu phi quan hệ NoSQL

4.1. Giới thiệu về NoSQL	59
4.2. Khái niệm và các trường hợp sử dụng NoSQL	70
4.3. Các mô hình dữ liệu trong NoSQL	79
4.4. Cơ sở dữ liệu phi quan hệ NoSQL - Phần 1: CAP theorem	115
4.5. CAP theorem	115
4.6. Amazon DynamoDB	128
4.7. Xử lý truy vấn SQL cho dữ liệu lớn với Presto	144

4.1. Giới thiệu về NoSQL

4.1.1. Các Kỹ Nguyên của Cơ Sở Dữ Liệu

Chúng ta có thể chia lịch sử phát triển của cơ sở dữ liệu thành ba kỷ nguyên chính, mỗi kỷ nguyên phản ánh những nhu cầu và thách thức khác nhau trong việc quản lý dữ liệu.



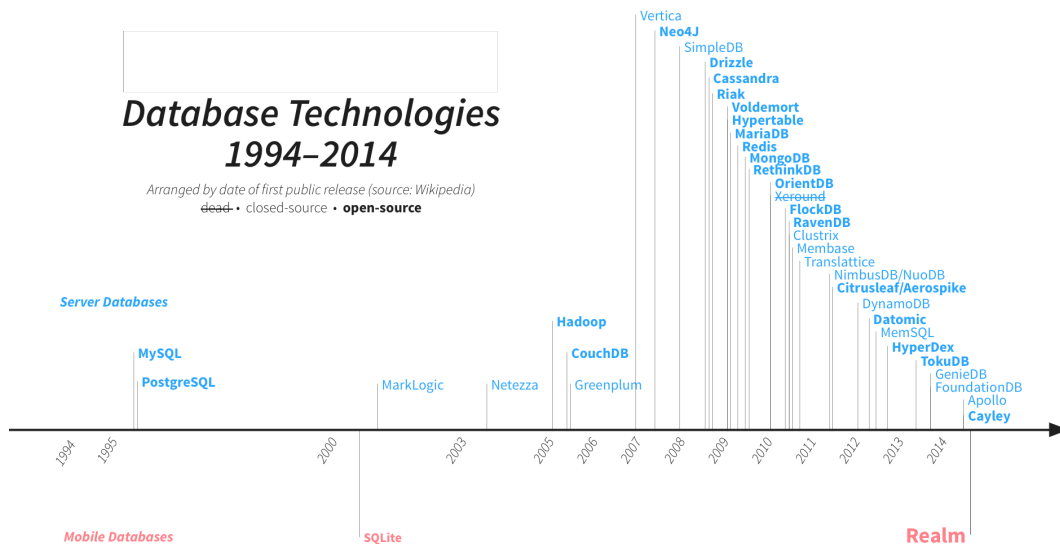
Hình 4.1 Biểu đồ phân chia thị phần công nghệ cơ sở dữ liệu

Nhìn vào biểu đồ phân chia thị phần công nghệ cơ sở dữ liệu, chúng ta thấy rõ sự chuyển dịch đáng kể qua các thời kỳ:

- **Giai đoạn 1985-1995:** Đây là kỷ nguyên mà hệ quản trị cơ sở dữ liệu quan hệ (RDBMS - Relational Database Management Systems) hoàn toàn thống trị. Với mô hình dữ liệu có cấu trúc chặt chẽ, khả năng đảm bảo tính toàn vẹn dữ liệu (ACID properties) và ngôn ngữ truy vấn SQL mạnh mẽ, RDBMS đã trở thành nền tảng cho hầu hết các ứng dụng doanh nghiệp thời bấy giờ. Các hệ thống như Oracle, SQL Server, hay sau này là MySQL và PostgreSQL, đã định hình cách chúng ta lưu trữ và truy xuất dữ liệu có tổ chức.
- **Giai đoạn 1995-2010:** Trong kỷ nguyên này, RDBMS vẫn giữ vai trò chủ đạo, nhưng chúng ta bắt đầu chứng kiến sự xuất hiện và phát triển của kho dữ liệu (Data Warehouse). Nhu cầu phân tích dữ liệu lớn hơn, phục vụ cho các quyết định kinh doanh và báo cáo, đã thúc đẩy sự ra đời của các hệ thống kho dữ liệu chuyên biệt. Kho dữ liệu được thiết kế để lưu trữ dữ liệu lịch sử, tổng hợp từ nhiều nguồn khác nhau, tối ưu cho các truy vấn phân tích phức tạp thay vì các giao dịch hàng ngày. Điều này cho thấy sự dịch chuyển từ chỉ quản lý dữ liệu giao dịch sang khai thác giá trị từ dữ liệu để hỗ trợ kinh doanh.
- **Giai đoạn 2010-Hiện tại:** Kỷ nguyên này đánh dấu sự bùng nổ của dữ liệu lớn (Big Data) và sự đa dạng hóa vượt bậc trong các loại hình dữ liệu (phi cấu trúc, bán cấu trúc). Điều này dẫn đến sự trỗi dậy mạnh mẽ của các hệ thống cơ sở dữ liệu NoSQL (Not Only SQL). NoSQL ra đời để giải quyết những hạn chế của RDBMS trong việc xử lý dữ liệu phi cấu trúc, khả năng mở rộng theo chiều ngang (horizontal scalability) và tính linh hoạt về lược đồ dữ liệu. Mặc dù RDBMS và Data Warehouse vẫn giữ vai trò quan trọng, đặc biệt cho các ứng dụng truyền thống và phân tích dữ liệu có cấu trúc, nhưng NoSQL đã chiếm một phần đáng kể trong thị trường, cho thấy sự cần thiết của các giải pháp chuyên biệt cho những thách thức dữ liệu mới.

Để minh họa rõ hơn cho sự đa dạng và tốc độ phát triển của các công nghệ cơ sở dữ liệu, chúng ta có thể nhìn vào dòng thời gian các công nghệ cơ sở dữ liệu từ năm 1994 đến 2014. Biểu đồ này cho thấy một cái nhìn trực quan về sự ra đời của hàng loạt các hệ thống cơ sở dữ liệu khác nhau:

- **Sự khởi đầu và phát triển của RDBMS mã nguồn mở:** Vào giữa những năm 1990, chúng ta thấy sự xuất hiện của MySQL và PostgreSQL, hai hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở đã trở thành trụ cột cho nhiều ứng dụng web và doanh nghiệp.
- **Sự đa dạng hóa theo mục đích sử dụng:** Bên cạnh các cơ sở dữ liệu máy chủ (Server Databases), chúng ta cũng thấy sự phát triển của cơ sở dữ liệu di động (Mobile Databases) như SQLite, đáp ứng nhu cầu lưu trữ dữ liệu trên các thiết bị cầm tay.



Hình 4.2 Các công nghệ cơ sở dữ liệu

- Kỷ nguyên Big Data và NoSQL:** Khoảng từ năm 2005 trở đi, đặc biệt là sau năm 2008, số lượng các công nghệ cơ sở dữ liệu mới tăng trưởng mạnh mẽ. Đây là thời điểm mà các hệ thống như Hadoop (một framework xử lý dữ liệu lớn), CouchDB (cơ sở dữ liệu tài liệu), và sau đó là hàng loạt các cơ sở dữ liệu NoSQL khác như Cassandra (dữ liệu dạng cột), MongoDB (tài liệu), Redis (key-value), Neo4J (đồ thị), v.v., ra đời. Sự xuất hiện của chúng phản ánh nhu cầu cấp thiết về khả năng xử lý dữ liệu lớn, tốc độ cao, và tính linh hoạt trong mô hình dữ liệu.
- Xu hướng mã nguồn mở:** Rất nhiều công nghệ mới trong giai đoạn này là mã nguồn mở, cho phép cộng đồng đóng góp và thúc đẩy sự đổi mới nhanh chóng.
- Tính động của thị trường:** Dòng thời gian cũng cho thấy một số công nghệ có thể bị khai tử hoặc không còn được phát triển tích cực, minh chứng cho tính cạnh tranh và sự thay đổi liên tục trong lĩnh vực này.

Tóm lại, lịch sử phát triển của cơ sở dữ liệu không phải là một quá trình thay thế hoàn toàn mà là sự bổ sung và đa dạng hóa. Mỗi kỷ nguyên mang đến những giải pháp mới để đáp ứng các yêu cầu ngày càng phức tạp của dữ liệu và ứng dụng, tạo nên một hệ sinh thái cơ sở dữ liệu phong phú và năng động như chúng ta thấy ngày nay. Việc hiểu rõ những “kỷ nguyên” này sẽ giúp chúng ta có cái nhìn toàn diện hơn về lý do tại sao chúng ta có nhiều

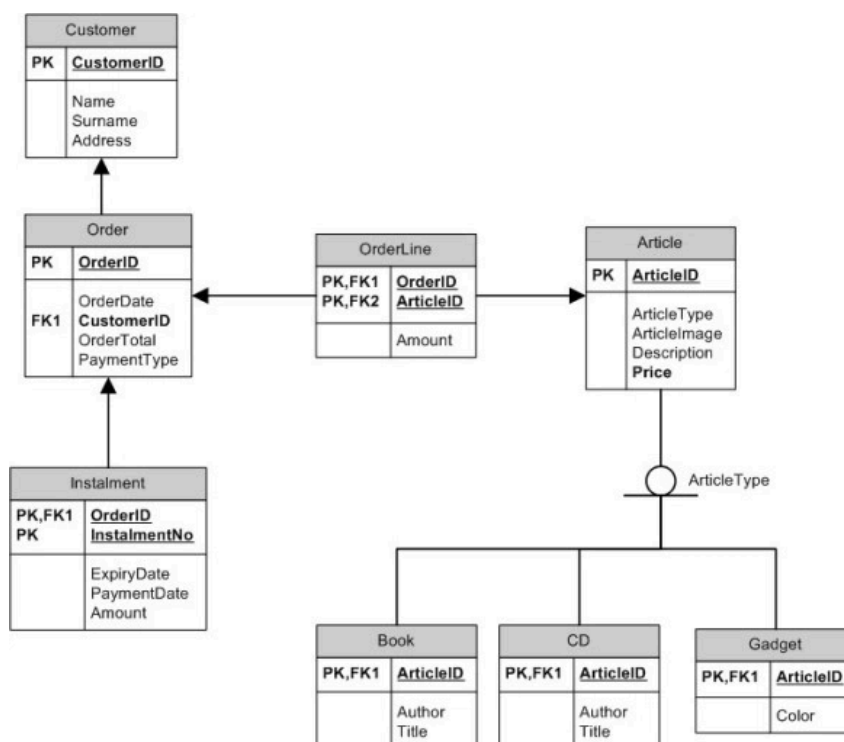
loại cơ sở dữ liệu khác nhau và khi nào nên sử dụng chúng.

4.1.2. Mô hình dữ liệu trước NoSQL

Trước hết chúng ta cần nhìn lại sự phát triển của các mô hình quản lý dữ liệu trước NoSQL. Trong nhiều thập kỷ, các hệ thống cơ sở dữ liệu quan hệ (Relational Databases) và các hệ thống kho dữ liệu (Data Warehouses) phục vụ cho phân tích trực tuyến (OLAP) đã là xương sống của hầu hết các ứng dụng và phân tích dữ liệu trong doanh nghiệp.

Cơ sở dữ liệu quan hệ (Relational Databases) và Xử lý giao dịch trực tuyến (OLTP)

Mô hình dữ liệu quan hệ đã thống trị trong lĩnh vực quản lý dữ liệu từ những năm 1970. Đặc trưng của mô hình này là việc tổ chức dữ liệu thành các bảng (tables) với các hàng (rows) và cột (columns), nơi các mối quan hệ giữa các bảng được định nghĩa rõ ràng thông qua khóa chính (Primary Key - PK) và khóa ngoại (Foreign Key - FK). Mục tiêu chính của các hệ thống này là hỗ trợ xử lý giao dịch trực tuyến (OLTP - Online Transaction Processing), nơi các giao dịch nhỏ, thường xuyên như thêm, sửa, xóa dữ liệu cần được thực hiện một cách nhanh chóng và đáng tin cậy.



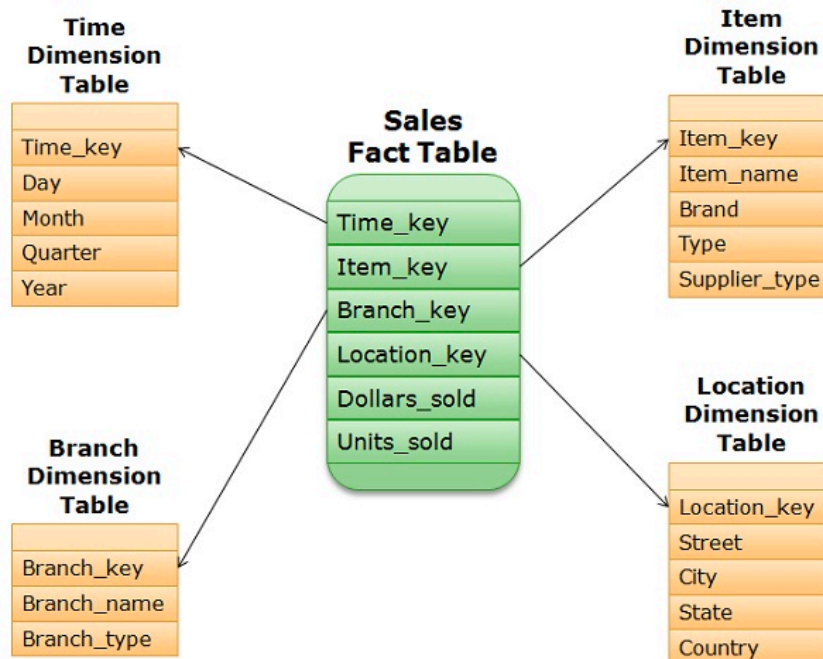
Hình 4.3 Một sơ đồ quan hệ

Hình trên thể hiện một sơ đồ quan hệ phức tạp thường bao gồm nhiều bảng như Customer, Order, Article, OrderLine, và Instalment. Các bảng này được thiết kế

theo các nguyên tắc chuẩn hóa (normalization) để giảm thiểu sự dư thừa dữ liệu và đảm bảo tính nhất quán. Ví dụ, bảng OrderLine liên kết Order và Article, cho phép chúng ta theo dõi chi tiết từng mặt hàng trong một đơn hàng. Các bảng con như Book, CD, Gadget chuyên biệt hóa từ Article thể hiện cách dữ liệu có thể được phân loại và quản lý một cách có cấu trúc. Mô hình này đảm bảo tính toàn vẹn dữ liệu thông qua các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability), vốn rất quan trọng cho các ứng dụng yêu cầu độ chính xác cao như hệ thống ngân hàng hay quản lý kho hàng.

Kho dữ liệu (Data Warehouses) và Xử lý phân tích trực tuyến (OLAP)

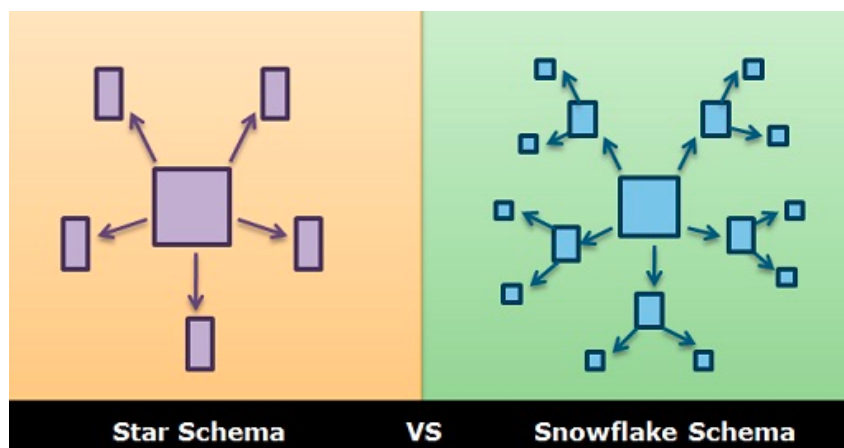
Trong khi các hệ thống OLTP tập trung vào các giao dịch riêng lẻ, các hệ thống kho dữ liệu và xử lý phân tích trực tuyến (OLAP - Online Analytical Processing) lại được thiết kế để phục vụ cho các nhu cầu phân tích dữ liệu phức tạp và ra quyết định kinh doanh. Mục tiêu của OLAP là tổng hợp và phân tích lượng lớn dữ liệu lịch sử từ nhiều nguồn khác nhau.



Hình 4.4 Star Schema

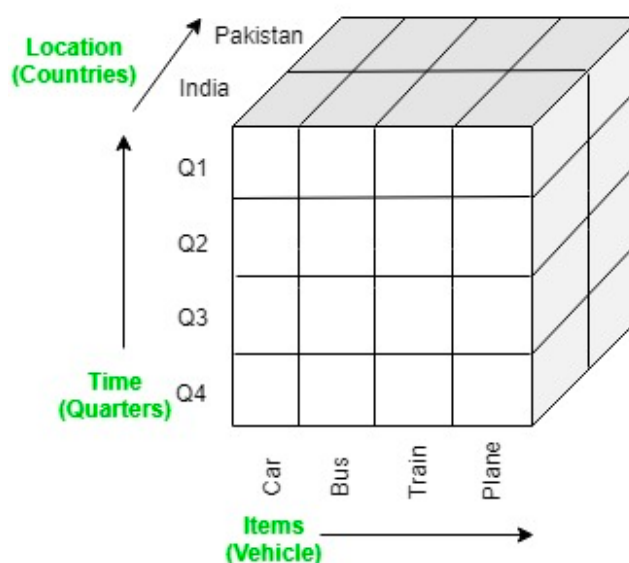
Một trong những lược đồ phổ biến nhất trong kho dữ liệu là **Star Schema (Lược đồ hình sao)**. Hình trên minh họa một Star Schema điển hình cho dữ liệu bán hàng, với bảng sự kiện (Fact Table) Sales Fact Table ở trung tâm. Bảng sự kiện này chứa các số đo (measures) như Dollars_sold và Units_sold, cùng với các khóa ngoại liên kết đến các bảng chiều (Dimension Tables) như Time Dimension, Item Dimension, Branch Dimension, và Location Dimension. Các bảng chiều này cung cấp ngữ cảnh, chẳng hạn

Time Dimension có thể chứa Day, Month, Quarter, Year. Để tối ưu hóa hiệu suất truy vấn phân tích, các bảng chiều trong Star Schema thường được phi chuẩn hóa (denormalized).



Hình 4.5 Star Schema và Snowflake Schema

Bên cạnh Star Schema, chúng ta còn có **Snowflake Schema (Lược đồ bông tuyết)**. Hình trên so sánh giữa Star Schema và Snowflake Schema cho thấy sự khác biệt cơ bản: trong khi Star Schema giữ các bảng chiều đơn giản và phi chuẩn hóa, Snowflake Schema lại chuẩn hóa các bảng chiều này sâu hơn, tạo ra một cấu trúc phức tạp hơn với nhiều bảng hơn. Điều này giúp giảm thiểu sự dư thừa dữ liệu nhưng có thể dẫn đến các truy vấn phức tạp hơn do cần nhiều phép nối (joins) hơn.



Hình 4.6 Data Cube

Để trực quan hóa và phân tích dữ liệu trong kho dữ liệu, khái niệm **Khối dữ liệu (Data Cube)** được sử dụng rộng rãi. Hình trên mô tả khối dữ liệu (Data Cube) được tổ chức theo nhiều chiều (dimensions), chẳng hạn như Location (Countries), Time (Quarters), và Items (Vehicle). Mô hình này cho phép người dùng thực hiện các thao tác phân tích đa chiều như drill-down (đi sâu vào chi tiết), roll-up (tổng hợp), slice (cắt lát), và dice (cắt khối), hỗ trợ đắc lực cho việc khám phá xu hướng và mẫu hình trong dữ liệu.

Tóm lại, các mô hình truyền thống này đã chứng tỏ hiệu quả vượt trội trong việc quản lý dữ liệu có cấu trúc và hỗ trợ các ứng dụng doanh nghiệp quan trọng. Tuy nhiên, sự bùng nổ của dữ liệu phi cấu trúc, bán cấu trúc, cùng với nhu cầu về khả năng mở rộng linh hoạt và hiệu suất cao trong môi trường phân tán, đã đặt ra những thách thức mới mà các hệ thống này đôi khi không thể đáp ứng một cách tối ưu, mở đường cho sự ra đời của các giải pháp NoSQL mà chúng ta sẽ tìm hiểu trong các phần tiếp theo.

4.1.3. RDBMS: one size fits all needs

Trong nhiều thập kỷ, Hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) đã được xem là nền tảng vững chắc và là giải pháp “một giải pháp chung cho mọi nhu cầu” (one size fits all) cho hầu hết các nhu cầu lưu trữ và quản lý dữ liệu. Với mô hình dữ liệu có cấu trúc rõ ràng, khả năng đảm bảo tính toàn vẹn dữ liệu thông qua các giao dịch ACID (Atomicity, Consistency, Isolation, Durability), và ngôn ngữ truy vấn SQL mạnh mẽ, RDBMS đã chứng minh được hiệu quả vượt trội trong nhiều ứng dụng truyền thống.

Tuy nhiên, trong bối cảnh kỷ nguyên dữ liệu lớn và sự phát triển nhanh chóng của các ứng dụng hiện đại, quan niệm này đang dần bị thách thức. Các yêu cầu và thách thức mà RDBMS truyền thống đang phải đối mặt bao gồm:

- **Tập dữ liệu lớn (Large Data Sets):** Với sự bùng nổ của dữ liệu, các hệ thống ngày nay cần xử lý hàng terabyte, petabyte, thậm chí exabyte dữ liệu. RDBMS truyền thống, đặc biệt là trên một nút đơn, thường gặp giới hạn về hiệu suất và khả năng lưu trữ khi quy mô dữ liệu tăng lên quá lớn.
- **Khả năng mở rộng (Scalability):** RDBMS truyền thống thường mở rộng theo chiều dọc (vertical scaling) bằng cách nâng cấp phần cứng của một máy chủ duy nhất. Phương pháp này có giới hạn và tốn kém. Việc mở rộng theo chiều ngang (horizontal scaling) trên nhiều máy chủ là một thách thức lớn đối với RDBMS do tính chất ràng buộc của lược đồ và các giao dịch phức tạp.
- **Độ tin cậy (Reliability):** Mặc dù RDBMS có cơ chế đảm bảo độ tin cậy cao, nhưng một hệ thống đơn nút vẫn tiềm ẩn rủi ro về điểm lỗi duy nhất (single point of failure). Trong các ứng dụng yêu cầu tính sẵn sàng cao 24/7, điều này có thể trở thành một hạn chế nghiêm trọng.

- **Mạng xã hội (Social Networks):** Các ứng dụng mạng xã hội tạo ra lượng lớn dữ liệu liên kết phức tạp (ví dụ: quan hệ bạn bè, bài đăng, lượt thích). RDBMS gặp khó khăn trong việc biểu diễn và truy vấn hiệu quả các mối quan hệ đồ thị này, cũng như xử lý tốc độ ghi/đọc cực cao.
- **Dữ liệu liên kết (Linked Data):** Tương tự như mạng xã hội, dữ liệu liên kết tập trung vào các mối quan hệ giữa các thực thể. RDBMS không được tối ưu cho việc duyệt và truy vấn các đồ thị dữ liệu lớn, nơi cấu trúc dữ liệu không cố định và các mối quan hệ là trọng tâm.
- **Linh hoạt với mô hình không cần lược đồ (Agile Schema Free):** Phát triển phần mềm hiện đại thường ưu tiên sự linh hoạt và khả năng thay đổi nhanh chóng. RDBMS yêu cầu lược đồ cố định, điều này có thể gây khó khăn khi cần thay đổi cấu trúc dữ liệu thường xuyên hoặc khi dữ liệu có tính chất bán cấu trúc/phi cấu trúc. Việc thay đổi lược đồ trong RDBMS thường đòi hỏi thời gian ngừng hoạt động và các thao tác phức tạp.
- **Dữ liệu dạng tài liệu (Document-Data):** Nhiều ứng dụng ngày nay xử lý dữ liệu dưới dạng tài liệu (ví dụ: JSON, XML), có cấu trúc linh hoạt và lồng ghép. RDBMS không được thiết kế để lưu trữ và truy vấn hiệu quả các tài liệu này một cách tự nhiên, thường yêu cầu ánh xạ phức tạp sang các bảng quan hệ.
- **OLAP/Kinh doanh thông minh (BI)/Kho dữ liệu (Data Warehouse):** Mặc dù RDBMS có thể được sử dụng cho các hệ thống OLAP và kho dữ liệu, nhưng chúng thường không tối ưu cho các truy vấn phân tích phức tạp trên tập dữ liệu rất lớn. Các hệ thống này đòi hỏi khả năng tổng hợp, phân tích đa chiều và hiệu suất cao cho các truy vấn đọc, điều mà các giải pháp chuyên biệt thường làm tốt hơn.

Tóm lại, trong khi RDBMS vẫn là một công nghệ mạnh mẽ và phù hợp cho nhiều trường hợp sử dụng truyền thống, sự xuất hiện của các yêu cầu dữ liệu mới về quy mô, tốc độ, tính linh hoạt và cấu trúc đã làm lộ rõ những giới hạn của mô hình “một kích cỡ phù hợp cho tất cả”. Điều này đã thúc đẩy sự phát triển của các hệ thống cơ sở dữ liệu phi quan hệ (NoSQL) và các giải pháp chuyên biệt khác để đáp ứng những thách thức này.

Hội nghị ICDE 2005

Trong lịch sử phát triển của các hệ quản trị cơ sở dữ liệu (DBMS) thương mại, đặc biệt là trong khoảng 25 năm trước thời điểm năm 2005, một cụm từ đã tóm gọn xu hướng chủ đạo: “một kích thước phù hợp cho tất cả” (one size fits all). Cụm từ này phản ánh thực tế rằng kiến trúc DBMS truyền thống, vốn được thiết kế và tối ưu hóa ban đầu cho các tác vụ

xử lý dữ liệu kinh doanh, đã được áp dụng rộng rãi để hỗ trợ nhiều ứng dụng tập trung vào dữ liệu khác nhau, mặc dù các ứng dụng này có đặc điểm và yêu cầu rất đa dạng.

Tuy nhiên, một bài báo khoa học quan trọng đã được trình bày tại Hội nghị Quốc tế về Kỹ thuật Dữ liệu (ICDE) năm 2005, với tiêu đề “One Size Fits All”: An Idea Whose Time Has Come and Gone” (Một kích thước phù hợp cho tất cả: Một ý tưởng đã đến và đi). Trong bài báo này, các tác giả đã lập luận mạnh mẽ rằng khái niệm “một kích thước phù hợp cho tất cả” không còn phù hợp với thị trường cơ sở dữ liệu hiện đại. Thay vào đó, họ dự đoán rằng thế giới thương mại sẽ chứng kiến sự phân mảnh, hình thành một tập hợp các công cụ cơ sở dữ liệu độc lập, mỗi công cụ được thiết kế và tối ưu hóa cho các nhu cầu và loại hình dữ liệu cụ thể.

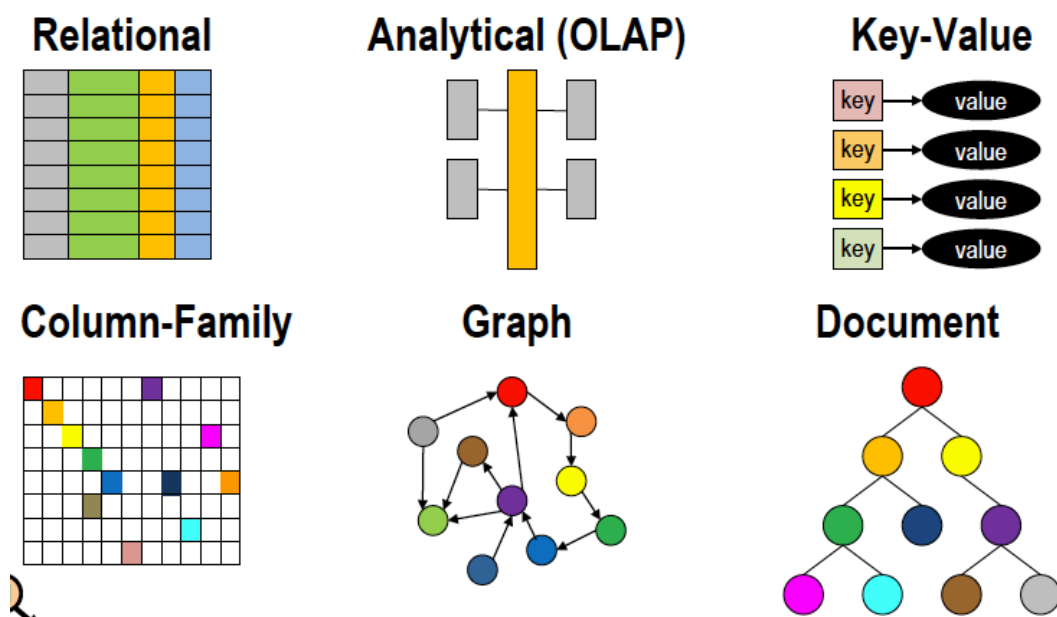
Thông tin chi tiết về bài báo này như sau:

- **Tác giả:** Michael Stonebraker (StreamBase Systems, Inc.) và Ugur Cetintemel (Brown University và StreamBase Systems, Inc.).
- **Xuất bản trong:** Kỷ yếu Hội nghị Quốc tế về Kỹ thuật Dữ liệu lần thứ 21 (ICDE '05 Proceedings of the 21st International Conference on Data Engineering).
- **Trang:** 2-11.
- **Thời gian hội nghị:** Từ ngày 05 đến ngày 08 tháng 4 năm 2005.
- **Địa điểm:** Washington, DC, Hoa Kỳ.
- **Nhà xuất bản:** IEEE Computer Society ©2005.
- **Mã số sách chuẩn quốc tế (ISBN):** 0-7695-2285-8.
- **Mã định danh đối tượng kỹ thuật số (DOI):** 10.1109/ICDE.2005.1.

4.1.4. Thời kỳ sau khi NoSQL ra đời

Sau giai đoạn bùng nổ của các hệ quản trị cơ sở dữ liệu NoSQL, chúng ta nhận thấy một sự chuyển dịch đáng kể trong cách tiếp cận và lựa chọn mô hình dữ liệu. “Sau NoSQL” không có nghĩa là NoSQL đã lỗi thời, mà là chúng ta đã bước vào một kỷ nguyên mà sự đa dạng của các mô hình dữ liệu được công nhận và ứng dụng rộng rãi, vượt ra ngoài khuôn khổ của cơ sở dữ liệu quan hệ truyền thống. Hình trên liệt kê các mô hình dữ liệu phổ biến hiện nay, mỗi mô hình đều có những ưu điểm và trường hợp sử dụng riêng biệt.

- **Quan hệ (Relational):** Đây là mô hình truyền thống và quen thuộc nhất, tổ chức dữ liệu dưới dạng các bảng (table) với hàng (row) và cột (column). Các bảng được liên kết với nhau thông qua các khóa (key), đảm bảo tính toàn vẹn và nhất quán của dữ



liệu. Mô hình này vẫn là lựa chọn hàng đầu cho các ứng dụng yêu cầu giao dịch phức tạp và tính nhất quán cao (ACID).

- Phân tích (OLAP - Online Analytical Processing):** Mô hình này được thiết kế đặc biệt cho các tác vụ phân tích dữ liệu phức tạp, thường liên quan đến việc tổng hợp và truy vấn dữ liệu từ nhiều chiều khác nhau (ví dụ: theo thời gian, sản phẩm, khu vực). Thay vì lưu trữ dữ liệu theo hàng như cơ sở dữ liệu quan hệ, OLAP thường sử dụng cấu trúc khối đa chiều (data cube) để tối ưu hóa hiệu suất cho các truy vấn phân tích.
- Khóa-Giá trị (Key-Value):** Đây là mô hình NoSQL đơn giản nhất, lưu trữ dữ liệu dưới dạng các cặp khóa-giá trị. Mỗi khóa là duy nhất và được sử dụng để truy xuất giá trị tương ứng. Mô hình này cực kỳ nhanh và có khả năng mở rộng cao, phù hợp cho các trường hợp sử dụng như bộ nhớ đệm (caching), quản lý phiên (session management) hoặc lưu trữ cấu hình.
- Họ Cột (Column-Family):** Mô hình này tổ chức dữ liệu theo các cột thay vì hàng. Điều này có nghĩa là các giá trị trong cùng một cột được lưu trữ gần nhau, tối ưu hóa cho các truy vấn chỉ cần một tập hợp con các cột. Mô hình họ cột rất hiệu quả cho các tập dữ liệu lớn, phân tán và thưa thớt, thường được sử dụng trong các ứng dụng Big Data và phân tích dữ liệu lớn.
- Đồ thị (Graph):** Mô hình đồ thị tập trung vào việc lưu trữ các thực thể (nodes) và mối quan hệ giữa chúng (edges). Mỗi node và edge có thể có các thuộc tính riêng. Mô

hình này lý tưởng cho việc phân tích các mối quan hệ phức tạp, chẳng hạn như mạng xã hội, hệ thống khuyến nghị, hoặc phát hiện gian lận.

- **Tài liệu (Document):** Mô hình tài liệu lưu trữ dữ liệu dưới dạng các “tài liệu” tự chứa, thường ở định dạng JSON, BSON hoặc XML. Mỗi tài liệu có thể có cấu trúc khác nhau, mang lại sự linh hoạt cao cho việc phát triển ứng dụng và thay đổi lược đồ dữ liệu. Mô hình này rất phổ biến cho các ứng dụng web, di động và các hệ thống cần lưu trữ dữ liệu bán cấu trúc.

Tóm lại, kỷ nguyên “Sau NoSQL” là kỷ nguyên của sự lựa chọn thông minh. Thay vì cố gắng ép buộc mọi loại dữ liệu vào một mô hình duy nhất, chúng ta có thể linh hoạt lựa chọn mô hình dữ liệu phù hợp nhất với yêu cầu cụ thể của ứng dụng, từ đó tối ưu hóa hiệu suất, khả năng mở rộng và chi phí. Điều này đòi hỏi các nhà phát triển và kiến trúc sư hệ thống phải có kiến thức sâu rộng về các mô hình dữ liệu khác nhau và biết cách kết hợp chúng một cách hiệu quả.

Tổng quan về NoSQL

Để hiểu rõ hơn về bức tranh tổng thể của các hệ quản trị cơ sở dữ liệu NoSQL, chúng ta có thể phân loại chúng dựa trên mô hình dữ liệu cơ bản mà chúng sử dụng. Dưới đây là bốn nhóm chính cùng với một số ví dụ điển hình:

- **Cơ sở dữ liệu tài liệu (Document Databases)** Nhóm này lưu trữ dữ liệu dưới dạng các tài liệu tự mô tả, thường là JSON, BSON hoặc XML. Mỗi tài liệu là một đơn vị độc lập, không yêu cầu lược đồ cố định, mang lại sự linh hoạt cao trong việc thay đổi cấu trúc dữ liệu. Các ví dụ nổi bật trong nhóm này bao gồm MongoDB, CouchDB và RavenDB.
- **Cơ sở dữ liệu cột (Column-Family Databases)** Còn được gọi là kho dữ liệu cột rộng (Wide-Column Stores), các hệ thống này tổ chức dữ liệu theo các hàng và cột. Tuy nhiên, khác với cơ sở dữ liệu quan hệ, chúng cho phép mỗi hàng có các cột khác nhau và các cột có thể được nhóm thành các họ cột (column families). Mô hình này tối ưu cho việc truy vấn trên các cột cụ thể trên tập dữ liệu lớn. Apache HBase và Cassandra là hai đại diện tiêu biểu của nhóm này.
- **Cơ sở dữ liệu khóa-giá trị (Key-Value Databases)** Đây là mô hình đơn giản nhất, lưu trữ dữ liệu dưới dạng các cặp khóa-giá trị. Mỗi khóa là duy nhất và được sử dụng để truy xuất giá trị tương ứng, trong đó giá trị có thể là bất kỳ loại dữ liệu nào (chuỗi, số, đối tượng phức tạp, v.v.). Các hệ thống như Redis, Oracle NoSQL Database, Project Voldemort và Riak (Basho) thuộc nhóm này.

- **Cơ sở dữ liệu đồ thị (Graph Databases)** Nhóm này được thiết kế đặc biệt để quản lý và truy vấn dữ liệu có mối quan hệ phức tạp. Dữ liệu được biểu diễn dưới dạng các nút (nodes) và cạnh (edges), nơi các nút đại diện cho thực thể và các cạnh thể hiện mối quan hệ giữa chúng. Mô hình này rất phù hợp cho các ứng dụng cần phân tích mạng lưới hoặc mối quan hệ sâu sắc, ví dụ điển hình là Neo4j.

4.2. Khái niệm và các trường hợp sử dụng NoSQL

4.2.1. Tại sao NoSQL?

Trong bối cảnh sự phát triển vượt bậc của các ứng dụng web và sự bùng nổ của nội dung do người dùng tạo ra, cùng với dữ liệu từ các nguồn mới như cảm biến và hệ thống giám sát, chúng ta đang đối mặt với một **khối lượng dữ liệu khổng lồ (big data)**. Điều này đặt ra những thách thức và đồng thời mở ra những cơ hội mới trong việc lưu trữ, phân tích và quản lý dữ liệu. Dữ liệu ngày càng trở nên **bán cấu trúc và phân tán**, khiến cho các kỹ thuật quản lý dữ liệu truyền thống, vốn yêu cầu định nghĩa schema trước và các tham chiếu quan hệ chặt chẽ, trở nên kém hiệu quả. Để giải quyết những vấn đề liên quan đến dữ liệu bán cấu trúc và khối lượng lớn này, một **lớp sản phẩm cơ sở dữ liệu mới đã ra đời, được gọi chung là NoSQL**.

1. Ứng dụng web có những nhu cầu khác biệt

Các ứng dụng web hiện đại thường có những yêu cầu đặc thù mà các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) truyền thống khó có thể đáp ứng một cách tối ưu. Cụ thể, những nhu cầu này bao gồm:

- **Khả năng mở rộng theo chiều ngang (Horizontal scalability) – giúp giảm chi phí:** NoSQL được thiết kế để **mở rộng quy mô bằng cách thêm các máy tính thông thường vào một cụm (cluster)**, thay vì phải nâng cấp lên các máy chủ mạnh mẽ và đắt tiền hơn (phương pháp mở rộng theo chiều dọc). Các cơ sở hạ tầng được mở rộng theo chiều ngang tại các tập đoàn công nghệ lớn như Google, Amazon, Facebook, eBay và Yahoo! có thể liên quan đến hàng nghìn, thậm chí hàng trăm nghìn máy chủ. Việc sử dụng các **máy tính thông thường (commodity systems)** làm giảm đáng kể chi phí đầu tư và vận hành.
- **Phân tán địa lý (Geographically distributed):** NoSQL ra đời trong thế giới của các ứng dụng Internet có khả năng mở rộng lớn và thường được thiết kế để hoạt động hiệu quả trong môi trường **điện toán phân tán và song song**. Điều này giúp hệ thống có thể hoạt động hiệu quả ngay cả khi các thành phần của nó được phân tán trên nhiều trung tâm dữ liệu khác nhau, đảm bảo khả năng truy cập và hiệu suất cho người dùng trên toàn cầu.

- **Tính đàn hồi (Elasticity):** NoSQL hỗ trợ **tăng hoặc giảm số lượng node trong cluster một cách linh hoạt** để đáp ứng sự thay đổi của tải công việc. Ví dụ, các hệ thống như Membase cho phép mở rộng và thu hẹp số lượng máy chủ trong một cụm mà không làm gián đoạn việc truy cập của client. Tương tự, MongoDB hỗ trợ **auto-sharding để mở rộng quy mô theo chiều ngang một cách dễ dàng**, tự động phân chia dữ liệu và phân bổ chúng trên các máy chủ mới.
- **Schema linh hoạt (Schema less, flexible schema) cho dữ liệu bán cấu trúc:** NoSQL là một thuật ngữ bao trùm cho tất cả các cơ sở dữ liệu và kho dữ liệu **không tuân theo các nguyên tắc RDBMS truyền thống**. Chúng được thiết kế để làm việc với các **tập dữ liệu bán cấu trúc và phân tán lớn**. Các cơ sở dữ liệu tài liệu (document databases) như MongoDB và CouchDB **linh hoạt trong việc lưu trữ các tài liệu với các tập thuộc tính khác nhau trong cùng một collection**, cho phép thêm thuộc tính mới một cách dễ dàng khi dữ liệu phát triển mà không cần thay đổi cấu trúc bảng. Ngoài ra, các cơ sở dữ liệu hướng cột (column-oriented databases) như HBase cũng rất phù hợp cho **dữ liệu thưa (sparse data sets)** vì chúng không cần lưu trữ các giá trị null, tiết kiệm không gian lưu trữ.
- **Dễ dàng hơn cho nhà phát triển (Easier for developers):** Sự linh hoạt về schema giúp nhà phát triển **không phải lo lắng về việc thay đổi schema liên tục** như trong RDBMS khi ứng dụng phát triển nhanh chóng. Nó cho phép lưu trữ dữ liệu theo cách tự nhiên hơn, giống với các đối tượng trong lập trình, thay vì phải chuyển đổi sang mô hình quan hệ cứng nhắc, giúp tăng tốc độ phát triển và giảm thiểu lỗi.
- **Lưu trữ dữ liệu không đồng nhất (Heterogeneous data storage):** Khác với RDBMS thường yêu cầu định nghĩa kiểu dữ liệu chặt chẽ, NoSQL cho phép **các trường dữ liệu có thể chứa nhiều loại giá trị khác nhau** (ví dụ: một trường có thể chứa chuỗi ký tự, sau đó lại chứa số nguyên). Điều này cho phép lưu trữ và quản lý các loại dữ liệu đa dạng trong cùng một hệ thống một cách linh hoạt.
- **Tính sẵn sàng cao (High Availability) / Khôi phục sau thảm họa (Disaster Recovery):** Nhiều hệ thống NoSQL được xây dựng với khả năng chịu lỗi và sẵn sàng cao làm ưu tiên hàng đầu, đặc biệt là các giải pháp dựa trên ý tưởng từ Amazon Dynamo. **Sao chép dữ liệu (Replication)** là một kỹ thuật phổ biến được sử dụng trong NoSQL để đảm bảo tính bền vững và khả năng phục hồi dữ liệu, giúp hệ thống tiếp tục hoạt động ngay cả khi một số node gặp sự cố.

2. Ứng dụng web không phải lúc nào cũng cần

Mặc dù NoSQL mang lại nhiều lợi ích, nhưng điều quan trọng là phải nhận thức rằng nó

không phải là giải pháp cho mọi vấn đề. Có những trường hợp mà các đặc tính của NoSQL không phải là ưu tiên hàng đầu hoặc thậm chí có thể là một hạn chế:

- **Giao dịch (Transaction):** Nhiều tập dữ liệu hiện đại, như **file log web traffic, cập nhật trạng thái mạng xã hội, dữ liệu giá chứng khoán**, chủ yếu được **ghi một lần và đọc nhiều lần**. Đối với những loại dữ liệu này, yêu cầu về giao dịch (transactional requirements) với tính chất ACID (Atomicity, Consistency, Isolation, Durability) là **rất hạn chế hoặc không có**. Trong những trường hợp này, việc sử dụng một hệ thống NoSQL không hỗ trợ giao dịch phức tạp sẽ không gây ảnh hưởng đáng kể.
- **Tính nhất quán mạnh (Strong consistency):** Định lý CAP (Consistency, Availability, Partition Tolerance) của Brewer chỉ ra rằng trong một hệ thống phân tán, **không thể đồng thời đạt được cả ba yếu tố này**. Chúng ta phải đánh đổi và hy sinh ít nhất một yếu tố để ưu tiên hai yếu tố còn lại. Nhiều ứng dụng web lớn ưu tiên **tính sẵn sàng (Availability) và khả năng chịu phân vùng (Partition Tolerance)** hơn là tính nhất quán mạnh. Vì vậy, mô hình **tính nhất quán cuối cùng (Eventual consistency)** thường được chấp nhận, trong đó dữ liệu có thể không nhất quán ngay lập tức trên tất cả các node sau một bản cập nhật, nhưng cuối cùng sẽ đồng bộ hóa sau một khoảng thời gian nhất định.
- **Truy vấn phức tạp (Complex queries):** Đối với các kho dữ liệu lớn, phân tán, thường có các **cách phân tích và truy cập dữ liệu được xác định trước và cụ thể**. Do đó, các truy vấn ad-hoc phức tạp và biểu thức truy vấn phong phú không phải là ưu tiên hàng đầu. Trong khi RDBMS mạnh về SQL và các truy vấn phức tạp, **NoSQL thường đơn giản hóa các khả năng truy vấn** để đạt được hiệu suất và khả năng mở rộng tối đa.

Như vậy, NoSQL không phải là giải pháp cho mọi vấn đề và chắc chắn có những hạn chế riêng. Tuy nhiên, nó là một công cụ mạnh mẽ, tuyệt vời cho những công việc cụ thể khi cần xử lý dữ liệu lớn và linh hoạt, đặc biệt trong môi trường ứng dụng web hiện đại.

4.2.2. SQL vs NoSQL

Trong bối cảnh phát triển nhanh chóng của công nghệ thông tin và sự bùng nổ của dữ liệu, việc lựa chọn hệ quản trị cơ sở dữ liệu phù hợp là một quyết định chiến lược quan trọng đối với bất kỳ hệ thống phần mềm nào. Hai mô hình phổ biến nhất hiện nay là SQL (còn gọi là cơ sở dữ liệu quan hệ) và NoSQL (cơ sở dữ liệu phi quan hệ). Mặc dù cả hai đều có mục tiêu chung là lưu trữ và quản lý dữ liệu, chúng lại có những triết lý thiết kế và ưu nhược điểm khác nhau, phù hợp với các loại ứng dụng và khối lượng dữ liệu đặc thù. Dưới đây là

bảng so sánh chi tiết giữa SQL và NoSQL dựa trên các tiêu chí quan trọng đưa ra cái nhìn tổng quan và sâu sắc hơn về hai mô hình này.

- **Dung lượng dữ liệu (Data Volume):**

- **SQL:** Các hệ thống cơ sở dữ liệu quan hệ truyền thống thường được thiết kế để xử lý dữ liệu trong phạm vi từ Gigabyte đến Terabyte. Việc mở rộng quy mô để xử lý lượng dữ liệu lớn hơn nhiều thường gặp phải những thách thức đáng kể về hiệu suất và chi phí.
- **NoSQL:** Được xây dựng để đáp ứng nhu cầu của “Big Data”, NoSQL có khả năng mở rộng vượt trội, có thể xử lý dữ liệu từ Petabyte (1 Petabyte = 1000 Terabyte) đến Exabyte (1 Exabyte = 1000 Petabyte) và thậm chí là Zetabyte (1 Zetabyte = 1000 Exabyte). Điều này làm cho NoSQL trở thành lựa chọn lý tưởng cho các ứng dụng với khối lượng dữ liệu khổng lồ và tốc độ tăng trưởng nhanh chóng.

- **Kiến trúc (Architecture):**

- **SQL:** Thường có kiến trúc tập trung (Centralized). Dữ liệu được lưu trữ trên một máy chủ duy nhất hoặc một cụm máy chủ được quản lý chặt chẽ. Việc mở rộng quy mô thường được thực hiện theo chiều dọc (vertical scaling) bằng cách nâng cấp phần cứng của máy chủ hiện có (ví dụ: thêm RAM, CPU mạnh hơn).
- **NoSQL:** Được xây dựng trên kiến trúc phân tán (Distributed). Dữ liệu được chia nhỏ và lưu trữ trên nhiều máy chủ khác nhau, cho phép mở rộng theo chiều ngang (horizontal scaling) bằng cách thêm nhiều máy chủ vào hệ thống. Kiến trúc này không chỉ tăng cường khả năng mở rộng mà còn cải thiện khả năng chịu lỗi và tính sẵn sàng của hệ thống.

- **Cấu trúc dữ liệu (Data Structure):**

- **SQL:** Yêu cầu dữ liệu phải có cấu trúc chặt chẽ (Structured). Dữ liệu được tổ chức thành các bảng với các hàng và cột được định nghĩa trước, tuân thủ một lược đồ (schema) cố định. Mọi dữ liệu mới phải phù hợp với lược đồ đã định nghĩa.
- **NoSQL:** Linh hoạt hơn nhiều, có thể xử lý cả dữ liệu bán cấu trúc (Semi-structured) như JSON, XML và dữ liệu phi cấu trúc (Unstructured) như văn bản, hình ảnh, video. Điều này giúp NoSQL phù hợp với các loại dữ liệu đa dạng, thường xuyên thay đổi và không có cấu trúc cố định.

- **Ngôn ngữ truy vấn (Query Language):**

- **SQL:** Sử dụng Ngôn ngữ truy vấn có cấu trúc (Structured Query Language) – một ngôn ngữ khai báo chuẩn hóa, mạnh mẽ và được sử dụng rộng rãi để định nghĩa, thao tác và truy vấn dữ liệu. SQL có cú pháp rõ ràng và dễ học.
- **NoSQL:** Không có một ngôn ngữ truy vấn khai báo chuẩn hóa chung. Mỗi loại cơ sở dữ liệu NoSQL (ví dụ: cơ sở dữ liệu tài liệu, khóa-giá trị, đồ thị) có thể có ngôn ngữ truy vấn riêng hoặc API để tương tác với dữ liệu, thường mang tính lập trình hơn và được tối ưu hóa cho mô hình dữ liệu cụ thể của chúng.

- **Mô hình dữ liệu (Data Model):**

- **SQL:** Có mô hình dữ liệu ổn định (Stable Data Model) hay còn gọi là “schema-on-write”. Lược đồ phải được định nghĩa rõ ràng và cố định trước khi dữ liệu được ghi vào cơ sở dữ liệu. Mọi thay đổi về lược đồ đều yêu cầu quá trình di chuyển dữ liệu (migration) phức tạp.
- **NoSQL:** Thường là “schema-less” (không lược đồ) hoặc “schema-on-read”. Điều này có nghĩa là lược đồ không cần được định nghĩa trước và có thể thay đổi linh hoạt. Dữ liệu có thể được ghi vào mà không cần tuân thủ một cấu trúc cố định, lược đồ sẽ được suy luận khi dữ liệu được đọc, mang lại sự linh hoạt cao cho việc phát triển ứng dụng.

- **Mối quan hệ (Relationships):**

- **SQL:** Xử lý các mối quan hệ phức tạp giữa các bảng thông qua khóa ngoại (foreign keys) và các phép nối (JOINS). Điều này đảm bảo tính toàn vẹn dữ liệu và giảm thiểu sự trùng lặp thông qua chuẩn hóa dữ liệu.
- **NoSQL:** Thường ưu tiên các mối quan hệ ít phức tạp hơn. Thay vì sử dụng JOINS, NoSQL thường sử dụng các cấu trúc nhúng (embedded structures) hoặc denormalization (phi chuẩn hóa) để lưu trữ dữ liệu liên quan cùng nhau trong một tài liệu hoặc bản ghi, nhằm tối ưu hóa hiệu suất truy vấn trong môi trường phân tán, nơi các phép nối có thể rất tốn kém.

- **Thuộc tính giao dịch (Transaction Property):**

- **SQL:** Tuân thủ thuộc tính ACID (Atomicity, Consistency, Isolation, Durability) một cách nghiêm ngặt. Điều này đảm bảo rằng các giao dịch được thực hiện một cách đáng tin cậy, toàn vẹn và nhất quán, ngay cả khi có lỗi hệ thống.

- **NoSQL:** Thường ưu tiên tính nhất quán cuối cùng (Eventual Consistency). Điều này có nghĩa là sau một khoảng thời gian ngắn, dữ liệu trên tất cả các bản sao sẽ trở nên nhất quán, nhưng trong thời gian chuyển tiếp, có thể có sự không nhất quán tạm thời giữa các nút. Đây là một sự đánh đổi để đạt được khả năng sẵn sàng cao và khả năng mở rộng.

- **Ưu tiên (Priority):**

- **SQL:** Ưu tiên hàng đầu là tính toàn vẹn của giao dịch (Transaction is priority) và độ tin cậy của dữ liệu. SQL được thiết kế để đảm bảo rằng mọi giao dịch đều được thực hiện một cách chính xác và đầy đủ.
- **NoSQL:** Ưu tiên cao hơn là khả năng sẵn sàng cao (High Availability) và khả năng mở rộng cao (High Scalability). NoSQL phù hợp với các ứng dụng cần xử lý lượng lớn yêu cầu đồng thời và không thể chấp nhận thời gian ngừng hoạt động, ngay cả khi điều đó có nghĩa là phải chấp nhận một mức độ nhất quán thấp hơn trong thời gian ngắn.

- **Thao tác dữ liệu (Data Manipulation):**

- **SQL:** Thao tác dữ liệu chủ yếu thông qua việc nối các bảng (Joins Tables) để kết hợp thông tin từ nhiều nguồn khác nhau, dựa trên các mối quan hệ đã định nghĩa.
- **NoSQL:** Thường sử dụng các cấu trúc nhúng (Embedded structures) để lưu trữ dữ liệu liên quan trong cùng một tài liệu hoặc bản ghi. Điều này giúp truy xuất dữ liệu nhanh chóng trong một lần đọc mà không cần thực hiện các phép nối phức tạp, tối ưu hóa cho các truy vấn đọc dữ liệu lớn.

4.2.3. Các trường hợp sử dụng NoSQL

Các hệ quản trị cơ sở dữ liệu NoSQL đã trở thành một lựa chọn mạnh mẽ và linh hoạt cho nhiều ứng dụng hiện đại, đặc biệt là khi đối mặt với những thách thức mà các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) truyền thống thường gặp phải. Dưới đây là những trường hợp sử dụng chính mà NoSQL thể hiện rõ ưu điểm vượt trội của mình.

Khối lượng dữ liệu khổng lồ ở quy mô lớn (Big volume)

Sự bùng nổ của nội dung do người dùng tạo ra, cùng với sự xuất hiện của các nguồn dữ liệu mới như cảm biến, GPS hay hệ thống theo dõi tự động, đã thúc đẩy một sự gia tăng nhanh chóng về khối lượng và loại dữ liệu được tạo ra, thao tác, phân tích và lưu trữ. Những tập dữ liệu khổng lồ này, thường được gọi là dữ liệu lớn (big data), đặt ra những thách thức

đáng kể nhưng cũng mở ra nhiều cơ hội mới trong việc quản lý và khai thác thông tin. Bất kỳ tập dữ liệu nào vượt quá vài terabyte đều có thể được phân loại là dữ liệu lớn, và đây cũng là ngưỡng mà các kỹ thuật RDBMS truyền thống bắt đầu gặp khó khăn trong việc xử lý hiệu quả.

Các tập đoàn công nghệ hàng đầu thế giới như Google, Amazon, Yahoo và Facebook là những ví dụ điển hình cho việc áp dụng NoSQL thành công. Cơ sở hạ tầng của họ thường bao gồm hàng nghìn, thậm chí hàng trăm nghìn máy chủ, được thiết kế để mở rộng theo chiều ngang một cách linh hoạt. Để minh họa cho quy mô dữ liệu mà các hệ thống này phải đối mặt, chúng ta có thể thấy Google xử lý tới 24 petabyte dữ liệu mỗi ngày. Facebook đã lưu trữ 1.5 petabyte ảnh, tương đương với 60 tỷ hình ảnh. Hệ thống nhắn tin của Facebook, được xây dựng trên HBase (một bản sao của Google Bigtable), xử lý hơn 135 tỷ tin nhắn mỗi tháng cho hơn 500 triệu người dùng tích cực. Twitter cũng tạo ra 12 TB dữ liệu tweet và tin nhắn trực tiếp mỗi ngày, tương đương hơn 4 petabyte mỗi năm. LinkedIn sử dụng Hadoop cho các tác vụ phân tích dữ liệu lớn, xử lý hơn 120 tỷ mối quan hệ mỗi ngày, yêu cầu hơn 16 TB dữ liệu trung gian.

Trong bối cảnh dữ liệu lớn, các cơ sở dữ liệu hướng cột (column-oriented databases) như HBase và Hypertable đặc biệt phù hợp để lưu trữ và mở rộng. Chúng có khả năng quản lý một số lượng lớn các cột và hàng mà không lãng phí không gian cho các giá trị null, điều này rất quan trọng khi xử lý các tập dữ liệu thưa thớt.

Khối lượng truy vấn cực lớn (Big velocity)

Đối với các tập dữ liệu lớn, việc thao tác đòi hỏi các quy trình song song mạnh mẽ, có khả năng phục hồi sau lỗi và cung cấp kết quả trong thời gian hợp lý. NoSQL được thiết kế đặc biệt để xử lý khối lượng công việc truy vấn cao, đặc biệt là các hoạt động đọc và ghi.

Cassandra nổi tiếng với tốc độ đọc và ghi cực nhanh, đây là một trong những lý do khiến nó được Facebook và Twitter sử dụng rộng rãi. Các hoạt động ghi trong Cassandra rất nhanh vì chúng chỉ đơn giản được thêm vào nhật ký cam kết (commit log) và không có khóa nào được áp dụng trong đường dẫn quan trọng. Các giải pháp NoSQL trong bộ nhớ như Redis và MongoDB là lựa chọn tốt cho các trường hợp cần ghi nhanh và thao tác dữ liệu theo thời gian thực, bởi vì chúng giữ dữ liệu trong bộ nhớ chính và chỉ ghi vào đĩa khi cần thiết. Redis được định nghĩa là một “máy chủ cấu trúc dữ liệu” vì nó hỗ trợ các cấu trúc dữ liệu phong phú như chuỗi, danh sách, tập hợp và tập hợp đã sắp xếp, cung cấp một tập hợp các lệnh thao tác dữ liệu rất đa dạng. MongoDB cung cấp khả năng truy vấn tương tự SQL và hỗ trợ các mô hình dữ liệu linh hoạt, làm cho nó phù hợp cho các ứng dụng web cần đọc và ghi nhanh. Thậm chí, các công cụ như HandlerSocket cho MySQL còn cho phép thực hiện 750.000 truy vấn tra cứu khóa chính mỗi giây bằng cách truy cập trực tiếp vào công cụ lưu trữ MySQL, bỏ qua lớp SQL.

Tính sẵn sàng cao (High availability)

Tính sẵn sàng là khả năng của hệ thống để phục vụ yêu cầu khi cần thiết. NoSQL đặc biệt mạnh về tính sẵn sàng cao trong môi trường phân tán. Amazon Dynamo, cùng với các bản sao mã nguồn mở như Apache Cassandra, Voldemort và Riak, tập trung vào việc cung cấp tính sẵn sàng và khả năng mở rộng vượt trội, đóng vai trò là xương sống cho các hệ thống phân tán chịu lỗi và có tính sẵn sàng cao của Amazon.

Một mô hình nhất quán thường được sử dụng trong các hệ thống phân tán là tính nhất quán cuối cùng (eventual consistency). Mô hình này ngụ ý rằng có thể có những khoảng thời gian không nhất quán nhỏ giữa các nút được sao chép khi dữ liệu được cập nhật, nhưng tất cả các bản sao cuối cùng sẽ trở nên nhất quán nếu không có thêm cập nhật nào được gửi. Mô hình này cho phép tính sẵn sàng cao ngay cả khi có các phân vùng mạng hoặc lỗi nút.

MongoDB hỗ trợ replica sets (một master và hai slave trong một tập hợp ba bản sao), cho phép phục hồi và chuyển đổi dự phòng tự động. Các shard cũng được sao chép để đảm bảo chuyển đổi dự phòng. HBase hỗ trợ cân bằng tải tự động, chuyển đổi dự phòng, nén và nhiều shard trên mỗi máy chủ, đồng thời hoạt động tốt với HDFS, cung cấp mức độ dự phòng và chịu lỗi bổ sung. Cassandra sử dụng mô hình ngang hàng (peer-to-peer), nghĩa là không có một master duy nhất; tất cả các nút đều có khả năng là master, giúp tăng cường khả năng mở rộng đọc và ghi và cho phép các nút hoạt động ngay cả trong trường hợp phân vùng. CouchDB là một hệ thống phân tán có tính sẵn sàng cao, ưu tiên tính sẵn sàng hơn tính nhất quán, sử dụng Kiểm soát đồng thời đa phiên bản (MVCC) và sao chép tăng dần/chịu lỗi.

Linh hoạt, tiến hóa lược đồ (Flexible, schema evolution)

Một trong những động lực chính đằng sau sự xuất hiện của NoSQL là việc dữ liệu ngày càng trở nên bán cấu trúc (semi-structured) và thưa thớt (sparse), điều này đặt ra câu hỏi về các kỹ thuật quản lý dữ liệu truyền thống dựa trên định nghĩa lược đồ trước và tham chiếu quan hệ. Nhiều cơ sở dữ liệu NoSQL quảng bá một kho dữ liệu phi lược đồ (schema-less), nơi sự tiến hóa là dễ dàng và tự nhiên.

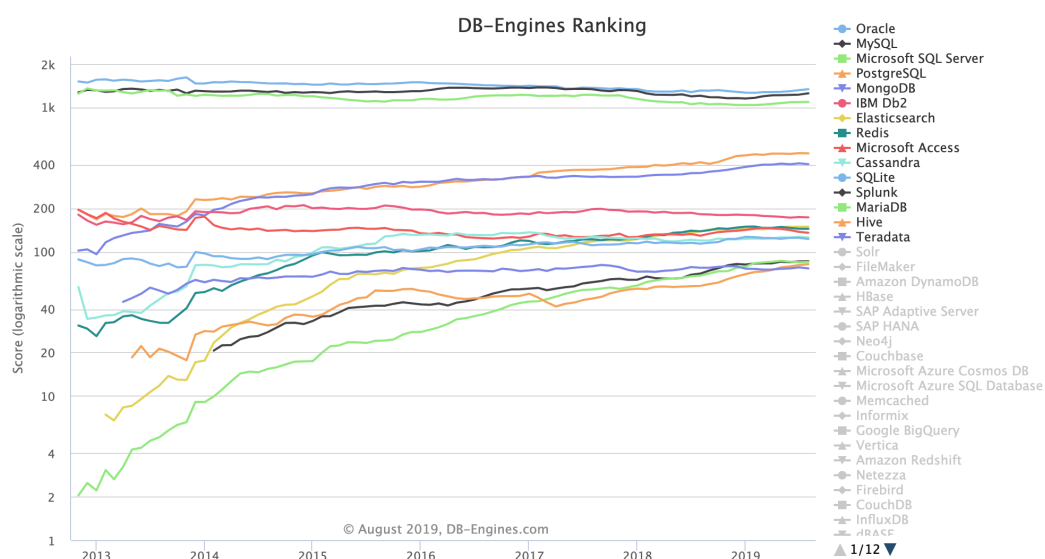
Các cơ sở dữ liệu tài liệu (document databases) như MongoDB và CouchDB rất linh hoạt về mặt lược đồ, cho phép các tài liệu tự chứa với các tập thuộc tính khác nhau được lưu trữ trong một tập hợp duy nhất. Điều này đặc biệt thuận lợi cho các tập dữ liệu thưa thớt vì các trường null không cần được lưu trữ, và việc thêm các trường mới khi tài liệu phát triển là điều đơn giản. MongoDB hỗ trợ BSON (Binary JSON), một định dạng nhị phân cho các cấu trúc giống JSON, cho phép nó lưu trữ các bộ sưu tập dữ liệu tùy ý được biểu diễn theo hệ thống phân cấp đối tượng giống JSON.

Cơ sở dữ liệu hướng cột (ví dụ: HBase, Hypertable) chỉ yêu cầu định nghĩa lược đồ tối

thiếu, chủ yếu là các column-family (nhóm các cột liên quan hợp lý). Bản thân các cột không cần định nghĩa trước và có thể lưu trữ bất kỳ loại dữ liệu nào. Chúng đặc biệt hiệu quả trong việc lưu trữ dữ liệu thưa thớt bằng cách không lưu trữ các giá trị null. Hơn nữa, các cơ sở dữ liệu hướng cột có thể duy trì nhiều phiên bản của mỗi ô dữ liệu (cell), hữu ích cho việc theo dõi sự thay đổi của dữ liệu theo thời gian.

Các kho khóa/giá trị (key/value stores) như Redis và Berkeley DB có khái niệm lược đồ rất hạn chế, chủ yếu là cặp khóa/giá trị, do đó sự tiến hóa lược đồ ít liên quan hơn. Berkeley DB không áp đặt bất kỳ giả định nào về lược đồ ngụ ý, cho phép các API cấp cao hơn lưu trữ dữ liệu có cấu trúc hoặc bán cấu trúc một cách linh hoạt. Các lược đồ linh hoạt này đặc biệt có lợi cho các ứng dụng web phát triển nhanh chóng, nơi các thay đổi thường xuyên và đáng kể đối với mô hình RDBMS cơ bản sẽ gây ra nhiều khó khăn và vấn đề di chuyển dữ liệu.

4.2.4. DB engines ranking according to their popularity (2019)



Biểu đồ trên minh họa xếp hạng các công cụ cơ sở dữ liệu dựa trên mức độ phổ biến của chúng tính đến tháng 8 năm 2019, theo dữ liệu từ DB-Engines.com. Trục tung biểu thị điểm số phổ biến theo thang logarit, cho phép chúng ta quan sát rõ ràng hơn sự thay đổi ở các công cụ có điểm số thấp và cao.

Quan sát tổng thể, chúng ta có thể thấy một số xu hướng rõ rệt trong giai đoạn từ năm 2013 đến năm 2019:

- **Nhóm dẫn đầu ổn định:** Các hệ quản trị cơ sở dữ liệu quan hệ truyền thống như Oracle, MySQL và Microsoft SQL Server tiếp tục duy trì vị trí dẫn đầu với điểm

số phổ biến rất cao, thường xuyên vượt ngưỡng 1000 điểm. Điều này cho thấy sự thống trị và tầm quan trọng không thể phủ nhận của chúng trong ngành công nghiệp. PostgreSQL cũng nằm trong nhóm này và thể hiện sự tăng trưởng ổn định, dần thu hẹp khoảng cách với các đối thủ hàng đầu.

- **Sự trỗi dậy của NoSQL và các công nghệ mới:** Một trong những điểm đáng chú ý nhất là sự tăng trưởng mạnh mẽ của các công cụ cơ sở dữ liệu NoSQL và các công nghệ mới hơn. MongoDB, một cơ sở dữ liệu tài liệu (document database), đã có sự tăng trưởng vượt bậc, từ mức điểm thấp vào năm 2013 lên đến hơn 200 điểm vào năm 2019, cho thấy sự chấp nhận rộng rãi của các giải pháp NoSQL trong các ứng dụng hiện đại. Tương tự, Elasticsearch (công cụ tìm kiếm và phân tích) và Redis (cơ sở dữ liệu trong bộ nhớ) cũng thể hiện đà tăng trưởng ấn tượng, phản ánh nhu cầu ngày càng cao về xử lý dữ liệu nhanh chóng và tìm kiếm hiệu quả.
- **Các công cụ khác:** IBM Db2 duy trì một vị trí vững chắc trong nhóm giữa, trong khi các công cụ như Cassandra, SQLite, MariaDB và Hive cũng cho thấy sự ổn định hoặc tăng trưởng nhẹ. Một số công cụ khác như Neo4j (cơ sở dữ liệu đồ thị) dù có điểm số thấp hơn nhưng cũng thể hiện xu hướng tăng trưởng liên tục, báo hiệu sự đa dạng hóa trong các loại hình cơ sở dữ liệu đang được quan tâm.

Nhìn chung, biểu đồ này cung cấp một cái nhìn tổng quan về bức tranh năng động của thị trường công cụ cơ sở dữ liệu. Mặc dù các hệ quản trị cơ sở dữ liệu quan hệ truyền thống vẫn giữ vững vị thế, sự phát triển và phổ biến của các giải pháp NoSQL và các công nghệ chuyên biệt khác đang định hình lại cách chúng ta lưu trữ và xử lý dữ liệu trong kỷ nguyên số. Điều này nhấn mạnh tầm quan trọng của việc hiểu biết về nhiều loại hình cơ sở dữ liệu khác nhau để đáp ứng các yêu cầu đa dạng của các ứng dụng hiện đại.

4.3. Các mô hình dữ liệu trong NoSQL

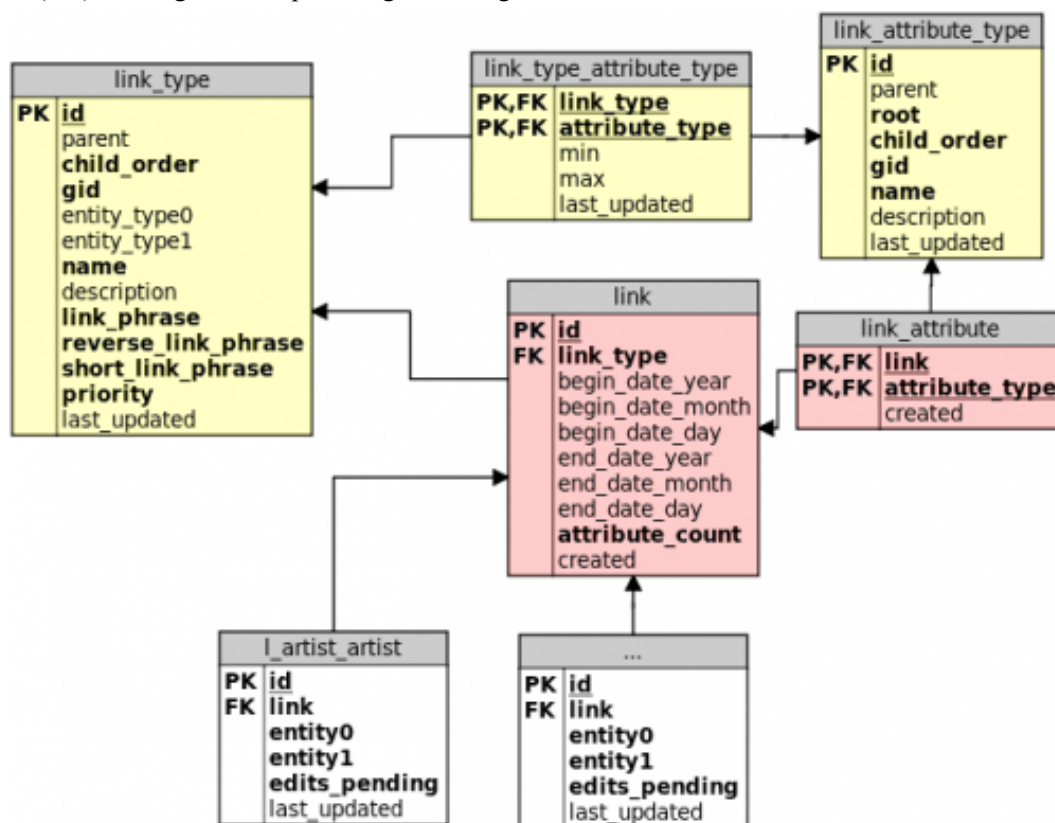
4.3.1. Mô hình dữ liệu quan hệ: Ôn tập

Mô hình dữ liệu quan hệ là một trong những mô hình tổ chức dữ liệu phổ biến và lâu đời nhất trong lĩnh vực công nghệ thông tin, đóng vai trò nền tảng cho hầu hết các hệ thống quản lý cơ sở dữ liệu truyền thống.

Trong mô hình này, dữ liệu thường được lưu trữ theo từng hàng (row store). Điều này có nghĩa là mỗi bản ghi (record) được tổ chức thành một hàng, với các thuộc tính (attributes) của bản ghi đó được sắp xếp trong các cột.

Ngôn ngữ truy vấn tiêu chuẩn (SQL) là công cụ chính để tương tác với cơ sở dữ liệu quan hệ, cho phép người dùng định nghĩa, thao tác và truy vấn dữ liệu một cách hiệu quả.

Một đặc điểm cốt lõi của mô hình dữ liệu quan hệ là yêu cầu định nghĩa cấu trúc dữ liệu (schema) một cách rõ ràng **trước khi** dữ liệu được thêm vào. Điều này đảm bảo tính nhất quán và toàn vẹn của dữ liệu. Hình sau minh họa một lược đồ cơ sở dữ liệu quan hệ điển hình, cho thấy các bảng (ví dụ: `link_type`, `link`) cùng với các khóa chính (PK) và khóa ngoại (FK) định nghĩa mối quan hệ giữa chúng.



Các thao tác kết nối (joins) là tính năng mạnh mẽ, cho phép hợp nhất dữ liệu từ nhiều bảng khác nhau dựa trên các mối quan hệ đã được định nghĩa. Kết quả của các truy vấn này cũng luôn là các bảng, duy trì tính đồng nhất của mô hình.

Ưu điểm: Mô hình dữ liệu quan hệ đã phát triển và trưởng thành, cung cấp các giao dịch ACID (Atomicity, Consistency, Isolation, Durability) mạnh mẽ, đảm bảo tính toàn vẹn và đáng tin cậy của dữ liệu. Bên cạnh đó, nó còn hỗ trợ kiểm soát bảo mật chi tiết (fine-grain security controls) ở cấp độ bảng, hàng hoặc cột. Đây là mô hình được sử dụng rộng rãi nhất trong ngành, với cộng đồng hỗ trợ lớn và nhiều công cụ phát triển.

Nhược điểm: Mô hình này yêu cầu mô hình hóa dữ liệu từ đầu (up-front data modeling), điều này có thể tốn thời gian và kém linh hoạt khi yêu cầu thay đổi cấu trúc dữ liệu. Ngoài ra, mô hình dữ liệu quan hệ có thể gặp khó khăn trong việc mở rộng quy mô (scale) một cách hiệu quả khi xử lý lượng dữ liệu rất lớn hoặc lưu lượng truy cập cao, đặc biệt là theo

chiều ngang (horizontal scaling).

Các hệ quản trị cơ sở dữ liệu quan hệ phổ biến bao gồm Oracle, MySQL, PostgreSQL, Microsoft SQL Server và IBM DB/2.

4.3.2. Mô hình dữ liệu Key/Value

key	value
firstName	Bugs
lastName	Bunny
location	Earth

PRODUCT	PRICE
WIDGET	\$118
GIZMO	\$88
TRINKET	\$37
THINGAMAJIG	\$18
DOODAD	\$60
TCHOTCHKE	\$999



PRODUCT	PRICE
TRINKET	\$37
THINGAMAJIG	\$18

PRODUCT	PRICE
GIZMO	\$88
DOODAD	\$60

PRODUCT	PRICE
WIDGET	\$118
TCHOTCHKE	\$999

Mô hình dữ liệu Key/Value là một trong những mô hình lưu trữ dữ liệu đơn giản và cơ bản nhất trong nhóm cơ sở dữ liệu NoSQL. Nó hoạt động dựa trên nguyên tắc lưu trữ dữ liệu dưới dạng các cặp khóa-giá trị (key-value pairs), trong đó mỗi khóa (key) là duy nhất và được sử dụng để truy cập đến giá trị (value) tương ứng.

Giao diện tương tác với mô hình này rất trực quan và đơn giản, thường chỉ bao gồm ba thao tác cơ bản:

- **GET**: Truy xuất giá trị dựa trên một khóa cụ thể.
- **PUT**: Lưu trữ một cặp khóa-giá trị mới hoặc cập nhật giá trị cho một khóa đã tồn tại.

- **DELETE**: Xóa một cặp khóa-giá trị khỏi hệ thống.

Một trong những ưu điểm nổi bật của mô hình Key/Value là tính linh hoạt cao của giá trị. Giá trị có thể chứa bất kỳ loại dữ liệu nào, từ các kiểu dữ liệu đơn giản như chuỗi văn bản, số, cho đến các cấu trúc phức tạp hơn như đối tượng JSON, XML, hoặc thậm chí là dữ liệu nhị phân (ví dụ: hình ảnh, video). Điều này cho phép người dùng lưu trữ đa dạng các loại thông tin mà không cần tuân thủ một lược đồ cố định trước. Chẳng hạn, một khóa “firstName” có thể liên kết với giá trị “Bugs”, “lastName” với “Bunny”, và “location” với “Earth”, tạo thành một bản ghi đơn giản nhưng đầy đủ thông tin.

Nhờ sự đơn giản trong cấu trúc và việc loại bỏ các phép nối (joins) phức tạp thường thấy trong cơ sở dữ liệu quan hệ, các hệ thống Key/Value thường đạt được hiệu suất truy xuất dữ liệu rất cao và khả năng mở rộng (scalability) vượt trội. Dữ liệu có thể dễ dàng được phân tán trên nhiều máy chủ, cho phép hệ thống xử lý lượng lớn dữ liệu và hàng triệu truy vấn đồng thời một cách hiệu quả. Khả năng phân chia dữ liệu thành các phần nhỏ hơn và phân tán chúng trên các nút khác nhau là một minh chứng rõ ràng cho khả năng mở rộng này.

Một số ví dụ điển hình về các hệ thống cơ sở dữ liệu sử dụng mô hình Key/Value bao gồm: Berkley DB, Memcache, DynamoDB, Redis và Riak. Các hệ thống này thường được ứng dụng rộng rãi trong các trường hợp cần lưu trữ cache, quản lý phiên người dùng, hoặc các tác vụ yêu cầu truy xuất dữ liệu cực nhanh và khả năng chịu tải cao.

Key/value vs. table

Trong bối cảnh các hệ quản trị cơ sở dữ liệu phi quan hệ (NoSQL), mô hình lưu trữ dữ liệu theo cặp khóa-giá trị (Key-Value Store) là một trong những mô hình cơ bản và phổ biến nhất. Để dễ hình dung, chúng ta có thể xem xét sự khác biệt và tương đồng giữa mô hình này với một bảng dữ liệu truyền thống.

Về bản chất, một hệ thống Key-Value có thể được hình dung như một bảng đơn giản chỉ với hai cột: một cột dành cho “Khóa” (Key) và một cột dành cho “Giá trị” (Value). Cụ thể hơn, cột “Khóa” thường được định nghĩa là một chuỗi ký tự (string datatype), đóng vai trò là định danh duy nhất cho mỗi mục dữ liệu. Trong khi đó, cột “Giá trị” có thể chứa bất kỳ loại dữ liệu nào, thường được gọi là BLOB (Binary Large Object) datatype. Điều này có nghĩa là giá trị có thể là một chuỗi văn bản, một số, một đối tượng JSON, một hình ảnh, một video, hoặc bất kỳ dạng dữ liệu nhị phân nào khác mà không cần tuân theo một cấu trúc cố định nào.

Giao diện tương tác với một Key-Value Store cực kỳ đơn giản, chỉ bao gồm ba thao tác cơ bản:

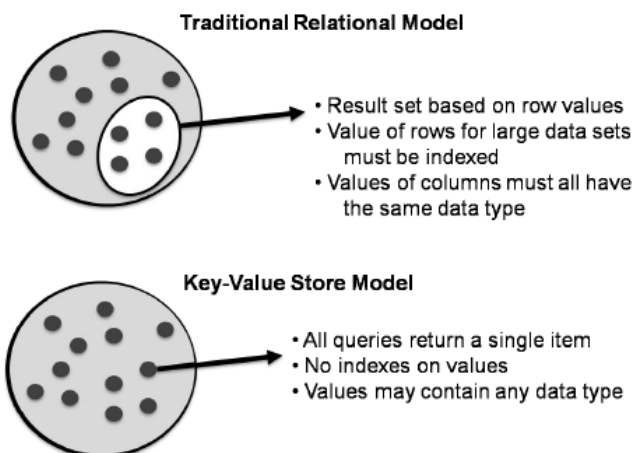
- **Thêm một cặp khóa-giá trị:** Người dùng cung cấp một khóa và một giá trị tương ứng để lưu trữ vào hệ thống.
- **Truy xuất giá trị theo khóa:** Khi người dùng cung cấp một khóa, hệ thống sẽ trả về giá trị được liên kết với khóa đó. Đây là thao tác truy vấn chính và hiệu quả nhất trong mô hình này.
- **Xóa một khóa:** Người dùng có thể xóa một cặp khóa-giá trị khỏi hệ thống bằng cách cung cấp khóa tương ứng.



Để minh họa rõ hơn, chúng ta có thể hình dung một Key-Value Store giống như một dãy các tủ khóa cá nhân. Mỗi chiếc chìa khóa (Key) là duy nhất và được dùng để mở một tủ khóa cụ thể. Bên trong mỗi tủ khóa (Value) là một không gian chứa đựng bất kỳ vật phẩm nào muốn cất giữ – có thể là sách, quần áo, hay các vật dụng cá nhân khác. Điều quan trọng là hệ thống không quan tâm đến nội dung bên trong tủ khóa, mà chỉ tập trung vào việc liên kết chìa khóa với tủ khóa và cho phép người dùng truy cập nội dung đó thông qua chìa khóa. Sự linh hoạt trong việc lưu trữ “bất kỳ dữ liệu nào” trong phần “Giá trị” chính là một trong những ưu điểm nổi bật của mô hình Key-Value, giúp nó phù hợp với nhiều ứng dụng yêu cầu khả năng mở rộng và hiệu suất cao trong việc truy xuất dữ liệu dựa trên định danh duy nhất.

Mô hình dữ liệu Khóa-Giá trị so với Mô hình dữ liệu Quan hệ

Trong lĩnh vực cơ sở dữ liệu, việc hiểu rõ sự khác biệt giữa các mô hình dữ liệu là rất quan trọng để lựa chọn giải pháp phù hợp cho từng bài toán cụ thể. Hai trong số các mô hình



phổ biến nhất là mô hình dữ liệu quan hệ truyền thống và mô hình lưu trữ cặp khóa-giá trị (Key-Value Store). Dưới đây là sự so sánh chi tiết về các đặc điểm cơ bản của chúng.

Mô hình Quan hệ Truyền thống (Traditional Relational Model)

Mô hình quan hệ là nền tảng của hầu hết các hệ quản trị cơ sở dữ liệu truyền thống (RDBMS) mà chúng ta vẫn thường sử dụng. Mô hình này tổ chức dữ liệu thành các bảng (table), với mỗi bảng bao gồm các hàng (row) và cột (column). Các đặc điểm nổi bật của mô hình quan hệ bao gồm:

- **Tập kết quả dựa trên giá trị hàng:** Khi thực hiện một truy vấn trong mô hình quan hệ, kết quả trả về thường là một tập hợp các hàng (bản ghi) thỏa mãn các điều kiện đã cho. Ví dụ, một truy vấn tìm kiếm thông tin sinh viên theo tên sẽ trả về một hoặc nhiều hàng dữ liệu tương ứng với các sinh viên có tên đó. Điều này cho phép chúng ta truy xuất các tập con dữ liệu dựa trên các tiêu chí phức tạp.
- **Giá trị hàng cho các tập dữ liệu lớn phải được lập chỉ mục:** Để đảm bảo hiệu suất truy vấn tối ưu, đặc biệt với các tập dữ liệu lớn, việc lập chỉ mục (indexing) cho các cột quan trọng là điều kiện tiên quyết. Chỉ mục giúp hệ thống nhanh chóng định vị các hàng dữ liệu mà không cần quét toàn bộ bảng, từ đó giảm đáng kể thời gian phản hồi cho các truy vấn tìm kiếm, sắp xếp hoặc kết nối bảng.
- **Giá trị của tất cả các cột phải có cùng kiểu dữ liệu:** Trong mô hình quan hệ, mỗi cột trong một bảng được định nghĩa với một kiểu dữ liệu cụ thể (ví dụ: số nguyên, chuỗi ký tự, ngày tháng, boolean). Tất cả các giá trị được lưu trữ trong cột đó phải tuân thủ kiểu dữ liệu đã định nghĩa. Điều này đảm bảo tính nhất quán, toàn vẹn của dữ liệu và cho phép thực hiện các phép toán chính xác trên dữ liệu.

Mô hình Lưu trữ Khóa-Giá trị (Key-Value Store Model)

Mô hình khóa-giá trị là một trong những mô hình cơ bản và đơn giản nhất trong nhóm cơ sở dữ liệu NoSQL. Trong mô hình này, dữ liệu được lưu trữ dưới dạng các cặp khóa duy nhất và giá trị tương ứng. Các đặc điểm chính của mô hình khóa-giá trị là:

- **Tất cả các truy vấn trả về một mục duy nhất:** Điểm khác biệt cơ bản so với mô hình quan hệ là cách thức truy vấn. Trong mô hình khóa-giá trị, chúng ta luôn truy vấn bằng cách cung cấp một khóa, và hệ thống sẽ trả về giá trị duy nhất được liên kết với khóa đó. Đây là một thao tác tra cứu trực tiếp, rất nhanh chóng và hiệu quả cho việc truy xuất từng mục dữ liệu cụ thể mà không cần tìm kiếm dựa trên nội dung của giá trị.
- **Không có chỉ mục trên giá trị:** Do việc truy cập dữ liệu chỉ thông qua khóa, mô hình khóa-giá trị thường không yêu cầu hoặc hỗ trợ việc lập chỉ mục trên các giá trị. Điều này giúp đơn giản hóa cấu trúc dữ liệu, giảm chi phí quản lý và tối ưu hóa hiệu suất cho các thao tác đọc/ghi dựa trên khóa.
- **Giá trị có thể chứa bất kỳ kiểu dữ liệu nào:** Một ưu điểm vượt trội của mô hình khóa-giá trị là tính linh hoạt trong việc lưu trữ giá trị. Giá trị liên kết với một khóa có thể là bất kỳ thứ gì: một chuỗi văn bản đơn giản, một đối tượng JSON phức tạp, một hình ảnh, một tệp nhị phân, hoặc thậm chí là một danh sách các mục. Cơ sở dữ liệu không áp đặt một lược đồ cố định cho nội dung của giá trị, mang lại sự linh hoạt cao cho nhà phát triển khi xử lý dữ liệu phi cấu trúc hoặc bán cấu trúc.

Tóm lại, mô hình quan hệ phù hợp với các ứng dụng yêu cầu cấu trúc dữ liệu chặt chẽ, tính toàn vẹn cao và các truy vấn phức tạp dựa trên nhiều tiêu chí. Ngược lại, mô hình khóa-giá trị lý tưởng cho các trường hợp cần tốc độ truy xuất dữ liệu cực nhanh dựa trên khóa, khả năng mở rộng lớn và sự linh hoạt trong việc lưu trữ các loại dữ liệu đa dạng mà không cần lược đồ cố định. Việc lựa chọn mô hình phù hợp phụ thuộc vào yêu cầu cụ thể về cấu trúc dữ liệu, hiệu suất truy vấn và khả năng mở rộng của ứng dụng.

Memcached

Memcached là một hệ thống bộ nhớ đệm (caching system) mã nguồn mở, hoạt động dựa trên cặp khóa-giá trị (key-value) và lưu trữ dữ liệu trực tiếp trong bộ nhớ chính (in-memory). Hệ thống này được thiết kế để tận dụng hiệu quả bộ nhớ RAM trên nhiều máy chủ web phân tán.

Mục tiêu chính của Memcached là tăng tốc các ứng dụng web động bằng cách giảm tải cho cơ sở dữ liệu. Điều này được thực hiện thông qua việc cung cấp một giao diện đơn giản

cho các bộ nhớ đệm RAM phân tán cao. Nhờ vậy, thời gian đọc dữ liệu thường rất nhanh, điển hình chỉ khoảng 30 mili giây.

Memcached được thiết kế để triển khai nhanh chóng và dễ dàng phát triển. Nó cung cấp các giao diện lập trình ứng dụng (APIs) bằng nhiều ngôn ngữ khác nhau, giúp các nhà phát triển dễ dàng tích hợp vào các hệ thống hiện có.

Redis

Redis, viết tắt của **Remote Dictionary Server**, là một hệ thống lưu trữ dữ liệu mã nguồn mở (open source) hoạt động dựa trên mô hình khóa-giá trị (key-value store) và chủ yếu lưu trữ dữ liệu trong bộ nhớ chính (in-memory). Một điểm đáng chú ý là Redis cung cấp tùy chọn về khả năng bền vững (optional durability), nghĩa là dữ liệu có thể được ghi ra đĩa để đảm bảo không bị mất khi hệ thống khởi động lại hoặc gặp sự cố, mặc dù hoạt động chính vẫn diễn ra trong RAM.

Mục tiêu cốt lõi của Redis là tối ưu hóa tốc độ đọc và ghi dữ liệu. Nhờ việc lưu trữ dữ liệu trực tiếp trong bộ nhớ RAM, Redis có thể đạt được hiệu suất vượt trội, đặc biệt đối với các thao tác trên các cấu trúc dữ liệu phổ biến.

Điểm mạnh nổi bật của Redis là khả năng hỗ trợ và thao tác trực tiếp trên nhiều cấu trúc dữ liệu phức tạp hơn là chỉ các giá trị đơn thuần. Cụ thể, Redis cho phép chúng ta lưu trữ và xử lý các danh sách (lists), tập hợp (sets) và bảng băm (hashes) ngay trong phần giá trị của cặp khóa-giá trị. Điều này giúp các nhà phát triển có thể thực hiện các thao tác phức tạp như thêm/bớt phần tử vào danh sách, kiểm tra sự tồn tại của phần tử trong tập hợp, hoặc cập nhật trường trong bảng băm một cách hiệu quả.

Ngoài ra, Redis còn tích hợp nhiều tính năng nâng cao mà các nhà phát triển thường xuyên sử dụng và đánh giá cao. Có thể kể đến:

- **Expiration (Thời gian hết hạn):** Cho phép tự động xóa dữ liệu sau một khoảng thời gian nhất định, rất hữu ích cho việc quản lý cache hoặc các phiên làm việc tạm thời.
- **Transactions (Giao dịch):** Đảm bảo một chuỗi các lệnh được thực thi một cách nguyên tử (atomic), nghĩa là tất cả các lệnh đều thành công hoặc tất cả đều thất bại, không có trạng thái trung gian.
- **Pub/Sub (Publish/Subscribe - Xuất bản/Đăng ký):** Một mô hình nhắn tin cho phép các ứng dụng gửi và nhận thông điệp theo thời gian thực, phục vụ cho các hệ thống thông báo hoặc chat.
- **Partitioning (Phân vùng):** Khả năng phân chia dữ liệu trên nhiều máy chủ khác nhau để mở rộng quy mô (scaling out) và xử lý lượng dữ liệu lớn hơn.

Những đặc điểm này làm cho Redis trở thành một lựa chọn mạnh mẽ và linh hoạt cho

nhiều ứng dụng khác nhau, từ caching, quản lý phiên, hàng đợi tin nhắn cho đến các cơ sở dữ liệu thời gian thực.

Amazon DynamoDB

Amazon DynamoDB là một dịch vụ cơ sở dữ liệu NoSQL được quản lý hoàn toàn, cung cấp hiệu suất cao và khả năng mở rộng linh hoạt, được thiết kế để hỗ trợ các ứng dụng có yêu cầu về độ trễ thấp và thông lượng cao.

- **Kho lưu trữ khóa-giá trị có khả năng mở rộng (Scalable key-value store):** DynamoDB hoạt động dựa trên mô hình dữ liệu khóa-giá trị, một dạng cơ sở dữ liệu NoSQL phổ biến, nơi mỗi mục dữ liệu được xác định duy nhất bởi một khóa. Điều này cho phép truy xuất dữ liệu nhanh chóng và hiệu quả. Điểm mạnh vượt trội của DynamoDB là khả năng mở rộng theo chiều ngang (horizontal scaling) một cách tự động và liền mạch, cho phép nó dễ dàng xử lý hàng triệu yêu cầu mỗi giây và lưu trữ petabyte dữ liệu mà không yêu cầu người dùng phải quản lý cơ sở hạ tầng.
- **Sản phẩm phát triển nhanh nhất trong lịch sử Amazon:** Kể từ khi ra mắt, DynamoDB đã chứng kiến tốc độ tăng trưởng đáng kinh ngạc, trở thành một trong những dịch vụ được sử dụng rộng rãi nhất trong hệ sinh thái Amazon Web Services (AWS). Sự phát triển nhanh chóng này phản ánh nhu cầu ngày càng tăng về các giải pháp cơ sở dữ liệu hiệu suất cao, có khả năng mở rộng và được quản lý hoàn toàn, đặc biệt trong bối cảnh các ứng dụng web, di động và IoT đang bùng nổ.
- **Tập trung vào thông lượng lưu trữ và thời gian đọc/ghi có thể dự đoán được:** Một trong những ưu điểm cốt lõi của DynamoDB là khả năng cung cấp thông lượng (throughput) cao cho cả hoạt động đọc và ghi dữ liệu. Hơn nữa, nó đảm bảo thời gian phản hồi (latency) thấp và có thể dự đoán được, ngay cả khi khối lượng công việc tăng lên đáng kể. Điều này cực kỳ quan trọng đối với các ứng dụng yêu cầu hiệu suất ổn định và độ trễ thấp, chẳng hạn như các hệ thống trò chơi trực tuyến, quảng cáo thời gian thực, hoặc các ứng dụng phân tích dữ liệu lớn.
- **Tích hợp chặt chẽ với S3 và Elastic MapReduce:** DynamoDB được thiết kế để tích hợp sâu rộng với các dịch vụ khác trong AWS, tạo nên một hệ sinh thái mạnh mẽ cho việc quản lý và phân tích dữ liệu. Cụ thể, nó có khả năng tích hợp mạnh mẽ với Amazon S3 (Simple Storage Service) để lưu trữ dữ liệu lớn, sao lưu, hoặc xuất dữ liệu cho các mục đích phân tích. Đồng thời, sự kết nối với Amazon Elastic MapReduce (EMR) cho phép người dùng dễ dàng thực hiện các tác vụ phân tích dữ liệu lớn phức tạp, xử lý dữ liệu từ DynamoDB bằng các framework như Apache Spark, Hive, hoặc Presto.

Riak

Riak là một hệ thống lưu trữ khóa-giá trị (key-value store) phân tán, mã nguồn mở, được phát triển và cung cấp các phiên bản thương mại cùng dịch vụ hỗ trợ bởi công ty Basho. Với mô hình lưu trữ khóa-giá trị, Riak cho phép người dùng lưu trữ và truy xuất dữ liệu một cách đơn giản thông qua các cặp khóa và giá trị, tương tự như cách hoạt động của một từ điển hoặc bảng băm.

Hệ thống này được thiết kế lấy cảm hứng từ Amazon Dynamo, một cơ sở dữ liệu phân tán nổi tiếng về tính sẵn sàng cao và khả năng chịu lỗi. Do đó, Riak cũng tập trung mạnh vào các đặc tính cốt lõi sau:

- **Tính sẵn sàng (Availability):** Đảm bảo hệ thống luôn hoạt động và có thể truy cập được ngay cả khi có sự cố xảy ra với một số thành phần.
- **Khả năng chịu lỗi (Fault-tolerance):** Hệ thống được thiết kế để tiếp tục hoạt động bình thường ngay cả khi một hoặc nhiều máy chủ gặp sự cố, giúp duy trì tính liên tục của dịch vụ.
- **Đơn giản trong vận hành (Operational simplicity):** Giúp việc triển khai, quản lý và bảo trì hệ thống trở nên dễ dàng hơn cho các nhà vận hành, giảm thiểu gánh nặng quản trị.
- **Khả năng mở rộng (Scalability):** Cho phép hệ thống dễ dàng mở rộng quy mô theo chiều ngang (thêm máy chủ) để đáp ứng nhu cầu lưu trữ và xử lý dữ liệu ngày càng tăng mà không làm giảm hiệu suất.

Để đạt được các mục tiêu trên, Riak hỗ trợ các tính năng quan trọng như:

- **Sao chép dữ liệu (Replication):** Dữ liệu được sao chép trên nhiều nút (node) khác nhau trong cụm, đảm bảo tính bền vững của dữ liệu và khả năng truy cập ngay cả khi một số nút không khả dụng.
- **Phân mảnh tự động (Auto-sharding):** Dữ liệu được tự động phân chia và phân phối trên các nút trong cụm, giúp cân bằng tải và tối ưu hiệu suất truy xuất.
- **Tái cân bằng khi có lỗi (Rebalancing on failures):** Khi một nút gặp sự cố hoặc được thêm/bớt khỏi cụm, Riak có khả năng tự động tái phân phối dữ liệu để duy trì sự cân bằng và hiệu suất tối ưu.

Ngoài ra, Riak còn cung cấp các tính năng nâng cao để hỗ trợ việc xử lý và truy vấn dữ liệu phức tạp hơn, bao gồm:

- **Hỗ trợ MapReduce:** Cho phép thực hiện các tác vụ phân tích dữ liệu lớn một cách phân tán, giúp xử lý các truy vấn phức tạp trên tập dữ liệu khổng lồ.
- **Tìm kiếm toàn văn (Fulltext search):** Cung cấp khả năng tìm kiếm nội dung bên trong các giá trị được lưu trữ, tương tự như các công cụ tìm kiếm thông thường.
- **Chỉ mục thứ cấp (Secondary indexes):** Cho phép người dùng tạo các chỉ mục trên các “thẻ giá trị” (value tags), giúp truy vấn dữ liệu linh hoạt hơn ngoài việc chỉ sử dụng khóa chính.

Riak được viết bằng ngôn ngữ lập trình ERLANG, một ngôn ngữ nổi tiếng với khả năng xây dựng các hệ thống phân tán, đồng thời và chịu lỗi cao, điều này góp phần vào sự mạnh mẽ và ổn định của Riak.

4.3.3. Column family store

Mô hình lưu trữ dữ liệu theo họ cột (Column family store) là một trong những loại cơ sở dữ liệu NoSQL phổ biến, được thiết kế để xử lý lượng lớn dữ liệu phân tán một cách hiệu quả. Mô hình này cung cấp một cách tiếp cận linh hoạt và mạnh mẽ để quản lý dữ liệu phi cấu trúc hoặc bán cấu trúc, đặc biệt phù hợp cho các ứng dụng cần khả năng mở rộng cao, hiệu suất truy vấn trên các tập hợp con của dữ liệu và khả năng lưu trữ phiên bản.

Đặc điểm chính của Column family store:

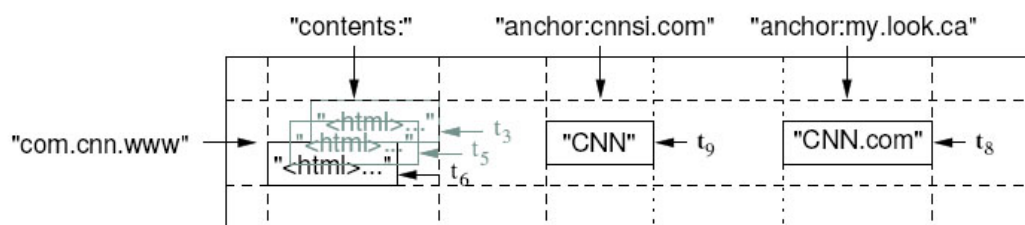
- **Lược đồ động (Dynamic schema) và Mô hình dữ liệu hướng cột (Column-oriented data model):**
 - **Lược đồ động** có nghĩa là không cần phải định nghĩa trước tất cả các cột cho mỗi hàng (row). Mỗi hàng có thể có các cột khác nhau, mang lại sự linh hoạt cao khi dữ liệu có cấu trúc không đồng nhất hoặc thay đổi thường xuyên. Điều này khác biệt đáng kể so với các cơ sở dữ liệu quan hệ truyền thống yêu cầu lược đồ cố định.
 - **Mô hình dữ liệu hướng cột** ám chỉ cách dữ liệu được lưu trữ vật lý. Thay vì lưu trữ dữ liệu theo hàng (như trong cơ sở dữ liệu quan hệ), dữ liệu được nhóm và lưu trữ theo cột hoặc các họ cột (column families). Cách tổ chức này đặc biệt hiệu quả cho các truy vấn phân tích chỉ cần truy cập một tập hợp con các cột, vì hệ thống có thể đọc trực tiếp các cột cần thiết mà không phải tải toàn bộ hàng.
- **Bản đồ đa chiều phân tán thưa thớt và bền vững (Sparse, distributed persistent multi-dimensional sorted map):**

- **Thưa thớt (Sparse):** Không phải tất cả các hàng đều cần có giá trị cho tất cả các cột. Nếu một cột không có dữ liệu cho một hàng cụ thể, nó sẽ không chiếm không gian lưu trữ, giúp tiết kiệm tài nguyên và tăng hiệu quả cho các tập dữ liệu lớn với nhiều thuộc tính tùy chọn.
- **Phân tán (Distributed):** Dữ liệu được trải rộng trên nhiều máy chủ (node) trong một cụm. Điều này đảm bảo khả năng mở rộng (scalability) và khả năng chịu lỗi (fault tolerance) cao, cho phép hệ thống xử lý lượng dữ liệu và lưu lượng truy cập lớn.
- **Bền vững (Persistent):** Dữ liệu được lưu trữ một cách lâu dài và an toàn, không bị mất khi hệ thống tắt hoặc gặp sự cố.
- **Bản đồ đa chiều được sắp xếp (Multi-dimensional sorted map):** Đây là cấu trúc dữ liệu cốt lõi. Dữ liệu được tổ chức như một bản đồ (map) với các khóa (key) được sắp xếp. “Đa chiều” ở đây ám chỉ việc dữ liệu được truy cập thông qua nhiều “chiều” khác nhau: khóa hàng (row key), họ cột (column family), tên cột (column name) và dấu thời gian (timestamp). Việc sắp xếp giúp tối ưu hóa các thao tác quét phạm vi (range scan) trên các khóa hàng hoặc khóa cột.
- **Cấu trúc truy cập dữ liệu: (row, column (family), timestamp) -> cell contents** Mỗi ô dữ liệu (cell contents) trong một Column family store được xác định duy nhất bởi ba thành phần chính:
 - **row (hàng):** Là khóa chính của một hàng, dùng để xác định một bản ghi duy nhất.
 - **column (family) (cột (họ cột)):** Dữ liệu được tổ chức thành các họ cột, và trong mỗi họ cột lại có các cột cụ thể. Ví dụ, một họ cột có thể là “contents” và các cột trong đó có thể là “html”, “text”, v.v. Hoặc tên cột có thể bao gồm cả họ cột như “anchor:cnnsi.com”.
 - **timestamp (dấu thời gian):** Mỗi giá trị trong một ô dữ liệu đều được gắn với một dấu thời gian. Điều này cho phép lưu trữ nhiều phiên bản của cùng một ô dữ liệu, giúp theo dõi lịch sử thay đổi và truy xuất các phiên bản cũ.

Ví dụ minh họa:

Hình trên minh họa cấu trúc dữ liệu của một Column family store. Chúng ta có thể thấy rõ các thành phần đã được mô tả:

- **Khóa hàng (Row Key):** Trong ví dụ này, khóa hàng là “com.cnn.www”, đại diện cho một trang web cụ thể.



• Họ cột/Cột (Column Family/Column):

- "contents:": Đây là một họ cột chứa nội dung của trang web. Trong hình, chúng ta thấy nhiều phiên bản của nội dung HTML ("`<html>...`") được lưu trữ, mỗi phiên bản có một dấu thời gian riêng biệt (t3, t5, t6). Điều này minh họa khả năng lưu trữ phiên bản của dữ liệu.
- "anchor:cnnsi.com" và "anchor:my.look.ca": Đây là các cột (có thể thuộc một họ cột "anchor" hoặc được coi là các cột độc lập) lưu trữ thông tin về các liên kết neo (anchor links) trỏ đến trang "com.cnn.www". Mỗi cột này cũng có giá trị và dấu thời gian riêng ("CNN" với t9, và "CNN.com" với t8).

- **Tính thưa thớt (Sparse nature):** có thể thấy rằng không phải mọi ô trong bảng đều có dữ liệu. Ví dụ, một hàng khác có thể không có cột "anchor:my.look.ca", và điều đó hoàn toàn bình thường trong mô hình này.

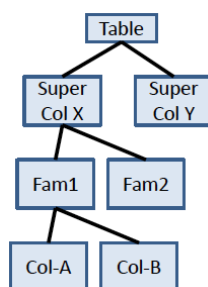
Column family store là một lựa chọn mạnh mẽ cho các ứng dụng cần khả năng mở rộng linh hoạt và xử lý dữ liệu lớn, đặc biệt khi cấu trúc dữ liệu có thể thay đổi hoặc không đồng nhất.

Column families

Trong các hệ thống cơ sở dữ liệu NoSQL, đặc biệt là các cơ sở dữ liệu định hướng cột (column-oriented databases), việc tổ chức dữ liệu theo cấu trúc "họ cột" (Column families) là một khái niệm cốt lõi nhằm tối ưu hóa hiệu suất và khả năng truy vấn.

Về bản chất, các cột (columns) có mối liên hệ logic hoặc thường được truy cập cùng nhau sẽ được **nhóm lại thành các "họ cột" (Column families)**. Việc nhóm này giúp cấu trúc dữ liệu một cách có tổ chức và dễ quản lý hơn.

Để tạo ra một cấp độ tổ chức cao hơn nữa, **các họ cột này lại có thể được nhóm lại thành các "siêu cột" (Super-Columns)**. Điều này hình thành một cấu trúc phân cấp rõ ràng, từ bảng (Table) xuống siêu cột, rồi đến họ cột và cuối cùng là các cột riêng lẻ, như minh họa trong sơ đồ cấu trúc dữ liệu.



Mục tiêu chính của việc tổ chức dữ liệu theo cấu trúc họ cột và siêu cột là để **nâng cao khả năng truy vấn dữ liệu**. Khi dữ liệu được nhóm lại một cách hợp lý, hệ thống có thể truy vấn tất cả các cột trong một họ cột hoặc thậm chí là trong một siêu cột một cách hiệu quả.

Việc **nhóm các dữ liệu tương tự nhau lại với nhau** không chỉ giúp quản lý dữ liệu dễ dàng hơn mà còn **cải thiện đáng kể tốc độ truy xuất và hiệu suất tổng thể** của hệ thống. Khi các dữ liệu liên quan nằm gần nhau về mặt vật lý hoặc logic, quá trình đọc và ghi sẽ được tối ưu hóa, giảm thiểu thời gian tìm kiếm và truy cập, từ đó mang lại trải nghiệm người dùng tốt hơn và hiệu quả hoạt động cao hơn cho ứng dụng.

Mô hình dữ liệu Column Family so với mô hình quan hệ

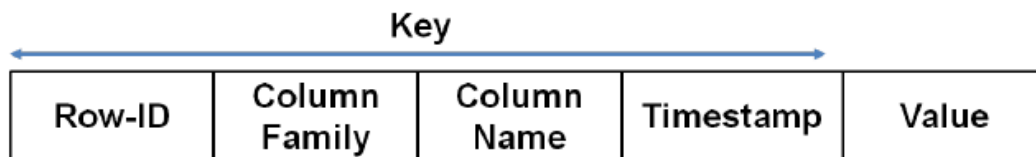
Khi so sánh mô hình dữ liệu Column Family với mô hình quan hệ truyền thống, chúng ta có thể thấy một số điểm khác biệt và ưu điểm nổi bật của Column Family, đặc biệt trong việc xử lý các tập dữ liệu lớn và linh hoạt.

Đầu tiên, mô hình Column Family được thiết kế để xử lý hiệu quả **ma trận thưa (sparse matrix)** và vẫn **bảo toàn cấu trúc bảng** một cách logic. Điều này có nghĩa là, mặc dù về mặt khái niệm, một hàng dữ liệu có thể chứa hàng triệu cột, nhưng trên thực tế, nó có thể rất thưa thớt. Không giống như cơ sở dữ liệu quan hệ nơi các ô trống (NULL) vẫn chiếm không gian lưu trữ, mô hình Column Family chỉ lưu trữ các cặp cột-giá trị thực sự tồn tại. Điều này giúp tối ưu hóa không gian lưu trữ và hiệu suất truy vấn cho các tập dữ liệu có nhiều thuộc tính không đồng nhất giữa các hàng.

Thứ hai, một số hệ thống Column Family có thể được xem là **lưu trữ kết hợp hàng/cột (hybrid row/column stores)**. Tùy thuộc vào cách triển khai và tối ưu hóa, chúng có thể lưu trữ dữ liệu theo hàng để truy xuất nhanh toàn bộ một bản ghi, hoặc lưu trữ theo cột để tối ưu hóa các truy vấn phân tích trên một tập hợp các thuộc tính cụ thể.

Một ưu điểm quan trọng khác là **số lượng cột có thể mở rộng** một cách linh hoạt. Điều này cho phép các nhà phát triển **thêm cột mới mà không cần thực hiện lệnh “alter table”** như trong cơ sở dữ liệu quan hệ. Trong môi trường phát triển nhanh và dữ liệu thay đổi liên tục, khả năng này giúp giảm đáng kể thời gian ngừng hoạt động và độ phức tạp trong việc

quản lý lược đồ (schema).



Để hiểu rõ hơn về cách dữ liệu được tổ chức trong mô hình Column Family, chúng ta hãy xem xét cấu trúc cơ bản của một mục dữ liệu như minh họa trong hình trên. Mỗi mục dữ liệu được xác định bởi một **khóa (Key)** bao gồm ba thành phần chính: Row-ID, Column Family và Column Name.

- Row-ID là định danh duy nhất cho một hàng dữ liệu.
- Column Family là một nhóm các cột có liên quan, tương tự như một bảng trong cơ sở dữ liệu quan hệ nhưng linh hoạt hơn về cấu trúc bên trong.
- Column Name là tên của thuộc tính cụ thể trong Column Family đó.

Ngoài ra, mỗi giá trị còn đi kèm với một Timestamp (dấu thời gian), cho phép quản lý các phiên bản dữ liệu hoặc theo dõi thời điểm dữ liệu được ghi. Cuối cùng, Value là giá trị thực tế của dữ liệu. Chính nhờ cấu trúc này mà mô hình Column Family có thể xử lý hiệu quả các ma trận dữ liệu thưa, vì chỉ những cặp (Row-ID, Column Family, Column Name) có giá trị mới được lưu trữ, giúp tiết kiệm không gian và tăng hiệu suất.

Bigtable

Bigtable là một hệ thống lưu trữ phân tán được thiết kế để quản lý lượng lớn dữ liệu có cấu trúc. Công trình nghiên cứu về Bigtable đã được công bố vào năm 2008 trên tạp chí uy tín *ACM Transactions on Computer Systems (TOCS)* với tiêu đề “Bigtable: A Distributed Storage System for Structured Data”. Đây là một bài báo khoa học mang tính bước ngoặt, thường được trích dẫn rộng rãi trong lĩnh vực hệ thống phân tán và cơ sở dữ liệu.

Các đặc điểm nổi bật của Bigtable bao gồm:

- **Khả năng chịu lỗi (Fault-tolerant) và bền vững (Persistent):** Bigtable được thiết kế để đảm bảo dữ liệu không bị mất mát ngay cả khi có lỗi phần cứng hoặc phần mềm xảy ra trong hệ thống. Dữ liệu được lưu trữ một cách bền vững, có nghĩa là chúng sẽ tồn tại và có thể truy cập được ngay cả khi hệ thống khởi động lại hoặc gặp sự cố. Điều này rất quan trọng đối với các ứng dụng yêu cầu tính sẵn sàng cao và độ tin cậy dữ liệu tuyệt đối.

- **Khả năng mở rộng (Scalable):** Đây là một trong những ưu điểm lớn nhất của Bigtable, cho phép hệ thống mở rộng quy mô một cách ấn tượng để đáp ứng nhu cầu lưu trữ và xử lý dữ liệu khổng lồ:
 - Nó có thể hoạt động trên hàng nghìn máy chủ, tạo thành một cụm lớn, phân tán.
 - Hỗ trợ lưu trữ hàng terabyte dữ liệu trong bộ nhớ (in-memory data) để truy cập nhanh chóng, tối ưu hóa hiệu suất đọc.
 - Có khả năng quản lý hàng petabyte dữ liệu trên đĩa, cho thấy khả năng lưu trữ dung lượng cực lớn.
 - Đạt được hiệu suất cao với hàng triệu thao tác đọc/ghi mỗi giây, đồng thời hỗ trợ các thao tác quét (scans) dữ liệu hiệu quả, giúp truy vấn và phân tích dữ liệu lớn một cách nhanh chóng.
- **Khả năng tự quản lý (Self-managing):** Bigtable giảm thiểu gánh nặng quản lý cho người vận hành nhờ các tính năng tự động:
 - Các máy chủ có thể được thêm hoặc bớt khỏi hệ thống một cách linh hoạt mà không làm gián đoạn hoạt động, cho phép mở rộng hoặc thu hẹp quy mô dễ dàng tùy theo nhu cầu.
 - Hệ thống có khả năng tự điều chỉnh để cân bằng tải giữa các máy chủ, đảm bảo rằng tài nguyên được sử dụng hiệu quả và không có máy chủ nào bị quá tải, từ đó duy trì hiệu suất ổn định và tối ưu.

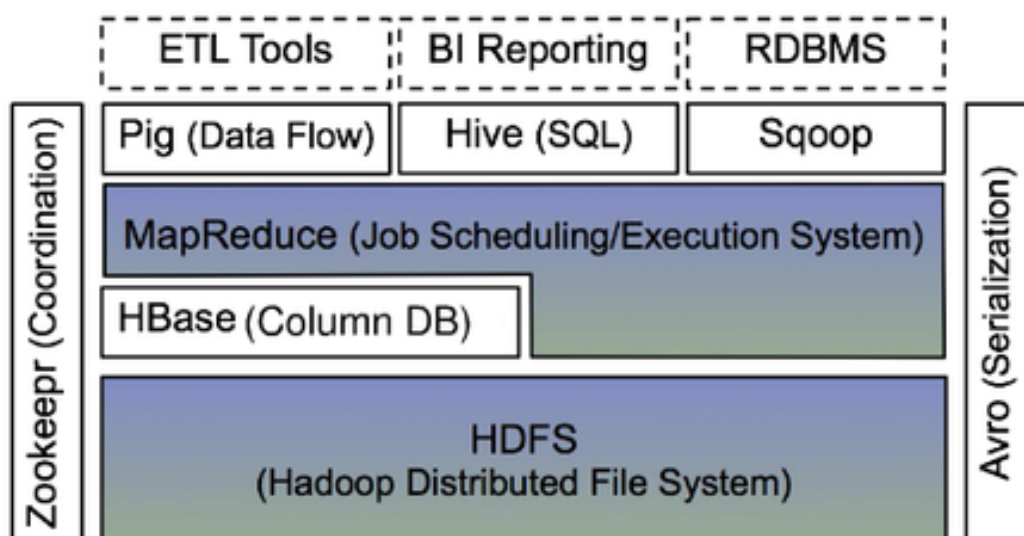
Những đặc điểm này đã giúp Bigtable trở thành một nền tảng lưu trữ mạnh mẽ, đáng tin cậy và hiệu quả cho nhiều dịch vụ của Google, đồng thời là nguồn cảm hứng cho nhiều hệ thống NoSQL khác sau này.

Apache Hbase

Apache Hbase là một hệ quản trị cơ sở dữ liệu phi quan hệ (NoSQL) phân tán, mã nguồn mở, được phát triển bằng ngôn ngữ lập trình Java. Hệ thống này được thiết kế dựa trên mô hình Bigtable của Google, một giải pháp lưu trữ dữ liệu có khả năng mở rộng cực kỳ cao, được sử dụng rộng rãi trong các ứng dụng của Google. Với kiến trúc là một cơ sở dữ liệu cột (column-oriented database), Hbase đặc biệt phù hợp cho việc lưu trữ và quản lý các tập dữ liệu lớn, thưa thớt (sparse datasets) và có cấu trúc linh hoạt (schema-less), nơi mà các hệ quản trị cơ sở dữ liệu quan hệ truyền thống gặp nhiều hạn chế.

Hbase là một thành phần cốt lõi trong hệ sinh thái Apache Hadoop. Nó hoạt động trên Hệ thống tệp phân tán Hadoop (HDFS - Hadoop Distributed File System), tận dụng khả năng

lưu trữ dữ liệu phân tán, có tính sẵn sàng cao và khả năng chịu lỗi của HDFS. Trong khi HDFS cung cấp khả năng lưu trữ tuần tự hiệu quả cho các tập dữ liệu lớn, Hbase bổ sung khả năng truy cập ngẫu nhiên (random access) tốc độ cao vào các hàng dữ liệu riêng lẻ, điều này rất quan trọng cho các ứng dụng yêu cầu truy xuất dữ liệu theo thời gian thực hoặc gần thời gian thực.



Hình trên mô tả sơ đồ kiến trúc hệ sinh thái Hadoop với vị trí và vai trò của Hbase. Ở tầng cơ sở, HDFS đóng vai trò là lớp lưu trữ dữ liệu nền tảng. Phía trên HDFS, chúng ta thấy MapReduce, một framework mạnh mẽ cho việc xử lý dữ liệu song song và phân tán, cùng với Hbase (được ghi chú là Column DB). Điều này cho thấy Hbase không chỉ là một kho lưu trữ mà còn là một phần tích hợp sâu sắc trong quy trình xử lý dữ liệu lớn. Các công cụ cấp cao hơn như Pig (cho luồng dữ liệu), Hive (cho truy vấn SQL trên Hadoop), và Sqoop (cho việc chuyển dữ liệu giữa Hadoop và các hệ thống khác) đều có thể tương tác với Hbase thông qua MapReduce hoặc trực tiếp để thực hiện các tác vụ phân tích và báo cáo. Ngoài ra, Zookeeper được sử dụng để điều phối các dịch vụ trong cụm, đảm bảo tính nhất quán và khả năng chịu lỗi, trong khi Avro hỗ trợ việc tuần tự hóa dữ liệu hiệu quả. Sự kết hợp này giúp Hbase trở thành một giải pháp mạnh mẽ cho các ứng dụng Big Data đòi hỏi cả khả năng lưu trữ lớn và truy cập dữ liệu nhanh chóng.

Apache Cassandra

Apache Cassandra là một hệ quản trị cơ sở dữ liệu NoSQL mã nguồn mở thuộc loại “column family database”. Đây là một trong những lựa chọn hàng đầu cho các ứng dụng yêu cầu khả năng chịu lỗi cao và khả năng mở rộng vượt trội trong môi trường dữ liệu lớn.

Cassandra được hỗ trợ mạnh mẽ bởi DataStax, một công ty chuyên cung cấp các sản phẩm và dịch vụ doanh nghiệp dựa trên Cassandra. Sự hỗ trợ này đảm bảo tính ổn định, bảo mật và sự phát triển liên tục cho hệ thống, giúp các tổ chức có thể triển khai và vận hành Cassandra một cách hiệu quả trong môi trường sản xuất.

Điểm nổi bật của Apache Cassandra là việc sử dụng mô hình phân tán ngang hàng (peer-to-peer). Trong mô hình này, không có một nút trung tâm nào đóng vai trò điều khiển, mà tất cả các nút trong cụm đều có vai trò như nhau. Điều này giúp loại bỏ điểm lỗi duy nhất (single point of failure), tăng cường khả năng chịu lỗi và đảm bảo tính sẵn sàng cao của dữ liệu, ngay cả khi một số nút gặp sự cố.

Cassandra nổi tiếng với khả năng mở rộng tuyến tính (linear scale out) ấn tượng. Điều này có nghĩa là khi thêm các nút mới vào cụm, hiệu suất của hệ thống sẽ tăng lên một cách tỷ lệ thuận, cho phép xử lý khối lượng công việc cực lớn. Khả năng này được minh chứng qua việc Cassandra có thể đạt tới hàng triệu lượt ghi mỗi giây, làm cho nó trở thành một giải pháp lý tưởng cho các ứng dụng cần xử lý dữ liệu lớn và có tốc độ tăng trưởng cao.

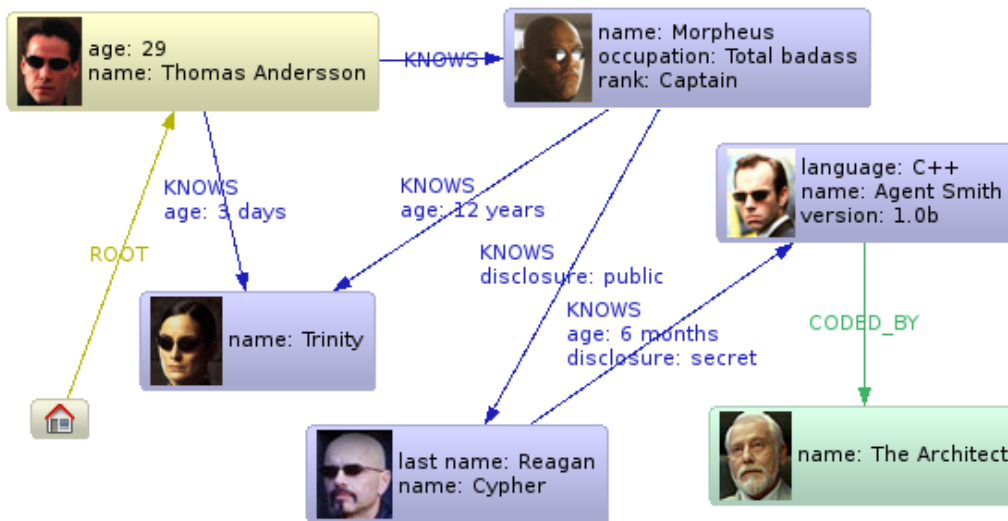
Về mặt kỹ thuật, Apache Cassandra được viết bằng ngôn ngữ lập trình Java. Nhờ đó, nó có khả năng tương thích và hoạt động rất hiệu quả với các công nghệ khác trong hệ sinh thái Big Data như Hệ thống tệp phân tán Hadoop (HDFS) và khung xử lý MapReduce. Sự tích hợp này giúp Cassandra trở thành một thành phần mạnh mẽ trong các kiến trúc dữ liệu lớn, hỗ trợ cả việc lưu trữ và phân tích dữ liệu quy mô lớn một cách hiệu quả.

4.3.4. Mô hình dữ liệu đồ thị (Graph Data Model)

Mô hình dữ liệu đồ thị là một cách tiếp cận mạnh mẽ để tổ chức và biểu diễn dữ liệu, đặc biệt hiệu quả khi dữ liệu có mối quan hệ phức tạp và đa dạng. Khác với các mô hình dữ liệu truyền thống như quan hệ hay tài liệu, mô hình đồ thị tập trung vào việc thể hiện các thực thể và mối liên kết giữa chúng một cách trực quan và linh hoạt.

Các khái niệm cốt lõi trong mô hình dữ liệu đồ thị bao gồm:

- **Các nút (Nodes):** Đây là các thực thể cơ bản trong mô hình đồ thị. Mỗi nút đại diện cho một đối tượng, một người, một địa điểm, một sự kiện, hoặc bất kỳ thực thể nào mà chúng ta muốn lưu trữ thông tin về.
- **Các mối quan hệ (Relationships):** Các mối quan hệ là các liên kết giữa các nút. Chúng mô tả cách các nút tương tác hoặc liên quan đến nhau. Mỗi quan hệ luôn có hướng (từ nút này đến nút kia) và thường có một loại (type) để mô tả bản chất của liên kết đó.
- **Các thuộc tính (Properties) trên cả nút và mối quan hệ:** Cả nút và mối quan hệ đều có thể có các thuộc tính. Thuộc tính là các cặp khóa-giá trị (key-value pairs) dùng để lưu trữ thông tin chi tiết về nút hoặc mối quan hệ đó.



Để minh họa rõ hơn, chúng ta hãy cùng xem xét ví dụ trong hình trên. Đây là một biểu đồ thể hiện các mối quan hệ giữa các nhân vật trong bộ phim “Ma Trận”:

- Các nút:** Trong biểu đồ này, mỗi hộp chữ nhật đại diện cho một nút. Ví dụ, “Thomas Andersson”, “Morpheus”, “Trinity”, “Agent Smith”, “Cypher”, và “The Architect” đều là các nút, đại diện cho các nhân vật. Ngoài ra, biểu tượng ngôi nhà có thể đại diện cho một nút “ROOT” hoặc một điểm khởi đầu nào đó.
- Các thuộc tính trên nút:** Mỗi nút có các thuộc tính riêng để mô tả chi tiết về thực thể đó. Chẳng hạn:
 - Nút “Thomas Andersson” có các thuộc tính: age: 29, name: Thomas Andersson.
 - Nút “Morpheus” có các thuộc tính: name: Morpheus, occupation: Total badass, rank: Captain.
 - Nút “Agent Smith” có các thuộc tính: language: C++, name: Agent Smith, version: 1.0b.
- Các mối quan hệ:** Các mũi tên nối giữa các nút chính là các mối quan hệ. Mỗi mũi tên có một nhãn (label) để chỉ loại mối quan hệ đó. Ví dụ:
 - Mũi tên từ “Thomas Andersson” đến “Morpheus” có nhãn “KNOWS”, cho thấy Thomas biết Morpheus.
 - Mũi tên từ “Thomas Andersson” đến “Trinity” cũng có nhãn “KNOWS”.

- Mũi tên từ “Agent Smith” đến “The Architect” có nhãn “CODED_BY”, ngụ ý Agent Smith được tạo ra bởi The Architect.
 - Mũi tên từ biểu tượng ngôi nhà đến “Thomas Andersson” có nhãn “ROOT”, có thể chỉ ra Thomas là điểm khởi đầu của một chuỗi quan hệ nào đó.
- **Các thuộc tính trên mỗi quan hệ:** Điều đặc biệt của mô hình đồ thị là các mối quan hệ cũng có thể mang các thuộc tính để cung cấp ngữ cảnh hoặc thông tin bổ sung về chính mối quan hệ đó. Ví dụ:
 - Mối quan hệ “KNOWS” từ “Thomas Andersson” đến “Trinity” có thuộc tính `age: 3 days`, có thể chỉ thời gian Thomas đã biết Trinity.
 - Mối quan hệ “KNOWS” từ “Morpheus” đến “Trinity” có thuộc tính `age: 12 years`.
 - Mối quan hệ “KNOWS” từ “Morpheus” đến “Cypher” có thuộc tính `disclosure: public`.
 - Mối quan hệ “KNOWS” từ “Agent Smith” đến “Cypher” có thuộc tính `age: 6 months` và `disclosure: secret`.

Qua ví dụ này, chúng ta có thể thấy mô hình dữ liệu đồ thị cho phép biểu diễn các mối quan hệ phức tạp và đa chiều một cách rất tự nhiên và trực quan, giúp việc truy vấn và phân tích các kết nối trở nên dễ dàng và hiệu quả hơn nhiều so với các mô hình dữ liệu khác.

Cơ sở dữ liệu đồ thị (Graph database store)

Cơ sở dữ liệu đồ thị là một loại hệ quản trị cơ sở dữ liệu NoSQL đặc biệt, được thiết kế để lưu trữ dữ liệu theo một cấu trúc đồ thị rõ ràng. Trong mô hình này, dữ liệu được tổ chức dưới dạng các *nút* (nodes) và các *cạnh* (edges) hoặc *quan hệ* (relationships) liên kết các nút đó. Mỗi nút thường đại diện cho một thực thể (ví dụ: một người, một địa điểm, một sản phẩm), trong khi các cạnh mô tả mối quan hệ giữa các thực thể (ví dụ: “kết bạn với”, “sống tại”, “mua”).

Một đặc điểm nổi bật của cơ sở dữ liệu đồ thị là mỗi nút đều “biết” trực tiếp các nút liên kết với nó thông qua các cạnh. Điều này có nghĩa là mỗi quan hệ giữa các thực thể được lưu trữ một cách tường minh và là một phần cốt lõi của cấu trúc dữ liệu, chứ không phải được suy ra thông qua các phép nối (joins) phức tạp như trong cơ sở dữ liệu quan hệ truyền thống. Sự liên kết trực tiếp này giúp việc truy cập và duyệt qua các mối quan hệ trở nên cực kỳ hiệu quả.

Do bản chất cấu trúc dữ liệu, các truy vấn trong cơ sở dữ liệu đồ thị thực chất là các thao tác duyệt đồ thị. Thay vì viết các câu lệnh truy vấn phức tạp để kết nối nhiều bảng, người

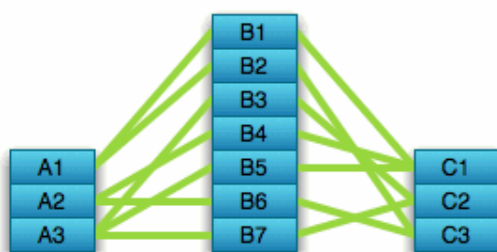
dùng sẽ “đi theo” các đường dẫn quan hệ từ nút này sang nút khác để tìm kiếm thông tin, khám phá các mẫu (patterns) hoặc phân tích các mối liên kết sâu sắc trong dữ liệu. Điều này làm cho cơ sở dữ liệu đồ thị trở nên mạnh mẽ đặc biệt trong các ứng dụng cần xử lý dữ liệu có tính kết nối cao và phức tạp, như mạng xã hội, hệ thống khuyến nghị, phát hiện gian lận, hoặc quản lý tri thức.

Một số hệ quản trị cơ sở dữ liệu đồ thị phổ biến hiện nay bao gồm Neo4j, OrientDB, AllegroGraph, và nhiều hệ thống khác.

So sánh với Cơ sở dữ liệu Quan hệ

Khi xem xét các hệ thống cơ sở dữ liệu NoSQL, một trong những cách hiệu quả để hiểu rõ ưu điểm của chúng là so sánh với cơ sở dữ liệu quan hệ truyền thống. Có hai khía cạnh chính mà các cơ sở dữ liệu NoSQL thường được tối ưu hóa hơn so với cơ sở dữ liệu quan hệ, đó là khả năng tập hợp dữ liệu (aggregation) và khả năng xử lý các kết nối (connections).

Tối ưu hóa cho việc kết tập dữ liệu (Optimized for aggregation)

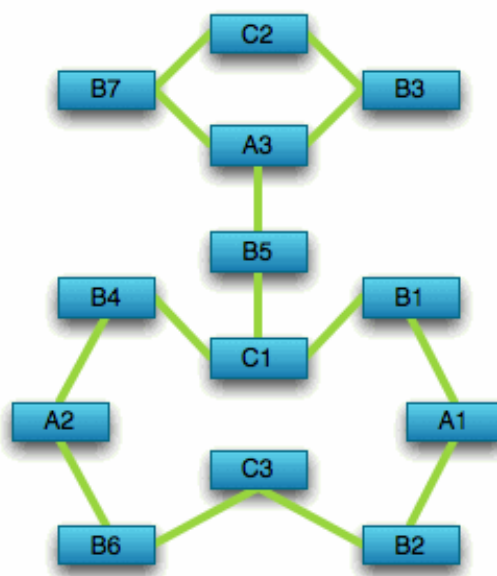


Trong cơ sở dữ liệu quan hệ, dữ liệu thường được chuẩn hóa và phân tách thành nhiều bảng khác nhau để tránh trùng lặp và đảm bảo tính toàn vẹn. Ví dụ, như minh họa trong hình trên, chúng ta có thể thấy dữ liệu được tổ chức thành các bảng riêng biệt như A, B và C. Khi cần truy vấn thông tin đầy đủ về một thực thể mà dữ liệu của nó nằm rải rác trên nhiều bảng, chúng ta phải thực hiện các phép nối (join) giữa các bảng này. Mặc dù phép nối là một công cụ mạnh mẽ, nhưng chúng có thể trở nên tốn kém về mặt hiệu suất khi số lượng bảng cần nối lớn hoặc khi dữ liệu rất lớn.

Tối ưu hóa cho các kết nối (Optimized for connections)

Trong khi cơ sở dữ liệu quan hệ biểu diễn mối quan hệ thông qua khóa ngoại (foreign keys) và các bảng nối (join tables), việc truy vấn các mối quan hệ sâu hoặc phức tạp có thể đòi hỏi nhiều phép nối liên tiếp, dẫn đến hiệu suất giảm sút.

Đối với các trường hợp mà mối quan hệ giữa các thực thể là trọng tâm của dữ liệu và các truy vấn, cơ sở dữ liệu đồ thị (Graph Databases) – một loại hình của NoSQL – thể hiện ưu thế vượt trội. Như được minh họa trong hình trên về cấu trúc đồ thị, cơ sở dữ liệu đồ thị lưu trữ dữ liệu dưới dạng các nút (nodes) và các cạnh (edges) nối các nút này lại với nhau.



Mỗi nút đại diện cho một thực thể (ví dụ: A1, B2, C3), và mỗi cạnh đại diện cho một mối quan hệ giữa các thực thể đó (ví dụ: mối quan hệ giữa A3 và B5).

Mô hình này cho phép duyệt qua các mối quan hệ một cách tự nhiên và cực kỳ hiệu quả. Thay vì thực hiện các phép nối tốn kém, cơ sở dữ liệu đồ thị có thể “đi theo” các cạnh trực tiếp từ nút này sang nút khác để khám phá các mối quan hệ. Điều này làm cho các truy vấn liên quan đến mạng lưới xã hội, hệ thống khuyến nghị, phát hiện gian lận hoặc bất kỳ kịch bản nào đòi hỏi việc phân tích các mối quan hệ phức tạp trở nên nhanh chóng và dễ dàng hơn rất nhiều so với việc sử dụng cơ sở dữ liệu quan hệ.

So sánh với Kho dữ liệu Khóa-Giá trị (Key-Value Stores)

Khi xem xét các loại cơ sở dữ liệu NoSQL khác nhau, việc hiểu rõ điểm mạnh và điểm yếu của từng loại là rất quan trọng. Trong phần này, chúng ta sẽ so sánh một cách tổng quan về cách thức hoạt động của một số loại cơ sở dữ liệu, đặc biệt là khi đối chiếu với kho dữ liệu Khóa-Giá trị (Key-Value Stores).

Tối ưu hóa cho các truy vấn tìm kiếm đơn giản (Optimized for simple look-ups)

Kho dữ liệu Khóa-Giá trị được thiết kế và tối ưu hóa đặc biệt cho các tác vụ truy vấn tìm kiếm đơn giản. Điều này có nghĩa là, khi người dùng có một khóa (key) và muốn lấy giá trị (value) tương ứng, hệ thống sẽ thực hiện việc này một cách cực kỳ nhanh chóng và hiệu quả. Cấu trúc của chúng rất đơn giản: mỗi khóa duy nhất sẽ ánh xạ tới một giá trị.

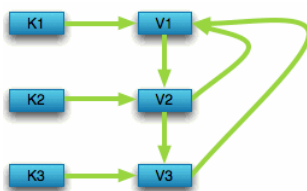
Hình trên minh họa cho cấu trúc này. Chúng ta có các khóa như K1, K2, K3, mỗi khóa được liên kết trực tiếp với một giá trị tương ứng là V1, V2, V3. Mặc dù các giá trị này có



thể chứa thông tin phức tạp hơn hoặc thậm chí là các tham chiếu đến các khóa khác (như V1 chứa K2, V2 chứa K1 và K3), nhưng bản chất của việc truy cập vẫn là thông qua khóa chính. Sự đơn giản này giúp Key-Value Stores đạt được hiệu suất cao trong các tác vụ đọc/ghi cơ bản, đặc biệt khi cần truy xuất dữ liệu nhanh chóng dựa trên một định danh duy nhất.

Tối ưu hóa cho việc duyệt qua dữ liệu có kết nối (Optimized for traversing connected data)

Ngược lại, một số loại cơ sở dữ liệu khác, điển hình là cơ sở dữ liệu đồ thị (Graph Databases), lại được tối ưu hóa cho một loại tác vụ hoàn toàn khác: duyệt qua các mối quan hệ và kết nối giữa các điểm dữ liệu. Trong thế giới thực, nhiều loại dữ liệu không tồn tại độc lập mà có mối liên hệ chặt chẽ với nhau, tạo thành một mạng lưới phức tạp.



Hình trên minh họa rõ ràng cấu trúc dữ liệu có kết nối này. Chúng ta có thể coi K1, K2, K3 là các nút (nodes) hoặc điểm khởi đầu, và V1, V2, V3 là các nút khác trong mạng lưới. Các mũi tên biểu thị các mối quan hệ (edges) giữa các nút này. Ví dụ, V1 có thể liên quan đến V2, V2 liên quan đến V3, và thậm chí V3 lại có mối liên hệ ngược trở lại V1, tạo thành một chu trình.

Trong một cơ sở dữ liệu đồ thị, việc tìm kiếm các đường đi, xác định các nút có ảnh hưởng, hoặc khám phá các mối quan hệ đa cấp (ví dụ: “bạn của bạn của bạn”) được thực hiện một cách tự nhiên và hiệu quả. Thay vì phải thực hiện nhiều truy vấn JOIN phức tạp như trong cơ sở dữ liệu quan hệ, hoặc phải tải toàn bộ dữ liệu vào bộ nhớ để xử lý các mối quan hệ như trong một số kho dữ liệu Key-Value lớn, cơ sở dữ liệu đồ thị được thiết kế để “đi theo” các cạnh (edges) một cách nhanh chóng. Điều này làm cho chúng trở thành lựa chọn lý tưởng cho các ứng dụng yêu cầu phân tích mạng xã hội, hệ thống khuyến nghị, phát hiện gian lận, hoặc bất kỳ lĩnh vực nào mà các mối quan hệ giữa dữ liệu là trọng tâm.

Tóm lại, trong khi Key-Value Stores xuất sắc trong việc truy xuất dữ liệu đơn lẻ dựa trên khóa, thì các cơ sở dữ liệu đồ thị lại vượt trội trong việc quản lý và duyệt qua các mạng lưới dữ liệu phức tạp, nơi các mối quan hệ đóng vai trò then chốt. Việc lựa chọn loại cơ sở dữ liệu phù hợp phụ thuộc vào bản chất của dữ liệu và các loại truy vấn mà ứng dụng cần thực

hiện thường xuyên nhất.

So sánh với Cơ sở dữ liệu tài liệu

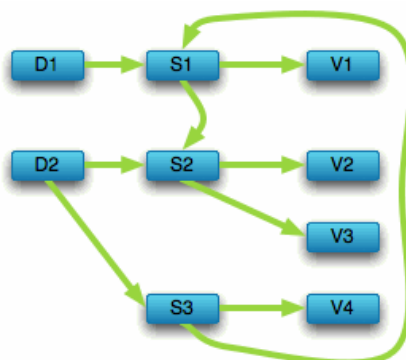
Khi tìm hiểu về các hệ quản trị cơ sở dữ liệu NoSQL, chúng ta thường gặp nhiều mô hình lưu trữ dữ liệu khác nhau, mỗi mô hình lại có những điểm mạnh riêng biệt. Trong phần này, chúng ta sẽ cùng phân tích sự khác biệt cơ bản trong cách tối ưu hóa dữ liệu giữa cơ sở dữ liệu tài liệu (Document Stores) và một mô hình khác, thường là cơ sở dữ liệu đồ thị (Graph Databases), dựa trên cách chúng biểu diễn và xử lý các mối quan hệ.



Cơ sở dữ liệu tài liệu được **tối ưu hóa cho các “cây” dữ liệu**. Điều này có nghĩa là chúng rất phù hợp để lưu trữ dữ liệu có cấu trúc phân cấp, nơi mỗi tài liệu (ví dụ: một tài liệu JSON hoặc BSON) có thể chứa các tài liệu con hoặc các mảng giá trị lồng ghép, tạo thành một cấu trúc giống như cây. Hình trên minh họa rõ ràng điều này: chúng ta thấy các tài liệu chính (D1, D2) chứa các phần tử con (S1, S2, S3) và các giá trị (V1, V2, V3, V4) được tổ chức theo từng cấp bậc. Mặc dù có thể có các tham chiếu chéo giữa các tài liệu (ví dụ: D2/S2 được nhúng trong D1/S1, hoặc D1/S1 được nhúng trong D2/S3), nhưng trọng tâm chính của mô hình này vẫn là việc quản lý từng tài liệu riêng lẻ một cách hiệu quả.

Tuy nhiên, khi nhu cầu của chúng ta không chỉ dừng lại ở việc quản lý các “cây” dữ liệu độc lập mà còn cần phải hiểu rõ các mối quan hệ phức tạp, đa chiều giữa chúng, một loại cơ sở dữ liệu khác sẽ thể hiện ưu thế vượt trội. Loại cơ sở dữ liệu này được **tối ưu hóa để “nhìn thấy cả khu rừng và từng cái cây, từng cành cây, và từng thân cây”**. Đây là một phép ẩn dụ rất hình tượng để mô tả khả năng của cơ sở dữ liệu đồ thị.

Trong khi cơ sở dữ liệu tài liệu tập trung vào việc lưu trữ và truy vấn các tài liệu độc lập, cơ sở dữ liệu đồ thị lại tập trung vào các mối quan hệ. Chúng không chỉ cho phép chúng ta lưu trữ các thực thể (tương tự như các “cây” hoặc “tài liệu” riêng lẻ, được gọi là các nút -



nodes) mà còn đặc biệt hiệu quả trong việc biểu diễn và truy vấn các kết nối, tương tác giữa các thực thể đó (được gọi là các cạnh - edges). Hình trên minh họa cách cùng một tập dữ liệu có thể được biểu diễn dưới dạng đồ thị: D1, D2, S1, S2, S3, V1, V2, V3, V4 trở thành các nút, và các mũi tên màu xanh lá cây biểu thị các mối quan hệ định hướng giữa chúng.

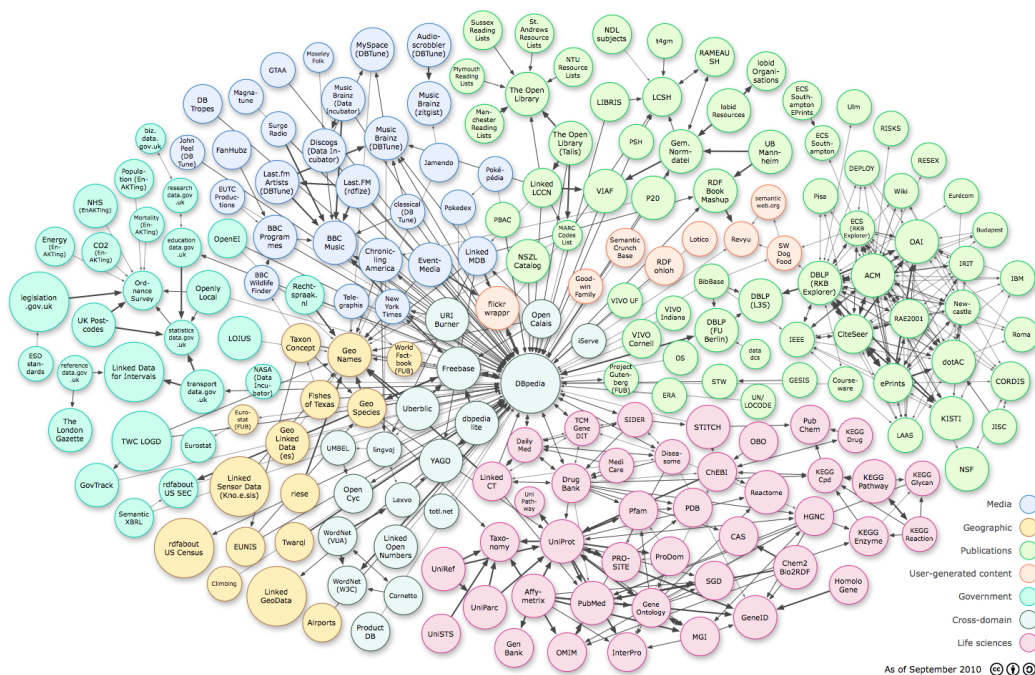
Khả năng “nhìn thấy cả khu rừng” có nghĩa là cơ sở dữ liệu đồ thị cho phép chúng ta dễ dàng khám phá các đường dẫn, chuỗi quan hệ, và các mẫu phức tạp trong dữ liệu mà việc này sẽ rất khó khăn hoặc kém hiệu quả khi sử dụng cơ sở dữ liệu tài liệu. Ví dụ, việc tìm kiếm tất cả các kết nối gián tiếp giữa D1 và V4, hoặc xác định các vòng lặp trong mối quan hệ, là những tác vụ mà cơ sở dữ liệu đồ thị được thiết kế để thực hiện một cách tự nhiên và hiệu quả cao.

Nếu dữ liệu chủ yếu là các cấu trúc phân cấp độc lập và các mối quan hệ giữa chúng không phải là trọng tâm chính của các truy vấn, cơ sở dữ liệu tài liệu có thể là lựa chọn phù hợp. Nhưng nếu các mối quan hệ giữa các phần tử dữ liệu là cốt lõi, và cần khả năng truy vấn, phân tích các kết nối phức tạp đó một cách hiệu quả, thì cơ sở dữ liệu đồ thị sẽ là giải pháp vượt trội, giúp ta không chỉ nhìn thấy từng “cây” dữ liệu mà còn hiểu rõ toàn bộ “khu rừng” các mối quan hệ.

Linking open data

Khái niệm “Linking Open Data” (LOD) là một trụ cột quan trọng trong việc phát triển Web ngữ nghĩa (Semantic Web), nhằm mục đích biến dữ liệu trên Internet từ các kho lưu trữ riêng lẻ thành một mạng lưới thông tin liên kết chặt chẽ và có thể được máy tính hiểu và xử lý. Mục tiêu chính của LOD là khuyến khích việc xuất bản dữ liệu có cấu trúc theo các nguyên tắc nhất định, cho phép các tập dữ liệu khác nhau được kết nối với nhau, từ đó tạo ra một kho tri thức khổng lồ và phong phú hơn.

Để minh họa cho quy mô và sự đa dạng của sáng kiến này, chúng ta có thể quan sát “Đám mây Dữ liệu Mở Liên kết” (Linked Open Data Cloud) được thể hiện trong hình trên.



Biểu đồ này, được chụp vào tháng 9 năm 2010, cho thấy một mạng lưới rộng lớn các tập dữ liệu đã được liên kết với nhau. Mỗi nút trong đám mây đại diện cho một tập dữ liệu (dataset) cụ thể, và các đường nối giữa chúng biểu thị các liên kết ngữ nghĩa giữa các mục dữ liệu.

Điểm đáng chú ý là sự đa dạng về lĩnh vực của các tập dữ liệu trong đám mây LOD. Chúng ta có thể thấy dữ liệu thuộc nhiều danh mục khác nhau, được phân biệt bằng màu sắc:

- **Media (Truyền thông):** Bao gồm các tập dữ liệu về phim ảnh, âm nhạc, tin tức.
- **Geographic (Địa lý):** Dữ liệu liên quan đến địa điểm, bản đồ.
- **Publications (Ấn phẩm):** Các bài báo khoa học, sách, thư viện.
- **User-generated content (Nội dung do người dùng tạo):** Dữ liệu từ các nền tảng cộng đồng.
- **Government (Chính phủ):** Dữ liệu công khai từ các cơ quan chính phủ.
- **Cross-domain (Đa lĩnh vực):** Các tập dữ liệu có thể liên kết với nhiều lĩnh vực khác nhau.
- **Life sciences (Khoa học đời sống):** Dữ liệu về sinh học, y học, dược phẩm.

Trong số các nút này, **DBpedia** nổi bật như một trung tâm quan trọng, đóng vai trò là một hub kết nối nhiều tập dữ liệu khác. DBpedia trích xuất thông tin có cấu trúc từ Wikipedia và cung cấp nó dưới dạng dữ liệu liên kết, giúp các tập dữ liệu khác dễ dàng liên kết đến các khái niệm và thực thể được định nghĩa trong Wikipedia.

Việc liên kết dữ liệu mở mang lại nhiều lợi ích to lớn. Nó không chỉ giúp người dùng và các ứng dụng dễ dàng khám phá và truy cập thông tin từ nhiều nguồn khác nhau mà còn cho phép kết hợp dữ liệu để tạo ra những hiểu biết mới và sâu sắc hơn. Khi dữ liệu được liên kết, máy tính có thể “hiểu” được mối quan hệ giữa các thực thể, từ đó hỗ trợ các truy vấn phức tạp và phát triển các ứng dụng thông minh hơn. Sáng kiến Linking Open Data là một dự án cộng đồng của W3C SWEO (Semantic Web Education and Outreach), nhấn mạnh tầm quan trọng của việc chuẩn hóa và hợp tác trong việc xây dựng một Web dữ liệu toàn cầu.

Neo4j

Neo4j là một hệ quản trị cơ sở dữ liệu đồ thị (Graph Database) hàng đầu, được thiết kế đặc biệt để dễ dàng sử dụng cho các nhà phát triển Java. Điều này có nghĩa là nó cung cấp các API và thư viện tích hợp chặt chẽ với hệ sinh thái Java, giúp việc phát triển ứng dụng trở nên thuận tiện và hiệu quả.

Một điểm quan trọng cần lưu ý là Neo4j là một hệ thống **dựa trên đĩa (disk-based)**, không chỉ hoạt động trong bộ nhớ RAM. Điều này đảm bảo rằng dữ liệu được lưu trữ một cách bền vững và có thể mở rộng quy mô vượt ra ngoài giới hạn của bộ nhớ vật lý, cho phép quản lý các tập dữ liệu lớn.

Về tính toàn vẹn và độ tin cậy của dữ liệu, Neo4j hỗ trợ **đầy đủ các thuộc tính ACID** (Atomicity, Consistency, Isolation, Durability). Điều này có nghĩa là mọi giao dịch trong Neo4j đều được đảm bảo thực hiện một cách nguyên tử (hoàn thành hoặc không hoàn thành), dữ liệu luôn nhất quán, các giao dịch được cô lập với nhau và dữ liệu đã cam kết sẽ được lưu trữ bền vững ngay cả khi có sự cố hệ thống. Đây là một yếu tố then chốt cho các ứng dụng yêu cầu độ tin cậy cao.

Đối với các môi trường sản xuất yêu cầu hoạt động liên tục, Neo4j cung cấp tính năng **khả năng sẵn sàng cao (High Availability)**, đặc biệt trong phiên bản Enterprise Edition. Tính năng này giúp đảm bảo rằng hệ thống luôn hoạt động ổn định, giảm thiểu thời gian ngừng hoạt động và duy trì khả năng truy cập dữ liệu ngay cả khi có lỗi xảy ra.

Về khả năng mở rộng, Neo4j thể hiện năng lực ấn tượng với khả năng quản lý lên đến **32 tỷ nút (Nodes), 32 tỷ mối quan hệ (Relationships) và 64 tỷ thuộc tính (Properties)**. Các nút đại diện cho các thực thể (ví dụ: người, địa điểm), các mối quan hệ mô tả sự kết nối giữa các nút (ví dụ: “quen biết”, “làm việc tại”), và các thuộc tính là thông tin chi tiết gắn liền với nút hoặc mối quan hệ. Khả năng này cho phép Neo4j xử lý các đồ thị dữ liệu cực

kỳ lớn và phức tạp.

Để tích hợp vào các ứng dụng, Neo4j cung cấp một **thư viện Java nhúng (Embedded Java library)**. Điều này cho phép các nhà phát triển chạy một phiên bản Neo4j nhỏ gọn ngay bên trong ứng dụng Java của họ, rất tiện lợi cho việc phát triển và thử nghiệm. Ngoài ra, Neo4j cũng cung cấp một **API REST** mạnh mẽ, cho phép các ứng dụng từ bất kỳ ngôn ngữ lập trình nào có thể tương tác với cơ sở dữ liệu thông qua các yêu cầu HTTP tiêu chuẩn, mang lại sự linh hoạt và khả năng tương thích cao.

4.3.5. Document store

Trong mô hình lưu trữ tài liệu (Document store), đơn vị cơ bản được lưu trữ là các “tài liệu” (documents), khác biệt hoàn toàn so với việc lưu trữ các giá trị đơn lẻ như trong cơ sở dữ liệu khóa-giá trị hay các bảng có cấu trúc cố định như trong cơ sở dữ liệu quan hệ truyền thống. Mỗi tài liệu là một đơn vị dữ liệu tự chứa, có thể có cấu trúc phức tạp và linh hoạt.

Các tài liệu này thường được định dạng dưới dạng JSON (JavaScript Object Notation) hoặc XML (Extensible Markup Language). Đây là những định dạng phổ biến, có cấu trúc phân cấp, dễ đọc và dễ dàng trao đổi dữ liệu giữa các hệ thống.

Mỗi tài liệu trong Document store được xác định duy nhất bằng một mã định danh (ID). Mã định danh này giúp chúng ta truy xuất tài liệu một cách nhanh chóng và chính xác.

Một điểm mạnh khác của Document store là khả năng cho phép lập chỉ mục (indexing) trên các thuộc tính (properties) bên trong tài liệu. Điều này cải thiện đáng kể hiệu suất khi chúng ta cần tìm kiếm hoặc truy vấn dữ liệu dựa trên các trường cụ thể trong tài liệu, thay vì phải quét toàn bộ dữ liệu.

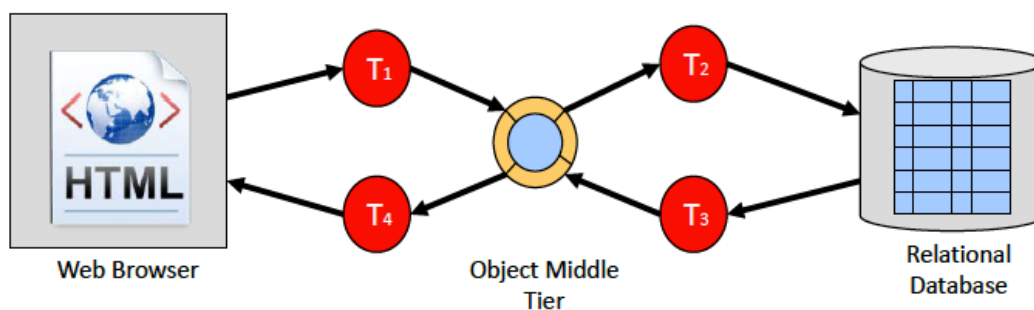
```
{
  person: {
    first_name: "Peter",
    last_name: "Peterson",
    addresses: [
      {street: "123 Peter St"},
      {street: "504 Not Peter St"}
    ],
  },
}
```

Để minh họa rõ hơn, quý vị có thể quan sát ví dụ trong hình trên. Đây là một tài liệu JSON biểu diễn thông tin của một “person” (người). Tài liệu này bao gồm các thuộc tính như `first_name` (tên), `last_name` (họ) và một mảng `addresses` (địa chỉ). Điều đáng chú ý là thuộc tính `addresses` lại chứa một mảng các đối tượng, mỗi đối tượng đại diện cho một địa chỉ cụ thể với thuộc tính `street`. Cấu trúc lồng ghép và linh hoạt này là đặc trưng

của mô hình tài liệu, cho phép chúng ta lưu trữ dữ liệu phức tạp và có cấu trúc không đồng nhất một cách tự nhiên.

Ánh xạ dữ liệu quan hệ

Trong kiến trúc của các ứng dụng hiện đại, đặc biệt là các ứng dụng web, dữ liệu thường cần được chuyển đổi và ánh xạ qua lại giữa nhiều định dạng và tầng khác nhau. Sơ đồ minh họa dưới đây trình bày một chu trình ánh xạ dữ liệu điển hình, thể hiện sự tương tác giữa trình duyệt web, tầng trung gian đối tượng và cơ sở dữ liệu quan hệ. Việc nắm vững các quá trình ánh xạ này là nền tảng quan trọng để thiết kế và phát triển các hệ thống phần mềm hiệu quả.



Chúng ta sẽ cùng tìm hiểu chi tiết bốn quá trình ánh xạ chính được đánh dấu từ T1 đến T4:

- **T1 – HTML thành Đối tượng (HTML into Objects):** Quá trình này diễn ra khi dữ liệu được người dùng nhập vào thông qua các biểu mẫu HTML trên trình duyệt web (Web Browser). Dữ liệu thô từ HTML sau đó được thu thập, phân tích và chuyển đổi thành các đối tượng (Objects) có cấu trúc trong tầng trung gian đối tượng (Object Middle Tier) của ứng dụng. Ví dụ, thông tin từ một biểu mẫu đăng ký người dùng sẽ được ánh xạ thành các thuộc tính của một đối tượng “User” trong bộ nhớ ứng dụng.
- **T2 – Đối tượng thành Bảng SQL (Objects into SQL Tables):** Sau khi dữ liệu đã được biểu diễn dưới dạng đối tượng trong tầng trung gian, quá trình T2 chịu trách nhiệm lưu trữ các đối tượng này vào cơ sở dữ liệu quan hệ (Relational Database). Điều này thường bao gồm việc ánh xạ các thuộc tính của đối tượng thành các cột trong một bảng SQL, và mỗi đối tượng sẽ tương ứng với một hàng (record) dữ liệu trong bảng đó. Đây là bước thiết yếu để đảm bảo tính bền vững và khả năng truy xuất dữ liệu.
- **T3 – Bảng thành Đối tượng (Tables into Objects):** Ngược lại với T2, quá trình T3 là việc truy xuất dữ liệu từ cơ sở dữ liệu quan hệ và chuyển đổi chúng trở lại thành

các đối tượng trong tầng trung gian. Khi ứng dụng cần hiển thị hoặc xử lý dữ liệu đã lưu trữ, nó sẽ gửi truy vấn SQL đến cơ sở dữ liệu, nhận về các hàng dữ liệu và ánh xạ chúng thành các đối tượng tương ứng trong bộ nhớ. Điều này cho phép tầng logic nghiệp vụ của ứng dụng làm việc với dữ liệu theo mô hình hướng đối tượng quen thuộc.

- **T4 – Đối tượng thành HTML (Objects into HTML):** Cuối cùng, khi các đối tượng đã được xử lý hoặc truy xuất từ cơ sở dữ liệu, quá trình T4 sẽ chuyển đổi dữ liệu từ các đối tượng này thành định dạng HTML để hiển thị trên trình duyệt web. Điều này bao gồm việc tạo ra các trang web động, nơi thông tin từ đối tượng (ví dụ: chi tiết sản phẩm, thông tin cá nhân) được nhúng vào cấu trúc HTML để người dùng có thể xem và tương tác.

Tóm lại, chu trình ánh xạ dữ liệu này minh họa cách dữ liệu di chuyển và được chuyển đổi giữa các tầng khác nhau của một ứng dụng, từ giao diện người dùng đến tầng logic nghiệp vụ và tầng lưu trữ dữ liệu. Việc quản lý hiệu quả các quá trình ánh xạ này là chìa khóa để xây dựng các hệ thống phần mềm mạnh mẽ và linh hoạt, đáp ứng được yêu cầu của người dùng và nghiệp vụ.

Dịch vụ Web ở tầng trung gian

Trong kiến trúc ứng dụng hiện đại, đặc biệt là các ứng dụng web, dịch vụ web (Web Service) thường đóng vai trò là một tầng trung gian quan trọng. Tầng này chịu trách nhiệm điều phối và thực hiện các quá trình chuyển đổi dữ liệu phức tạp giữa các thành phần khác nhau của hệ thống, từ giao diện người dùng đến cơ sở dữ liệu và các hệ thống bên ngoài. Sự hiện diện của dịch vụ web ở tầng giữa giúp đảm bảo tính linh hoạt, khả năng mở rộng và khả năng tương tác của ứng dụng.

Chúng ta có thể hình dung vai trò này thông qua một loạt các tác vụ chuyển đổi dữ liệu cơ bản sau:

- **T1 – Chuyển đổi HTML thành Đối tượng Java (HTML into Java Objects):** Khi người dùng tương tác với giao diện web (HTML), ví dụ như điền vào một biểu mẫu và gửi dữ liệu, dịch vụ web ở tầng giữa sẽ tiếp nhận thông tin này. Dữ liệu từ HTML (thường là chuỗi văn bản) cần được phân tích cú pháp và chuyển đổi thành các đối tượng Java có cấu trúc. Các đối tượng này đại diện cho dữ liệu theo cách mà logic nghiệp vụ của ứng dụng có thể dễ dàng xử lý.
- **T2 – Chuyển đổi Đối tượng Java thành Bảng SQL (Java Objects into SQL Tables):** Sau khi dữ liệu được biểu diễn dưới dạng đối tượng Java và được xử lý bởi logic nghiệp vụ, chúng thường cần được lưu trữ lâu dài. Tầng trung gian sẽ thực

hiện việc ánh xạ các thuộc tính của đối tượng Java thành các cột tương ứng trong bảng cơ sở dữ liệu quan hệ (SQL Tables). Quá trình này đảm bảo dữ liệu được lưu trữ một cách nhất quán và có thể truy vấn được.

- **T3 – Chuyển đổi Bảng thành Đối tượng (Tables into Objects):** Ngược lại với T2, khi ứng dụng cần truy xuất dữ liệu từ cơ sở dữ liệu, các hàng và cột từ bảng SQL sẽ được truy vấn. Dịch vụ web ở tầng giữa sẽ chịu trách nhiệm chuyển đổi dữ liệu thô này trở lại thành các đối tượng Java. Điều này cho phép logic nghiệp vụ làm việc với dữ liệu theo mô hình hướng đối tượng quen thuộc, thay vì phải thao tác trực tiếp với cấu trúc bảng.
- **T4 – Chuyển đổi Đối tượng thành HTML (Objects into HTML):** Sau khi dữ liệu được xử lý và sẵn sàng để hiển thị cho người dùng, các đối tượng Java cần được chuyển đổi thành định dạng HTML. Quá trình này bao gồm việc tạo ra các thẻ HTML, định dạng dữ liệu và nhúng chúng vào các mẫu giao diện để trình duyệt có thể hiển thị một cách trực quan và thân thiện với người dùng.
- **T5 – Chuyển đổi Đối tượng thành XML (Objects to XML):** Trong nhiều trường hợp, dịch vụ web cần trao đổi dữ liệu với các hệ thống hoặc ứng dụng khác. XML (Extensible Markup Language) là một định dạng phổ biến cho việc trao đổi dữ liệu. Tầng trung gian sẽ thực hiện việc tuần tự hóa (serialize) các đối tượng Java thành cấu trúc XML, giúp dữ liệu có thể được truyền tải và hiểu bởi các hệ thống khác nhau.
- **T6 – Chuyển đổi XML thành Đối tượng (XML to Objects):** Tương tự, khi dịch vụ web nhận dữ liệu từ một hệ thống bên ngoài dưới định dạng XML, nó cần phải phân tích cú pháp (parse) XML đó và chuyển đổi ngược lại thành các đối tượng Java. Điều này cho phép ứng dụng tích hợp và xử lý dữ liệu đến từ các nguồn bên ngoài một cách hiệu quả.

Nhìn chung, các tác vụ này minh họa rõ ràng vai trò của “Object Middle Tier” (Tầng đối tượng trung gian) trong kiến trúc ứng dụng. Tầng này, nơi các đối tượng Java được xử lý, đóng vai trò là cầu nối quan trọng giữa “Relational Database” (Cơ sở dữ liệu quan hệ) và các lớp trình bày (HTML) hoặc các giao diện trao đổi dữ liệu (XML). Dịch vụ web chính là thành phần cốt lõi giúp thực hiện các quá trình chuyển đổi và điều phối dữ liệu phức tạp này, đảm bảo sự vận hành trơn tru của toàn bộ hệ thống.

Thảo luận

Ánh xạ đối tượng-quan hệ (Object-Relational Mapping - ORM) đã trở thành một trong những thành phần phức tạp nhất trong quá trình xây dựng các ứng dụng hiện đại ngày nay.

ORM là một kỹ thuật cho phép các nhà phát triển làm việc với cơ sở dữ liệu quan hệ bằng cách sử dụng các đối tượng trong ngôn ngữ lập trình hướng đối tượng, thay vì phải viết các câu lệnh SQL trực tiếp. Mặc dù mang lại nhiều lợi ích đáng kể như tăng tốc độ phát triển, giảm thiểu lỗi và cải thiện khả năng bảo trì mã nguồn, sự phức tạp của ORM phát sinh từ việc phải ánh xạ các cấu trúc dữ liệu khác biệt (đối tượng và bảng cơ sở dữ liệu), xử lý các mối quan hệ phức tạp giữa các thực thể, quản lý hiệu suất và tối ưu hóa các truy vấn. Các ví dụ điển hình cho thấy sự phổ biến và đồng thời cả sự phức tạp của ORM bao gồm Khung công tác Hibernate của Java (Java Hibernate Framework) và đặc tả JPA (Java Persistence API), vốn là những công cụ mạnh mẽ nhưng cũng đòi hỏi sự hiểu biết sâu sắc và kinh nghiệm để có thể sử dụng hiệu quả.

Để tránh rơi vào vòng xoáy của sự phức tạp này, một nguyên tắc quan trọng mà chúng ta cần ghi nhớ là hãy giữ cho kiến trúc ứng dụng thật đơn giản. Một kiến trúc đơn giản không chỉ dễ hiểu, dễ bảo trì mà còn giảm thiểu đáng kể khả năng phát sinh lỗi và giúp quá trình phát triển trở nên trôi chảy hơn. Đôi khi, việc cố gắng giải quyết mọi vấn đề bằng các công cụ hoặc mô hình quá phức tạp lại vô tình tạo ra nhiều vấn đề hơn là giải quyết chúng. Sự đơn giản thường là chìa khóa dẫn đến sự bền vững và hiệu quả trong phát triển phần mềm.

Ánh xạ Tài liệu (Document mapping)

Trong lĩnh vực quản lý dữ liệu, đặc biệt là với các hệ quản trị cơ sở dữ liệu hướng tài liệu (NoSQL Document Databases), khái niệm “Ánh xạ Tài liệu” (Document mapping) đóng vai trò then chốt, giúp đơn giản hóa đáng kể quá trình tương tác giữa ứng dụng và cơ sở dữ liệu. Chúng ta sẽ cùng tìm hiểu về cách thức hoạt động và những lợi ích mà phương pháp này mang lại.

Tài liệu trong cơ sở dữ liệu và Tài liệu trong ứng dụng

Điểm cốt lõi của ánh xạ tài liệu là sự tương đồng về cấu trúc dữ liệu giữa cách ứng dụng của chúng ta mô hình hóa thông tin và cách cơ sở dữ liệu lưu trữ chúng. Thay vì phải chuyển đổi các đối tượng phức tạp trong ứng dụng thành nhiều bảng và hàng riêng lẻ trong cơ sở dữ liệu quan hệ, các hệ thống hướng tài liệu cho phép chúng ta lưu trữ toàn bộ một đối tượng hoặc một cấu trúc dữ liệu phức tạp (ví dụ: một tài liệu JSON hoặc BSON) như một “tài liệu” duy nhất trong cơ sở dữ liệu. Điều này có nghĩa là, một đối tượng sản phẩm với các thuộc tính như tên, mô tả, giá, danh sách các biến thể và đánh giá của khách hàng có thể được lưu trữ nguyên vẹn như một tài liệu duy nhất.

Không có tầng trung gian đối tượng

Một trong những lợi ích lớn nhất của ánh xạ tài liệu là việc loại bỏ hoặc giảm thiểu đáng kể nhu cầu về một “tầng trung gian đối tượng” phức tạp. Trong các hệ thống cơ sở dữ liệu quan hệ truyền thống, chúng ta thường cần một lớp Ánh xạ Đối tượng-Quan hệ (ORM) để chuyển đổi giữa các đối tượng trong mã ứng dụng và các bảng trong cơ sở dữ liệu. Quá trình

này thường gặp phải vấn đề “không khớp trở ngại đối tượng-quan hệ” (object-relational impedance mismatch), đòi hỏi nhiều công sức để quản lý và tối ưu hóa. Với ánh xạ tài liệu, vì cấu trúc dữ liệu trong ứng dụng và cơ sở dữ liệu đã tương đồng, tầng trung gian này trở nên không cần thiết, giúp đơn giản hóa kiến trúc và giảm thiểu mã nguồn.

Không có “phân mảnh” (shredding)

“Phân mảnh” là quá trình chia nhỏ một đối tượng hoặc một tài liệu phức tạp thành nhiều phần nhỏ hơn để lưu trữ chúng trong các bảng khác nhau của cơ sở dữ liệu quan hệ. Ví dụ, một đơn hàng có thể được chia thành bảng `Orders` và bảng `OrderItems`. Ánh xạ tài liệu loại bỏ hoàn toàn quá trình phân mảnh này. Toàn bộ tài liệu được lưu trữ như một đơn vị duy nhất, giữ nguyên cấu trúc phân cấp và các mối quan hệ bên trong nó.

Không cần “tái hợp” (reassembly)

Là hệ quả trực tiếp của việc không phân mảnh, chúng ta cũng không cần phải “tái hợp” dữ liệu khi truy vấn. Khi một tài liệu được lưu trữ nguyên vẹn, việc truy xuất nó cũng đơn giản là lấy toàn bộ tài liệu đó ra. Điều này giúp giảm đáng kể độ phức tạp của các truy vấn và tăng hiệu suất đọc, vì không cần phải thực hiện các phép nối (JOIN) phức tạp giữa nhiều bảng để xây dựng lại đối tượng ban đầu.

Đơn giản!

Tóm lại, toàn bộ quá trình ánh xạ tài liệu hướng đến sự đơn giản. Từ cách mô hình hóa dữ liệu, lưu trữ, cho đến truy vấn, mọi thứ đều trở nên trực quan và dễ quản lý hơn. Sự tương đồng giữa cấu trúc dữ liệu trong ứng dụng và cơ sở dữ liệu giúp các nhà phát triển làm việc hiệu quả hơn, giảm thiểu lỗi và tăng tốc độ phát triển.

Tầng ứng dụng (Application Layer)

Quá trình ánh xạ tài liệu này diễn ra chủ yếu tại hoặc rất gần với tầng ứng dụng. Điều này có nghĩa là các nhà phát triển có thể làm việc trực tiếp với các đối tượng dữ liệu quen thuộc trong mã nguồn của họ, và hệ thống cơ sở dữ liệu sẽ tự động xử lý việc lưu trữ và truy xuất các đối tượng đó dưới dạng tài liệu. Sự tích hợp chặt chẽ này giúp giảm bớt gánh nặng về quản lý dữ liệu cho tầng ứng dụng, cho phép các nhà phát triển tập trung hơn vào logic nghiệp vụ cốt lõi.

MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu NoSQL phổ biến, được phát triển bởi công ty 10gen (nay là MongoDB Inc.). Đây là một kho dữ liệu mã nguồn mở, được thiết kế đặc biệt để lưu trữ dữ liệu dưới dạng tài liệu (document) theo định dạng JSON (JavaScript Object Notation), hoặc chính xác hơn là BSON (Binary JSON), một dạng nhị phân của JSON. Việc sử dụng cấu trúc tài liệu linh hoạt này cho phép MongoDB dễ dàng lưu trữ các loại dữ liệu phức tạp và không đồng nhất, vốn là một ưu điểm nổi bật của các cơ sở dữ liệu NoSQL so

với mô hình quan hệ truyền thống.

Một trong những đặc điểm cốt lõi của MongoDB là khả năng mở rộng theo chiều ngang (scale out) thông qua mô hình master-slave (hay còn gọi là primary-secondary trong thuật ngữ của MongoDB). Mô hình này cho phép dữ liệu được phân tán và sao chép trên nhiều máy chủ, đảm bảo tính sẵn sàng cao và khả năng chịu lỗi. Khi hệ thống cần xử lý lượng dữ liệu lớn hơn hoặc lượng truy cập tăng lên, chúng ta có thể dễ dàng thêm các máy chủ mới vào cụm để mở rộng năng lực mà không cần nâng cấp phần cứng của một máy chủ duy nhất.

MongoDB còn được hưởng lợi từ một cộng đồng nhà phát triển rất mạnh mẽ và năng động. Sự hỗ trợ từ cộng đồng rộng lớn này không chỉ cung cấp nguồn tài liệu phong phú, các công cụ hữu ích mà còn đảm bảo việc phát triển và cải tiến liên tục của hệ thống, giúp người dùng dễ dàng tìm kiếm giải pháp và hỗ trợ khi gặp vấn đề trong quá trình triển khai và vận hành.

Để giải quyết bài toán về dữ liệu lớn (big data) và hiệu suất cao, MongoDB tích hợp sẵn tính năng phân mảnh (sharding) một cách tự động. Sharding là kỹ thuật phân chia dữ liệu thành các tập hợp nhỏ hơn (gọi là shard) và lưu trữ chúng trên các máy chủ khác nhau. Điều này giúp phân tán tải trọng truy vấn và lưu trữ, cho phép MongoDB quản lý các tập dữ liệu khổng lồ vượt quá khả năng của một máy chủ đơn lẻ. Tính năng tự động của sharding giúp giảm thiểu đáng kể công sức quản trị và tối ưu hóa hiệu suất.

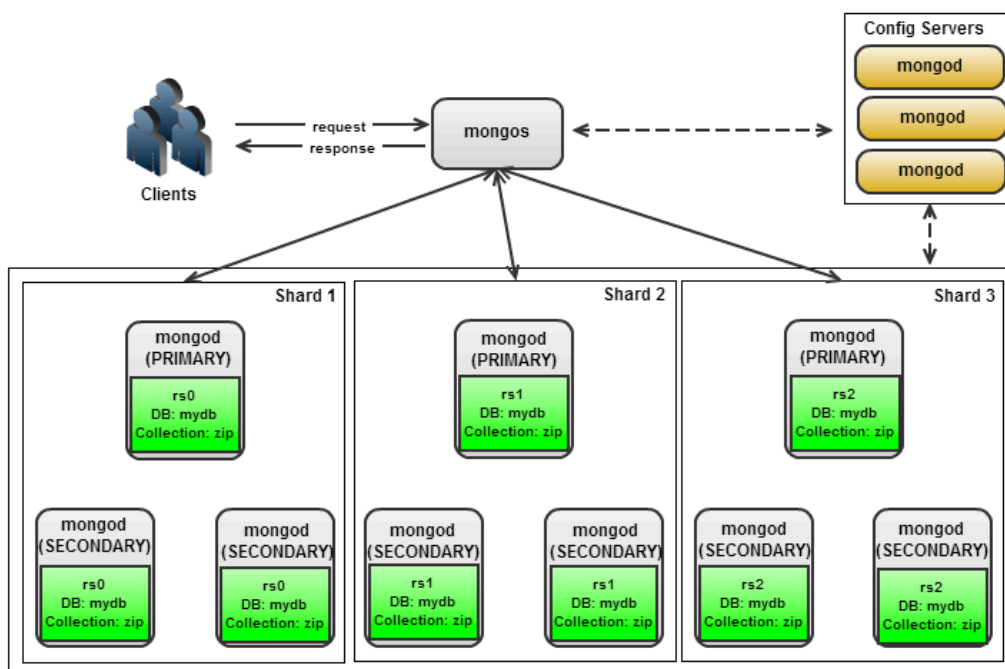
Về mặt triển khai, MongoDB được xây dựng chủ yếu bằng ngôn ngữ lập trình C++. Điều này góp phần mang lại hiệu suất cao và khả năng xử lý mạnh mẽ. Hơn nữa, MongoDB cung cấp một hệ sinh thái API (Application Programming Interface) rất đa dạng, hỗ trợ nhiều ngôn ngữ lập trình phổ biến như C++, JavaScript, Java, Perl, Python, và nhiều ngôn ngữ khác. Sự đa dạng này giúp các nhà phát triển dễ dàng tích hợp MongoDB vào các ứng dụng của họ, bất kể ngôn ngữ lập trình mà họ đang sử dụng, từ đó thúc đẩy quá trình phát triển và triển khai ứng dụng.

Kiến trúc MongoDB

MongoDB được thiết kế với kiến trúc linh hoạt và mạnh mẽ, cho phép nó xử lý lượng dữ liệu lớn và đáp ứng nhu cầu về hiệu suất cao cũng như khả năng chịu lỗi. Để đạt được điều này, MongoDB sử dụng hai thành phần cốt lõi là Replica Set và Sharding.

- **Replica Set** Replica Set (Bộ bản sao) là một nhóm các tiến trình mongod duy trì cùng một tập hợp dữ liệu. Về cơ bản, nó tạo ra các bản sao dữ liệu trên nhiều máy chủ (node) khác nhau. Trong một Replica Set, sẽ có một node chính (Primary) chịu trách nhiệm xử lý tất cả các thao tác ghi, và các node phụ (Secondary) duy trì một bản sao của dữ liệu từ node chính.

Mục đích chính của Replica Set bao gồm:



- **An toàn dữ liệu (Data safety):** Bằng cách sao chép dữ liệu trên nhiều node, Replica Set giúp bảo vệ dữ liệu khỏi mất mát trong trường hợp một node bị lỗi.
- **Tính sẵn sàng cao (High availability):** Nếu node Primary gặp sự cố, một trong các node Secondary sẽ tự động được bầu chọn làm Primary mới, đảm bảo hệ thống luôn hoạt động liên tục mà không bị gián đoạn đáng kể.
- **Phục hồi sau thảm họa (Disaster recovery):** Với các bản sao dữ liệu được phân tán, hệ thống có khả năng phục hồi nhanh chóng sau các sự cố nghiêm trọng như mất điện toàn bộ trung tâm dữ liệu.
- **Bảo trì (Maintenance):** Cho phép thực hiện các hoạt động bảo trì như nâng cấp phần cứng hoặc phần mềm trên từng node mà không làm gián đoạn hoạt động của toàn bộ hệ thống.
- **Mở rộng khả năng đọc (Read scaling):** Các truy vấn đọc có thể được phân phối đến các node Secondary, giúp giảm tải cho node Primary và tăng cường hiệu suất đọc tổng thể của hệ thống.

Như minh họa trong hình, mỗi “Shard” (phân đoạn dữ liệu) thực chất là một Replica Set, bao gồm một mongod (PRIMARY) và nhiều mongod (SECONDARY), đảm bảo tính sẵn sàng và an toàn dữ liệu cho từng phân đoạn.

- **Sharding** Sharding (Phân mảnh) là một phương pháp phân phối dữ liệu trên nhiều máy chủ để hỗ trợ các tập dữ liệu lớn và các ứng dụng có thông lượng cao. Thay vì

lưu trữ toàn bộ dữ liệu trên một máy chủ duy nhất, Sharding chia dữ liệu thành các “phân đoạn” (partitions) và lưu trữ mỗi phân đoạn trên một máy chủ riêng biệt hoặc một nhóm máy chủ (Replica Set).

Lợi ích chính của Sharding là:

- **Mở rộng theo chiều ngang (Horizontal scale):** Sharding cho phép mở rộng dung lượng lưu trữ và khả năng xử lý bằng cách thêm nhiều máy chủ hơn vào hệ thống, thay vì phải nâng cấp máy chủ hiện có (mở rộng theo chiều dọc). Điều này giúp hệ thống dễ dàng đáp ứng được sự tăng trưởng không ngừng của dữ liệu và lưu lượng truy cập.

Trong kiến trúc tổng thể, như hình vẽ mô tả, các Clients (người dùng/ứng dụng) gửi yêu cầu đến mongos (một bộ định tuyến truy vấn). mongos có nhiệm vụ định tuyến các truy vấn đến các Shard phù hợp. Mỗi Shard như đã đề cập, là một Replica Set, chứa một phần dữ liệu của toàn bộ cơ sở dữ liệu. Config Servers lưu trữ siêu dữ liệu (metadata) về cấu trúc của cluster, giúp mongos biết dữ liệu nào nằm ở shard nào. Sự kết hợp giữa Replica Set và Sharding tạo nên một kiến trúc mạnh mẽ, có khả năng mở rộng và chịu lỗi cao cho MongoDB.

Apache CouchDB

Apache CouchDB là một dự án cơ sở dữ liệu mã nguồn mở thuộc Tổ chức Apache Software Foundation. Đây là một hệ quản trị cơ sở dữ liệu NoSQL, cụ thể là một kho dữ liệu tài liệu (document data store) được thiết kế để lưu trữ dữ liệu dưới định dạng JSON. Điều này có nghĩa là mỗi bản ghi trong CouchDB được lưu trữ dưới dạng một tài liệu JSON, mang lại sự linh hoạt cao trong việc định nghĩa cấu trúc dữ liệu.

CouchDB được phát triển bằng ngôn ngữ lập trình Erlang, một ngôn ngữ nổi tiếng với khả năng xử lý đồng thời (concurrency) và xây dựng các hệ thống phân tán có độ tin cậy cao.

Một trong những đặc điểm nổi bật của Apache CouchDB là việc sử dụng API RESTful dựa trên JSON. Điều này cho phép các ứng dụng tương tác với cơ sở dữ liệu thông qua các yêu cầu HTTP tiêu chuẩn, giúp việc tích hợp và phát triển trở nên dễ dàng và quen thuộc với các nhà phát triển web.

Về cơ chế lập chỉ mục, CouchDB sử dụng cấu trúc B-Tree để quản lý các chỉ mục. Đặc biệt, nó áp dụng cơ chế phiên bản hóa “shadowing b-tree”, một dạng của kiểm soát đồng thời đa phiên bản (MVCC - Multi-Version Concurrency Control). Điều này cho phép CouchDB duy trì các phiên bản cũ của tài liệu, hỗ trợ khả năng ghi không khóa (non-blocking writes) và đảm bảo tính toàn vẹn dữ liệu.

Apache CouchDB hỗ trợ đầy đủ các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo rằng các giao dịch dữ liệu được thực hiện một cách đáng tin cậy và nhất quán, một yếu tố quan trọng đối với nhiều ứng dụng doanh nghiệp.

CouchDB cung cấp một mô hình xem (View model) mạnh mẽ, cho phép người dùng định nghĩa các cách thức khác nhau để truy vấn và tổng hợp dữ liệu. Các view này thường được xây dựng bằng JavaScript và sử dụng mô hình MapReduce để xử lý và tạo ra các chỉ mục tùy chỉnh, giúp tối ưu hóa hiệu suất truy vấn.

Tính năng nén dữ liệu (Data compaction) là một cơ chế quan trọng giúp CouchDB quản lý không gian lưu trữ hiệu quả. Khi các tài liệu được cập nhật hoặc xóa, các phiên bản cũ vẫn có thể tồn tại. Quá trình nén sẽ loại bỏ các phiên bản không còn cần thiết và các khoảng trống, giúp giải phóng không gian đĩa và cải thiện hiệu suất.

Cuối cùng, CouchDB cũng tích hợp các tính năng bảo mật cần thiết, bao gồm cơ chế xác thực và phân quyền, cho phép kiểm soát quyền truy cập vào dữ liệu và các thao tác trên cơ sở dữ liệu.

4.4. Cơ sở dữ liệu phi quan hệ NoSQL - Phần 1: CAP theorem

4.5. CAP theorem

4.5.1. Vấn đề mở rộng RDBMS truyền thống

Hệ quản trị cơ sở dữ liệu quan hệ truyền thống (RDBMS) là nền tảng của nhiều ứng dụng quan trọng. Tuy nhiên, khi lượng dữ liệu và số lượng yêu cầu từ người dùng tăng lên đáng kể, việc đảm bảo hiệu suất và khả năng đáp ứng của hệ thống trở thành một thách thức lớn. Để giải quyết vấn đề này, RDBMS truyền thống có thể được mở rộng theo hai phương pháp chính: mở rộng theo chiều dọc (Vertical Scaling) và mở rộng theo chiều ngang (Horizontal Scaling).

Mở rộng theo chiều dọc (Vertical Scaling hay Scale Up): Phương pháp này tập trung vào việc tăng cường tài nguyên của một máy chủ đơn lẻ. Điều này thường được thực hiện bằng cách nâng cấp phần cứng cho máy chủ hiện có, ví dụ như trang bị bộ vi xử lý (CPU) nhanh hơn, bổ sung thêm bộ nhớ truy cập ngẫu nhiên (RAM) lớn hơn, hoặc sử dụng ổ đĩa có dung lượng cao hơn và tốc độ truy xuất nhanh hơn. Ưu điểm của phương pháp này là sự đơn giản trong triển khai, vì không yêu cầu thay đổi kiến trúc ứng dụng hoặc cơ sở dữ liệu một cách phức tạp. Tuy nhiên, mở rộng theo chiều dọc có những giới hạn cố hữu. Một máy chủ vật lý chỉ có thể được cấu hình với một lượng CPU, RAM và ổ đĩa nhất định. Đến một ngưỡng nào đó, việc nâng cấp thêm sẽ không còn khả thi hoặc không mang lại hiệu quả đáng kể, do giới hạn về công nghệ phần cứng và chi phí đầu tư.

Mở rộng theo chiều ngang (Horizontal Scaling hay Scale Out): Ngược lại với mở

rộng theo chiều dọc, mở rộng theo chiều ngang liên quan đến việc bổ sung thêm các máy chủ vào hệ thống để phân tán tải. Phương pháp này cho phép hệ thống xử lý lượng dữ liệu và yêu cầu lớn hơn nhiều so với khả năng của một máy chủ đơn lẻ. Để triển khai mở rộng theo chiều ngang cho RDBMS, cần áp dụng các kỹ thuật phức tạp hơn như phân mảnh cơ sở dữ liệu (database sharding) và có thể là nhân bản dữ liệu (replication).

- **Phân mảnh (Sharding):** Là quá trình chia nhỏ một cơ sở dữ liệu lớn thành nhiều phần nhỏ hơn, gọi là “shard”, và phân tán chúng trên nhiều máy chủ khác nhau. Mỗi shard chứa một tập hợp con của dữ liệu và có thể được quản lý độc lập, giúp phân tán tải và tăng cường khả năng xử lý song song.
- **Nhân bản (Replication):** Là việc tạo ra các bản sao của dữ liệu trên nhiều máy chủ khác nhau. Điều này không chỉ tăng cường khả năng chịu lỗi của hệ thống (bằng cách cung cấp các bản sao dự phòng) mà còn có thể cải thiện hiệu suất đọc bằng cách phân phối các yêu cầu đọc đến các bản sao khác nhau. Tuy nhiên, mở rộng theo chiều ngang cũng có những thách thức riêng. Hiệu quả của nó có thể bị giới hạn bởi tỷ lệ giữa các thao tác đọc và ghi (Read-to-Write ratio). Các thao tác ghi thường phức tạp hơn khi dữ liệu được phân tán và cần được đồng bộ hóa trên nhiều máy chủ để duy trì tính nhất quán. Ngoài ra, chi phí giao tiếp (communication overhead) giữa các máy chủ để duy trì tính nhất quán của dữ liệu và phối hợp các hoạt động cũng là một yếu tố quan trọng cần xem xét, có thể ảnh hưởng đến hiệu suất tổng thể của hệ thống.

4.5.2. Giải pháp hiện tại và thách thức

Data sharding

Trong các hệ thống dữ liệu lớn, việc xử lý toàn bộ dữ liệu trên một máy chủ duy nhất có thể gây ra tắc nghẽn và hạn chế hiệu suất. Để giải quyết vấn đề này, một kỹ thuật phổ biến được áp dụng là **phân mảnh dữ liệu (data sharding)**, đôi khi còn được gọi là **phân dải (striping)**. Kỹ thuật này bao gồm việc chia nhỏ một tập dữ liệu lớn thành nhiều phần nhỏ hơn, gọi là các “mảnh” (shards) hoặc “dải” (stripes). Mục tiêu chính của việc phân mảnh là cho phép nhiều truy cập được thực hiện một cách đồng thời hoặc song song trên các mảnh dữ liệu khác nhau. Điều này giúp phân tán tải trọng xử lý và cải thiện đáng kể khả năng mở rộng của hệ thống.

Hãy hình dung một tệp dữ liệu lớn được chia thành nhiều mảnh nhỏ, ví dụ từ Mảnh 1 đến Mảnh 5. Kỹ thuật phân mảnh cho phép chúng ta truy cập đồng thời vào các mảnh dữ liệu này. Chẳng hạn, Mảnh 1, Mảnh 3 và Mảnh 5 có thể được truy cập và xử lý cùng lúc bởi các tiến trình hoặc máy chủ khác nhau. Điều này minh họa rõ ràng cách thức phân mảnh giúp tăng cường khả năng xử lý song song và giảm thời gian chờ đợi.

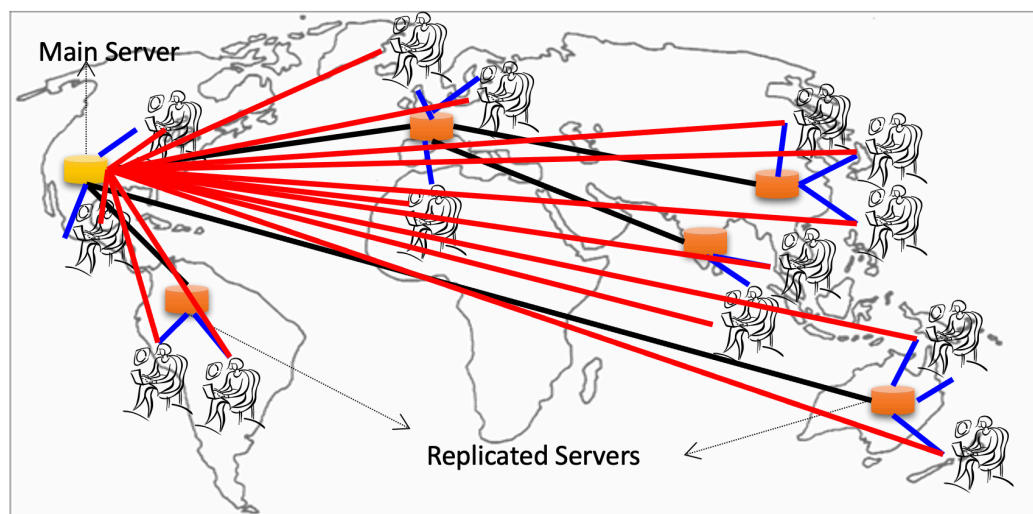
Mặc dù việc phân mảnh dữ liệu mang lại lợi ích rõ rệt về hiệu suất và khả năng mở rộng

cho các truy vấn đơn giản (ví dụ: truy vấn dựa trên khóa chính), nhưng nó cũng đặt ra một thách thức đáng kể: **Liệu kỹ thuật này có thể mở rộng hiệu quả cho việc xử lý các truy vấn phức tạp?** Các truy vấn phức tạp thường liên quan đến việc kết hợp dữ liệu từ nhiều bảng hoặc nhiều phần khác nhau của cùng một bảng. Khi dữ liệu đã bị phân mảnh và nằm rải rác trên nhiều máy chủ hoặc phân vùng khác nhau, việc thực hiện các phép nối (joins) hoặc tổng hợp (aggregations) trên toàn bộ tập dữ liệu trở nên phức tạp hơn rất nhiều. Điều này đòi hỏi các cơ chế phối hợp tinh vi giữa các máy chủ chứa các mảnh dữ liệu, có thể dẫn đến độ trễ và giảm hiệu suất tổng thể.

Sao chép dữ liệu

Kỹ thuật sao chép dữ liệu (Data replicating) là một phương pháp thiết yếu trong việc thiết kế và triển khai các hệ thống phân tán hiện đại. Về cơ bản, nó liên quan đến việc tạo và duy trì nhiều bản sao của cùng một tập dữ liệu trên các máy chủ khác nhau. Việc áp dụng kỹ thuật này mang lại nhiều lợi ích quan trọng, bao gồm:

- **Tránh các nút thắt cổ chai về hiệu suất:** Khi dữ liệu được sao chép và phân tán trên nhiều máy chủ, các yêu cầu truy vấn (đặc biệt là các yêu cầu đọc) có thể được phân phối đồng đều giữa các bản sao. Điều này giúp giảm tải cho từng máy chủ riêng lẻ, từ đó cải thiện đáng kể thời gian phản hồi và nâng cao hiệu suất tổng thể của hệ thống, đặc biệt trong các tình huống có lưu lượng truy cập cao.
- **Tránh các điểm lỗi đơn lẻ (Single Point of Failure - SPOF):** Một trong những rủi ro lớn nhất của hệ thống tập trung là sự phụ thuộc vào một thành phần duy nhất. Khi dữ liệu được sao chép, nếu một máy chủ hoặc một trung tâm dữ liệu gặp sự cố (ví dụ: hỏng hóc phần cứng, mất điện, thiên tai), các bản sao dữ liệu trên các máy chủ khác vẫn có thể tiếp tục hoạt động và phục vụ yêu cầu. Điều này đảm bảo tính liên tục của dịch vụ và giảm thiểu thời gian ngừng hoạt động.
- **Nâng cao khả năng mở rộng và tính sẵn sàng:**
 - **Khả năng mở rộng (Scalability):** Việc sao chép dữ liệu cho phép hệ thống dễ dàng mở rộng theo chiều ngang (scale out) bằng cách thêm các máy chủ mới để lưu trữ thêm bản sao dữ liệu. Điều này giúp hệ thống dễ dàng thích nghi với sự tăng trưởng về khối lượng dữ liệu và số lượng người dùng mà không cần phải nâng cấp phần cứng của một máy chủ duy nhất.
 - **Tính sẵn sàng (Availability):** Với nhiều bản sao dữ liệu được phân tán, hệ thống có khả năng duy trì hoạt động và phục vụ người dùng ngay cả khi một phần của hệ thống gặp sự cố. Điều này đặc biệt quan trọng đối với các ứng dụng yêu cầu thời gian hoạt động liên tục cao (24/7).



Hình ảnh bản đồ thế giới trên minh họa cho ý tưởng rằng các máy chủ chứa dữ liệu sao chép có thể được phân tán rộng khắp về mặt địa lý. Việc đặt các bản sao dữ liệu ở nhiều vị trí khác nhau trên toàn cầu không chỉ giúp giảm độ trễ cho người dùng ở các khu vực địa lý khác nhau mà còn tăng cường khả năng chống chịu của hệ thống trước các sự kiện thảm họa cục bộ, đảm bảo rằng dịch vụ luôn sẵn sàng cho người dùng trên khắp thế giới.

Tính nhất quán trở thành một thách thức

Trong bối cảnh các hệ thống phân tán hiện đại, đặc biệt là khi dữ liệu được sao chép trên nhiều máy chủ để tăng cường khả năng sẵn sàng và cải thiện hiệu suất, việc duy trì tính nhất quán của dữ liệu trở thành một trong những thách thức lớn nhất. Tính nhất quán ở đây có nghĩa là tất cả các bản sao của dữ liệu đều hiển thị cùng một giá trị tại một thời điểm nhất định, bất kể bản sao đó được truy vấn từ máy chủ nào.

Hãy xem xét một ví dụ điển hình trong một ứng dụng thương mại điện tử, nơi cơ sở dữ liệu ngân hàng đã được sao chép trên hai máy chủ khác nhau. Đây là một kịch bản phổ biến để đảm bảo rằng các giao dịch tài chính có thể được xử lý liên tục ngay cả khi một máy chủ gặp sự cố.

Tuy nhiên, việc duy trì tính nhất quán giữa các bản sao dữ liệu này lại là một vấn đề phức tạp. Giả sử có hai sự kiện cần được áp dụng lên cơ sở dữ liệu:

- **Sự kiện 1:** Cộng thêm 1000 đô la vào tài khoản.
- **Sự kiện 2:** Cộng thêm 5% lãi suất vào số dư hiện tại.

Nếu hai máy chủ nhận và xử lý các sự kiện này không đồng bộ hoặc theo một thứ tự khác nhau, điều gì sẽ xảy ra? Ví dụ, máy chủ A có thể xử lý Sự kiện 1 trước rồi đến Sự kiện

2, trong khi máy chủ B lại xử lý Sự kiện 2 trước rồi mới đến Sự kiện 1 (do độ trễ mạng hoặc lỗi hệ thống). Hoặc tệ hơn, một máy chủ có thể nhận được một sự kiện nhưng máy chủ kia lại chưa nhận được ngay lập tức.

Trong trường hợp này, số dư cuối cùng trên máy chủ A có thể khác với số dư trên máy chủ B. Đối với một cơ sở dữ liệu ngân hàng, sự không nhất quán này là không thể chấp nhận được, vì nó có thể dẫn đến sai lệch về số dư tài khoản, mất mát giao dịch, hoặc các vấn đề nghiêm trọng khác ảnh hưởng đến niềm tin của người dùng và tính toàn vẹn của hệ thống tài chính. Do đó, việc thiết kế và triển khai các cơ chế đảm bảo tính nhất quán trong các hệ thống sao chép dữ liệu là một yêu cầu tối quan trọng nhưng cũng đầy thách thức.

The Two-Phase Commit Protocol

Giao thức cam kết hai pha (Two-Phase Commit Protocol - 2PC) là một cơ chế quan trọng được sử dụng trong các hệ thống cơ sở dữ liệu phân tán nhằm đảm bảo tính nguyên tử (atomicity) và tính nhất quán (consistency) của các giao dịch. Điều này có nghĩa là một giao dịch phân tán, dù được thực hiện trên nhiều máy chủ cơ sở dữ liệu khác nhau, sẽ hoặc hoàn tất thành công trên tất cả các máy chủ, hoặc hoàn toàn bị hủy bỏ trên tất cả các máy chủ, không có trường hợp nào giao dịch chỉ thành công một phần.

Giao thức 2PC hoạt động thông qua hai giai đoạn chính, được điều phối bởi một máy chủ đóng vai trò điều phối viên (coordinator) và các máy chủ cơ sở dữ liệu khác đóng vai trò tham gia (participants), ví dụ như Máy chủ cơ sở dữ liệu 1, Máy chủ cơ sở dữ liệu 2 và Máy chủ cơ sở dữ liệu 3.

Giai đoạn I: Chuẩn bị (Phase I: Prepare)

Trong giai đoạn đầu tiên này, điều phối viên sẽ gửi một thông điệp “chuẩn bị” (prepare request) đến tất cả các máy chủ tham gia. Khi nhận được yêu cầu này, mỗi máy chủ tham gia sẽ thực hiện các bước cần thiết để chuẩn bị cho việc cam kết giao dịch. Điều này bao gồm việc kiểm tra xem giao dịch có thể được thực hiện thành công hay không, ví dụ như kiểm tra tài nguyên, khóa dữ liệu cần thiết, và đảm bảo không có xung đột.

Sau khi kiểm tra, mỗi máy chủ tham gia sẽ gửi một phản hồi về cho điều phối viên:

- Nếu máy chủ có thể thực hiện giao dịch và sẵn sàng cam kết, nó sẽ gửi một phiếu “có” (vote “yes”).
- Nếu máy chủ không thể thực hiện giao dịch vì bất kỳ lý do gì (ví dụ: thiếu tài nguyên, lỗi, xung đột), nó sẽ gửi một phiếu “không” (vote “no”).

Giai đoạn II: Cam kết (Phase II: Commit)

Sau khi thu thập tất cả các phiếu bầu từ các máy chủ tham gia, điều phối viên sẽ đưa ra quyết định cuối cùng:

- Nếu tất cả các máy chủ tham gia đều gửi phiếu “có”, điều phối viên sẽ gửi một thông điệp “cam kết” (commit command) đến tất cả các máy chủ. Khi nhận được lệnh này, mỗi máy chủ sẽ hoàn tất giao dịch và lưu trữ các thay đổi vĩnh viễn.
- Nếu có bất kỳ máy chủ nào gửi phiếu “không”, hoặc nếu điều phối viên không nhận được phản hồi từ một máy chủ nào đó trong một khoảng thời gian nhất định, điều phối viên sẽ gửi một thông điệp “hủy bỏ” (abort command) đến tất cả các máy chủ. Khi nhận được lệnh này, mỗi máy chủ sẽ hoàn tác (rollback) mọi thay đổi đã chuẩn bị cho giao dịch, đảm bảo rằng giao dịch không để lại dấu vết nào.

Mặc dù giao thức 2PC hiệu quả trong việc đảm bảo tính nguyên tử và tính nhất quán nghiêm ngặt (strict consistency) cho các giao dịch phân tán, đây cũng chính là điểm hạn chế của nó. Tính nhất quán nghiêm ngặt đòi hỏi tất cả các máy chủ phải đồng bộ hóa hoàn toàn và chờ đợi xác nhận từ nhau, điều này tạo ra một điểm nghẽn tiềm năng và làm tăng độ trễ. Do đó, 2PC thường được xem là một giao thức hạn chế khả năng mở rộng (limits scalability) trong các hệ thống phân tán lớn, nơi yêu cầu hiệu suất cao và khả năng chịu lỗi tốt.

4.5.3. Định lý CAP

Định lý CAP

Trong bối cảnh các hệ thống cơ sở dữ liệu phân tán, việc hiểu rõ những giới hạn cố hữu là điều vô cùng quan trọng để thiết kế và triển khai các giải pháp hiệu quả. Những giới hạn này được mô tả một cách súc tích thông qua một khái niệm nền tảng được gọi là Định lý CAP. Định lý CAP giúp chúng ta nhận diện và đưa ra các quyết định đánh đổi (trade-offs) cần thiết khi xây dựng các hệ thống phân tán.

Định lý CAP đề cập đến ba thuộc tính mong muốn của một hệ thống phân tán, đó là:

- **Tính nhất quán (Consistency - C):** Thuộc tính này đảm bảo rằng tại bất kỳ thời điểm nào, mọi nút trong hệ thống đều nhìn thấy cùng một phiên bản dữ liệu. Điều này có nghĩa là, sau khi một thao tác ghi dữ liệu được thực hiện thành công, tất cả các yêu cầu đọc tiếp theo, dù được gửi đến nút nào, cũng sẽ trả về dữ liệu mới nhất đã được ghi. Đây thường được hiểu là “tính nhất quán chặt chẽ” (strict consistency).
- **Tính sẵn sàng (Availability - A):** Tính sẵn sàng đòi hỏi hệ thống phải luôn hoạt động và có khả năng phản hồi các yêu cầu từ người dùng hoặc các ứng dụng khác, ngay cả khi có một số nút trong cụm bị lỗi (ví dụ: bị sập, ngừng hoạt động) hoặc một phần cứng/phần mềm nào đó gặp sự cố do nâng cấp. Một hệ thống có tính sẵn sàng cao sẽ không bao giờ từ chối yêu cầu mà luôn cố gắng trả về một phản hồi hợp lệ.
- **Khả năng chịu lỗi phân vùng (Partition Tolerance - P):** Thuộc tính này chỉ ra rằng

hệ thống vẫn phải tiếp tục hoạt động bình thường ngay cả khi mạng bị chia cắt (network partitions). Sự chia cắt mạng xảy ra khi có lỗi trong giao tiếp giữa các nút, khiến một nhóm các nút không thể liên lạc được với nhóm nút khác. Trong tình huống này, hệ thống vẫn phải duy trì khả năng hoạt động và xử lý các yêu cầu trong từng phân vùng riêng biệt.

Định lý CAP phát biểu rằng: **Bất kỳ cơ sở dữ liệu phân tán nào có dữ liệu được chia sẻ, chỉ có thể đạt được tối đa hai trong ba thuộc tính mong muốn là Tính nhất quán (C), Tính sẵn sàng (A) hoặc Khả năng chịu lỗi phân vùng (P).**

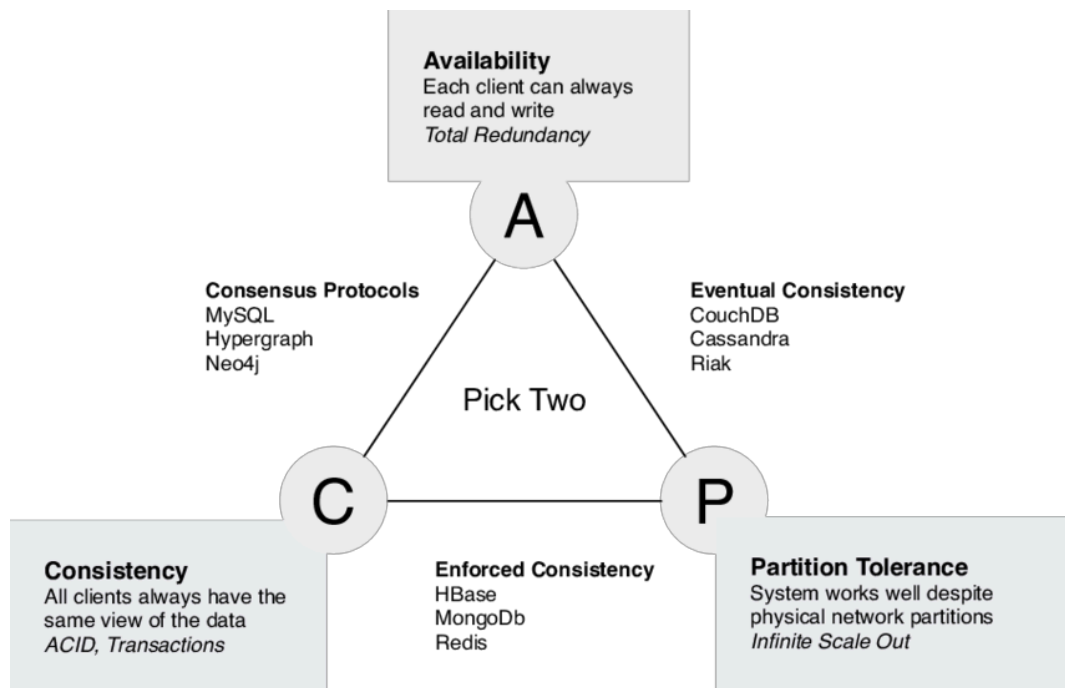
Điều này có nghĩa là, khi thiết kế một hệ thống phân tán, chúng ta buộc phải đưa ra sự đánh đổi. Chúng ta không thể đồng thời có cả ba thuộc tính C, A và P một cách tuyệt đối. Thay vào đó, chúng ta phải lựa chọn ưu tiên hai trong số ba thuộc tính này, và chấp nhận hy sinh một phần hoặc toàn bộ thuộc tính còn lại. Đây là những đánh đổi cốt lõi trong các hệ thống phân tán, được Eric Brewer đề xuất lần đầu tiên tại hội nghị PODC vào năm 2000. Việc hiểu rõ định lý này là chìa khóa để lựa chọn kiến trúc phù hợp nhất cho các yêu cầu cụ thể của ứng dụng.

Ý nghĩa của định lý CAP

Định lý CAP là một trong những khái niệm nền tảng và quan trọng nhất trong lĩnh vực hệ thống phân tán. Nó mô tả ba thuộc tính chính mà một hệ thống lưu trữ dữ liệu phân tán có thể cố gắng đạt được, và chỉ ra rằng không thể đồng thời đảm bảo cả ba thuộc tính này cùng một lúc. Ba thuộc tính đó là: Tính nhất quán (Consistency), Tính sẵn sàng (Availability), và Khả năng chịu lỗi phân vùng (Partition Tolerance).

1. Tính nhất quán (Consistency - C) Tính nhất quán đảm bảo rằng tất cả các client trong hệ thống luôn có cùng một cái nhìn về dữ liệu. Điều này có nghĩa là, sau khi một thao tác ghi dữ liệu được thực hiện thành công, mọi thao tác đọc tiếp theo trên bất kỳ node nào của hệ thống đều phải trả về dữ liệu đã được cập nhật mới nhất. Nếu một thao tác đọc không thể trả về dữ liệu mới nhất, hệ thống sẽ báo lỗi. Trong các hệ thống cơ sở dữ liệu truyền thống, tính nhất quán thường được đảm bảo thông qua các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability) và các cơ chế giao dịch (Transactions). Mục tiêu là duy trì trạng thái dữ liệu hợp lệ và đồng bộ trên toàn bộ hệ thống.

2. Tính sẵn sàng (Availability - A) Tính sẵn sàng đề cập đến khả năng của hệ thống luôn hoạt động và phản hồi các yêu cầu từ client. Điều này có nghĩa là, mỗi client luôn có thể thực hiện các thao tác đọc và ghi dữ liệu thành công, ngay cả khi một phần của hệ thống gặp sự cố. Hệ thống được thiết kế để có tính sẵn sàng cao thường sử dụng các cơ chế dự phòng (Total Redundancy) để đảm bảo rằng nếu một node bị lỗi, các node khác có thể tiếp quản và tiếp tục phục vụ yêu cầu mà không làm gián đoạn hoạt động chung.



3. Khả năng chịu lỗi phân vùng (Partition Tolerance - P) Khả năng chịu lỗi phân vùng là khả năng của hệ thống tiếp tục hoạt động một cách chính xác và hiệu quả ngay cả khi có sự cố mạng làm gián đoạn liên lạc giữa các node (gọi là phân vùng mạng vật lý). Trong một hệ thống phân tán, các node có thể nằm ở các vị trí địa lý khác nhau và giao tiếp qua mạng. Khi mạng gặp sự cố, một số node có thể bị cô lập khỏi các node khác, tạo thành các “phân vùng”. Một hệ thống có khả năng chịu lỗi phân vùng tốt sẽ tiếp tục xử lý các yêu cầu trong mỗi phân vùng mà không bị dừng hoàn toàn. Điều này đặc biệt quan trọng đối với các hệ thống lớn, phân tán rộng rãi, nơi sự cố mạng là điều không thể tránh khỏi. Khả năng chịu lỗi phân vùng cũng là yếu tố then chốt cho việc mở rộng hệ thống (Infinite Scale Out) vì nó cho phép hệ thống phát triển mà không bị giới hạn bởi các điểm lỗi đơn lẻ.

Ý nghĩa cốt lõi của Định lý CAP: Định lý CAP khẳng định rằng trong một hệ thống phân tán, khi xảy ra sự cố phân vùng mạng (P), chúng ta chỉ có thể chọn ưu tiên giữa Tính nhất quán (C) hoặc Tính sẵn sàng (A). Chúng ta không thể có cả ba cùng một lúc.

- Nếu chọn ưu tiên **C và A**, hệ thống sẽ không thể chịu lỗi phân vùng. Khi có phân vùng, hệ thống sẽ phải dừng hoạt động hoặc từ chối yêu cầu để đảm bảo tính nhất quán và sẵn sàng.
- Nếu chọn ưu tiên **C và P**, hệ thống sẽ phải hy sinh tính sẵn sàng. Khi có phân vùng, để đảm bảo tính nhất quán, hệ thống có thể từ chối các yêu cầu ghi hoặc đọc từ các node bị cô lập cho đến khi phân vùng được khắc phục.

- Nếu chọn ưu tiên **A và P**, hệ thống sẽ phải hy sinh tính nhất quán. Khi có phân vùng, hệ thống sẽ tiếp tục phục vụ các yêu cầu để đảm bảo tính sẵn sàng, nhưng các node trong các phân vùng khác nhau có thể có cái nhìn không đồng bộ về dữ liệu.

Định lý CAP giúp đưa ra quyết định thiết kế phù hợp với yêu cầu nghiệp vụ và môi trường hoạt động của ứng dụng.

Chứng minh giản lược của định lý CAP

Để minh họa một cách đơn giản định lý CAP, chúng ta hãy xem xét một hệ thống phân tán chỉ bao gồm hai nút (node).

Giả sử có một client gửi yêu cầu ghi dữ liệu lên nút A. Nút A sẽ xử lý yêu cầu này và phản hồi lại cho client. Để đảm bảo tính nhất quán của dữ liệu trên toàn hệ thống, nút A cần phải cập nhật dữ liệu vừa ghi cho nút B. Tương tự, nút B cũng sẽ chờ đợi để được cập nhật dữ liệu mới nhất từ nút A. Quá trình này có thể được hình dung như một chu trình đồng bộ hóa liên tục giữa các nút để duy trì trạng thái nhất quán.

Tuy nhiên, vấn đề phát sinh khi có một sự cố mạng xảy ra, tạo ra một phân vùng mạng (network partition) giữa nút A và nút B. Trong tình huống này, hai nút không thể giao tiếp với nhau. Điều này dẫn đến việc nút A không thể cập nhật dữ liệu cho nút B, và nút B cũng không thể nhận được dữ liệu mới từ A.

Kết quả là, hệ thống của chúng ta không thể chịu lỗi phân vùng (Not Partition Tolerant). Điều này minh chứng rằng, trong một hệ thống phân tán, nếu chúng ta ưu tiên duy trì tính nhất quán (Consistency) và tính khả dụng (Availability) trong điều kiện hoạt động bình thường, chúng ta sẽ phải đánh đổi khả năng chịu lỗi phân vùng khi có sự cố mạng. Đây chính là cốt lõi của định lý CAP: một hệ thống phân tán chỉ có thể đảm bảo tối đa hai trong ba thuộc tính Consistency, Availability và Partition Tolerance tại một thời điểm.

Hệ quả của định lý CAP

Cơ sở dữ liệu quan hệ (Relational Database) được xây dựng dựa trên các nguyên tắc **ACID** (Atomicity – Tính nguyên tử, Consistency – Tính nhất quán, Isolation – Tính cô lập, Durability – Tính bền vững). Điều này ngụ ý rằng, một hệ thống cơ sở dữ liệu quan hệ phân tán thực sự lý tưởng sẽ cần phải đảm bảo đồng thời cả ba yếu tố: **tính sẵn sàng (Availability), tính nhất quán (Consistency) và khả năng chịu lỗi phân vùng (Partition Tolerance)**.

Tuy nhiên, thật không may, việc đạt được đồng thời cả ba yếu tố này là **bất khả thi** theo định lý CAP.

Trong bối cảnh các hệ thống cơ sở dữ liệu quy mô lớn:

Khi các công ty công nghệ hàng đầu như Google và Amazon thiết kế các hệ thống cơ sở

dữ liệu quy mô lớn, tính sẵn sàng 24/7 (24/7 Availability) luôn là một yếu tố then chốt. Đối với họ, chỉ vài phút gián đoạn hoạt động cũng có thể đồng nghĩa với việc mất đi hàng triệu đô la doanh thu.

Hơn nữa, khi mở rộng quy mô cơ sở dữ liệu theo chiều ngang (horizontally scaling) lên đến hàng nghìn máy chủ, khả năng xảy ra lỗi ở một nút (node) hoặc lỗi mạng (network failure) tăng lên đáng kể. Trong một hệ thống phân tán lớn, việc các thành phần bị lỗi là điều không thể tránh khỏi và cần được tính đến.

Do đó, để đảm bảo mạnh mẽ về tính sẵn sàng (Availability) và khả năng chịu lỗi phân vùng (Partition Tolerance) – hai yếu tố cực kỳ quan trọng đối với các hệ thống quy mô lớn và phân tán – các công ty này buộc phải hy sinh tính nhất quán “ng nghiêm ngặt” (strict Consistency). Đây chính là hệ quả trực tiếp từ định lý CAP, khi phải lựa chọn hai trong ba yếu tố A, C, P. Thay vào đó, họ thường chấp nhận các mô hình nhất quán yếu hơn (eventual consistency) để đạt được mục tiêu về tính sẵn sàng và khả năng hoạt động liên tục.

4.5.4. Các mô hình consistency

Đánh Đổi Giữa Các Mức Độ Nhất Quán (Trading-Off Consistency)

Trong quá trình thiết kế và phát triển hệ thống, đặc biệt là các hệ thống phân tán, việc duy trì tính nhất quán (consistency) là một thách thức lớn. Điều quan trọng là chúng ta cần tìm ra điểm cân bằng phù hợp giữa mức độ nghiêm ngặt của tính nhất quán với khả năng sẵn sàng (availability) và khả năng mở rộng (scalability) của hệ thống.

Mức độ nhất quán được coi là ‘đủ tốt’ (good-enough consistency) không phải là một tiêu chuẩn cố định mà hoàn toàn phụ thuộc vào yêu cầu cụ thể của ứng dụng mà quý vị đang xây dựng. Chẳng hạn, một hệ thống giao dịch tài chính sẽ yêu cầu tính nhất quán rất cao, trong khi một ứng dụng mạng xã hội có thể chấp nhận một mức độ nhất quán lỏng lẻo hơn để đảm bảo khả năng phản hồi nhanh và mở rộng quy mô dễ dàng.

Để làm rõ hơn, chúng ta có thể phân biệt giữa hai cách tiếp cận chính:

- **Tính nhất quán lỏng lẻo (Loose Consistency):**

- Cách tiếp cận này thường dễ dàng hơn trong việc triển khai.
- Nó cũng mang lại hiệu quả cao hơn về mặt hiệu năng và tài nguyên, cho phép hệ thống xử lý nhiều yêu cầu hơn và mở rộng dễ dàng hơn.

- **Tính nhất quán nghiêm ngặt (Strict Consistency):**

- Ngược lại, việc đạt được tính nhất quán nghiêm ngặt thường rất khó khăn để triển khai trong các hệ thống phân tán lớn.

- Hơn nữa, nó thường kém hiệu quả hơn do yêu cầu đồng bộ hóa chặt chẽ giữa tất cả các bản sao dữ liệu, dẫn đến độ trễ cao hơn và khả năng mở rộng bị hạn chế.

Các Thuộc Tính BASE

Định lý CAP đã chỉ ra rằng không thể đồng thời đảm bảo tính Nhất quán (Consistency) nghiêm ngặt và tính Sẵn sàng (Availability) trong một hệ thống phân tán khi có sự cố phân vùng mạng (network partitions). Sự hạn chế này đã thúc đẩy sự phát triển của các hệ quản trị cơ sở dữ liệu với các đảm bảo ACID được nới lỏng hơn, đặc biệt là trong bối cảnh các hệ thống phân tán quy mô lớn.

Cụ thể, các hệ thống cơ sở dữ liệu này thường áp dụng các thuộc tính BASE, một bộ nguyên tắc được thiết kế để ưu tiên tính sẵn sàng và khả năng chịu lỗi trong môi trường phân tán:

- **Basically Available (Sẵn sàng Cơ bản):** Thuộc tính này nhấn mạnh rằng hệ thống sẽ luôn đảm bảo tính Sẵn sàng (Availability). Điều này có nghĩa là hệ thống sẽ luôn phản hồi các yêu cầu, ngay cả khi một số phần của hệ thống đang gặp sự cố hoặc dữ liệu chưa hoàn toàn nhất quán. Mục tiêu là duy trì hoạt động liên tục và khả năng truy cập.
- **Soft-State (Trạng thái Mềm):** Trạng thái của hệ thống có thể thay đổi theo thời gian, ngay cả khi không có đầu vào mới. Điều này ngụ ý rằng trạng thái của dữ liệu không được đảm bảo là nhất quán ngay lập tức và có thể thay đổi do các quá trình ngầm định như đồng bộ hóa hoặc lan truyền dữ liệu. Dữ liệu có thể không ổn định cho đến khi đạt được sự nhất quán cuối cùng.
- **Eventual Consistency (Nhất quán Cuối cùng):** Hệ thống đảm bảo rằng dữ liệu sẽ đạt được trạng thái nhất quán tại một thời điểm nào đó trong tương lai, sau khi tất cả các thao tác ghi đã được lan truyền và xử lý trên toàn bộ hệ thống. Mặc dù dữ liệu có thể không nhất quán ngay lập tức sau một thao tác ghi, nhưng cuối cùng nó sẽ hội tụ về một trạng thái nhất quán khi hệ thống ổn định.

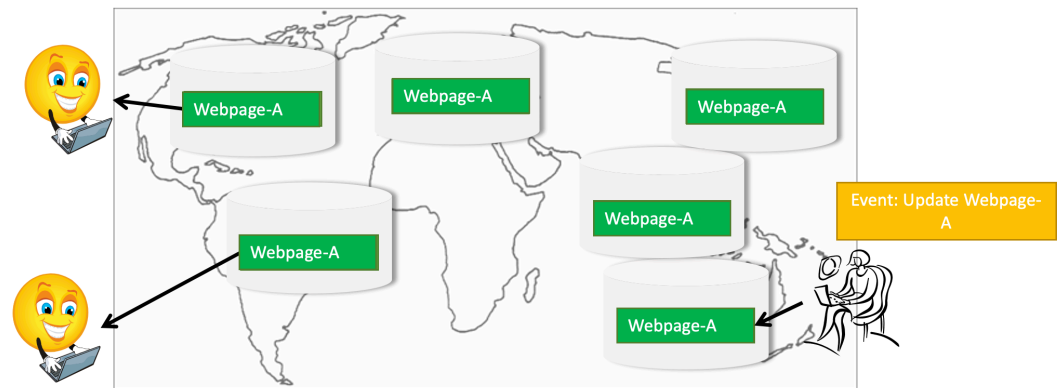
Tính nhất quán cuối cùng (Eventual Consistency)

Trong các hệ thống cơ sở dữ liệu phân tán, một trong những mô hình nhất quán quan trọng mà chúng ta cần tìm hiểu là **Tính nhất quán cuối cùng (Eventual Consistency)**. Một cơ sở dữ liệu được định nghĩa là có tính nhất quán cuối cùng nếu:

- Tất cả các bản sao (replicas) của dữ liệu sẽ dần dần trở nên nhất quán với nhau trong trường hợp không có các cập nhật mới.

Điều này có nghĩa là, nếu hệ thống ngừng nhận các yêu cầu ghi dữ liệu mới, cuối cùng tất cả các bản sao dữ liệu sẽ hội tụ về cùng một trạng thái. Sẽ có một khoảng thời gian trễ nhất định trước khi sự nhất quán hoàn toàn được thiết lập trên toàn bộ hệ thống.

Để dễ hình dung, chúng ta hãy xem xét một ví dụ cụ thể. Hãy tưởng tượng một hệ thống cơ sở dữ liệu phân tán toàn cầu, với các bản sao dữ liệu được đặt tại nhiều khu vực địa lý khác nhau trên bản đồ thế giới.



Khi một sự kiện như “Cập nhật Trang web A” (Update Webpage-A) xảy ra tại một vị trí nào đó, ví dụ như ở Bắc Mỹ, thay đổi này sẽ được ghi nhận vào bản sao dữ liệu tại khu vực đó trước tiên. Tuy nhiên, trong một hệ thống áp dụng mô hình nhất quán cuối cùng, sự thay đổi này không lập tức được đồng bộ hóa tới tất cả các bản sao khác trên toàn cầu, chẳng hạn như các bản sao ở Châu Âu hay Châu Á.

Sẽ có một khoảng thời gian trễ nhất định để bản cập nhật này được truyền tải và áp dụng lên các bản sao còn lại. Trong khoảng thời gian này, nếu một người dùng truy cập Trang web A từ Châu Âu, họ có thể vẫn thấy phiên bản cũ của trang web, trong khi người dùng ở Bắc Mỹ đã thấy phiên bản mới.

Tuy nhiên, nếu không có thêm bất kỳ cập nhật mới nào khác cho Trang web A, cuối cùng tất cả các bản sao dữ liệu trên toàn cầu sẽ nhận được và áp dụng bản cập nhật này. Khi đó, tất cả người dùng, dù truy cập từ bất kỳ đâu, đều sẽ thấy cùng một phiên bản mới nhất của Trang web A. Đây chính là bản chất của “tính nhất quán cuối cùng”: sự nhất quán sẽ đạt được, nhưng không phải ngay lập tức mà cần một khoảng thời gian nhất định.

Tính nhất quán đọc sau ghi (ví dụ: Amazon S3)

Trong các hệ thống phân tán, đặc biệt là những hệ thống có dữ liệu được sao chép trên nhiều máy chủ hoặc trung tâm dữ liệu khác nhau, việc đảm bảo tính nhất quán của dữ liệu là một thách thức lớn. Một trong những mô hình nhất quán quan trọng cần xem xét là tính nhất quán đọc sau ghi (Read-after-write consistency).

Giả sử một tình huống phổ biến trong các hệ thống phân tán toàn cầu, như được minh họa trên bản đồ thế giới trong ví dụ trước, nơi dữ liệu được sao chép đến nhiều vị trí địa lý khác nhau để tăng cường khả năng sẵn sàng và giảm độ trễ truy cập cho người dùng trên toàn cầu. Điều gì sẽ xảy ra nếu một máy khách truy cập dữ liệu từ các bản sao khác nhau?

Hãy xem xét một sự kiện cụ thể: một người dùng thực hiện cập nhật “Trang web-A”. Ban đầu, bản cập nhật này có thể chỉ được ghi vào một bản sao dữ liệu cụ thể (ví dụ, bản sao gần với người dùng nhất). Nếu ngay sau đó, cùng một người dùng này cố gắng đọc lại “Trang web-A”, nhưng yêu cầu đọc của họ lại được định tuyến đến một bản sao khác chưa nhận được bản cập nhật mới nhất, họ có thể sẽ thấy phiên bản cũ của trang web. Điều này gây ra trải nghiệm không nhất quán và khó chịu cho người dùng, vì họ không thấy được chính những thay đổi mà mình vừa thực hiện.

Để giải quyết vấn đề này, các giao thức như “Đọc bản ghi của chính bạn” (Read Your Own Writes - RYOW) có thể được áp dụng. RYOW là một mô hình nhất quán đảm bảo rằng một khi một máy khách đã thực hiện một thao tác ghi, bất kỳ thao tác đọc nào tiếp theo của *chính máy khách đó* sẽ luôn phản ánh bản ghi mới nhất. Điều này có nghĩa là, ngay cả khi dữ liệu đã được sao chép đến nhiều vị trí, hệ thống sẽ đảm bảo rằng người dùng luôn thấy được phiên bản dữ liệu mà họ vừa cập nhật. Các hệ thống lưu trữ đối tượng lớn như Amazon S3 thường cung cấp mức độ nhất quán này cho một số loại hoạt động nhất định, giúp người dùng có trải nghiệm nhất quán hơn khi tương tác với dữ liệu của chính họ trong môi trường phân tán.

4.6. Amazon DynamoDB

4.6.1. Giới thiệu & Nguyên tắc thiết kế

Amazon DynamoDB

Amazon DynamoDB là một dịch vụ quản trị cơ sở dữ liệu NoSQL hoàn chỉnh được thiết kế để cung cấp hiệu suất cao và khả năng mở rộng linh hoạt. Khi tìm hiểu về DynamoDB, chúng ta cần nắm vững một số đặc điểm cốt lõi sau:

Đầu tiên, DynamoDB nổi bật với **giao diện đơn giản**. Đây là một kho dữ liệu dạng **khóa-giá trị (Key/value store)**, có nghĩa là dữ liệu được lưu trữ và truy xuất dựa trên các cặp khóa duy nhất và giá trị tương ứng. Sự đơn giản này giúp việc tương tác với cơ sở dữ liệu trở nên dễ dàng và hiệu quả.

Một nguyên tắc thiết kế quan trọng của DynamoDB là có thể giảm tính nhất quán mạnh (strong consistency) để đạt được tính sẵn sàng cao (availability)**. Điều này có nghĩa là trong một số trường hợp, khi đọc dữ liệu ngay sau khi ghi, có thể nhận được một phiên bản dữ liệu cũ hơn một chút. Tuy nhiên, đổi lại, hệ thống luôn đảm bảo khả năng hoạt động và phản hồi các yêu cầu, ngay cả khi có sự cố mạng hoặc lỗi nút.

DynamoDB được thiết kế như một kho dữ liệu **“luôn có thể ghi” (always writable)**. Điều này có nghĩa là các thao tác cập nhật (updates) sẽ không bị từ chối do lỗi hệ thống hoặc các thao tác ghi đồng thời. Hệ thống ưu tiên việc chấp nhận dữ liệu mới để đảm bảo tính sẵn sàng ghi cao nhất.

Để đạt được điều này, việc **giải quyết xung đột được thực hiện trong quá trình đọc thay vì ghi**. Khi có nhiều bản ghi đồng thời hoặc các bản sao dữ liệu không đồng bộ, DynamoDB sẽ không chặn các thao tác ghi. Thay vào đó, khi một yêu cầu đọc được thực hiện, hệ thống sẽ tổng hợp các phiên bản dữ liệu khác nhau và giải quyết xung đột để trả về phiên bản phù hợp nhất (ví dụ: phiên bản mới nhất hoặc phiên bản được ưu tiên theo quy tắc định trước).

Cuối cùng, DynamoDB hoạt động như **một cơ sở hạ tầng trong một miền quản trị duy nhất, nơi tất cả các nút được giả định là đáng tin cậy**. Điều này ngụ ý rằng hệ thống được triển khai và quản lý trong môi trường của Amazon, nơi các thành phần máy chủ (nút) được tin cậy và không có hành vi độc hại. Điều này giúp đơn giản hóa việc quản lý và đảm bảo an ninh trong phạm vi hệ thống.

Những cân nhắc khi thiết kế

Khi xây dựng một hệ thống phân tán quy mô lớn như Amazon DynamoDB, có một số nguyên tắc thiết kế cốt lõi cần được xem xét kỹ lưỡng để đảm bảo hiệu suất, độ tin cậy và khả năng mở rộng.

- **Khả năng mở rộng tăng dần (Incremental scalability)** Một trong những nguyên tắc cốt lõi khi thiết kế các hệ thống phân tán quy mô lớn là khả năng mở rộng một cách tăng dần. Điều này có nghĩa là hệ thống có thể được mở rộng bằng cách bổ sung từng phần nhỏ tài nguyên (ví dụ: các nút máy chủ) mà không yêu cầu một sự thay đổi kiến trúc lớn hay gây gián đoạn hoạt động. Nguyên tắc này đảm bảo tính liên tục của dịch vụ và tối ưu hóa chi phí đầu tư, cho phép hệ thống phát triển linh hoạt theo nhu cầu.
- **Tính đối xứng (Symmetry)** Nguyên tắc đối xứng quy định rằng mọi nút (node) trong hệ thống DynamoDB đều phải có cùng bộ trách nhiệm với các nút ngang hàng khác. Điều này đơn giản hóa việc quản lý hệ thống, tăng cường khả năng chịu lỗi (vì không có điểm lỗi đơn lẻ nào do một nút đặc biệt đảm nhiệm) và giúp việc mở rộng trở nên dễ dàng hơn khi các nút mới chỉ cần sao chép chức năng hiện có.
- **Tính phi tập trung (Decentralization)** Trong các hệ thống phân tán trước đây, việc kiểm soát tập trung thường dẫn đến lỗi đơn lẻ (single point of failure) là nguyên nhân của các sự cố ngừng hoạt động. Do đó, một mục tiêu quan trọng trong thiết kế DynamoDB là tối đa hóa tính phi tập trung. Điều này có nghĩa là không có một thành phần điều khiển trung tâm nào, giúp loại bỏ các điểm lỗi đơn lẻ và tăng cường đáng kể khả năng phục hồi cũng như tính sẵn sàng của hệ thống.
- **Tính không đồng nhất (Heterogeneity)** Tính không đồng nhất là một yếu tố thiết yếu, cho phép hệ thống tích hợp các nút mới với dung lượng hoặc hiệu suất cao hơn mà không yêu cầu phải nâng cấp đồng loạt tất cả các máy chủ hiện có. Điều này mang lại sự linh hoạt trong việc nâng cấp phần cứng, tối ưu hóa chi phí và cho phép hệ thống phát triển dần dần theo thời gian mà không gây gián đoạn.

4.6.2. Kiến trúc hệ thống & Cơ chế lỗi

Kiến trúc hệ thống

Kiến trúc hệ thống của một cơ sở dữ liệu phân tán như Amazon DynamoDB là yếu tố cốt lõi quyết định khả năng mở rộng, tính sẵn sàng cao và độ bền vững của dữ liệu. Để đạt được những mục tiêu này, DynamoDB đã triển khai một loạt các cơ chế phức tạp, đảm bảo rằng dữ liệu luôn sẵn sàng và an toàn ngay cả khi có sự cố xảy ra.

Phân vùng là một kỹ thuật thiết yếu trong các hệ thống cơ sở dữ liệu phân tán, cho phép chia nhỏ dữ liệu thành các phần nhỏ hơn và phân tán chúng trên nhiều máy chủ hoặc nút lưu trữ. Trong DynamoDB, quá trình phân vùng được thực hiện tự động dựa trên khóa phân vùng (partition key) mà người dùng định nghĩa. Khi dữ liệu được ghi vào DynamoDB, hệ thống sẽ sử dụng khóa phân vùng để xác định vị trí lưu trữ dữ liệu đó. Điều này mang lại nhiều lợi ích:

- **Khả năng mở rộng:** Cho phép hệ thống xử lý lượng dữ liệu và lưu lượng truy cập lớn hơn nhiều so với một máy chủ đơn lẻ.
- **Hiệu suất:** Các truy vấn có thể được thực hiện song song trên các phân vùng khác nhau, cải thiện đáng kể thời gian phản hồi.
- **Tính sẵn sàng:** Sự cố của một phân vùng hoặc một nút không ảnh hưởng đến toàn bộ hệ thống, giúp duy trì hoạt động liên tục.

Đảm bảo tính sẵn sàng cao cho các thao tác ghi là cực kỳ quan trọng để dữ liệu không bị mất và hệ thống luôn phản hồi. DynamoDB đạt được điều này thông qua cơ chế sao chép dữ liệu (replication). Mỗi mục dữ liệu (item) trong DynamoDB được sao chép tự động và lưu trữ trên nhiều nút lưu trữ khác nhau, thường là trên các Vùng sẵn sàng (Availability Zones) riêng biệt trong cùng một khu vực (Region).

Khi một thao tác ghi được thực hiện, dữ liệu sẽ được ghi vào nhiều bản sao. DynamoDB sử dụng một mô hình đồng thuận nội bộ để đảm bảo rằng thao tác ghi được xác nhận chỉ khi nó đã được ghi thành công vào một số lượng bản sao nhất định. Điều này giúp đảm bảo rằng ngay cả khi một hoặc hai bản sao gặp sự cố, dữ liệu vẫn an toàn và có thể được truy xuất từ các bản sao còn lại, duy trì tính bền vững và sẵn sàng của dữ liệu.

Trong một hệ thống phân tán, các lỗi tạm thời như sự cố mạng chập chờn, nút bị treo trong thời gian ngắn, hoặc quá tải tạm thời là điều không thể tránh khỏi. DynamoDB được thiết kế để xử lý các tình huống này một cách mạnh mẽ:

- **Thử lại (Retries):** Các yêu cầu gửi đến DynamoDB có thể được tự động thử lại nếu gặp lỗi tạm thời, thường đi kèm với chiến lược “exponential backoff” (tăng dần thời gian chờ giữa các lần thử lại) để tránh làm trầm trọng thêm tình trạng quá tải.
- **Thời gian chờ (Timeouts):** Hệ thống sử dụng các ngưỡng thời gian chờ để xác định khi nào một nút hoặc một yêu cầu không phản hồi, cho phép chuyển hướng yêu cầu đến các bản sao khỏe mạnh khác.

- **Tự phục hồi:** Nhờ cơ chế sao chép và phân tán, các lỗi tạm thời ở một nút thường không gây gián đoạn dịch vụ vì các yêu cầu có thể được chuyển hướng đến các bản sao khác đang hoạt động.

Các lỗi vĩnh viễn, chẳng hạn như hỏng ổ đĩa cứng, lỗi phần cứng nghiêm trọng của máy chủ, hoặc mất toàn bộ một nút lưu trữ, đòi hỏi một cơ chế phục hồi mạnh mẽ hơn. DynamoDB tự động phát hiện các lỗi vĩnh viễn này và kích hoạt quá trình phục hồi:

- **Phát hiện lỗi:** Hệ thống liên tục giám sát tình trạng của các nút.
- **Tái sao chép (Re-replication):** Khi một nút được xác định là đã hỏng vĩnh viễn, DynamoDB sẽ tự động tạo ra các bản sao mới của dữ liệu trên các nút khỏe mạnh khác để khôi phục lại số lượng bản sao mong muốn. Quá trình này diễn ra hoàn toàn tự động và minh bạch đối với người dùng, đảm bảo rằng dữ liệu luôn có đủ số lượng bản sao để duy trì tính bền vững và sẵn sàng.

Để một hệ thống phân tán hoạt động hiệu quả, điều quan trọng là phải biết được những nút nào đang hoạt động và những nút nào đã gặp sự cố. DynamoDB sử dụng các giao thức phức tạp để quản lý thành viên và phát hiện lỗi trong cụm:

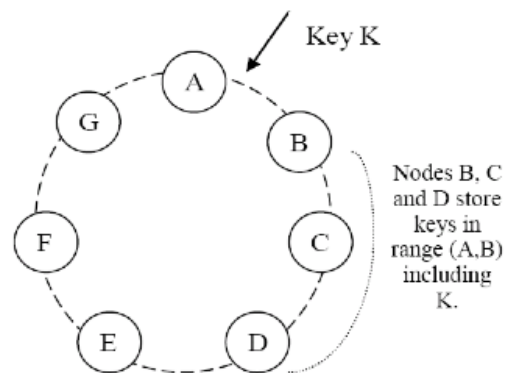
- **Quản lý thành viên:** Các nút trong cụm liên tục trao đổi thông tin với nhau để duy trì một danh sách cập nhật về các thành viên đang hoạt động. Khi một nút mới tham gia hoặc một nút rời đi (do lỗi hoặc bảo trì), thông tin này sẽ được cập nhật trên toàn hệ thống.
- **Phát hiện lỗi:** Các cơ chế như “heartbeat” (gửi tín hiệu định kỳ) hoặc giao thức “gossip” (trao đổi thông tin về trạng thái của các nút khác) được sử dụng để nhanh chóng xác định các nút không phản hồi. Khi một nút được xác định là đã hỏng, thông tin này sẽ được lan truyền khắp cụm, cho phép hệ thống thực hiện các hành động cần thiết như chuyển hướng yêu cầu hoặc kích hoạt quá trình tái sao chép.

Những cơ chế này cùng nhau tạo nên một kiến trúc mạnh mẽ, cho phép Amazon DynamoDB cung cấp một dịch vụ cơ sở dữ liệu NoSQL có khả năng mở rộng, độ bền và tính sẵn sàng vượt trội.

Thuật toán phân vùng

Trong các hệ thống phân tán quy mô lớn như Amazon DynamoDB, việc phân chia dữ liệu (partitioning) là một yếu tố cốt lõi để đảm bảo khả năng mở rộng (scalability), hiệu năng cao và tính sẵn sàng (availability). Một trong những kỹ thuật quan trọng được sử dụng cho mục đích này là **Consistent Hashing**.

Consistent Hashing là một kỹ thuật phân tán dữ liệu cho phép ánh xạ cả khóa dữ liệu (data keys) và các nút (nodes) lưu trữ dữ liệu vào cùng một không gian băm (hash space) cố định, thường được hình dung như một “vòng tròn” (ring). Khi một khóa dữ liệu được băm, nó sẽ rơi vào một điểm trên vòng tròn. Dữ liệu tương ứng với khóa đó sau đó sẽ được lưu trữ trên nút chịu trách nhiệm cho phân đoạn của vòng tròn mà khóa đó thuộc về. Ưu điểm nổi bật của Consistent Hashing là khi có một nút được thêm vào hoặc loại bỏ khỏi hệ thống, chỉ một phần nhỏ dữ liệu cần được di chuyển, thay vì phải phân phối lại toàn bộ dữ liệu, giúp giảm thiểu đáng kể chi phí vận hành và duy trì tính sẵn sàng.



Hình trên minh họa một vòng tròn Consistent Hashing với các nút từ A đến G. Khi một khóa K được băm và ánh xạ tới một vị trí trên vòng tròn, các nút chịu trách nhiệm lưu trữ khóa này sẽ được xác định. Ví dụ, như chú thích trên hình, các nút B, C và D có thể chịu trách nhiệm lưu trữ các khóa nằm trong phạm vi từ A đến B (bao gồm cả K), điều này thường ám chỉ cơ chế sao chép dữ liệu (replication) để đảm bảo tính bền vững và sẵn sàng.

Amazon DynamoDB được thiết kế như một **DHT (Distributed Hash Table) “zero-hop”**. Điều này có nghĩa là khi một yêu cầu truy vấn dữ liệu được gửi đến DynamoDB, hệ thống có thể xác định trực tiếp (hoặc gần như trực tiếp) nút lưu trữ dữ liệu đó mà không cần phải thông qua nhiều bước chuyển tiếp (hops) giữa các nút khác nhau. Khả năng “zero-hop” này giúp giảm độ trễ (latency) và tăng hiệu quả truy cập dữ liệu, vì không có sự chậm trễ do việc định tuyến yêu cầu qua nhiều tầng nút.

Tuy nhiên, một thách thức lớn trong việc triển khai Consistent Hashing và các hệ thống DHT là **mỗi nút trong hệ thống phải duy trì một cái nhìn cập nhật và chính xác về trạng thái của toàn bộ vòng tròn**. Điều này bao gồm việc biết được nút nào đang hoạt động, nút nào đã rời đi, và phân đoạn nào của vòng tròn mà mỗi nút đang chịu trách nhiệm. Việc đảm bảo tất cả các nút có cùng một “bản đồ” về hệ thống là cực kỳ quan trọng để định tuyến yêu cầu chính xác và duy trì tính nhất quán của dữ liệu. Câu hỏi đặt ra là: làm thế nào để các nút có thể duy trì cái nhìn cập nhật này một cách hiệu quả trong một môi trường phân tán luôn thay đổi? Đây là một vấn đề phức tạp đòi hỏi các cơ chế đồng thuận và phát hiện lỗi mạnh mẽ.

Các nút ảo

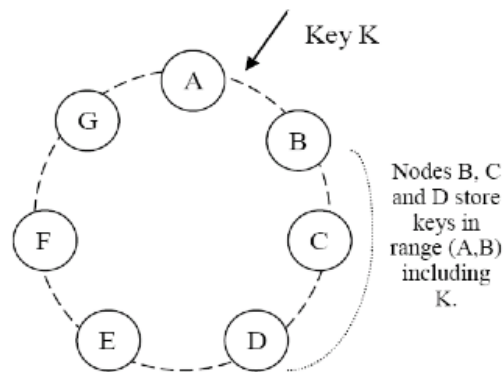
- Mỗi nút vật lý có thể chịu trách nhiệm cho nhiều hơn một nút ảo.
 - Mỗi nút vật lý sở hữu nhiều nút ảo.
 - Các máy chủ có hiệu năng cao hơn sẽ được gán nhiều nút ảo hơn.
 - Các nút ảo được phân tán trên toàn bộ vòng tròn (ring).
- Ưu điểm của việc sử dụng các nút ảo:
 - Nếu một nút trở nên không khả dụng, tải trọng mà nút đó đang xử lý sẽ được phân tán đều đặn cho các nút còn lại đang hoạt động.
 - Khi một nút trở lại trạng thái hoạt động, hoặc một nút mới được thêm vào hệ thống, nút mới này sẽ tiếp nhận một lượng tải trọng tương đương từ mỗi nút khác đang hoạt động.
 - Số lượng nút ảo mà một nút vật lý chịu trách nhiệm có thể được quyết định dựa trên dung lượng của nó, có tính đến sự không đồng nhất (heterogeneity) trong hạ tầng vật lý.

4.6.3. Quản lý dữ liệu & Đảm bảo tính khả dụng

Replication

Trong các hệ thống cơ sở dữ liệu phân tán hiện đại, đặc biệt là những hệ thống được thiết kế để đạt được tính khả dụng cao và bền vững như Amazon DynamoDB, khái niệm nhân bản dữ liệu (Replication) đóng vai trò cực kỳ quan trọng.

Về cơ bản, nhân bản dữ liệu là quá trình sao chép mỗi mục dữ liệu (data item) trên nhiều máy chủ (host) khác nhau. Số lượng máy chủ mà một mục dữ liệu được nhân bản tới được ký hiệu là N . N này không chỉ đơn thuần là một con số, mà nó đại diện cho một “danh sách ưu tiên” (preference list). Danh sách ưu tiên này chính là tập hợp các nút (node) chịu trách nhiệm lưu trữ một khóa (key) cụ thể. Điều này có nghĩa là, khi một khóa dữ liệu được ghi vào hệ thống, nó sẽ không chỉ được lưu trữ tại một vị trí duy nhất mà sẽ được sao chép và phân tán đến N nút trong danh sách ưu tiên của nó.



Để hình dung rõ hơn về cơ chế này, chúng ta có thể quan sát sơ đồ vòng tròn minh họa trong hình. Trong sơ đồ này, các nút A, B, C, D, E, F, G tạo thành một vòng tròn phân tán, đại diện cho các máy chủ trong hệ thống. Giả sử chúng ta có một khóa K cần được lưu trữ. Theo nguyên tắc phân tán và nhân bản, khóa K này sẽ được gán cho một nút cụ thể, ví dụ như nút A trong hình. Tuy nhiên, để đảm bảo tính khả dụng và bền vững của dữ liệu, khóa K này (hoặc các khóa trong phạm vi liên quan đến K) sẽ được nhân bản và lưu trữ trên các nút khác trong danh sách ưu tiên của nó. Như minh họa, các nút B, C và D được chỉ định là các nút chịu trách nhiệm lưu trữ các khóa trong phạm vi từ A đến B (bao gồm cả K). Điều này có nghĩa là, ngay cả khi nút A gặp sự cố hoặc không thể truy cập được, dữ liệu liên quan đến khóa K vẫn có thể được truy cập từ các nút B, C hoặc D, nhờ vào cơ chế nhân bản và danh sách ưu tiên này. Cơ chế này giúp tăng cường đáng kể khả năng chịu lỗi và tính bền vững của hệ thống.

Quorum

Trong các hệ thống phân tán, đặc biệt là các hệ thống cơ sở dữ liệu NoSQL như Amazon DynamoDB, việc quản lý dữ liệu trên nhiều bản sao (replica) là rất quan trọng để đảm bảo tính khả dụng và bền vững. Khái niệm Quorum đóng vai trò trung tâm trong việc cân bằng giữa tính nhất quán (consistency), tính khả dụng (availability) và hiệu suất (performance).

Để hiểu rõ hơn về Quorum, chúng ta cần nắm vững các định nghĩa sau:

- **N**: Đây là tổng số bản sao (replica) của mỗi cặp khóa/giá trị (key/value pair) trong hệ thống. Nói cách khác, khi một dữ liệu được lưu trữ, nó sẽ được nhân bản và phân tán trên N nút (node) khác nhau.
- **R**: Là số lượng nút tối thiểu phải tham gia vào một thao tác đọc (read operation) thành công. Để một yêu cầu đọc được coi là hoàn tất và trả về dữ liệu đáng tin cậy, hệ thống phải nhận được phản hồi từ ít nhất R bản sao.
- **W**: Là số lượng nút tối thiểu phải tham gia vào một thao tác ghi (write operation) thành công. Tương tự, để một yêu cầu ghi được xác nhận là thành công, hệ thống phải đảm bảo rằng dữ liệu đã được ghi thành công lên ít nhất W bản sao.

Một hệ thống được gọi là hoạt động theo mô hình Quorum (Quorum-like system) khi nó tuân thủ điều kiện cốt lõi sau:

$$R + W > N$$

Điều kiện này là chìa khóa để đảm bảo tính nhất quán mạnh mẽ (strong consistency) trong hệ thống phân tán. Khi tổng số bản sao cần thiết cho một thao tác đọc (R) và một thao tác ghi (W) lớn hơn tổng số bản sao (N), điều này đảm bảo rằng luôn có sự chồng lấp (overlap) giữa tập hợp các bản sao được đọc và tập hợp các bản sao được ghi. Nói cách khác, bất kỳ thao tác đọc nào cũng sẽ luôn chạm vào ít nhất một bản sao đã nhận được dữ liệu mới nhất từ thao tác ghi gần nhất, từ đó ngăn chặn việc đọc phải dữ liệu cũ (stale data).

Tuy nhiên, trong mô hình này, một điểm quan trọng cần lưu ý là độ trễ (latency) của một thao tác đọc (get) hoặc ghi (put) sẽ được quyết định bởi bản sao chậm nhất trong số R (hoặc W) bản sao đã phản hồi. Điều này có nghĩa là nếu một trong các bản sao cần thiết để hoàn thành thao tác bị chậm trễ, toàn bộ thao tác sẽ bị chậm theo. Vì lý do này, trong thực tế, R và W thường được cấu hình nhỏ hơn N. Việc này giúp cải thiện độ trễ và hiệu suất tổng thể của hệ thống, mặc dù có thể phải đánh đổi một phần với tính nhất quán mạnh mẽ tuyệt đối trong một số trường hợp cụ thể (ví dụ, nếu $R+W \leq N$, có thể có trường hợp đọc dữ liệu cũ nếu không có cơ chế bổ sung). Việc lựa chọn giá trị R và W phù hợp là một quyết định quan trọng, phụ thuộc vào yêu cầu cụ thể về tính nhất quán và hiệu suất của ứng dụng.

Lỗi tạm thời: Sloppy Quorum và Hinted Handoff

Trong bối cảnh các hệ thống phân tán quy mô lớn như Amazon DynamoDB, việc xử lý các lỗi tạm thời của các node là một yếu tố then chốt để đảm bảo tính khả dụng cao. Khi một node tạm thời không hoạt động hoặc không thể truy cập được, hệ thống cần có cơ chế để tiếp tục hoạt động mà không bị gián đoạn, tránh làm ảnh hưởng đến trải nghiệm người dùng.

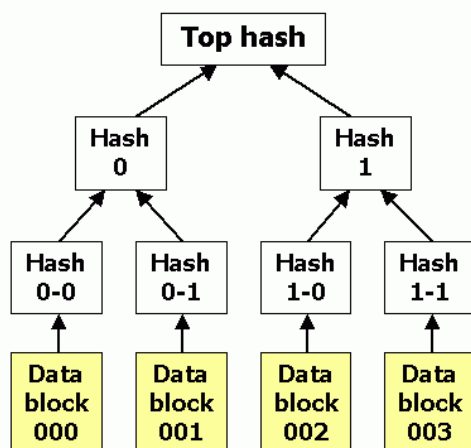
Hãy xem xét một ví dụ cụ thể với $N = 3$, tức là mỗi bản ghi dữ liệu được sao chép trên ba node khác nhau để đảm bảo tính bền vững và khả dụng. Giả sử trong quá trình ghi dữ liệu, node B tạm thời bị lỗi hoặc không thể truy cập được. Thay vì dừng lại và chờ đợi node B phục hồi, hệ thống sẽ áp dụng cơ chế **Sloppy Quorum**. Theo đó, bản sao dữ liệu (replica) sẽ được gửi tới một node khác đang hoạt động và có khả năng lưu trữ, ví dụ như node E. Điều này cho phép thao tác ghi hoàn tất thành công mà không bị ảnh hưởng bởi sự cố tạm thời của node B, đảm bảo rằng hệ thống luôn có thể ghi dữ liệu (“always writeable”). Đây là một điểm khác biệt quan trọng so với các hệ thống yêu cầu quorum nghiêm ngặt, nơi việc ghi có thể thất bại nếu các node được chỉ định không khả dụng.

Node E, khi nhận bản sao dữ liệu này, sẽ được “gợi ý” (hinted) rằng bản sao này thực chất thuộc về node B. Đây là cơ chế **Hinted Handoff**. Khi node B phục hồi và trở lại trạng thái hoạt động bình thường, node E sẽ chủ động chuyển giao bản sao dữ liệu mà nó đang tạm giữ cho node B. Quá trình này đảm bảo rằng dữ liệu cuối cùng sẽ được đặt đúng vào vị trí của nó, duy trì tính nhất quán và bền vững của hệ thống mà không làm gián đoạn các thao tác ghi.

Tóm lại, sự kết hợp giữa Sloppy Quorum và Hinted Handoff là một chiến lược hiệu quả để đối phó với các lỗi tạm thời trong hệ thống phân tán. Chúng cho phép hệ thống duy trì khả năng ghi dữ liệu liên tục, ngay cả khi một số node gặp sự cố, đồng thời đảm bảo rằng dữ liệu sẽ được đồng bộ hóa chính xác khi các node bị lỗi phục hồi. Đây là một trong những nguyên tắc cốt lõi giúp DynamoDB đạt được cam kết về tính khả dụng cao và khả năng chịu lỗi mạnh mẽ.

Đồng bộ hóa bản sao (Replica Synchronization)

Để đảm bảo tính nhất quán và khả dụng của dữ liệu trong các hệ thống phân tán như Amazon DynamoDB, việc đồng bộ hóa giữa các bản sao (replica) là vô cùng quan trọng. Một trong những kỹ thuật hiệu quả được sử dụng cho mục đích này là Cây Merkle.



Cây Merkle

Cây Merkle, hay còn gọi là cây băm (hash tree), là một cấu trúc dữ liệu dạng cây trong đó mỗi nút chứa một giá trị băm. Cấu trúc này được xây dựng theo nguyên tắc sau:

- **Nút lá (Leaf nodes):** Các nút ở tầng thấp nhất của cây là các nút lá. Mỗi nút lá chứa giá trị băm của một khối dữ liệu hoặc một khóa riêng lẻ. Ví dụ, như minh họa trong hình, các “Data block” (khối dữ liệu) ở tầng dưới cùng sẽ được băm để tạo ra các “Hash 0-0”, “Hash 0-1”, “Hash 1-0”, “Hash 1-1”.
- **Nút cha (Parent nodes):** Các nút ở các tầng cao hơn trong cây là các nút cha. Mỗi nút cha chứa giá trị băm được tính toán từ giá trị băm của các nút con trực tiếp của nó. Quá trình này tiếp tục cho đến khi đạt đến nút gốc (root node) hay còn gọi là “Top hash”, đại diện cho toàn bộ dữ liệu trong cây.

Ưu điểm của Cây Merkle

Cây Merkle mang lại nhiều lợi ích đáng kể trong việc kiểm tra tính toàn vẹn và đồng bộ hóa dữ liệu giữa các bản sao:

- **Kiểm tra độc lập từng nhánh:** Một trong những ưu điểm nổi bật nhất là khả năng kiểm tra tính nhất quán của từng nhánh cây một cách độc lập. Điều này có nghĩa là khi cần xác minh tính toàn vẹn của một phần dữ liệu, các nút không cần phải tải về và so sánh toàn bộ cây Merkle. Thay vào đó, chúng chỉ cần so sánh các giá trị băm của các nút tương ứng trên cùng một nhánh. Nếu hai nút gốc của cây Merkle từ hai bản sao khác nhau có giá trị băm khác nhau, hệ thống có thể đi sâu vào các nhánh con để

xác định chính xác vị trí của sự không nhất quán.

- **Giảm lượng dữ liệu truyền tải:** Khả năng kiểm tra độc lập này giúp giảm đáng kể lượng dữ liệu cần truyền tải trong quá trình kiểm tra sự không nhất quán giữa các bản sao. Thay vì phải truyền tải toàn bộ dữ liệu hoặc toàn bộ cây băm, hệ thống chỉ cần truyền tải các giá trị băm của các nút trên các nhánh có sự khác biệt. Khi phát hiện một sự khác biệt ở một nút băm cấp cao, hệ thống có thể đi sâu vào nhánh đó để xác định chính xác khối dữ liệu nào bị lỗi mà không cần phải truyền tải toàn bộ dữ liệu. Điều này đặc biệt quan trọng trong các hệ thống phân tán lớn, nơi việc tối ưu hóa băng thông mạng là yếu tố then chốt để duy trì hiệu suất và tính khả dụng.

Phiên bản hóa dữ liệu

Trong các hệ thống phân tán, việc quản lý và duy trì tính nhất quán của dữ liệu khi có nhiều bản sao (replicas) là một thách thức lớn. Khái niệm phiên bản hóa dữ liệu (data versioning) ra đời để giải quyết các vấn đề phát sinh từ sự không đồng bộ tạm thời giữa các bản sao này.

Cụ thể, có hai tình huống phổ biến cần được lưu ý:

- **Lệnh put () có thể hoàn tất trước khi cập nhật được áp dụng trên tất cả các bản sao:** Khi một ứng dụng thực hiện lệnh ghi dữ liệu (put ()), hệ thống có thể trả về thông báo thành công cho người gọi ngay cả khi bản cập nhật đó chưa được lan truyền và áp dụng trên tất cả các bản sao dữ liệu. Điều này thường được thực hiện để tối ưu hóa hiệu suất và độ trễ, nhưng có thể dẫn đến việc các bản sao khác nhau của cùng một dữ liệu có trạng thái không đồng nhất trong một khoảng thời gian ngắn.
- **Lệnh get () có thể trả về nhiều phiên bản của cùng một đối tượng:** Do sự không đồng bộ nêu trên hoặc do các thao tác ghi đồng thời, khi một ứng dụng thực hiện lệnh đọc dữ liệu (get ()), hệ thống có thể trả về nhiều hơn một phiên bản của cùng một đối tượng. Mỗi phiên bản này có thể đại diện cho một trạng thái khác nhau của dữ liệu tại các bản sao khác nhau, hoặc là kết quả của các cập nhật đồng thời mà hệ thống chưa thể hòa giải ngay lập tức.

Thách thức chính phát sinh từ việc này là sự tồn tại của các “nhánh phiên bản” (distinct version sub-histories) riêng biệt cho cùng một đối tượng dữ liệu. Mỗi nhánh phiên bản này đại diện cho một chuỗi các cập nhật độc lập, và hệ thống cần có cơ chế để hòa giải (reconcile) chúng, tức là hợp nhất các phiên bản này để đạt được một trạng thái nhất quán và chính xác.

Giải pháp phổ biến để giải quyết thách thức này là sử dụng **đồng hồ vector (vector clocks)**. Đồng hồ vector là một cơ chế cho phép hệ thống theo dõi mối quan hệ nhân quả (causality) giữa các phiên bản khác nhau của cùng một đối tượng. Bằng cách gán một đồng hồ vector cho mỗi phiên bản dữ liệu, hệ thống có thể xác định được thứ tự logic của các sự kiện cập nhật, ngay cả khi không có một đồng hồ toàn cục duy nhất. Điều này giúp hệ thống hiểu được phiên bản nào “xảy ra sau” phiên bản nào, từ đó hỗ trợ quá trình hòa giải các phiên bản một cách hiệu quả, đảm bảo tính nhất quán cuối cùng của dữ liệu trong môi trường phân tán.

Đồng hồ Vector (Vector Clock)

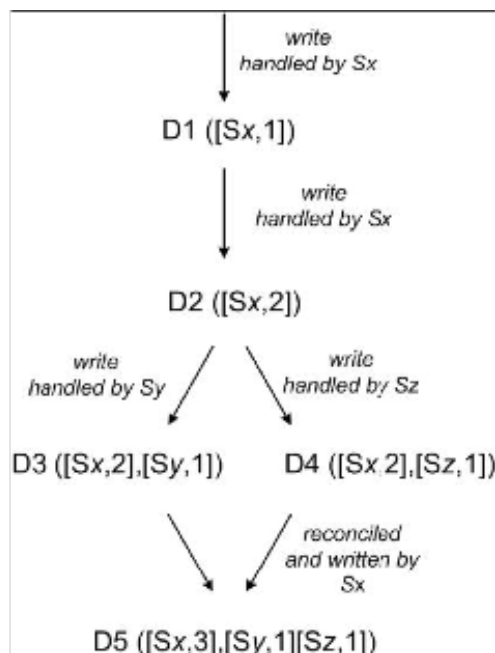
Trong các hệ thống phân tán, đặc biệt là khi quản lý dữ liệu và đảm bảo tính khả dụng như trong Amazon DynamoDB, việc theo dõi thứ tự các sự kiện và phiên bản dữ liệu là vô cùng quan trọng để giải quyết xung đột và duy trì tính nhất quán. Một trong những cơ chế hiệu quả được sử dụng cho mục đích này là **Đồng hồ Vector (Vector Clock)**.

Về bản chất, một đồng hồ vector là một danh sách các cặp (nút, bộ đếm). Ở đây, “nút” (node) thường đại diện cho một máy chủ hoặc một bản sao (replica) trong hệ thống phân tán, nơi dữ liệu được lưu trữ hoặc cập nhật. “Bộ đếm” (counter) là một giá trị số nguyên, tăng lên mỗi khi có một sự kiện hoặc cập nhật xảy ra trên nút đó.

Mỗi phiên bản của một đối tượng dữ liệu trong hệ thống đều được gắn kèm với một đồng hồ vector riêng. Điều này cho phép chúng ta theo dõi lịch sử và mối quan hệ nhân quả giữa các phiên bản khác nhau của cùng một đối tượng.

Quy tắc so sánh giữa hai đồng hồ vector là cốt lõi để xác định mối quan hệ giữa các phiên bản dữ liệu. Giả sử chúng ta có hai phiên bản của một đối tượng, phiên bản thứ nhất và phiên bản thứ hai, mỗi phiên bản đều có đồng hồ vector riêng của mình. Nếu tất cả các bộ đếm trong đồng hồ vector của phiên bản thứ nhất đều nhỏ hơn hoặc bằng các bộ đếm tương ứng trong đồng hồ vector của phiên bản thứ hai (tức là, với mỗi nút N có trong đồng hồ của phiên bản thứ nhất, giá trị bộ đếm của N trong đồng hồ của phiên bản thứ nhất nhỏ hơn hoặc bằng giá trị bộ đếm của N trong đồng hồ của phiên bản thứ hai, và đồng hồ của phiên bản thứ hai có thể có thêm các nút khác), thì chúng ta có thể kết luận rằng phiên bản thứ nhất là “tổ tiên” của phiên bản thứ hai. Điều này có nghĩa là phiên bản thứ nhất đã xảy ra trước và các thay đổi của nó đã được bao gồm hoặc kế thừa hoàn toàn trong phiên bản thứ hai. Trong trường hợp này, phiên bản thứ nhất có thể được “quên đi” hoặc loại bỏ, vì phiên bản thứ hai đã đại diện cho trạng thái mới nhất và đầy đủ hơn.

Ví dụ về Vector Clock



Hình trên trình bày một ví dụ cụ thể về cách thức hoạt động của vector clock trong việc theo dõi các phiên bản dữ liệu và mối quan hệ nhân quả giữa chúng trong một hệ thống phân tán.

Chúng ta hãy cùng phân tích từng bước trong sơ đồ:

- **D1 ([Sx,1]):** Đây là phiên bản dữ liệu ban đầu. Một thao tác ghi (write) được xử lý bởi nút (node) Sx, và bộ đếm (counter) tương ứng với Sx được khởi tạo là 1. Vector clock tại thời điểm này chỉ chứa một cặp (nút, bộ đếm) là [Sx, 1].
- **D2 ([Sx,2]):** Sau đó, một thao tác ghi khác lại được xử lý bởi cùng nút Sx. Bộ đếm của Sx trong vector clock được tăng lên 2. Vector clock hiện tại là [Sx, 2].
- **Từ D2, hệ thống phát sinh hai thao tác ghi đồng thời (concurrent writes):**
 - **D3 ([Sx,2],[Sy,1]):** Một thao tác ghi được xử lý bởi nút Sy. Phiên bản dữ liệu này kế thừa vector clock từ D2 ([Sx, 2]) và bổ sung thêm cặp [Sy, 1] do Sy là nút mới thực hiện ghi.
 - **D4 ([Sx,2],[Sz,1]):** Tương tự, một thao tác ghi khác được xử lý bởi nút Sz. Phiên bản này cũng kế thừa vector clock từ D2 ([Sx, 2]) và thêm cặp [Sz, 1]. Tại thời

điểm này, D3 và D4 là các phiên bản dữ liệu đồng thời, không có mối quan hệ nhân quả trực tiếp với nhau (chúng đều bắt nguồn từ D2).

- **D5 ([Sx,3],[Sy,1],[Sz,1]):** Đây là kết quả của quá trình hòa giải (reconciliation) giữa D3 và D4, sau đó có thêm một thao tác ghi được xử lý bởi nút Sx.

– Khi hòa giải D3 và D4, vector clock kết quả sẽ lấy giá trị lớn nhất cho mỗi nút từ cả hai vector clock. Cụ thể:

- * Đối với Sx: giá trị từ D3 là 2, từ D4 là 2. $\text{Max}(2,2) = 2$.
- * Đối với Sy: giá trị từ D3 là 1, từ D4 không có. $\text{Max}(1, \text{null}) = 1$.
- * Đối với Sz: giá trị từ D3 không có, từ D4 là 1. $\text{Max}(\text{null}, 1) = 1$.

– Như vậy, vector clock sau hòa giải (trước khi có ghi mới của Sx) sẽ là $[Sx, 2], [Sy, 1], [Sz, 1]$.

– Tiếp theo, một thao tác ghi được xử lý bởi nút Sx, làm tăng bộ đếm của Sx lên 3. Do đó, vector clock cuối cùng cho D5 là $[Sx, 3], [Sy, 1], [Sz, 1]$.

Một điểm quan trọng cần lưu ý trong các hệ thống thực tế là kích thước của vector clock có thể tăng lên theo thời gian. Để quản lý điều này, khi số lượng các cặp (nút, bộ đếm) trong vector clock đạt đến một ngưỡng nhất định (ví dụ, 10 cặp), cặp cũ nhất sẽ được loại bỏ khỏi vector clock. Điều này giúp kiểm soát bộ nhớ và hiệu suất mà không làm mất đi quá nhiều thông tin về mối quan hệ nhân quả gần đây.

4.6.4. Tổng kết & Đánh giá

Tóm tắt kỹ thuật

Trong quá trình thiết kế và vận hành các hệ thống phân tán quy mô lớn như Amazon DynamoDB, việc giải quyết các thách thức kỹ thuật phức tạp là điều cốt yếu để đảm bảo hiệu suất, tính sẵn sàng và độ bền. Dưới đây là tổng hợp các vấn đề chính, kỹ thuật được áp dụng và lợi ích mà chúng mang lại:

1. Phân vùng (Partitioning): Để quản lý lượng dữ liệu khổng lồ và phân phối tải hiệu quả, hệ thống cần có cơ chế phân vùng dữ liệu. Kỹ thuật được sử dụng là **Băm nhất quán (Consistent Hashing)**. Kỹ thuật này cho phép hệ thống đạt được **Khả năng mở rộng tăng dần (Incremental Scalability)**, nghĩa là có thể thêm hoặc bớt các nút một cách linh hoạt

mà không gây ra sự di chuyển dữ liệu lớn, giúp hệ thống thích ứng với sự thay đổi về quy mô một cách hiệu quả.

2. Tính sẵn sàng cao cho các thao tác ghi (High Availability for writes): Đảm bảo rằng các thao tác ghi luôn có thể thực hiện được, ngay cả khi một số nút gặp sự cố tạm thời, là một yêu cầu quan trọng. Hệ thống sử dụng **Đồng hồ vector (Vector clocks)** kết hợp với cơ chế **hòa giải trong quá trình đọc (reconciliation during reads)**. Ưu điểm nổi bật của phương pháp này là **kích thước phiên bản (version size) không phụ thuộc vào tốc độ cập nhật (update rates)**, giúp quản lý các phiên bản dữ liệu một cách hiệu quả mà không làm tăng đáng kể chi phí lưu trữ metadata.

3. Xử lý các lỗi tạm thời (Handling temporary failures): Trong môi trường phân tán, các lỗi tạm thời của nút là không thể tránh khỏi. Để đối phó với vấn đề này, hệ thống áp dụng kỹ thuật **Quorum lỏng lẻo (Sloppy Quorum)** và **chuyển giao gợi ý (hinted handoff)**. Sự kết hợp này **cung cấp tính sẵn sàng cao và đảm bảo độ bền (durability guarantee) ngay cả khi một số bản sao (replicas) không khả dụng**, bằng cách cho phép ghi dữ liệu vào các nút thay thế và sau đó chuyển giao dữ liệu về đúng vị trí khi nút gốc phục hồi.

4. Phục hồi từ các lỗi vĩnh viễn (Recovering from permanent failures): Khi một nút gặp lỗi vĩnh viễn hoặc các bản sao dữ liệu bị lệch pha theo thời gian, cần có cơ chế để đồng bộ hóa chúng. Kỹ thuật được sử dụng là **Chống phân rã (Anti-entropy) bằng cách sử dụng cây Merkle (Merkle trees)**. Kỹ thuật này giúp **đồng bộ hóa các bản sao khác biệt trong nền (in the background)** một cách hiệu quả, giảm thiểu lượng dữ liệu cần truyền tải và duy trì tính nhất quán của hệ thống.

5. Phát hiện thành viên và lỗi (Membership and failure detection): Để các nút trong hệ thống biết được trạng thái của nhau và phát hiện các nút bị lỗi, hệ thống sử dụng **Giao thức thành viên và phát hiện lỗi dựa trên Gossip (Gossip-based membership protocol and failure detection)**. Lợi ích chính của phương pháp này là **duy trì tính đối xứng (symmetry)** và **tránh có một kho lưu trữ tập trung (centralized registry) để lưu trữ thông tin thành viên và trạng thái hoạt động (liveness information) của nút**, từ đó loại bỏ điểm lỗi đơn và tăng cường khả năng chịu lỗi của hệ thống.

Tổng kết về DynamoDB

DynamoDB là một kho dữ liệu được thiết kế với tính sẵn sàng cao và khả năng mở rộng vượt trội, phục vụ cho nền tảng thương mại điện tử quy mô lớn của Amazon.com.

Hệ thống này đã chứng minh được hiệu quả trong việc xử lý các tình huống phức tạp như lỗi máy chủ, sự cố tại trung tâm dữ liệu và các vấn đề phân vùng mạng.

Một trong những ưu điểm nổi bật của DynamoDB là khả năng mở rộng linh hoạt và tăng

dẫn. Điều này cho phép các nhà cung cấp dịch vụ dễ dàng điều chỉnh quy mô tài nguyên (mở rộng hoặc thu hẹp) một cách chủ động, dựa trên biến động của tải yêu cầu thực tế.

Ngoài ra, DynamoDB còn cung cấp cho các nhà cung cấp dịch vụ quyền tùy chỉnh sâu rộng hệ thống lưu trữ của mình thông qua việc điều chỉnh các tham số quan trọng như N , R và W . Điều này giúp tối ưu hóa hiệu suất và độ tin cậy theo yêu cầu cụ thể của ứng dụng.

4.7. Xử lý truy vấn SQL cho dữ liệu lớn với Presto

4.7.1. Giới thiệu & Động lực

Lịch sử

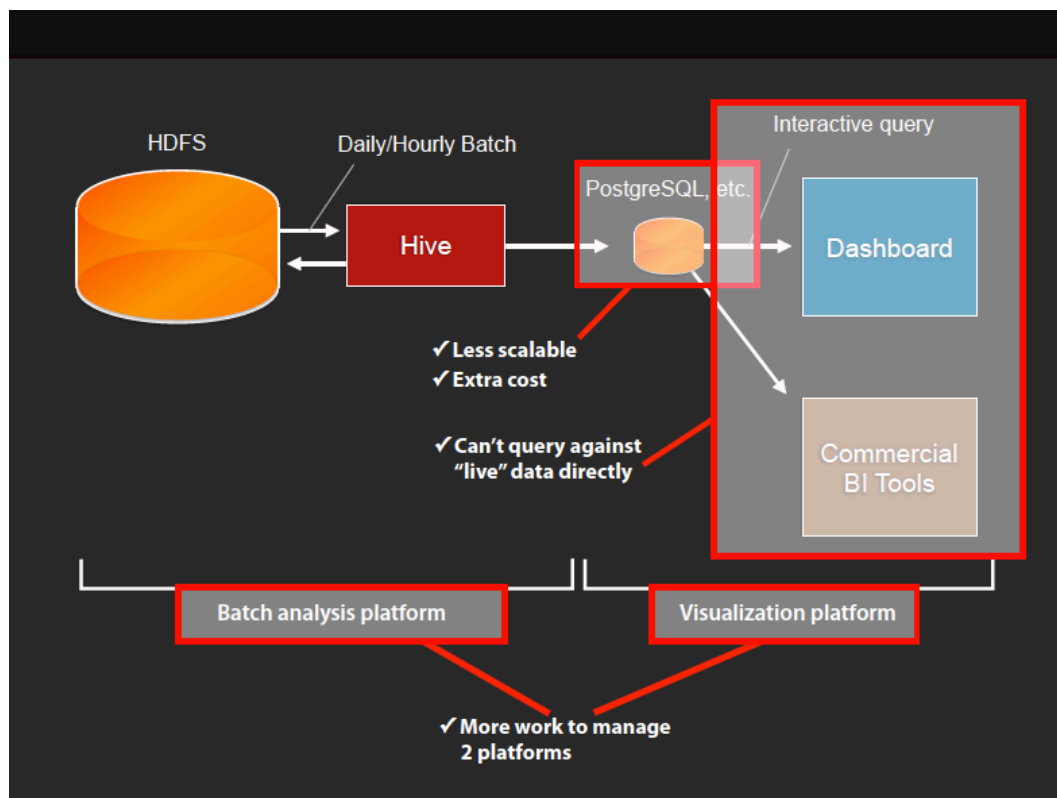
- **Mùa thu năm 2012:** Dự án được khởi động tại Facebook. Mục tiêu thiết kế là xử lý các truy vấn tương tác (interactive query) với tốc độ tương đương các kho dữ liệu thương mại (commercial data warehouse) và khả năng mở rộng (scalability) để đáp ứng quy mô dữ liệu khổng lồ của Facebook.
- **Mùa đông năm 2013:** Dự án được phát hành dưới dạng mã nguồn mở (open sourced).
 - Trong vòng 6 tháng, đã có hơn 30 người đóng góp (contributors).
 - Đáng chú ý, những người đóng góp này bao gồm cả các cá nhân và tổ chức bên ngoài Facebook.
- **Năm 2019:** Số lượng người đóng góp đã vượt mốc 300.

Động lực

Trong bối cảnh phân tích dữ liệu lớn hiện nay, chúng ta thường xuyên đối mặt với nhiều thách thức đáng kể, đặc biệt khi cần trích xuất thông tin chi tiết một cách nhanh chóng và hiệu quả từ lượng lớn dữ liệu. Những thách thức này chính là động lực thúc đẩy sự ra đời của các giải pháp mới, trong đó có Presto.

Một trong những vấn đề cốt lõi mà chúng ta thường gặp phải là khả năng trực quan hóa dữ liệu trực tiếp từ Hệ thống tệp phân tán Hadoop (HDFS) bằng cách sử dụng các bảng điều khiển (dashboards) hoặc công cụ Business Intelligence (BI). Lý do chính cho khó khăn này là Hive, mặc dù rất mạnh mẽ cho các tác vụ xử lý theo lô (batch processing), lại quá chậm để đáp ứng các truy vấn tương tác (interactive queries). Điều này khiến chúng ta không thể có phản hồi ngay lập tức khi cần khám phá dữ liệu. Hơn nữa, kết nối ODBC (Open Database Connectivity) với Hive đôi khi không khả dụng hoặc không ổn định, gây cản trở lớn cho việc tích hợp với các công cụ BI.

Để khắc phục hạn chế về tốc độ của Hive, một giải pháp phổ biến trước đây là lưu trữ kết quả của các tác vụ xử lý theo lô hàng ngày vào một cơ sở dữ liệu tương tác như PostgreSQL hoặc Redshift. Mục đích là để có được phản hồi nhanh chóng cho các truy vấn. Tuy nhiên, phương pháp này lại có nhiều nhược điểm: các cơ sở dữ liệu tương tác thường có chi phí cao hơn và khả năng mở rộng (scalability) kém hơn nhiều so với HDFS. Điều này dẫn đến việc phải quản lý hai nền tảng riêng biệt – một cho phân tích hàng loạt và một cho trực quan hóa – làm tăng thêm sự phức tạp và chi phí vận hành.



Thêm vào đó, không phải tất cả dữ liệu đều được lưu trữ trong HDFS. Nhiều tổ chức có dữ liệu phân tán trên nhiều hệ thống khác nhau như Cassandra, MySQL, hoặc các cơ sở dữ liệu thương mại khác. Để phân tích toàn bộ dữ liệu này, chúng ta thường phải sao chép chúng vào HDFS, một quy trình tốn thời gian và tài nguyên, đồng thời tạo ra các bản sao dữ liệu không cần thiết.

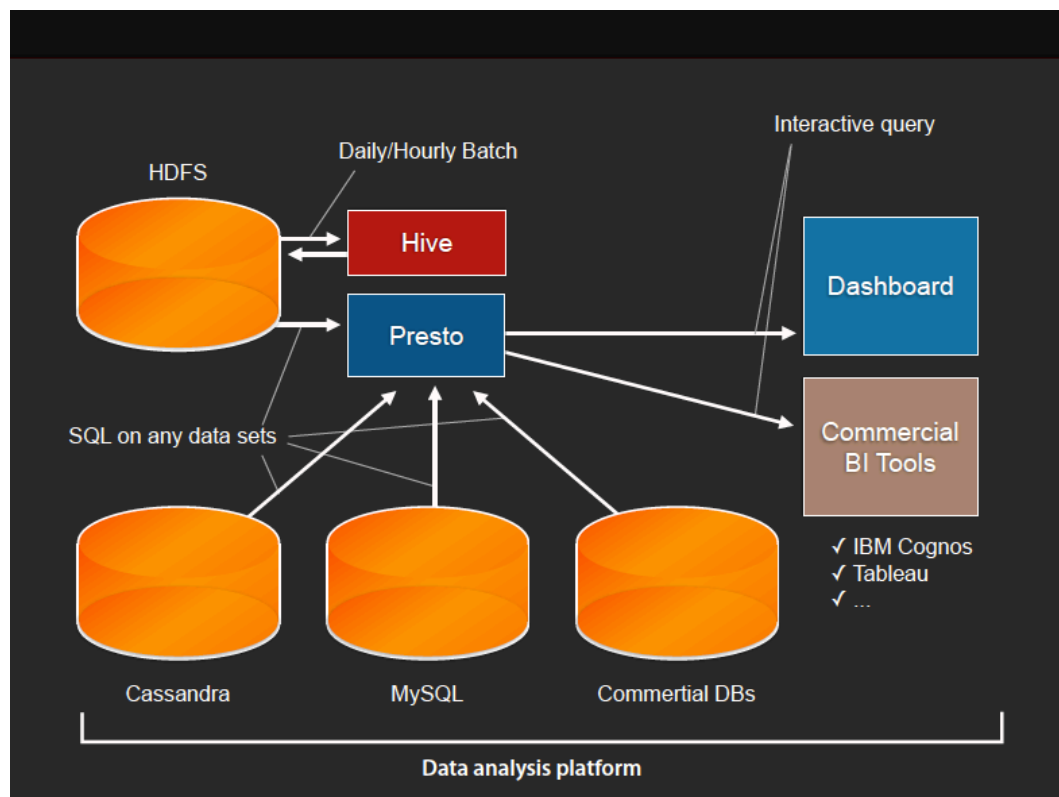
Những vấn đề trên đã tạo ra một nhu cầu cấp thiết về một công cụ có khả năng truy vấn dữ liệu lớn một cách nhanh chóng và dễ dàng, không chỉ trên HDFS mà còn trên nhiều nguồn dữ liệu khác nhau, nhằm mục đích trích xuất thông tin chi tiết một cách hiệu quả. Đây chính là động lực chính đằng sau sự phát triển của Presto, một giải pháp được thiết kế để giải quyết những thách thức này và mang lại khả năng phân tích dữ liệu tương tác trên quy mô lớn.

4.7.2. Tổng quan Presto

Presto có thể làm gì?

Presto là một công cụ truy vấn SQL phân tán mã nguồn mở, đã được Facebook triển khai và vận hành trong môi trường sản xuất từ năm 2013. Điều này chứng tỏ sự ổn định và hiệu quả của nó trong các hệ thống quy mô lớn. Presto cung cấp một giao diện tuân thủ chuẩn

ANSI SQL, giúp người dùng quen thuộc với SQL có thể dễ dàng sử dụng.



Một trong những khả năng nổi bật của Presto là cho phép truy vấn dữ liệu một cách tương tác, với thời gian phản hồi từ vài mili-giây đến vài phút. Điều này đặc biệt hữu ích cho các tác vụ phân tích dữ liệu tức thời hoặc khám phá dữ liệu. Tuy nhiên, cần lưu ý rằng Presto được thiết kế chủ yếu cho việc truy vấn tương tác, trong khi các công cụ như MapReduce và Hive vẫn đóng vai trò quan trọng và cần thiết cho các quy trình ETL (Extract, Transform, Load) dữ liệu lớn.

Presto cũng hỗ trợ việc truy vấn dữ liệu thông qua các công cụ Business Intelligence (BI) thương mại hoặc các bảng điều khiển (dashboards) phổ biến. Nhờ khả năng kết nối ODBC/JDBC đáng tin cậy, Presto có thể dễ dàng tích hợp vào hệ sinh thái phân tích dữ liệu hiện có của doanh nghiệp.

Điểm mạnh khác của Presto là khả năng truy vấn dữ liệu trên nhiều nguồn khác nhau. Nó có thể kết nối và truy vấn dữ liệu từ các hệ thống như Hive, HBase, Cassandra, hoặc thậm chí các cơ sở dữ liệu thương mại truyền thống. Khả năng này được thực hiện thông qua một cơ chế plugin linh hoạt, cho phép mở rộng để hỗ trợ nhiều nguồn dữ liệu mới.

Cuối cùng, Presto giúp tích hợp quá trình phân tích dữ liệu theo lô (batch analysis) và trực quan hóa vào một nền tảng phân tích dữ liệu duy nhất. Điều này tạo điều kiện thuận lợi

cho các nhà phân tích dữ liệu, giúp họ có thể thực hiện toàn bộ quy trình từ truy vấn đến hiển thị kết quả trên cùng một môi trường, nâng cao hiệu quả công việc.

Triển khai Presto

Presto, một công cụ truy vấn SQL phân tán, được phát triển và triển khai lần đầu tiên tại Facebook vào năm 2013. Ngay từ những ngày đầu, Presto đã chứng tỏ khả năng vượt trội trong việc xử lý dữ liệu quy mô lớn và đáp ứng nhu cầu phân tích nhanh chóng.

Tại Facebook, hệ thống Presto được triển khai trên nhiều khu vực địa lý khác nhau, cho thấy tính linh hoạt và khả năng phân tán của nó. Hệ thống này đã được mở rộng đáng kể, đạt tới quy mô 1.000 nút (nodes), minh chứng cho khả năng mở rộng (scalability) ấn tượng. Với hơn 1.000 nhân viên sử dụng Presto một cách tích cực, họ thực hiện hơn 30.000 truy vấn mỗi ngày. Điều này cho thấy Presto là một công cụ không thể thiếu trong các hoạt động phân tích dữ liệu hàng ngày của công ty. Đặc biệt, Presto có khả năng xử lý một lượng dữ liệu khổng lồ, lên tới 1 petabyte (PB) mỗi ngày, đáp ứng nhu cầu phân tích dữ liệu lớn của một tổ chức tầm cỡ như Facebook.

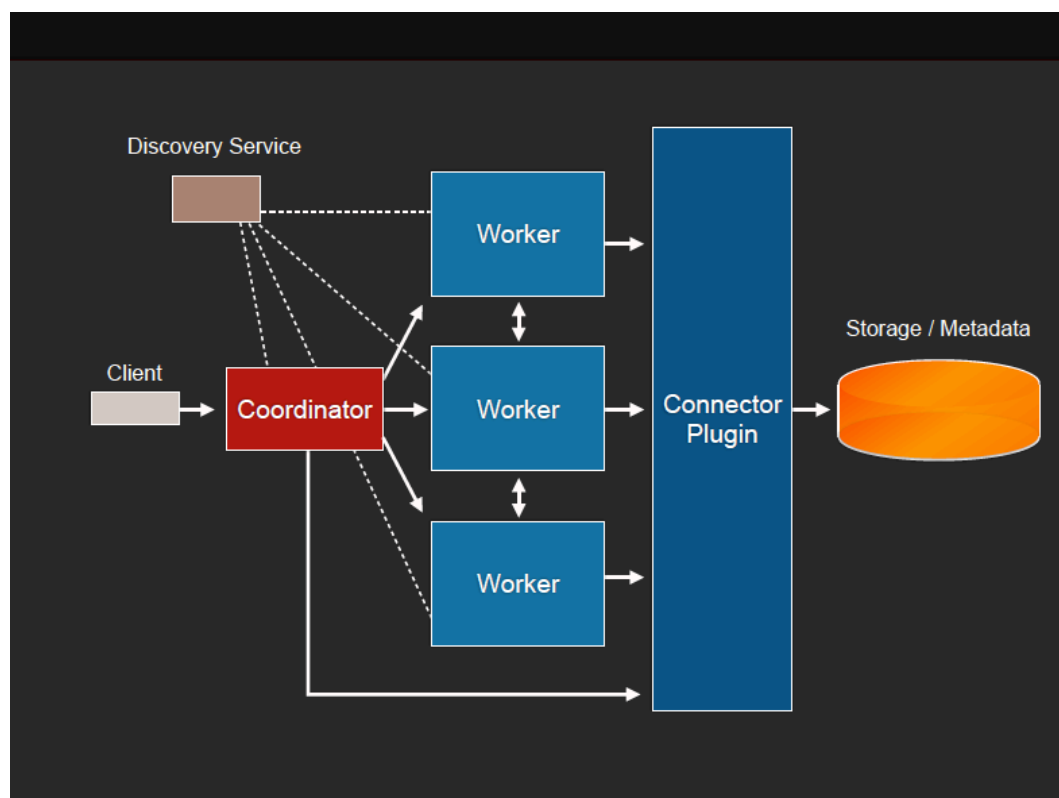
Ngoài Facebook, Presto còn được nhiều công ty công nghệ hàng đầu khác tin dùng và triển khai rộng rãi, bao gồm LinkedIn, Netflix, Airbnb, Teradata, và được tích hợp trong các dịch vụ đám mây như Amazon Athena. Sự chấp nhận rộng rãi này khẳng định vị thế của Presto như một giải pháp truy vấn dữ liệu lớn mạnh mẽ, hiệu quả và được cộng đồng đón nhận.

Presto architecture

Presto là một công cụ truy vấn SQL phân tán được thiết kế để chạy các truy vấn phân tích trên các nguồn dữ liệu lớn. Để hiểu rõ cách Presto hoạt động, chúng ta cần nắm vững kiến trúc của nó và luồng xử lý một truy vấn.

Các thành phần chính trong kiến trúc Presto:

1. **Client:** Đây là giao diện mà người dùng tương tác để gửi các truy vấn SQL đến Presto. Client có thể là một công cụ dòng lệnh (CLI), một ứng dụng BI (Business Intelligence), hoặc bất kỳ ứng dụng nào có khả năng kết nối và gửi truy vấn qua giao thức HTTP.
2. **Discovery Service:** Dịch vụ này đóng vai trò như một cơ chế đăng ký và khám phá cho các nút trong cụm Presto. Các Coordinator và Worker sẽ đăng ký với Discovery Service để các thành phần khác có thể tìm thấy chúng. Điều này đảm bảo tính linh hoạt và khả năng mở rộng của cụm.
3. **Coordinator:** Là trái tim của cụm Presto. Coordinator chịu trách nhiệm nhận các truy vấn từ Client, phân tích cú pháp, tối ưu hóa, lập kế hoạch thực thi truy vấn (query plan),



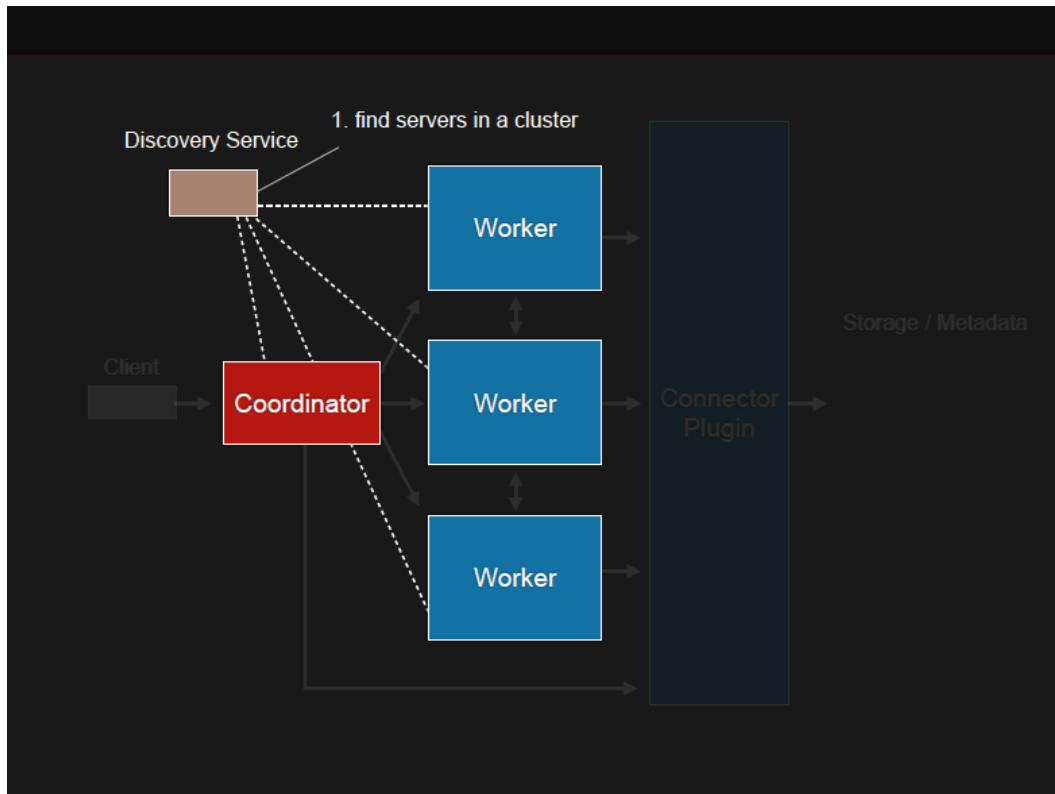
và phân phối các tác vụ (tasks) cho các Worker. Nó cũng theo dõi tiến độ thực thi của các tác vụ và quản lý tài nguyên.

4. **Worker:** Các Worker là các nút thực thi trong cụm Presto. Chúng nhận các tác vụ từ Coordinator, đọc dữ liệu từ các nguồn dữ liệu thông qua Connector Plugin, thực hiện các phép tính trong bộ nhớ (in-memory processing), và gửi kết quả trung gian hoặc cuối cùng trở lại Coordinator hoặc trực tiếp cho Client.
5. **Connector Plugin:** Đây là một thành phần mở rộng quan trọng của Presto, cho phép nó kết nối với nhiều hệ thống lưu trữ và cơ sở dữ liệu khác nhau (ví dụ: Hive, Cassandra, MySQL, PostgreSQL, Kafka, v.v.). Mỗi Connector Plugin cung cấp một giao diện để Presto có thể đọc siêu dữ liệu (metadata) như lược đồ bảng (table schema) và truy cập dữ liệu thực tế.
6. **Storage / Metadata:** Đại diện cho các nguồn dữ liệu thực tế mà Presto có thể truy vấn, cùng với siêu dữ liệu mô tả cấu trúc của dữ liệu đó.

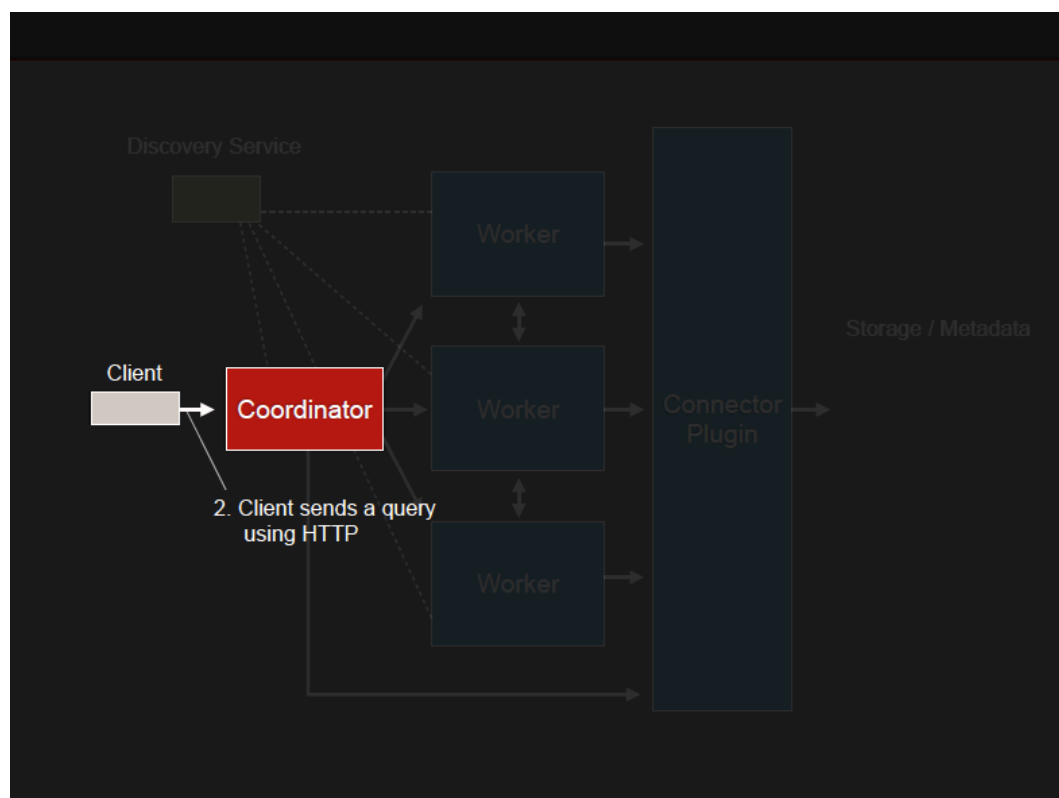
Luồng thực thi một truy vấn trong Presto:

Để minh họa cách các thành phần này phối hợp với nhau, chúng ta hãy xem xét luồng xử lý một truy vấn SQL:

1. **Khám phá dịch vụ:** Khi một cụm Presto khởi động, các Coordinator và Worker sẽ đăng ký với Discovery Service để các thành phần khác có thể tìm thấy và giao tiếp với chúng. Điều này đảm bảo rằng Coordinator biết được những Worker nào đang hoạt động và sẵn sàng nhận tác vụ.



2. **Gửi truy vấn:** Người dùng thông qua Client gửi một truy vấn SQL đến Coordinator. Việc giao tiếp này thường được thực hiện qua giao thức HTTP.
3. **Lập kế hoạch truy vấn:** Coordinator nhận truy vấn, phân tích cú pháp và ngữ nghĩa. Để xây dựng kế hoạch thực thi tối ưu, Coordinator sẽ tương tác với Connector Plugin tương ứng để lấy siêu dữ liệu (ví dụ: lược đồ bảng, thông tin phân vùng) từ nguồn dữ liệu. Dựa trên thông tin này, Coordinator tạo ra một kế hoạch truy vấn phân tán, chia nhỏ truy vấn lớn thành nhiều tác vụ nhỏ hơn.
4. **Phân phối tác vụ:** Coordinator sau đó phân phối các tác vụ đã được lập kế hoạch này đến các Worker có sẵn trong cụm. Mỗi tác vụ được giao cho một Worker cụ thể để thực hiện một phần của truy vấn.

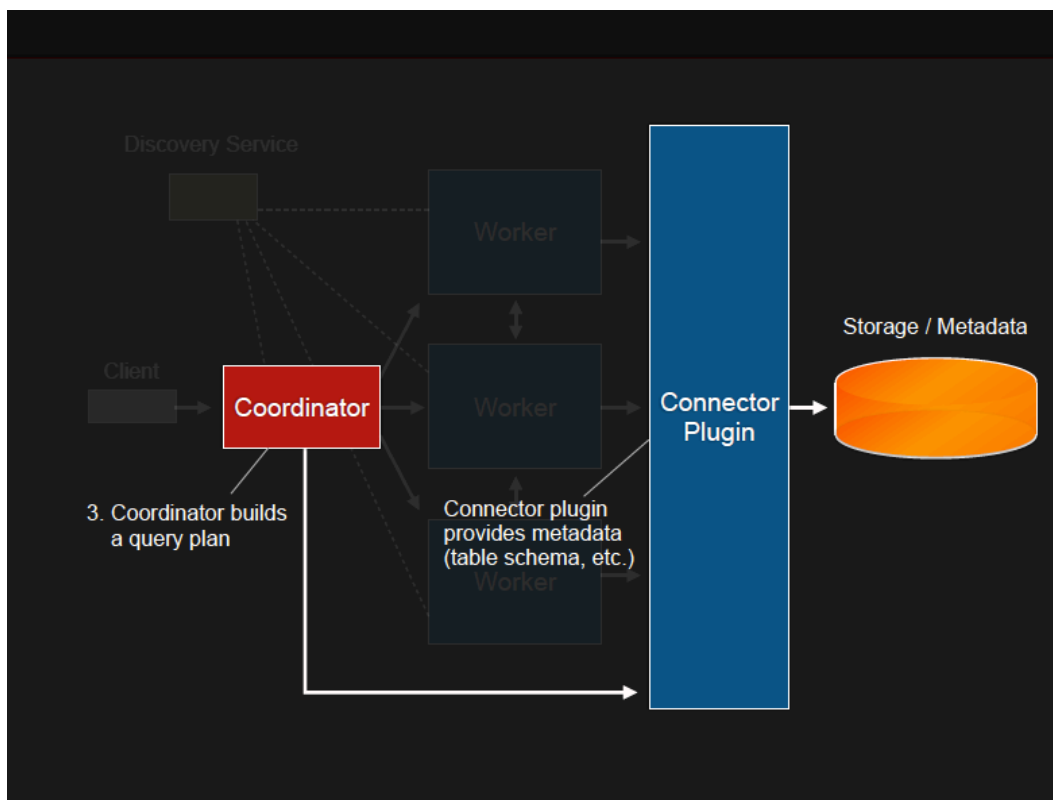


5. **Đọc dữ liệu:** Các Worker nhận tác vụ và sử dụng Connector Plugin để kết nối trực tiếp với nguồn dữ liệu và đọc các phần dữ liệu cần thiết cho tác vụ của mình.
6. **Thực thi tác vụ trong bộ nhớ:** Sau khi đọc dữ liệu, các Worker thực hiện các phép tính và xử lý dữ liệu hoàn toàn trong bộ nhớ (in-memory). Điều này giúp Presto đạt được hiệu suất truy vấn rất cao, tránh việc ghi dữ liệu trung gian xuống đĩa.
7. **Trả về kết quả:** Khi các Worker hoàn thành tác vụ của mình, chúng sẽ gửi kết quả trung gian hoặc kết quả cuối cùng trở lại Coordinator. Trong một số trường hợp, đặc biệt là với các truy vấn đơn giản hoặc khi kết quả đã được tổng hợp, Client có thể nhận kết quả trực tiếp từ một Worker. Coordinator sẽ tổng hợp các kết quả từ các Worker (nếu cần) và gửi kết quả cuối cùng trở lại Client.

Quá trình này cho thấy Presto tận dụng kiến trúc phân tán và xử lý trong bộ nhớ để cung cấp khả năng truy vấn dữ liệu lớn một cách nhanh chóng và hiệu quả.

Các Connector trong Presto

Trong kiến trúc của Presto, các *connector* đóng vai trò là những plugin thiết yếu, cho phép Presto tương tác với nhiều hệ thống lưu trữ và nguồn dữ liệu khác nhau. Về cơ bản,



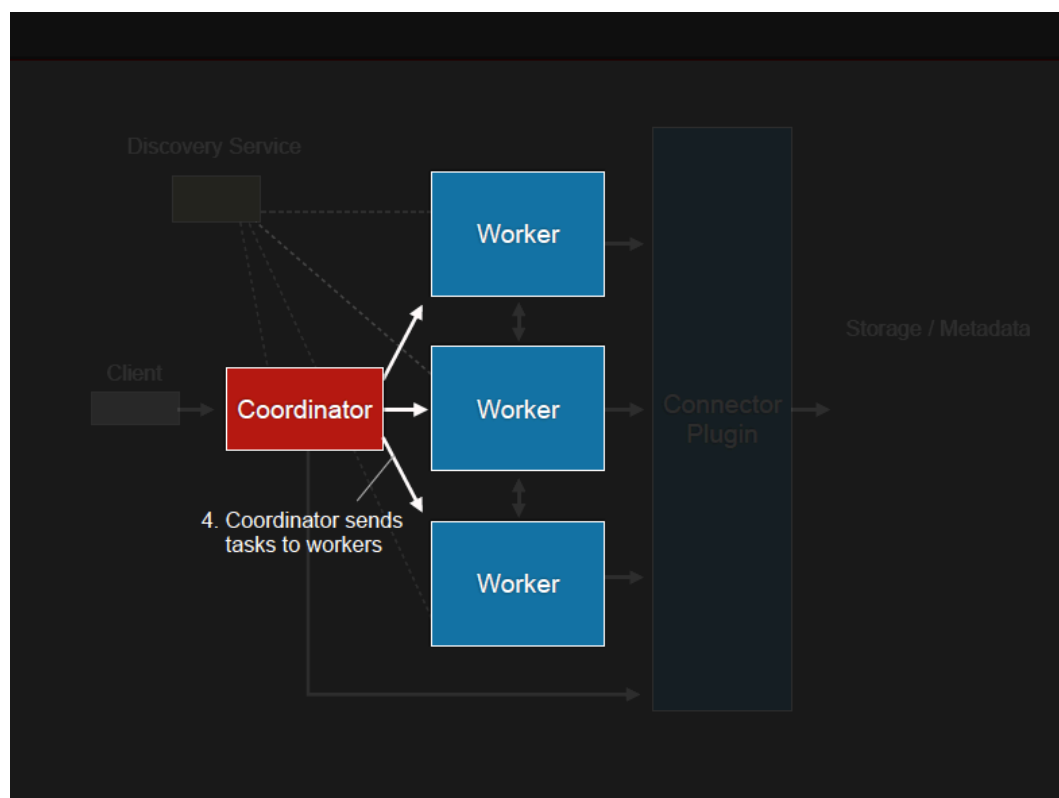
chúng được phát triển bằng ngôn ngữ Java, điều này mang lại sự linh hoạt và khả năng mở rộng cho hệ thống.

Chức năng chính của các connector là cung cấp quyền truy cập vào cả dữ liệu lưu trữ và siêu dữ liệu (metadata) của chúng. Cụ thể hơn:

- Chúng cung cấp lược đồ bảng (table schema) cho các *coordinator*. Điều này rất quan trọng để coordinator có thể lập kế hoạch và tối ưu hóa truy vấn một cách hiệu quả, vì nó cần biết cấu trúc của dữ liệu mà nó sẽ xử lý.
- Chúng cung cấp các hàng dữ liệu (table rows) cho các *worker*. Các worker là thành phần thực hiện công việc xử lý dữ liệu thực tế, do đó chúng cần dữ liệu thô từ nguồn lưu trữ thông qua các connector.

Presto hỗ trợ nhiều loại connector khác nhau để kết nối với các hệ thống dữ liệu phổ biến. Một số ví dụ điển hình bao gồm:

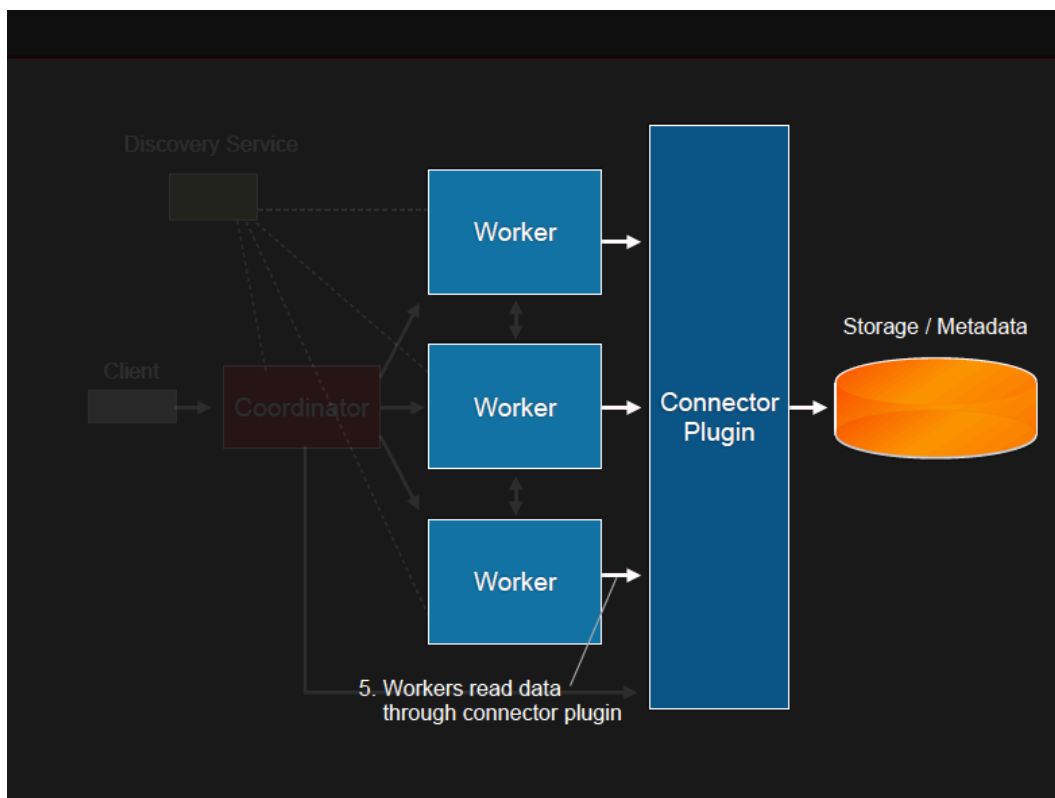
- **Hive connector:** Đây là một trong những connector quan trọng nhất, cho phép Presto truy vấn dữ liệu được lưu trữ trong HDFS và siêu dữ liệu được quản lý bởi Hive Metastore. Như có thể thấy trong sơ đồ minh họa, Hive connector đóng vai trò cầu



nối giữa các worker của Presto và hệ sinh thái Hive.

- **Cassandra connector:** Tương tự, connector này cho phép Presto kết nối và truy vấn dữ liệu từ cơ sở dữ liệu NoSQL Cassandra.
- **MySQL thông qua JDBC connector:** Mặc dù vẫn đang trong giai đoạn tiền phát hành (prerelease), sự tồn tại của connector này cho thấy khả năng mở rộng của Presto để kết nối với các cơ sở dữ liệu quan hệ truyền thống thông qua giao diện JDBC.
- Ngoài ra, Presto còn cho phép người dùng tự phát triển các connector của riêng mình, mở ra khả năng tích hợp với bất kỳ hệ thống dữ liệu nào mà doanh nghiệp đang sử dụng.

Điều đáng chú ý là Presto có khả năng sử dụng nhiều connector khác nhau trong cùng một truy vấn. Điều này có nghĩa là có thể kết hợp dữ liệu từ Hive, Cassandra và các nguồn khác trong một truy vấn duy nhất, mang lại sự linh hoạt vượt trội trong việc phân tích dữ liệu đa nguồn.



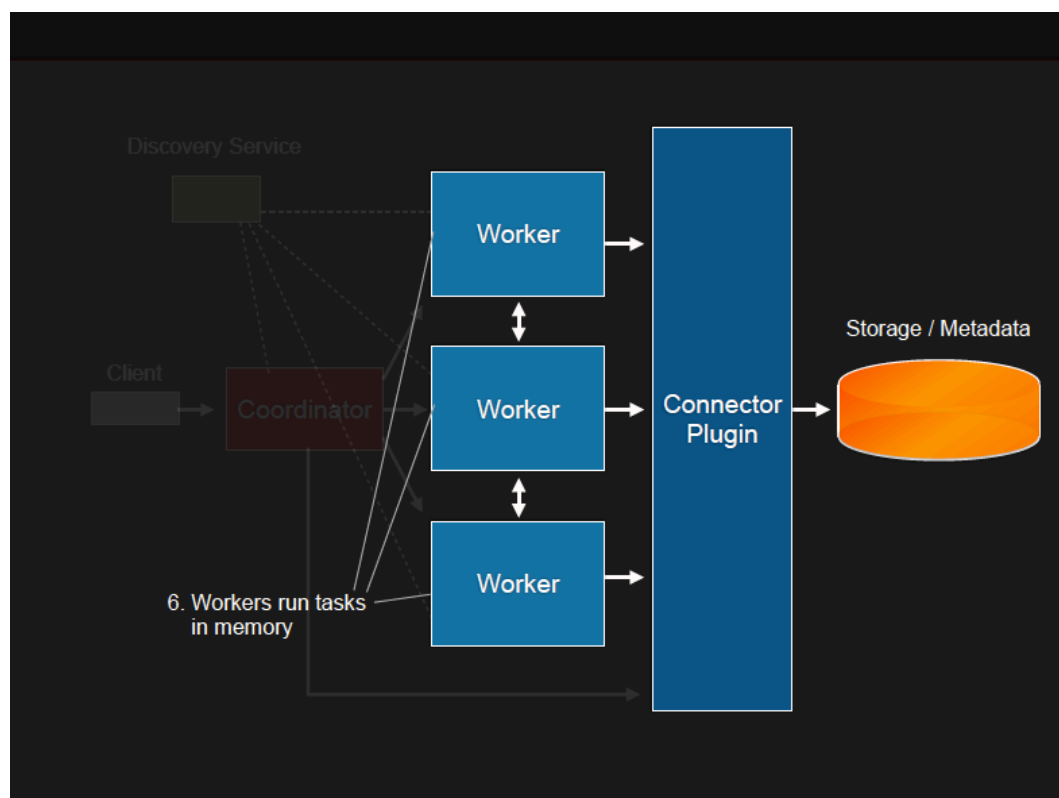
Kiến trúc phân tán

Presto được xây dựng trên một kiến trúc phân tán mạnh mẽ, cho phép nó xử lý các truy vấn dữ liệu quy mô lớn một cách hiệu quả. Kiến trúc này bao gồm ba loại máy chủ chính, mỗi loại đóng một vai trò quan trọng trong quá trình vận hành của hệ thống.

Đầu tiên, chúng ta có **Coordinator**. Đây là bộ não của cụm Presto, chịu trách nhiệm tiếp nhận các truy vấn SQL từ phía client. Sau khi nhận truy vấn, Coordinator sẽ thực hiện phân tích cú pháp, lập kế hoạch thực thi tối ưu, và phân phối các tác vụ nhỏ hơn đến các máy chủ Worker. Trong các môi trường yêu cầu tính sẵn sàng cao (High Availability - HA), có thể triển khai nhiều Coordinator để đảm bảo rằng hệ thống vẫn hoạt động liên tục ngay cả khi một Coordinator gặp sự cố.

Thứ hai là các **Worker**. Đây là những máy chủ thực hiện công việc xử lý dữ liệu thực tế. Các Worker nhận các tác vụ được phân công từ Coordinator, thực thi chúng, và trả về kết quả. Trong quá trình thực thi các truy vấn phức tạp, các Worker cũng có thể giao tiếp trực tiếp với nhau để trao đổi dữ liệu trung gian, tối ưu hóa hiệu suất.

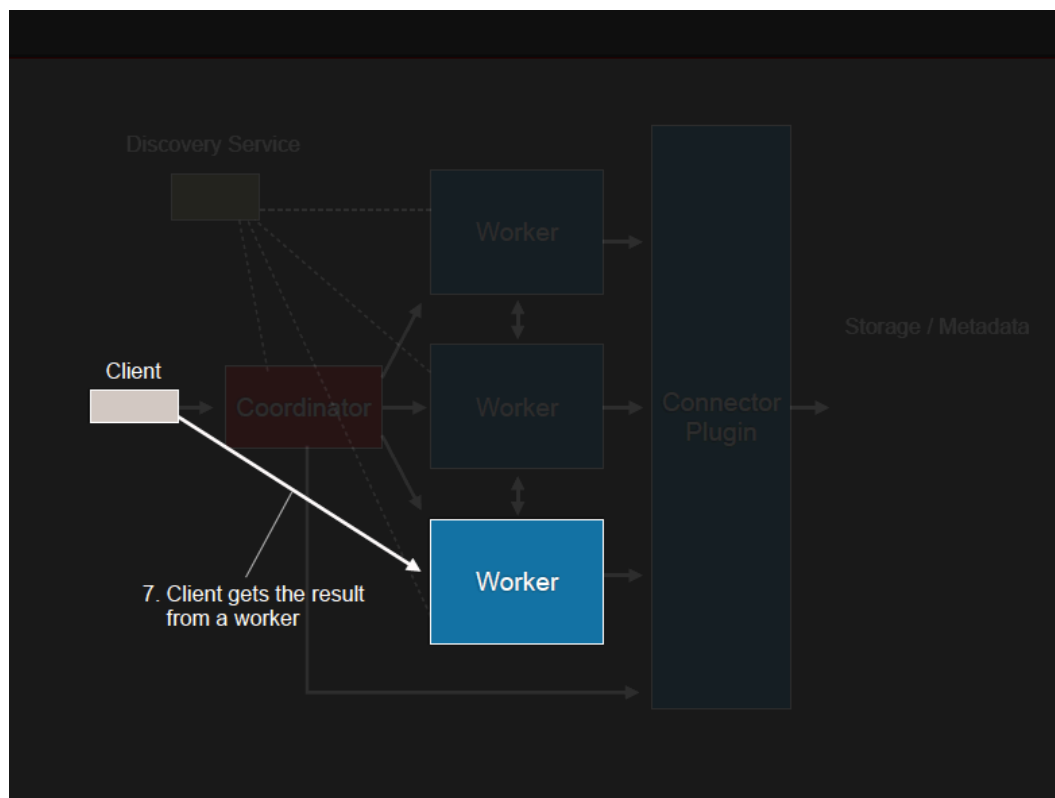
Cuối cùng, **Discovery Service** đóng vai trò như một dịch vụ đăng ký và khám phá. Khi các Coordinator và Worker khởi động, chúng sẽ đăng ký sự hiện diện của mình với Discovery



Service. Điều này cho phép các thành phần khác trong cụm dễ dàng tìm thấy và giao tiếp với nhau, đảm bảo sự phối hợp nhịp nhàng trong toàn bộ hệ thống.

Một điểm cốt lõi trong thiết kế của Presto là cách nó tương tác với dữ liệu. Presto **không phải là một hệ quản trị cơ sở dữ liệu** theo nghĩa truyền thống; nó không lưu trữ dữ liệu của riêng mình. Thay vào đó, Presto được thiết kế để cung cấp một giao diện SQL mạnh mẽ cho các kho dữ liệu hiện có. Việc này được thực hiện thông qua các **connector plugin**. Mỗi connector được phát triển để tương tác với một loại nguồn dữ liệu cụ thể (ví dụ: Hive, Cassandra, MySQL, Kafka, v.v.). Khi một truy vấn được thực thi, các Worker sẽ sử dụng connector phù hợp để truy xuất dữ liệu hoặc metadata trực tiếp từ các hệ thống lưu trữ bên ngoài. Điều này mang lại sự linh hoạt vượt trội, cho phép Presto truy vấn dữ liệu từ nhiều nguồn khác nhau bằng một ngôn ngữ SQL thống nhất.

Về giao thức giao tiếp, Presto sử dụng **HTTP kết hợp với JSON** cho việc tương tác với client. Giao thức này phổ biến và dễ dàng tích hợp, cho phép Presto kết nối với nhiều ứng dụng và ngôn ngữ lập trình khác nhau. Để hỗ trợ các nhà phát triển, Presto cung cấp các thư viện (language bindings) cho nhiều ngôn ngữ phổ biến như Ruby, Python, PHP, Java (thông qua JDBC), R, và Node.JS, giúp việc xây dựng các ứng dụng tương tác với Presto trở nên thuận tiện hơn.



4.7.3. Mô hình thực thi & Tối ưu hóa

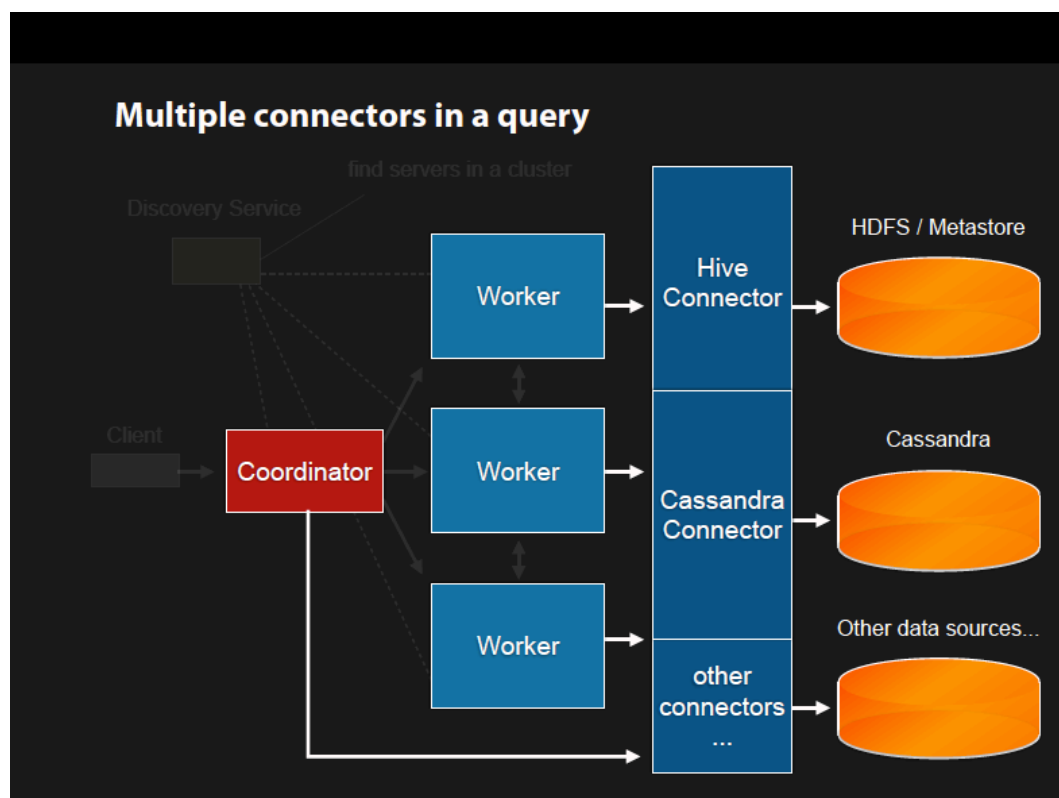
Mô hình thực thi của Presto

Presto là một công cụ truy vấn SQL phân tán được thiết kế để xử lý các tập dữ liệu lớn một cách nhanh chóng. Để hiểu rõ hơn về cách Presto hoạt động, chúng ta cần nắm vững mô hình thực thi của nó.

Đầu tiên, điều quan trọng cần nhấn mạnh là **Presto KHÔNG phải là MapReduce**. Mặc dù cả hai đều là các framework xử lý dữ liệu phân tán, Presto được tối ưu hóa cho các truy vấn tương tác và phân tích ad-hoc với độ trễ thấp, trong khi MapReduce thường được sử dụng cho các tác vụ xử lý theo lô (batch processing) với độ trễ cao hơn.

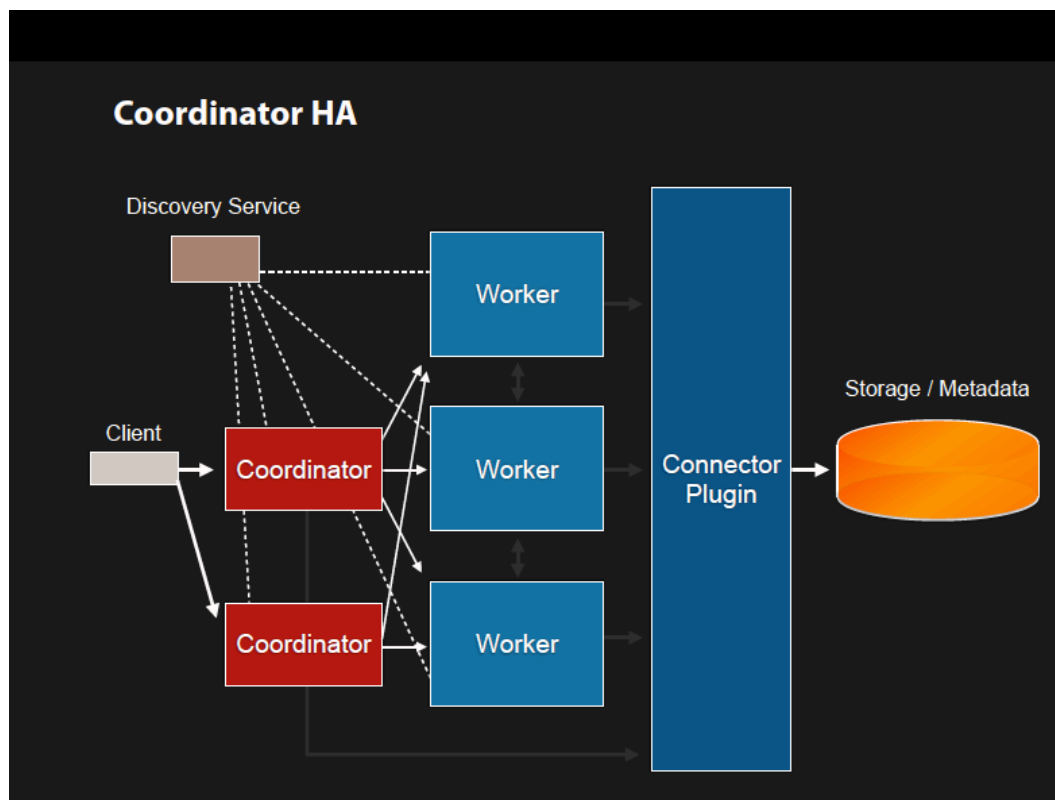
Kế hoạch truy vấn của Presto được xây dựng dựa trên **Đồ thị Định hướng Không Chu trình (DAG - Directed Acyclic Graph)**. Điều này có nghĩa là các bước thực thi của một truy vấn được biểu diễn dưới dạng một chuỗi các thao tác phụ thuộc lẫn nhau, không có vòng lặp. Cách tiếp cận này tương tự như các hệ thống xử lý dữ liệu hiện đại như Apache Tez hoặc các cơ sở dữ liệu xử lý song song lớn (MPP - Massively Parallel Processing) truyền thống, cho phép tối ưu hóa và thực thi hiệu quả hơn so với mô hình tuyến tính của MapReduce.

Vậy, một truy vấn được thực thi trong Presto như thế nào?



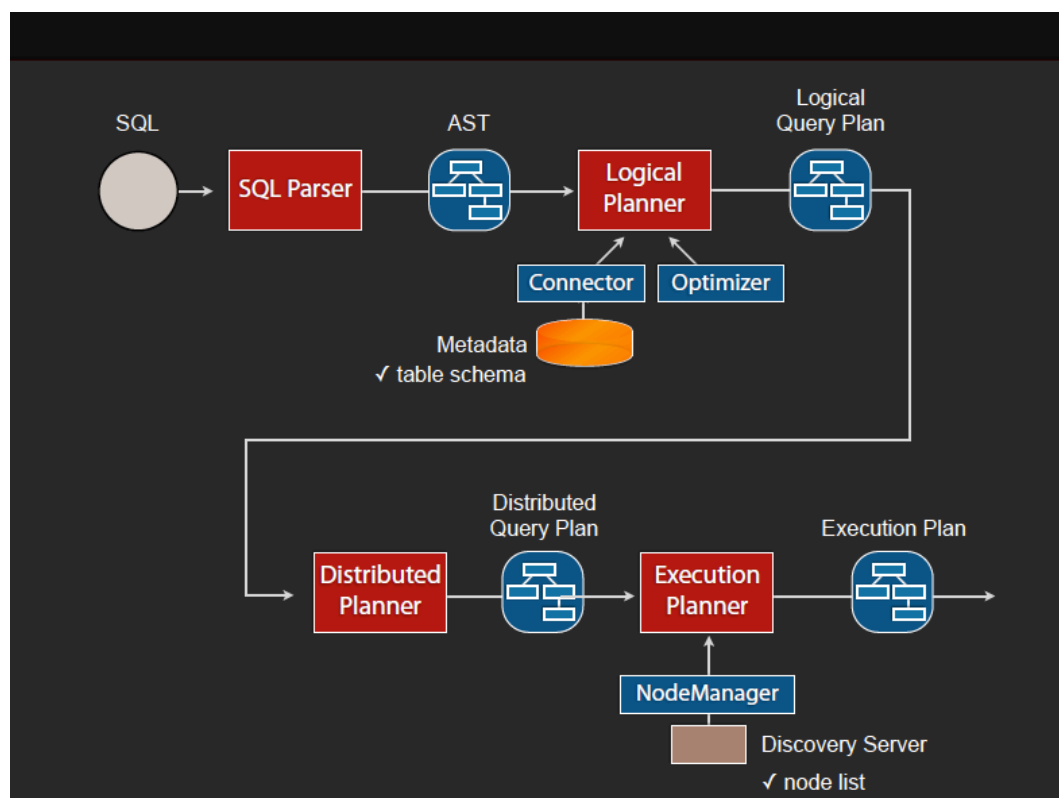
Quá trình thực thi một truy vấn trong Presto được phân chia rõ ràng giữa hai thành phần chính: **Coordinator** và **Workers**.

- **Coordinator (Bộ điều phối):** Coordinator là bộ não của Presto, chịu trách nhiệm tiếp nhận truy vấn, phân tích, lập kế hoạch và điều phối toàn bộ quá trình thực thi. Các chức năng chính của Coordinator bao gồm:
 - **SQL Parser (Bộ phân tích cú pháp SQL):** Khi một truy vấn SQL được gửi đến Presto, SQL Parser sẽ phân tích cú pháp của câu lệnh, chuyển đổi nó thành một Cây Cú pháp Trừu tượng (AST - Abstract Syntax Tree).
 - **Query Planner (Bộ lập kế hoạch truy vấn):** Dựa trên AST, bộ lập kế hoạch này sẽ tạo ra một kế hoạch truy vấn logic. Quá trình này bao gồm:
 - * **Logical Planner (Bộ lập kế hoạch logic):** Tạo ra một kế hoạch truy vấn logic ban đầu. Trong giai đoạn này, Logical Planner tương tác với các Connector để lấy siêu dữ liệu (metadata), chẳng hạn như lược đồ bảng (table schema), và sử dụng Optimizer để tối ưu hóa kế hoạch này, tạo ra một Kế hoạch Truy vấn Logic (Logical Query Plan) được tối ưu.
 - * **Distributed Planner (Bộ lập kế hoạch phân tán):** Chuyển đổi Kế hoạch



Truy vấn Logic thành một Kế hoạch Truy vấn Phân tán (Distributed Query Plan), xác định cách thức các thao tác sẽ được chia nhỏ và thực thi trên các Worker khác nhau.

- **Execution Planner (Bộ lập kế hoạch thực thi):** Cuối cùng, bộ lập kế hoạch thực thi sẽ tạo ra một Kế hoạch Thực thi (Execution Plan) chi tiết, bao gồm việc phân công các tác vụ cụ thể cho từng Worker. Quá trình này thường tương tác với NodeManager và Discovery Server để có được danh sách các node Worker đang hoạt động và sẵn sàng thực thi.
- **Workers (Các nút thực thi):** Các Worker là nơi thực hiện các tác vụ tính toán thực tế. Mỗi Worker nhận các tác vụ được giao từ Coordinator và xử lý dữ liệu.
 - **Task execution scheduler (Bộ lập lịch thực thi tác vụ):** Mỗi Worker có một bộ lập lịch riêng để quản lý và thực thi các tác vụ được giao. Các Worker sẽ đọc dữ liệu từ các nguồn dữ liệu được kết nối, thực hiện các phép toán (ví dụ: lọc, tổng hợp, kết nối) và trả về kết quả cho Coordinator hoặc chuyển tiếp dữ liệu trung gian cho các Worker khác theo Kế hoạch Thực thi.



Mô hình phân chia vai trò rõ ràng giữa Coordinator và Workers, cùng với việc sử dụng DAG cho kế hoạch truy vấn, cho phép Presto xử lý các truy vấn phức tạp một cách hiệu quả bằng cách phân chia công việc và tận dụng khả năng xử lý song song của một cụm máy tính.

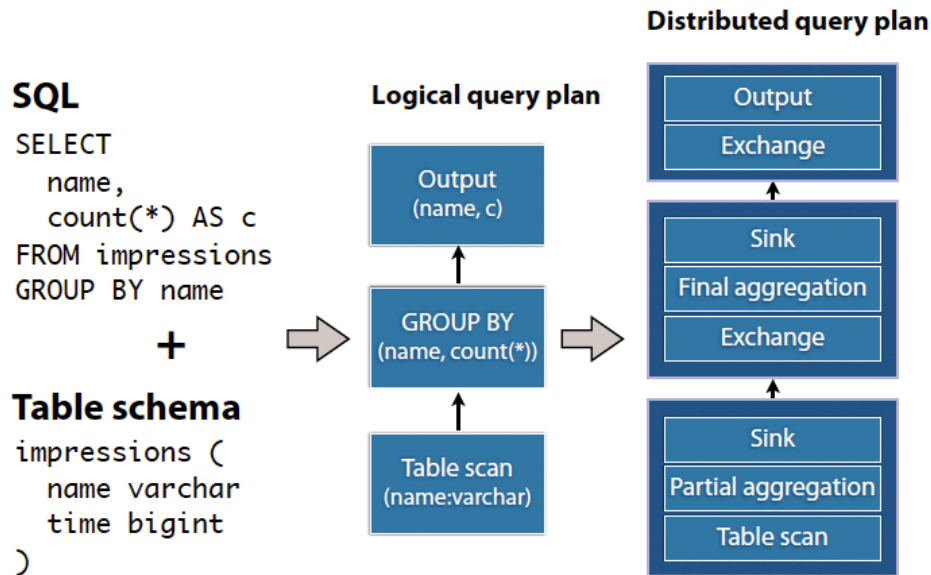
Query planner

Trong các hệ thống quản lý cơ sở dữ liệu phân tán như Presto, bộ phận Query Planner đóng vai trò cốt lõi. Nhiệm vụ chính của nó là chuyển đổi một câu lệnh SQL mà người dùng nhập vào thành một kế hoạch thực thi tối ưu, có thể chạy hiệu quả trên một kiến trúc phân tán. Quá trình này thường diễn ra qua hai bước chính: tạo kế hoạch truy vấn logic và sau đó là kế hoạch truy vấn phân tán.

Kế hoạch truy vấn logic (Logical query plan)

Khi Query Planner nhận được một câu lệnh SQL cùng với lược đồ bảng liên quan, bước đầu tiên là xây dựng một kế hoạch truy vấn logic. Kế hoạch này biểu diễn các phép toán cần thực hiện một cách trừu tượng, không phụ thuộc vào cách thức dữ liệu được lưu trữ hay xử lý phân tán. Ví dụ, với câu lệnh SQL `SELECT name, count(*) AS c FROM impressions GROUP BY name` và lược đồ bảng `impressions (name varchar, time bigint)`, kế hoạch logic sẽ bao gồm các bước tuần tự như sau:

Query Planner



- **Table scan (name:varchar):** Quét toàn bộ dữ liệu từ bảng impressions, tập trung vào cột name cần thiết cho phép nhóm.
- ****GROUP BY (name, count(*)):**** Thực hiện phép nhóm dữ liệu theo cột name và đếm số lượng bản ghi cho mỗi nhóm.
- **Output (name, c):** Trả về kết quả cuối cùng bao gồm tên (name) và số lượng đếm (c).

Kế hoạch logic này cung cấp một cái nhìn tổng quan về các phép toán cần thiết để giải quyết truy vấn.

Kế hoạch truy vấn phân tán (Distributed query plan)

Từ kế hoạch truy vấn logic, Query Planner sẽ tiến hành phân tán hóa các phép toán để tận dụng kiến trúc song song của hệ thống. Điều này liên quan đến việc chia nhỏ các phép toán lớn thành các giai đoạn (stages) nhỏ hơn và xác định cách dữ liệu được trao đổi giữa các giai đoạn này. Mục tiêu là tối ưu hóa hiệu suất bằng cách thực hiện các phần của truy vấn trên các worker khác nhau một cách đồng thời.

Hãy xem xét ví dụ về phép GROUP BY trong kế hoạch logic. Trong môi trường phân tán, phép toán này thường được chia thành hai phần:

- **Partial aggregation (Tổng hợp một phần):** Được thực hiện cục bộ trên từng worker,

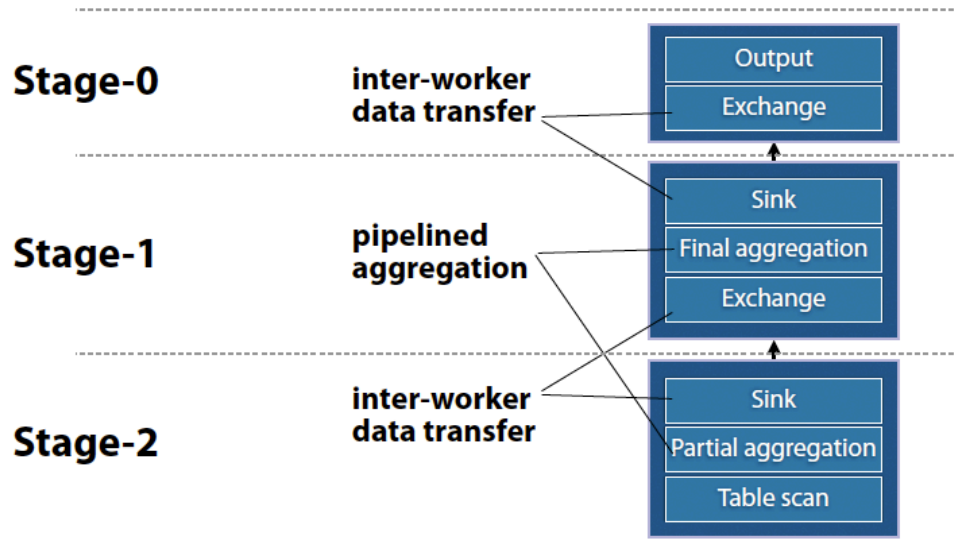
nơi dữ liệu được quét (Table scan) và tổng hợp sơ bộ. Ví dụ, mỗi worker sẽ đếm số lượng name mà nó đang xử lý. Kết quả của tổng hợp một phần này sau đó được gửi đến một điểm nhận dữ liệu (Sink).

- **Exchange:** Là cơ chế để di chuyển dữ liệu giữa các worker hoặc giữa các giai đoạn khác nhau. Dữ liệu từ các Sink của các worker thực hiện tổng hợp một phần sẽ được tập hợp lại thông qua Exchange.
- **Final aggregation (Tổng hợp cuối cùng):** Sau khi dữ liệu từ các worker khác nhau được tập hợp lại thông qua Exchange, phép tổng hợp cuối cùng sẽ được thực hiện để cho ra kết quả chính xác.
- **Output:** Cuối cùng, kết quả đã được tổng hợp hoàn chỉnh sẽ được trả về.

Các giai đoạn trong kế hoạch truy vấn phân tán (Stages)

Kế hoạch truy vấn phân tán được tổ chức thành các giai đoạn, mỗi giai đoạn thực hiện một tập hợp các phép toán và chịu trách nhiệm chuyển dữ liệu đến giai đoạn tiếp theo.

Query Planner - Stages



- **Stage-2:** Thường là giai đoạn đầu tiên trong luồng xử lý. Tại đây, các worker sẽ thực hiện Table scan để đọc dữ liệu từ nguồn. Ngay sau đó, Partial aggregation

được áp dụng để tổng hợp dữ liệu cục bộ. Kết quả của giai đoạn này được gửi đi thông qua Sink và được chuyển tiếp đến các worker ở giai đoạn tiếp theo thông qua `inter-worker data transfer`.

- **Stage-1:** Giai đoạn này nhận dữ liệu từ Stage-2 thông qua một Exchange. Tại đây, `Final aggregation` được thực hiện để hoàn tất quá trình tổng hợp dữ liệu từ tất cả các nguồn. Quá trình này có thể bao gồm `pipelined aggregation`, nơi dữ liệu được xử lý liên tục khi nó đến. Tương tự, kết quả của Stage-1 cũng được gửi đi qua Sink và chuyển tiếp đến Stage-0 thông qua `inter-worker data transfer`.
- **Stage-0:** Đây là giai đoạn cuối cùng. Nó nhận dữ liệu đã được tổng hợp hoàn chỉnh từ Stage-1 thông qua một Exchange và chịu trách nhiệm định dạng, sau đó trả về kết quả cuối cùng (Output) cho người dùng.

Việc phân chia thành các giai đoạn và sử dụng các cơ chế trao đổi dữ liệu giữa các worker là chìa khóa để Presto có thể xử lý các truy vấn phức tạp trên lượng lớn dữ liệu một cách hiệu quả và song song.

Các Giai Đoạn

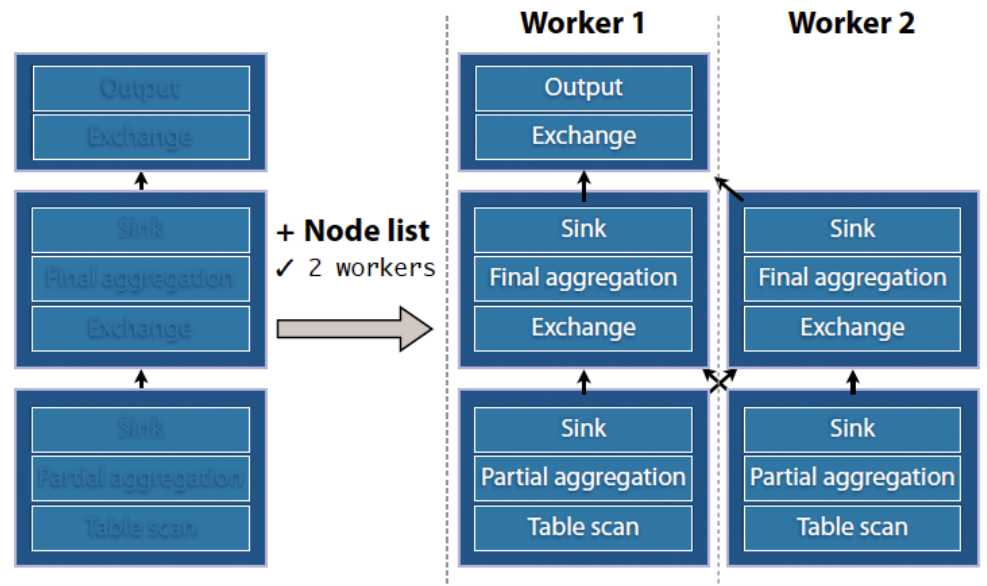
- Một giai đoạn là một phần của kế hoạch thực thi có thể được thực hiện song song trên các worker (máy chủ làm việc).
 - Các worker thực hiện cùng một tác vụ tính toán trên các tập dữ liệu đầu vào khác nhau.
 - Việc truyền dữ liệu đệm trong bộ nhớ (thường được gọi là `shuffles`) giữa các giai đoạn được thực hiện để cho phép trao đổi dữ liệu.
- Tuy nhiên, các thao tác shuffle thường làm tăng độ trễ, tiêu tốn bộ nhớ đệm đáng kể và gây ra chi phí CPU cao.

Execution Planner

Trong các hệ thống xử lý truy vấn phân tán như Presto, giai đoạn lập kế hoạch thực thi (Execution Planner) đóng vai trò vô cùng quan trọng. Nhiệm vụ chính của nó là chuyển đổi một kế hoạch truy vấn logic hoặc đã được tối ưu hóa thành một kế hoạch thực thi vật lý, có thể được phân phối và chạy hiệu quả trên một cụm các máy chủ (worker nodes). Hình sau minh họa quá trình này.

Phần bên trái của hình thể hiện một kế hoạch thực thi logic ban đầu. Đây là một chuỗi các bước xử lý dữ liệu theo thứ tự:

Execution Planner



- **Table Scan (Quét bảng):** Bước đầu tiên là đọc dữ liệu từ bảng nguồn.
- **Partial Aggregation (Tổng hợp cục bộ/từng phần):** Sau khi quét dữ liệu, một phép tổng hợp ban đầu được thực hiện. Ví dụ, nếu muốn tính tổng doanh thu theo khu vực, bước này có thể tính tổng doanh thu cho từng phần dữ liệu được đọc.
- **Sink (Điểm thu thập):** Đây là một điểm mà dữ liệu được thu thập hoặc chuyển tiếp.
- **Exchange (Trao đổi dữ liệu):** Đại diện cho việc di chuyển dữ liệu giữa các giai đoạn hoặc giữa các node trong hệ thống phân tán.
- **Final Aggregation (Tổng hợp cuối cùng):** Sau khi dữ liệu từ các phép tổng hợp cục bộ được tập hợp lại, phép tổng hợp cuối cùng sẽ được thực hiện để cho ra kết quả cuối cùng.
- **Output (Đầu ra):** Bước cuối cùng là xuất kết quả của truy vấn.

Kế hoạch logic này, mặc dù mô tả đúng các bước cần thiết, nhưng chưa chỉ ra cách thức chúng sẽ được thực thi trên một hệ thống phân tán. Đây chính là lúc Execution Planner phát huy tác dụng.

Execution Planner sẽ nhận kế hoạch logic này cùng với danh sách các node có sẵn để thực thi (trong ví dụ này là 2 worker). Dựa trên thông tin này, nó sẽ tạo ra một kế hoạch thực thi vật lý được phân tán, như được thể hiện ở phía bên phải của hình.

Chúng ta có thể thấy kế hoạch ban đầu đã được nhân bản và phân chia công việc cho hai worker: Worker 1 và Worker 2.

- Các bước Table Scan và Partial Aggregation được thực hiện song song trên cả hai worker. Điều này có nghĩa là dữ liệu nguồn đã được phân vùng, và mỗi worker sẽ xử lý một phần dữ liệu của riêng mình một cách độc lập.
- Sau khi thực hiện tổng hợp cục bộ, dữ liệu từ cả Worker 1 và Worker 2 cần được trao đổi (Exchange) và tập hợp lại để thực hiện Final Aggregation. Các mũi tên chéo giữa các worker ở giai đoạn này minh họa rõ nét quá trình trao đổi dữ liệu này. Đây thường được gọi là “shuffle” hoặc “redistribution” của dữ liệu, một bước cần thiết để đảm bảo tất cả dữ liệu liên quan đến một nhóm tổng hợp cụ thể đều nằm trên cùng một worker để tính toán chính xác.
- Cuối cùng, các bước Final Aggregation, Exchange và Output cũng được phân phối. Dữ liệu từ các phép tổng hợp cuối cùng trên cả hai worker sẽ được tập hợp lại và xuất ra kết quả cuối cùng.

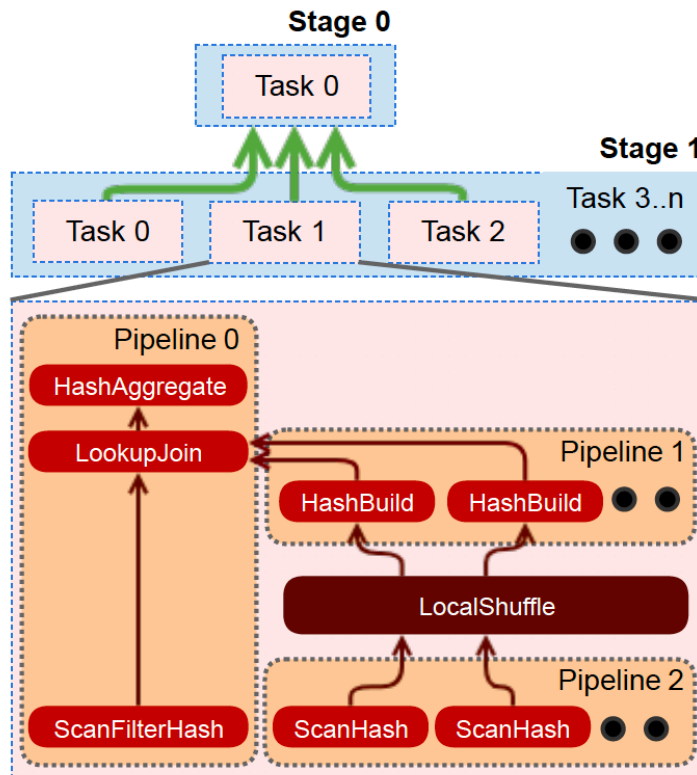
Tóm lại, Execution Planner có nhiệm vụ biến một kế hoạch truy vấn trừu tượng thành một tập hợp các tác vụ cụ thể, được phân bổ và điều phối trên các worker node, tận dụng tối đa khả năng xử lý song song của hệ thống phân tán. Điều này giúp Presto và các hệ thống tương tự đạt được hiệu suất cao khi xử lý các truy vấn trên lượng lớn dữ liệu.

Tasks

Trong mô hình thực thi của Presto, sau khi kế hoạch truy vấn được phân chia thành các giai đoạn (stages), điều phối viên (coordinator) sẽ phân phối các giai đoạn này tới các worker dưới dạng các đơn vị công việc có thể thực thi được, mà chúng ta gọi là các **tác vụ (tasks)**.

Mỗi tác vụ được thiết kế để thực thi trên một worker cụ thể và thuộc về một giai đoạn nhất định. Điều này có nghĩa là, tại một thời điểm, một worker sẽ chịu trách nhiệm thực thi một tác vụ duy nhất cho một giai đoạn cụ thể của truy vấn. Nguyên tắc “1 tác vụ / 1 worker / 1 giai đoạn” này là nền tảng cho việc phân phối công việc và cho phép các tác vụ được thực thi một cách song song trên nhiều worker, tối ưu hóa hiệu suất xử lý.

Hình trên cho thấy rằng mỗi worker (ví dụ Worker 1, Worker 2) có thể được giao nhiều tác vụ, nhưng mỗi tác vụ này lại thuộc về một giai đoạn khác nhau hoặc được xử lý độc lập.



Lập lịch

Để thực thi một truy vấn, bộ lập lịch (Scheduler) trong Presto đưa ra hai nhóm quyết định lập lịch quan trọng, nhằm tối ưu hóa hiệu suất và tài nguyên hệ thống.

Lập lịch Giai đoạn (Stage Scheduling)

Đây là nhóm quyết định đầu tiên, liên quan đến cách thức các giai đoạn (stages) của một truy vấn được điều phối. Có hai chiến lược chính:

- **Đồng thời (All-at-once):**

- Chiến lược này được thiết kế để giảm thiểu thời gian thực thi tổng thể (wall clock time) của truy vấn. Bộ lập lịch sẽ cố gắng lập lịch cho tất cả các giai đoạn của quá trình thực thi diễn ra đồng thời, tức là song song với nhau.
- Ưu điểm nổi bật của phương pháp này là dữ liệu được xử lý ngay khi có sẵn, không cần chờ đợi các giai đoạn trước hoàn tất hoàn toàn. Điều này giúp tăng tốc độ xử lý và phản hồi.
- Do đó, chiến lược “Đồng thời” đặc biệt phù hợp và mang lại lợi ích đáng kể cho các trường hợp sử dụng nhạy cảm về độ trễ (latency-sensitive use).

cases). Điển hình là các ứng dụng yêu cầu phản hồi nhanh như Phân tích tương tác (Interactive Analytics), Phân tích dành cho nhà phát triển/nhà quảng cáo (Developer/Advertiser Analytics), và Kiểm thử A/B (A/B Testing), nơi mà việc có kết quả nhanh chóng là yếu tố then chốt.

- **Theo pha (Phased):**

- Ngược lại với chiến lược “Đồng thời”, phương pháp “Theo pha” thực hiện lập lịch các giai đoạn một cách tuần tự hoặc theo một trình tự nhất định. Khi một giai đoạn được xác định là cần được lập lịch theo chính sách đã định, bộ lập lịch sẽ bắt đầu gán các tác vụ (tasks) cho giai đoạn đó tới các nút làm việc (worker nodes).
- Lợi ích chính của chiến lược “Theo pha” là khả năng cải thiện hiệu quả sử dụng bộ nhớ (memory efficiency). Bằng cách xử lý từng giai đoạn một hoặc theo nhóm nhỏ, hệ thống có thể quản lý tài nguyên bộ nhớ tốt hơn, tránh tình trạng quá tải khi xử lý các tập dữ liệu lớn.
- Điều này làm cho chiến lược “Theo pha” trở nên lý tưởng cho các trường hợp sử dụng Phân tích theo lô (Batch Analytics), nơi khối lượng dữ liệu lớn và việc tối ưu hóa tài nguyên bộ nhớ quan trọng hơn việc đạt được độ trễ thấp nhất.

Lập lịch tác vụ

Trong quá trình thực thi một truy vấn, các giai đoạn (stages) được phân loại thành hai nhóm chính để phục vụ cho việc lập lịch tác vụ hiệu quả: giai đoạn lá (leaf stages) và giai đoạn trung gian (intermediate stages). Việc phân loại này giúp bộ lập lịch tác vụ (task scheduler) đưa ra các quyết định tối ưu về vị trí và số lượng tác vụ cần được thực thi.

Giai đoạn lá (Leaf Stages)

Các giai đoạn lá thường là những giai đoạn đầu tiên trong luồng thực thi, chịu trách nhiệm đọc dữ liệu trực tiếp từ các nguồn lưu trữ. Đối với các giai đoạn này, bộ lập lịch tác vụ cần xem xét kỹ lưỡng các ràng buộc do mạng lưới và các trình kết nối (connectors) đặt ra khi gán tác vụ cho các nút worker.

- **Tận dụng tính cục bộ của dữ liệu:** Một trong những nguyên tắc quan trọng là ưu tiên đặt các tác vụ trên các nút worker cùng vị trí với các nút lưu trữ dữ liệu (co-located workers and storage nodes). Điều này nhằm mục đích tận dụng tối đa tính cục bộ của dữ liệu (data locality), giảm thiểu việc truyền dữ liệu qua mạng. Khi dữ liệu xử lý trực tiếp tại nơi lưu trữ, chi phí I/O mạng sẽ giảm đáng kể, từ đó cải thiện hiệu suất thực thi tổng thể.

- **Ràng buộc cách tổ chức dữ liệu của trình kết nối:** Ngoài ra, các ràng buộc về cách tổ chức dữ liệu của trình kết nối (Connector Data Layout constraints) cũng được tính đến. Mỗi trình kết nối có thể có cách tổ chức và truy cập dữ liệu riêng biệt (ví dụ: dữ liệu được phân vùng, nén, hoặc lưu trữ theo định dạng cụ thể). Bộ lập lịch cần đảm bảo rằng các tác vụ được gán phù hợp với cấu trúc này để việc đọc dữ liệu diễn ra hiệu quả nhất, tránh các thao tác không cần thiết hoặc kém hiệu quả.

Giai đoạn trung gian (Intermediate Stages)

Ngược lại với giai đoạn lá, các giai đoạn trung gian xử lý dữ liệu được tạo ra bởi các giai đoạn trước đó, không còn phụ thuộc trực tiếp vào vị trí lưu trữ ban đầu của dữ liệu nguồn.

- **Linh hoạt trong việc đặt tác vụ:** Các tác vụ cho giai đoạn trung gian có thể được đặt trên bất kỳ nút worker nào. Điều này mang lại sự linh hoạt lớn hơn cho bộ lập lịch, cho phép nó phân phối tải công việc một cách cân bằng hơn trên toàn bộ cụm. Dữ liệu mà các giai đoạn này xử lý thường đã được di chuyển hoặc được tạo ra trong bộ nhớ bởi các tác vụ upstream, do đó, ràng buộc về tính cục bộ dữ liệu ban đầu không còn là yếu tố quyết định.
- **Quyết định số lượng tác vụ:** Mặc dù có sự linh hoạt trong việc đặt tác vụ, công cụ thực thi vẫn cần quyết định số lượng tác vụ được lập lịch cho mỗi giai đoạn trung gian. Quyết định này rất quan trọng để tối ưu hóa mức độ song song (parallelism) và đảm bảo rằng tài nguyên được sử dụng hiệu quả mà không gây ra tình trạng quá tải hoặc thiếu hụt tài nguyên, từ đó ảnh hưởng đến hiệu suất tổng thể của truy vấn.

Lập lịch Split

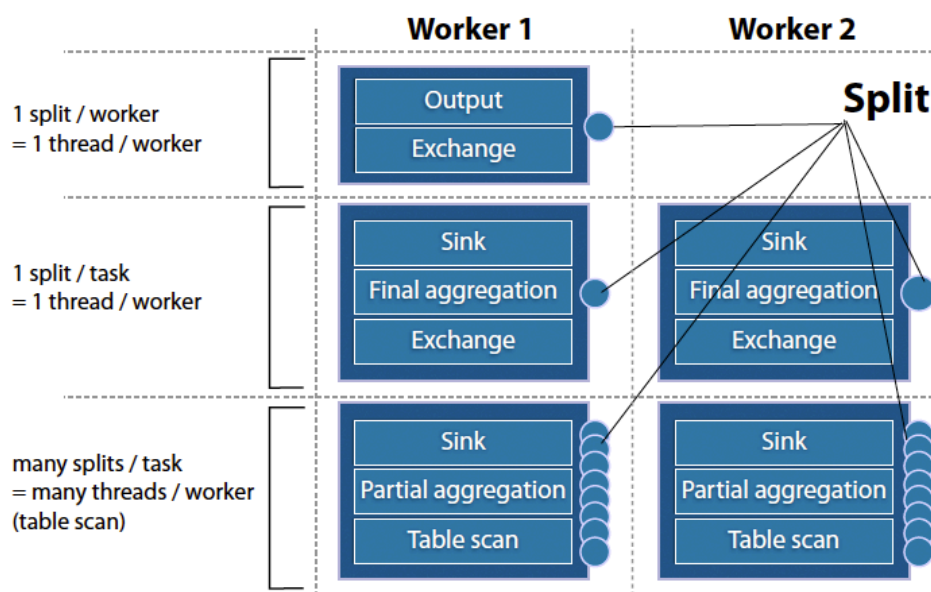
Trong các hệ thống xử lý dữ liệu phân tán như Presto, khái niệm “split” đóng vai trò trung tâm trong việc quản lý và thực thi các truy vấn. Một split có thể được hiểu là một định danh trừu tượng cho một khối dữ liệu có thể định địa chỉ. Khối dữ liệu này có thể nằm trong một hệ thống lưu trữ bên ngoài, ví dụ như HDFS, hoặc là kết quả trung gian được tạo ra bởi các worker khác trong quá trình thực thi truy vấn.

Để minh họa, khi đọc dữ liệu từ HDFS, một split sẽ bao gồm đường dẫn đến tệp và các offset chỉ định một vùng cụ thể trong tệp đó. Điều này cho phép hệ thống truy cập và xử lý các phần nhỏ của dữ liệu một cách độc lập.

Gán Split (Split Assignment)

Việc gán các split cho các tác vụ (task) được thực hiện một cách “lười biếng” (lazily). Điều này có nghĩa là các split không được liệt kê và gán toàn bộ ngay từ đầu khi truy vấn bắt đầu. Thay vào đó, chúng được liệt kê và gán dần dần khi truy vấn đang thực thi. Cách tiếp cận này mang lại một số lợi ích đáng kể:

Execution Planner - Split



- **Khả năng tạo kết quả sớm:** Các truy vấn có thể bắt đầu tạo ra kết quả mà không cần phải chờ đợi toàn bộ dữ liệu được xử lý. Điều này đặc biệt hữu ích cho các truy vấn tương tác, nơi người dùng mong muốn nhận được phản hồi nhanh chóng.
- **Cân bằng tải hiệu quả:** Các split được gán cho worker có hàng đợi ngắn nhất tại thời điểm đó. Cơ chế này giúp phân phối công việc một cách đồng đều giữa các worker, tối ưu hóa việc sử dụng tài nguyên và giảm thiểu thời gian chờ đợi.
- **Giảm sử dụng bộ nhớ siêu dữ liệu:** Bằng cách không liệt kê tất cả các split ngay từ đầu, coordinator (bộ điều phối) sẽ giảm được lượng bộ nhớ cần thiết để lưu trữ siêu dữ liệu về các split. Điều này đặc biệt quan trọng đối với các truy vấn xử lý lượng dữ liệu khổng lồ, nơi số lượng split có thể lên tới hàng triệu.

Mô hình này cho phép Presto xử lý dữ liệu một cách linh hoạt và hiệu quả, từ việc đọc dữ liệu thô (table scan) với nhiều split trên mỗi tác vụ/luồng, đến việc tổng hợp cuối cùng hoặc xuất kết quả, nơi một split có thể tương ứng với một tác vụ hoặc thậm chí một luồng xử lý.

Tối ưu hóa truy vấn: cách tổ chức dữ liệu

Trong quá trình tối ưu hóa truy vấn, bộ tối ưu hóa (optimizer) đóng vai trò quan trọng

trong việc tận dụng cách tổ chức vật lý của dữ liệu. Các thuộc tính của cách tổ chức dữ liệu mà bộ tối ưu hóa có thể khai thác bao gồm: phân vùng (partitioning), sắp xếp (sorting), gom nhóm (grouping) và chỉ mục (indexes).

Điều đáng chú ý là một bảng có thể có nhiều bố cục khác nhau, mỗi bố cục mang những thuộc tính riêng biệt. Hơn nữa, các bố cục này có thể chỉ chứa một tập con các cột hoặc dữ liệu từ bảng gốc, giúp tối ưu hóa việc truy xuất.

Dựa trên các bố cục có sẵn, bộ tối ưu hóa sẽ lựa chọn bố cục tối ưu nhất cho từng truy vấn cụ thể nhằm đạt được hiệu suất cao nhất. Do đó, một trong những phương pháp hiệu quả để điều chỉnh và cải thiện hiệu suất của các truy vấn là bằng cách bổ sung các bố cục vật lý mới, phù hợp với nhu cầu truy xuất dữ liệu.

Tối ưu hóa truy vấn: Đẩy vị từ xuống (Predicate Pushdown)

Trong quá trình tối ưu hóa truy vấn, một kỹ thuật quan trọng được áp dụng là “đẩy vị từ xuống” (predicate pushdown). Kỹ thuật này cho phép bộ tối ưu hóa (optimizer) di chuyển các vị từ (predicates) – tức là các điều kiện lọc dữ liệu, đặc biệt là các điều kiện về khoảng giá trị (range predicates) và điều kiện bằng (equality predicates) – xuống gần hơn với nguồn dữ liệu thông qua các bộ kết nối (connectors). Mục tiêu chính của việc này là nâng cao hiệu quả lọc dữ liệu.

Khi các vị từ được đẩy xuống, quá trình lọc sẽ diễn ra sớm hơn, ngay tại nơi dữ liệu được truy xuất hoặc trước khi dữ liệu được truyền qua mạng hoặc xử lý bởi các thành phần khác của hệ thống. Điều này giúp giảm đáng kể lượng dữ liệu cần được xử lý ở các giai đoạn sau của truy vấn, từ đó cải thiện hiệu suất tổng thể.

Để hỗ trợ việc này, công cụ truy vấn (query engine) cung cấp cho các bộ kết nối một ràng buộc (constraint) gồm hai phần:

- **Miền giá trị (Domain of values):** Bao gồm các thông tin về khoảng giá trị có thể có của dữ liệu (ví dụ: dữ liệu nằm trong một khoảng số cụ thể) và khả năng chấp nhận giá trị rỗng (nullability) của cột.
- **Vị từ “hộp đen” (Black box predicate) để lọc:** Đây là một vị từ tổng quát hơn, có thể phức tạp hơn, mà bộ kết nối cần áp dụng để lọc dữ liệu. Mặc dù có thể không phải lúc nào bộ kết nối cũng có thể tối ưu hóa hoàn toàn vị từ này một cách nội bộ, nhưng nó vẫn phải thực hiện việc lọc dựa trên vị từ được cung cấp.

Ví dụ

Để minh họa cách Presto chuyển đổi một truy vấn SQL thành kế hoạch thực thi phân tán, chúng ta hãy cùng xem xét ví dụ sau.

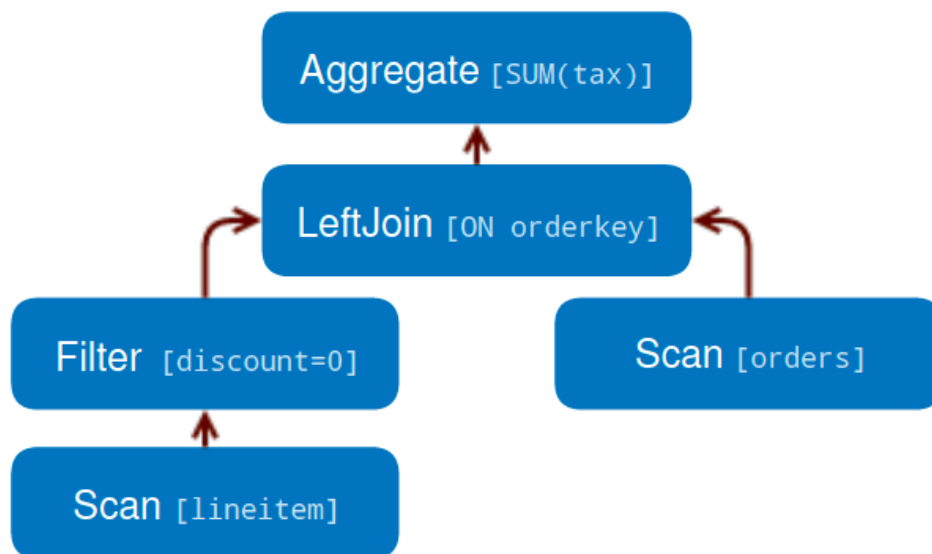
Truy vấn SQL:

```
SELECT
    orders.orderkey, SUM(tax)
FROM orders
LEFT JOIN lineitem
ON orders.orderkey = lineitem.orderkey
WHERE discount = 0
GROUP BY orders.orderkey
```

Truy vấn này có mục đích tính tổng thuế (SUM(tax)) cho mỗi orderkey từ bảng orders, sau khi kết hợp với bảng lineitem dựa trên orderkey, và chỉ bao gồm các dòng có discount bằng 0.

Kế hoạch Logic (Logical Plan):

Dựa trên truy vấn SQL đã cho, Presto sẽ xây dựng một kế hoạch logic, hay còn gọi là kế hoạch truy vấn, như hình minh họa. Kế hoạch này biểu diễn các phép toán cần thực hiện và mối quan hệ phụ thuộc giữa chúng, mà không đi sâu vào chi tiết về cách chúng sẽ được phân tán hay thực thi vật lý.



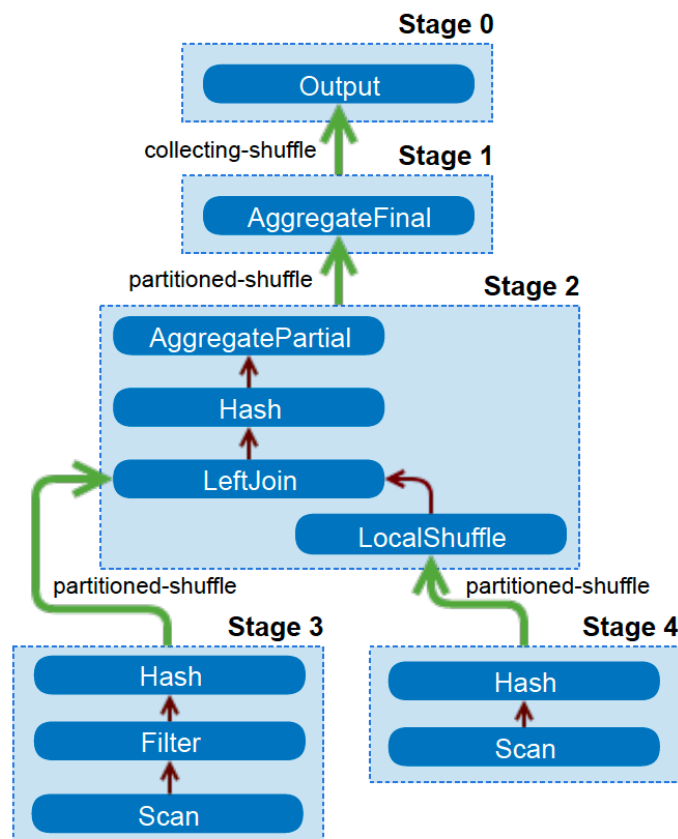
- **Scan [lineitem]** và **Scan [orders]**: Đây là các toán tử cơ bản để đọc dữ liệu từ hai bảng nguồn là lineitem và orders.
- **Filter [discount=0]**: Dữ liệu từ bảng lineitem sẽ được áp dụng một điều kiện lọc để chỉ giữ lại các bản ghi có giá trị discount bằng 0.

- **LeftJoin [ON orderkey]:** Kết quả từ việc lọc bảng `lineitem` và dữ liệu từ bảng `orders` sẽ được kết hợp thông qua phép nối trái (Left Join) dựa trên khóa `orderkey`.
- **Aggregate [SUM(tax)]:** Cuối cùng, kết quả của phép nối sẽ được nhóm theo `orders.orderkey` và tính tổng `tax` cho mỗi nhóm.

Kế hoạch logic này cung cấp một cái nhìn trừu tượng về luồng xử lý dữ liệu của truy vấn.

Kế hoạch Thực thi Phân tán (Distributed Plan):

Để thực thi truy vấn trên một hệ thống phân tán như Presto, kế hoạch logic sẽ được chuyển đổi thành một kế hoạch thực thi phân tán chi tiết hơn. Kế hoạch này được chia thành nhiều giai đoạn (stages) và bao gồm các hoạt động xáo trộn dữ liệu (shuffles) để di chuyển dữ liệu giữa các worker trong cụm.



• Giai đoạn 3 (Stage 3):

- Giai đoạn này bắt đầu bằng việc quét (Scan) bảng `lineitem`.

- Tiếp theo, toán tử lọc (Filter) được áp dụng để loại bỏ các bản ghi có discount khác 0.
- Dữ liệu sau khi lọc được băm (Hash) dựa trên orderkey. Mục đích của việc băm này là để phân vùng dữ liệu, chuẩn bị cho phép nối phân tán.
- Dữ liệu từ giai đoạn này được gửi đến Giai đoạn 2 thông qua một hoạt động partitioned-shuffle. Điều này đảm bảo rằng các bản ghi có cùng orderkey sẽ được gửi đến cùng một worker trong Giai đoạn 2, tạo điều kiện cho phép nối hiệu quả.

- **Giai đoạn 4 (Stage 4):**

- Giai đoạn này chịu trách nhiệm quét (Scan) bảng orders.
- Tương tự như Giai đoạn 3, dữ liệu từ bảng orders cũng được băm (Hash) dựa trên orderkey và gửi đến Giai đoạn 2 thông qua một hoạt động partitioned-shuffle.

- **Giai đoạn 2 (Stage 2):**

- Giai đoạn này nhận dữ liệu đã được phân vùng từ Giai đoạn 3 và Giai đoạn 4.
- Thực hiện phép nối trái (LeftJoin) giữa dữ liệu lineitem và orders trên các worker tương ứng.
- Sau phép nối, dữ liệu có thể trải qua một LocalShuffle (xáo trộn cục bộ). Hoạt động này giúp tối ưu hóa việc phân phối dữ liệu trên cùng một worker trước khi thực hiện các phép toán tiếp theo, đặc biệt là chuẩn bị cho việc tổng hợp.
- Tiếp theo, dữ liệu được băm (Hash) một lần nữa và thực hiện AggregatePartial (tổng hợp một phần). Đây là bước đầu tiên của quá trình tổng hợp phân tán, nơi mỗi worker tính toán tổng tax cục bộ cho các orderkey mà nó đang xử lý.
- Kết quả của AggregatePartial được gửi đến Giai đoạn 1 thông qua một hoạt động partitioned-shuffle.

- **Giai đoạn 1 (Stage 1):**

- Giai đoạn này nhận các kết quả tổng hợp một phần từ Giai đoạn 2.
- Thực hiện AggregateFinal (tổng hợp cuối cùng) để kết hợp tất cả các tổng cục bộ thành tổng cuối cùng cho mỗi orderkey.

- Kết quả cuối cùng được gửi đến Giai đoạn 0 thông qua một hoạt động `collecting-shuffle`. Điều này có nghĩa là tất cả các kết quả cuối cùng được tập hợp về một điểm duy nhất (thường là coordinator hoặc một worker cụ thể) để trả về cho người dùng.

- **Giai đoạn 0 (Stage 0):**

- Đây là giai đoạn cuối cùng, chịu trách nhiệm thu thập tất cả các kết quả và trả về đầu ra cho người dùng.

Mục đích của việc chia nhỏ kế hoạch thành các giai đoạn và sử dụng các hoạt động xáo trộn dữ liệu là để tận dụng tối đa khả năng xử lý song song của hệ thống phân tán. Mỗi giai đoạn có thể được thực thi độc lập trên nhiều worker, và dữ liệu được di chuyển giữa các giai đoạn một cách hiệu quả để đảm bảo tính đúng đắn và hiệu suất của truy vấn. Các hoạt động `partitioned-shuffle` giúp đảm bảo rằng dữ liệu cần thiết cho một phép toán (như join hoặc aggregate) được tập trung tại cùng một worker, trong khi `collecting-shuffle` thu thập kết quả cuối cùng.

Mapreduce vs Presto

Để hiểu rõ hơn về sự khác biệt trong mô hình thực thi giữa MapReduce và Presto, chúng ta hãy cùng xem xét sơ đồ minh họa dưới đây. Sơ đồ này làm nổi bật cách thức xử lý dữ liệu và quản lý tài nguyên của hai hệ thống.

- **Mô hình thực thi của MapReduce:**

MapReduce hoạt động theo một mô hình xử lý theo lô (batch processing) với các giai đoạn được phân tách rõ ràng.

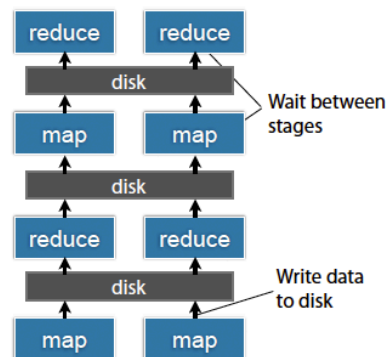
- **Ghi dữ liệu ra đĩa (Write data to disk):** Sau mỗi giai đoạn (ví dụ: giai đoạn Map), dữ liệu trung gian sẽ được ghi xuống đĩa. Điều này đảm bảo tính bền vững và khả năng chịu lỗi, cho phép phục hồi công việc nếu có sự cố xảy ra.
- **Thời gian chờ giữa các giai đoạn (Wait between stages):** Do dữ liệu phải được ghi xuống đĩa và sau đó đọc lại bởi giai đoạn tiếp theo, sẽ có một khoảng thời gian chờ đáng kể giữa các giai đoạn. Điều này làm tăng độ trễ tổng thể của các truy vấn, đặc biệt là đối với các truy vấn tương tác hoặc các truy vấn có nhiều giai đoạn.

- **Mô hình thực thi của Presto:**

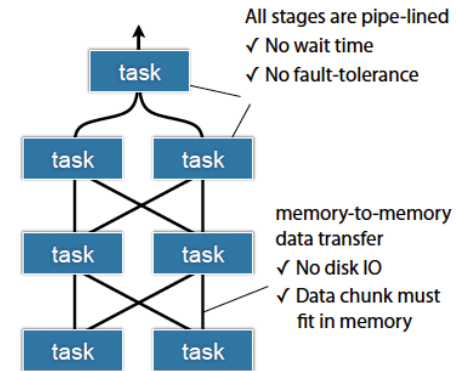
Ngược lại, Presto được thiết kế để tối ưu hóa cho các truy vấn tương tác và có độ trễ thấp, thể hiện ở phía bên phải của sơ đồ.

MapReduce vs. Presto

MapReduce



Presto



- Tất cả các giai đoạn đều được xử lý theo kiểu đường ống (All stages are pipe-lined):** Presto sử dụng mô hình xử lý theo đường ống, nghĩa là dữ liệu được truyền trực tiếp từ một tác vụ (task) ở giai đoạn này sang tác vụ ở giai đoạn tiếp theo ngay khi nó sẵn sàng, mà không cần chờ toàn bộ giai đoạn hoàn thành.
- Không có thời gian chờ (No wait time):** Nhờ cơ chế đường ống, không có thời gian chờ đáng kể giữa các giai đoạn, giúp giảm đáng kể độ trễ của truy vấn.
- Truyền dữ liệu trực tiếp trong bộ nhớ (memory-to-memory data transfer):** Dữ liệu trung gian được truyền trực tiếp giữa các tác vụ trong bộ nhớ, loại bỏ nhu cầu ghi và đọc từ đĩa.
- Không có I/O đĩa (No disk IO):** Đây là một điểm khác biệt cốt lõi so với MapReduce. Việc tránh I/O đĩa giúp Presto đạt được hiệu suất cao hơn nhiều cho các truy vấn.
- Khối dữ liệu phải nằm gọn trong bộ nhớ (Data chunk must fit in memory):** Do dữ liệu được xử lý trong bộ nhớ, mỗi khối dữ liệu trung gian phải đủ nhỏ để nằm gọn trong bộ nhớ khả dụng. Đây là một hạn chế cần lưu ý khi xử lý các tập dữ liệu cực lớn.
- Không hỗ trợ cơ chế chịu lỗi (No fault-tolerance):** Một đánh đổi quan trọng của việc xử lý trong bộ nhớ và không ghi dữ liệu trung gian ra đĩa là Presto không hỗ trợ

cơ chế chịu lỗi ở cấp độ tác vụ như MapReduce. Nếu một tác vụ thất bại, toàn bộ truy vấn có thể cần phải được khởi động lại.

Trong khi MapReduce ưu tiên tính bền vững và khả năng chịu lỗi thông qua việc ghi dữ liệu ra đĩa, dẫn đến độ trễ cao hơn, thì Presto lại tập trung vào hiệu suất và độ trễ thấp bằng cách xử lý dữ liệu hoàn toàn trong bộ nhớ và sử dụng cơ chế đường ống, phù hợp hơn cho các tác vụ phân tích dữ liệu tương tác.

Thực thi truy vấn

Trong quá trình thực thi truy vấn, hệ thống Presto áp dụng một mô hình được thiết kế để tối ưu hóa hiệu suất và khả năng xử lý song song.

Đầu tiên, một truy vấn SQL được chuyển đổi thành các thành phần nhỏ hơn, bao gồm các **giai đoạn (stages)**, các **tác vụ (tasks)** và các **phân đoạn (splits)**. Việc phân chia này là nền tảng cho việc quản lý và thực thi truy vấn một cách hiệu quả trên kiến trúc phân tán.

Một đặc điểm nổi bật trong mô hình thực thi của Presto là khả năng **thực thi song song tất cả các tác vụ**. Điều này mang lại những lợi ích và đặc điểm quan trọng sau:

- **Không có thời gian chờ giữa các giai đoạn:** Dữ liệu được xử lý theo kiểu **đường ống (pipelined)**. Điều này có nghĩa là các giai đoạn tiếp theo có thể bắt đầu xử lý dữ liệu ngay khi dữ liệu từ giai đoạn trước được tạo ra, mà không cần chờ toàn bộ giai đoạn trước hoàn thành. Cơ chế này giúp giảm đáng kể độ trễ tổng thể của truy vấn.
- **Tính toàn vẹn khi thất bại:** Nếu một tác vụ bất kỳ trong quá trình thực thi gặp lỗi, toàn bộ truy vấn sẽ bị hủy bỏ và tất cả các tác vụ liên quan cũng sẽ thất bại đồng thời. Điều này đảm bảo tính nhất quán của kết quả truy vấn.

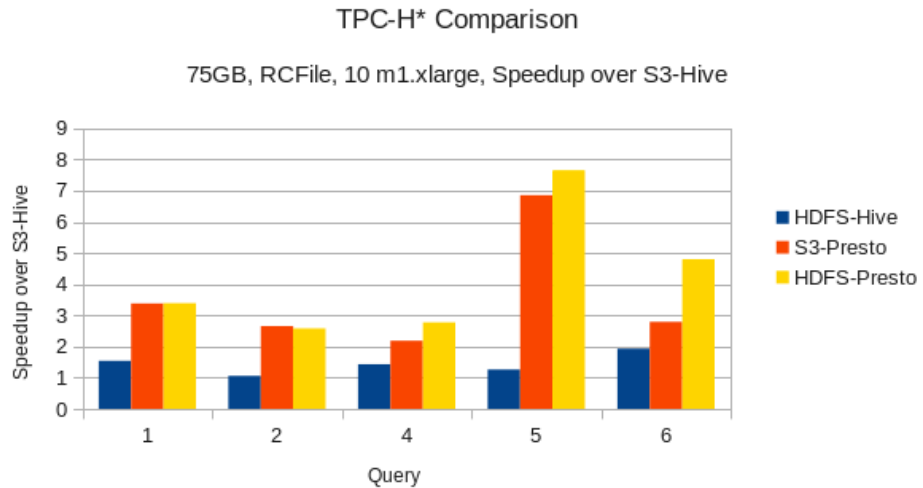
Để đạt được hiệu suất cao, Presto ưu tiên **truyền dữ liệu trực tiếp giữa các vùng bộ nhớ (memory-to-memory data transfer)**. Phương pháp này mang lại những ưu điểm và giới hạn nhất định:

- **Không có hoạt động nhập/xuất đĩa (disk I/O):** Việc truyền dữ liệu trực tiếp trong bộ nhớ giúp loại bỏ nhu cầu ghi và đọc dữ liệu từ đĩa, từ đó giảm đáng kể độ trễ và tăng tốc độ xử lý.
- **Giới hạn bộ nhớ:** Do phụ thuộc vào bộ nhớ, nếu lượng dữ liệu tổng hợp (aggregated data) vượt quá khả năng chứa của bộ nhớ hệ thống, truy vấn sẽ thất bại. Đây là một đặc điểm thiết kế nhằm đảm bảo hiệu suất cao bằng cách giữ toàn bộ dữ liệu làm việc trong bộ nhớ.

- **Quản lý bộ nhớ mạnh mẽ:** Cần lưu ý rằng, khi một truy vấn thất bại do vấn đề về bộ nhớ, chỉ riêng truy vấn đó bị dừng (query dies) chứ tiến trình xử lý (worker) của hệ thống vẫn tiếp tục hoạt động bình thường. Điều này là nhờ vào cơ chế quản lý bộ nhớ chặt chẽ, đảm bảo việc tiêu thụ bộ nhớ của tất cả các truy vấn được kiểm soát và quản lý hoàn toàn, giúp duy trì sự ổn định và khả năng phục hồi của hệ thống.

4.7.4. Đánh giá hiệu năng & Ứng dụng

Qubole Presto benchmark



Biểu đồ này trình bày kết quả đánh giá hiệu năng của Presto do Qubole thực hiện, tập trung vào việc so sánh hiệu suất giữa các cấu hình khác nhau dựa trên bộ dữ liệu TPC-H*. Cụ thể, bài kiểm tra được thực hiện trên một bộ dữ liệu 75GB, sử dụng định dạng tệp RCFile, và triển khai trên 10 instance loại m1.xlarge. Trục tung của biểu đồ thể hiện “Speedup over S3-Hive”, tức là mức độ tăng tốc so với hiệu suất của Hive khi dữ liệu được lưu trữ trên S3. Điều này có nghĩa là S3-Hive được coi là đường cơ sở (baseline) với giá trị chuẩn bằng 1.

Chúng ta có thể quan sát ba cấu hình chính được so sánh:

- **HDFS-Hive (màu xanh dương):** Đại diện cho hiệu suất của Hive khi dữ liệu được lưu trữ trên HDFS.
- **S3-Presto (màu cam):** Đại diện cho hiệu suất của Presto khi dữ liệu được lưu trữ trên S3.
- **HDFS-Presto (màu vàng):** Đại diện cho hiệu suất của Presto khi dữ liệu được lưu trữ trên HDFS.

Từ biểu đồ, chúng ta có thể rút ra một số nhận xét quan trọng:

1. **Hiệu suất của Hive:** Cấu hình HDFS-Hive (màu xanh dương) thường cho thấy tốc độ tăng tốc thấp hơn 1 so với S3-Hive trên hầu hết các truy vấn (ví dụ: truy vấn 1, 2, 4, 5), điều này cho thấy trong môi trường thử nghiệm này, Hive trên HDFS không vượt trội hơn đáng kể so với Hive trên S3, và thậm chí còn chậm hơn ở một số trường hợp.
2. **Ưu thế vượt trội của Presto:** Cả S3-Presto (màu cam) và HDFS-Presto (màu vàng) đều thể hiện sự vượt trội đáng kể về hiệu suất so với S3-Hive trên tất cả các truy vấn được kiểm tra.
 - Đối với **S3-Presto**, chúng ta thấy mức tăng tốc ấn tượng, đặc biệt là ở truy vấn 5, đạt gần 7 lần so với S3-Hive. Các truy vấn khác như 1, 2, 4, 6 cũng cho thấy mức tăng tốc từ 2.5 đến hơn 3 lần.
 - **HDFS-Presto** thường có hiệu suất tương đương hoặc nhanh hơn một chút so với S3-Presto. Đáng chú ý nhất là ở truy vấn 5, HDFS-Presto đạt mức tăng tốc gần 8 lần, là hiệu suất cao nhất trong tất cả các cấu hình. Điều này cho thấy Presto có thể tận dụng tốt lợi thế của việc truy cập dữ liệu cục bộ trên HDFS.
3. **Ảnh hưởng của vị trí lưu trữ dữ liệu:** Mặc dù HDFS-Presto thường nhanh hơn S3-Presto, sự khác biệt này không quá lớn ở nhiều truy vấn. Điều này minh họa rằng Presto được thiết kế để hoạt động hiệu quả ngay cả khi dữ liệu được lưu trữ trên các dịch vụ lưu trữ đối tượng như S3, giảm thiểu tác động của độ trễ mạng.

Kết quả benchmark của Qubole cho thấy Presto mang lại sự cải thiện hiệu suất đáng kể so với Hive cho các tác vụ phân tích dữ liệu trên bộ dữ liệu TPC-H*, bất kể dữ liệu được lưu trữ trên HDFS hay S3. Điều này khẳng định Presto là một lựa chọn mạnh mẽ cho các ứng dụng yêu cầu truy vấn dữ liệu nhanh chóng và hiệu quả.

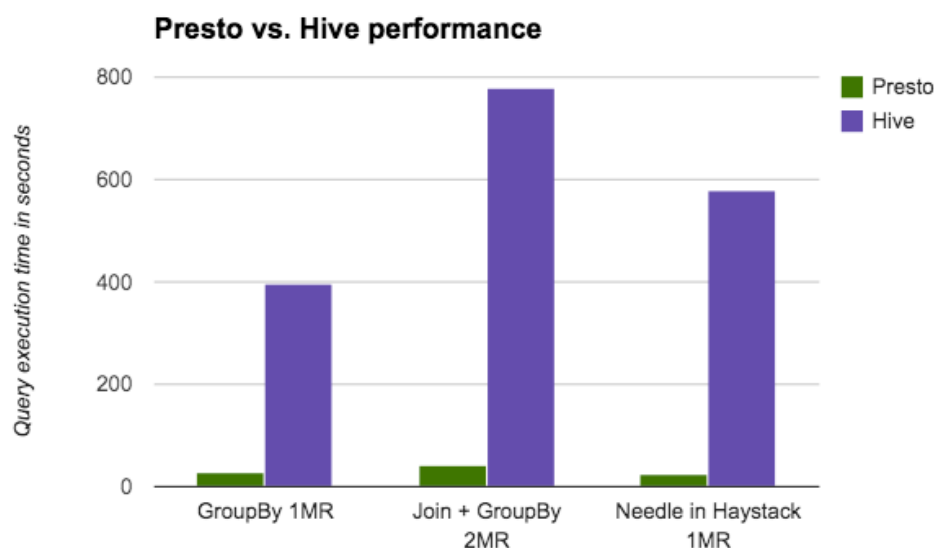
Đánh giá hiệu năng Presto của Netflix

Để đánh giá hiệu năng của Presto trong môi trường thực tế, Netflix đã tiến hành một loạt các thử nghiệm benchmark với cấu hình hệ thống bao gồm 40 node loại m2.4xlarge. Dữ liệu đầu vào được sử dụng là các tệp định dạng Parquet, lưu trữ trên Amazon S3, với kích thước dao động từ 140GB đến 210GB.

Trong phần đánh giá hiệu năng tổng thể, Netflix đã so sánh Presto với Hive thông qua ba loại truy vấn điển hình:

- Truy vấn GroupBy: Nhóm dữ liệu theo một tiêu chí.

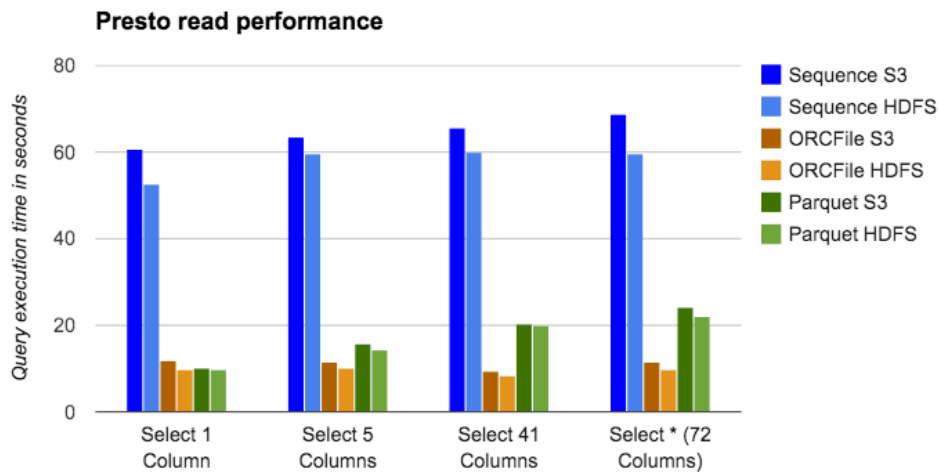
- Truy vấn Join kết hợp với GroupBy: Kết hợp dữ liệu từ nhiều bảng và sau đó nhóm lại.
- Truy vấn Needle in a Haystack: Tìm kiếm một giá trị cụ thể trong toàn bộ bảng, tương đương với một thao tác quét bảng đầy đủ (table scan).



Kết quả được minh họa trong biểu đồ trên. Biểu đồ cho thấy Presto vượt trội đáng kể so với Hive về thời gian thực thi truy vấn. Cụ thể, Presto hoàn thành các truy vấn GroupBy, Join + GroupBy và Needle in Haystack với thời gian chỉ bằng một phần nhỏ so với Hive. Điều này khẳng định khả năng của Presto trong việc xử lý các truy vấn phân tích dữ liệu lớn một cách nhanh chóng và hiệu quả hơn nhiều so với các công cụ truyền thống như Hive, đặc biệt trong các tác vụ yêu cầu độ trễ thấp.

Ngoài ra, Netflix cũng đánh giá hiệu năng đọc dữ liệu của Presto với các định dạng tệp và vị trí lưu trữ khác nhau, được trình bày trong biểu đồ trên. Các định dạng tệp được thử nghiệm bao gồm Sequence, ORCFile và Parquet, trên cả S3 và HDFS. Các truy vấn được thực hiện là các truy vấn SELECT với số lượng cột khác nhau (1 cột, 5 cột, 41 cột và tất cả 72 cột).

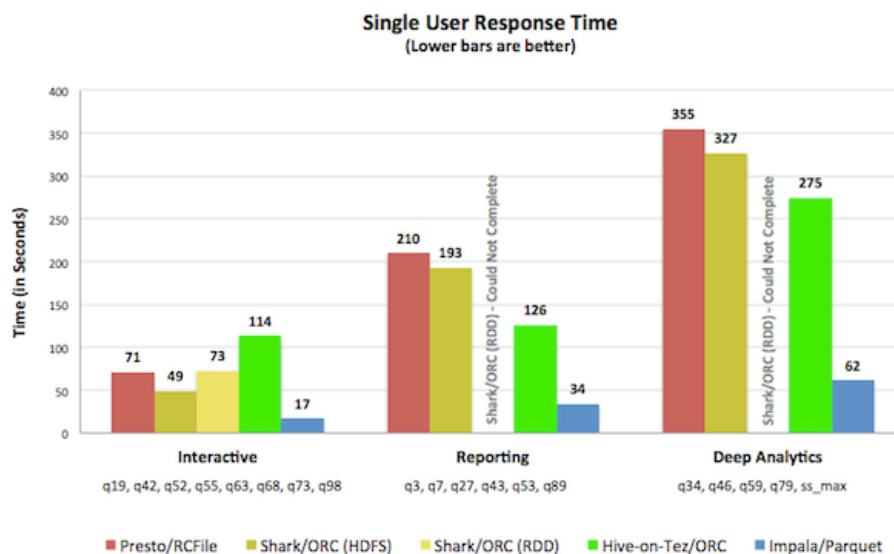
Từ biểu đồ, chúng ta có thể nhận thấy rằng định dạng Parquet mang lại hiệu năng đọc vượt trội so với Sequence và ORCFile, đặc biệt khi dữ liệu được lưu trữ trên S3. Điều này là do Parquet là định dạng cột (columnar format) được tối ưu hóa cho việc đọc dữ liệu phân tích, cho phép Presto chỉ đọc các cột cần thiết, giảm đáng kể lượng dữ liệu phải xử lý. Mặc dù hiệu năng trên HDFS cũng tốt, nhưng S3 với Parquet vẫn cho thấy lợi thế rõ rệt trong nhiều trường hợp. Hiệu năng đọc cũng thay đổi tùy thuộc vào số lượng cột được chọn; việc



chọn ít cột hơn thường dẫn đến thời gian thực thi nhanh hơn, điều này là hợp lý vì Presto cần xử lý ít dữ liệu hơn.

Cloudera benchmark

Để đánh giá hiệu năng của các công cụ truy vấn dữ liệu lớn, chúng ta sẽ cùng xem xét kết quả từ một bộ benchmark được thực hiện bởi Cloudera. Bộ benchmark này so sánh hiệu suất của Presto, Shark/ORC (trên HDFS và RDD), Hive-on-Tez/ORC và Impala/Parquet trong các kịch bản khác nhau.



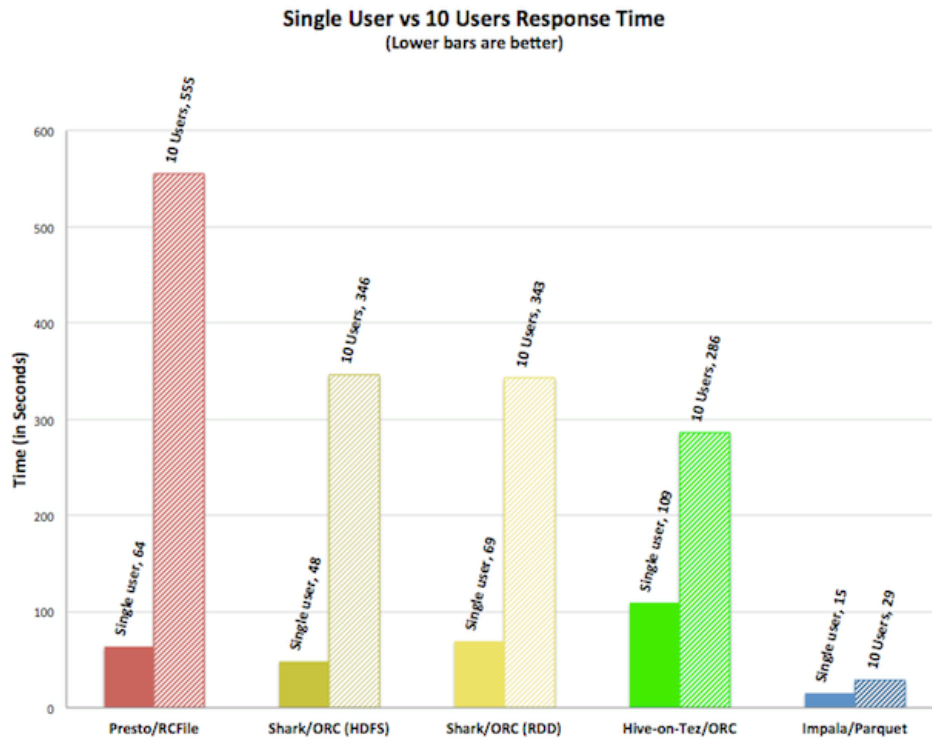
Biểu đồ trên minh họa thời gian phản hồi cho một người dùng duy nhất (Single User

Response Time) đối với các loại truy vấn khác nhau. Lưu ý rằng, các thanh biểu đồ càng thấp thì hiệu năng càng tốt.

- **Truy vấn tương tác (Interactive):** Trong nhóm truy vấn này, Impala/Parquet thể hiện hiệu suất vượt trội với thời gian phản hồi rất thấp (ví dụ: 17 giây cho truy vấn q98). Presto/RCFile cũng cho thấy hiệu suất cạnh tranh (ví dụ: 71 giây cho truy vấn q19), trong khi Shark/ORC và Hive-on-Tez/ORC có thời gian phản hồi cao hơn đáng kể.
- **Truy vấn báo cáo (Reporting):** Tương tự, Impala/Parquet tiếp tục dẫn đầu với thời gian phản hồi ấn tượng (ví dụ: 34 giây cho truy vấn q89). Presto/RCFile và Shark/ORC (HDFS) có thời gian phản hồi cao hơn. Đáng chú ý là Shark/ORC (RDD) và Hive-on-Tez/ORC không thể hoàn thành một số truy vấn trong nhóm này, điều này cho thấy những hạn chế về khả năng xử lý hoặc độ ổn định của chúng trong một số trường hợp.
- **Phân tích chuyên sâu (Deep Analytics):** Trong các truy vấn phức tạp này, Impala/Parquet vẫn duy trì hiệu suất tốt nhất (ví dụ: 62 giây cho truy vấn ss_max). Presto/RCFile có thời gian phản hồi cao nhất trong nhóm này (ví dụ: 355 giây cho truy vấn q34). Tương tự như nhóm Reporting, Shark/ORC (RDD) và Hive-on-Tez/ORC cũng gặp vấn đề không hoàn thành được một số truy vấn, điều này nhấn mạnh rằng chúng có thể không phù hợp cho các tác vụ phân tích chuyên sâu đòi hỏi cao.

Biểu đồ trên so sánh thời gian phản hồi giữa kịch bản một người dùng và kịch bản 10 người dùng đồng thời (Single User vs 10 Users Response Time). Biểu đồ này giúp chúng ta đánh giá khả năng mở rộng (scalability) của các công cụ khi tải đồng thời tăng lên. Một lần nữa, các thanh biểu đồ càng thấp thì hiệu năng càng tốt.

- **Impala/Parquet:** Thể hiện khả năng mở rộng rất tốt. Thời gian phản hồi chỉ tăng nhẹ từ 15 giây (một người dùng) lên 29 giây (10 người dùng), cho thấy hiệu suất ổn định dưới tải cao.
- **Shark/ORC (HDFS) và Shark/ORC (RDD):** Cũng cho thấy sự gia tăng thời gian phản hồi khi có nhiều người dùng hơn, nhưng mức độ tăng không quá đột biến so với Presto.
- **Hive-on-Tez/ORC:** Thời gian phản hồi tăng từ 109 giây (một người dùng) lên 286 giây (10 người dùng), cho thấy sự suy giảm hiệu suất đáng kể khi có tải đồng thời.
- **Presto/RCFile:** Đây là trường hợp đáng chú ý nhất. Thời gian phản hồi của Presto tăng vọt từ 64 giây (một người dùng) lên tới 555 giây (10 người dùng). Điều này chỉ ra rằng Presto/RCFile trong cấu hình benchmark này có thể gặp khó khăn đáng kể



trong việc xử lý các truy vấn đồng thời, dẫn đến hiệu suất suy giảm nghiêm trọng khi số lượng người dùng tăng lên.

Tổng kết lại, từ bộ benchmark của Cloudera, chúng ta có thể thấy Impala/Parquet nổi bật về hiệu suất trong cả kịch bản một người dùng và khả năng mở rộng dưới tải đồng thời. Presto/RCFile cho hiệu suất tốt ở kịch bản một người dùng cho các truy vấn tương tác, nhưng lại gặp thách thức lớn về khả năng mở rộng khi có nhiều người dùng đồng thời. Các công cụ như Shark/ORC (RDD) và Hive-on-Tez/ORC cũng bộc lộ những hạn chế nhất định, đặc biệt là việc không hoàn thành được một số truy vấn phức tạp. Những kết quả này cung cấp cái nhìn quan trọng về điểm mạnh và điểm yếu của từng công cụ trong các tình huống sử dụng thực tế.

Tăng tốc hơn nữa: Dữ liệu với tốc độ của Presto ORC

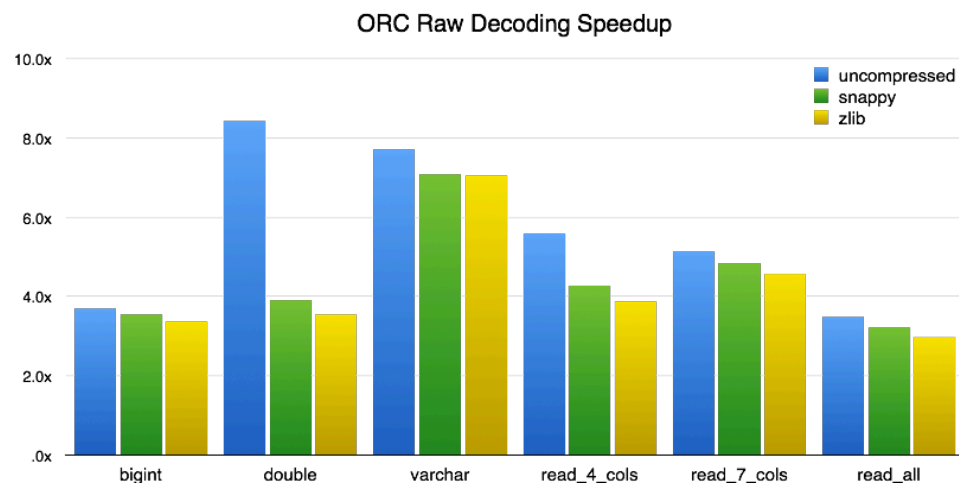
Để minh họa cho khả năng tối ưu hiệu suất của Presto, chúng ta sẽ xem xét một thử nghiệm so sánh tốc độ đọc dữ liệu. Thử nghiệm này được thực hiện trên một tập dữ liệu lớn, cụ thể là 6 triệu hàng (rows), sử dụng bộ dữ liệu chuẩn TPC-H, một tiêu chuẩn phổ biến để đánh giá hiệu năng của các hệ thống quản lý cơ sở dữ liệu và phân tích dữ liệu.

Mục tiêu của thử nghiệm là đánh giá hiệu suất khi đọc các tập hợp cột khác nhau từ dữ

liệu. Điều này rất quan trọng vì trong các tác vụ phân tích thực tế, không phải lúc nào chúng ta cũng cần đọc toàn bộ các cột của một bảng; việc chỉ đọc các cột cần thiết có thể cải thiện đáng kể hiệu suất.

Điểm mấu chốt của thử nghiệm này là sự so sánh giữa hai công cụ đọc định dạng ORC (Optimized Row Columnar) khác nhau:

- **Công cụ đọc ORC dựa trên Hive cũ (Old Hive-based ORC reader):** Đây là công cụ đọc ORC truyền thống được sử dụng trong hệ sinh thái Hadoop, đặc biệt là với Hive.
- **Công cụ đọc ORC mới của Presto (New Presto ORC reader):** Đây là công cụ đọc được tối ưu hóa và phát triển riêng cho Presto, hứa hẹn mang lại hiệu suất vượt trội.



Biểu đồ trên minh họa kết quả của thử nghiệm này. Mặc dù các nhãn cụ thể trên biểu đồ không được hiển thị rõ ràng, chúng ta có thể quan sát xu hướng chung: các cột màu xanh lam (đại diện cho công cụ đọc ORC dựa trên Hive cũ) thường cao hơn đáng kể so với các cột màu xanh lá cây và vàng (đại diện cho công cụ đọc ORC mới của Presto) trong hầu hết các trường hợp. Điều này cho thấy rằng, khi đọc cùng một lượng dữ liệu (6 triệu hàng) và các tập hợp cột khác nhau, công cụ đọc ORC mới của Presto đã giảm đáng kể thời gian cần thiết để hoàn thành tác vụ. Kết quả này khẳng định khả năng của Presto trong việc xử lý dữ liệu nhanh hơn, đặc biệt là khi làm việc với định dạng ORC, giúp tăng tốc độ truy vấn và phân tích dữ liệu lên một tầm cao mới.

Chương 5

Hệ thống truyền thông điệp phân tán

5.1. Mục tiêu học tập	183
5.2. Giới thiệu về hệ thống xuất bản - đăng ký	183
5.3. Tổ chức và quản lý dữ liệu trong Kafka	186
5.4. Sản xuất và tiêu thụ dữ liệu trong Kafka	189
5.5. Tóm tắt chương	193
5.6. Thuật ngữ quan trọng	193
5.7. Câu hỏi ôn tập	194

5.1. Mục tiêu học tập

Sau khi học xong chương này, sinh viên có thể:

- Hiểu được nguyên lý hoạt động của mô hình pub-sub và vai trò của nó trong các hệ thống phân tán hiện đại
- Nắm vững kiến trúc và các thành phần cốt lõi của Apache Kafka như topic, partition, consumer group
- Phân tích được cơ chế đảm bảo độ tin cậy và khả năng mở rộng trong Kafka thông qua replication và partition
- Thiết kế được hệ thống message broker đáp ứng các yêu cầu về khả năng mở rộng, độ tin cậy và hiệu năng
- Triển khai và vận hành được hệ thống Kafka trong môi trường thực tế, bao gồm cấu hình, monitoring và xử lý sự cố

5.2. Giới thiệu về hệ thống xuất bản - đăng ký

Trong các hệ thống phân tán hiện đại, việc truyền thông điệp giữa các thành phần có thể được thực hiện qua hai mô hình chính: point-to-point messaging và publish-subscribe messaging. Mỗi mô hình có những ưu điểm và trường hợp sử dụng riêng:

Point-to-Point Messaging Mô hình này thiết lập kênh truyền thông trực tiếp giữa người gửi và người nhận. Thông điệp được gửi đến một hàng đợi (queue) và chỉ được tiêu thụ bởi một người nhận duy nhất. Phù hợp cho các tác vụ cần xử lý chính xác một lần như xử lý đơn hàng, giao dịch tài chính.

Publish-Subscribe Messaging Mô hình này tách biệt hoàn toàn người gửi (publisher) và người nhận (subscriber) thông qua một broker trung gian. Một thông điệp có thể được nhận bởi nhiều subscriber, phù hợp cho các tác vụ như phân phối sự kiện, cập nhật trạng thái hệ thống.

5.2.1. Kiến trúc và nguyên lý hoạt động

Hệ thống xuất bản - đăng ký (pub-sub) là một mô hình truyền thông bất đồng bộ tiên tiến. Trong mô hình này, publisher không cần biết về subscriber và ngược lại, tất cả giao tiếp đều thông qua broker trung gian. Kiến trúc này mang lại ba lợi thế chính:

Tính linh hoạt Publisher và subscriber hoạt động độc lập, có thể thêm/bớt/thay đổi mà không ảnh hưởng lẫn nhau. Ví dụ, một hệ thống giao dịch chứng khoán có thể thêm các subscriber mới (như ứng dụng phân tích kỹ thuật, cảnh báo giá) mà không cần thay đổi phía publisher.

Tính mở rộng Broker có thể được triển khai phân tán để tăng khả năng xử lý. Ví dụ, một hệ thống IoT có thể mở rộng từ vài nghìn lên hàng triệu thiết bị bằng cách thêm broker mà không cần thay đổi logic ứng dụng.

Tính chịu lỗi Broker lưu trữ tạm thời thông điệp, đảm bảo delivery ngay cả khi subscriber tạm thời offline. Ví dụ, trong hệ thống thanh toán, các thông báo giao dịch vẫn được lưu và gửi lại khi dịch vụ thông báo khách hàng phục hồi sau sự cố.

5.2.2. Các mô hình triển khai

Có ba mô hình triển khai pub-sub phổ biến, mỗi mô hình phù hợp với các yêu cầu khác nhau:

Topic-based Thông điệp được phân loại và định tuyến theo chủ đề. Publisher gửi message vào topic cụ thể, subscriber đăng ký các topic họ quan tâm. Ví dụ: trong hệ thống trading, các topic có thể là "stock.price", "stock.order", "stock.trade".

Content-based Thông điệp được định tuyến dựa trên nội dung thực tế, sử dụng các bộ lọc. Ví dụ: subscriber có thể đăng ký nhận thông báo chỉ khi giá cổ phiếu vượt ngưỡng định trước hoặc khối lượng giao dịch bất thường.

Type-based Định tuyến dựa trên kiểu dữ liệu của thông điệp, thường được sử dụng trong các hệ thống hướng đối tượng. Ví dụ: trong hệ thống CRM, các subscriber có thể đăng ký nhận các sự kiện kiểu CustomerCreated, OrderPlaced.

5.2.3. Ứng dụng và Pattern phổ biến

Hệ thống pub-sub được ứng dụng rộng rãi trong nhiều lĩnh vực, với các pattern thiết kế đặc thù. Trong kiến trúc hướng sự kiện (Event-Driven Architecture), các hệ thống giám sát an ninh mạng sử dụng pub-sub để phân phối các sự kiện bảo mật như login thất bại hoặc truy cập bất thường tới các hệ thống phân tích và phản ứng. Pattern Event Sourcing kết hợp với CQRS cho phép lưu trữ mọi thay đổi trạng thái dưới dạng chuỗi sự kiện bất biến, đảm bảo tính nhất quán và khả năng truy vết.

Trong tích hợp microservices, hệ thống e-commerce sử dụng pub-sub để đồng bộ dữ liệu giữa các service như order, inventory, shipping và notification. Các pattern phổ biến bao gồm Saga pattern để quản lý giao dịch phân tán và Outbox pattern để đảm bảo tính nhất quán của dữ liệu khi có sự cố mạng.

Đối với xử lý dữ liệu IoT, nhà máy thông minh sử dụng pub-sub để thu thập và xử lý dữ liệu từ hàng nghìn cảm biến, hỗ trợ bảo trì dự đoán. Edge Processing kết hợp với Stream Processing cho phép xử lý và tổng hợp dữ liệu theo thời gian thực ngay tại nguồn dữ liệu.

Trong phân tích dữ liệu thời gian thực, hệ thống phát hiện gian lận sử dụng pub-sub để xử lý luồng giao dịch theo thời gian thực. Lambda Architecture kết hợp xử lý batch và streaming để cân bằng giữa độ trễ và độ chính xác, đảm bảo hệ thống có thể phản ứng nhanh chóng với các mối đe dọa mới.

5.2.4. Thách thức và Giải pháp

Triển khai hệ thống pub-sub quy mô lớn đặt ra nhiều thách thức quan trọng cần được giải quyết. Thách thức đầu tiên là đảm bảo độ tin cậy của message, đặc biệt trong điều kiện lỗi mạng hoặc crash hệ thống. Để giải quyết vấn đề này, các hệ thống hiện đại sử dụng cơ chế acknowledgment, message deduplication và dead letter queue để đảm bảo không có message nào bị mất hoặc trùng lặp.

Thách thức thứ hai liên quan đến việc duy trì thứ tự message khi có nhiều producer và consumer xử lý song song. Trong môi trường phân tán, việc đảm bảo thứ tự xử lý message là rất quan trọng đối với các ứng dụng như giao dịch tài chính hoặc xử lý đơn hàng. Các giải pháp phổ biến bao gồm sử dụng partition key, sequence number và các thuật toán đồng thuận phân tán để duy trì tính nhất quán.

Cuối cùng, thách thức về hiệu năng và khả năng mở rộng đòi hỏi hệ thống phải xử lý được khối lượng message lớn với độ trễ thấp. Để đạt được mục tiêu này, các hệ thống sử

dụng kỹ thuật phân vùng dữ liệu, caching thông minh và auto-scaling dựa trên tải để tối ưu hóa hiệu suất và đảm bảo khả năng mở rộng theo nhu cầu thực tế.

5.2.5. Giới thiệu về Apache Kafka

Apache Kafka là một nền tảng truyền tải dữ liệu theo thời gian thực, được xây dựng để xử lý khối lượng lớn thông điệp với độ tin cậy và hiệu suất cao. Được phát triển ban đầu tại LinkedIn và sau đó mở mã nguồn thành dự án thuộc Apache Software Foundation, Kafka đã nhanh chóng trở thành một trong những công cụ quan trọng trong hệ sinh thái Big Data.

Apache Kafka bao gồm ba thành phần chính:

Producer Producer là các ứng dụng hoặc quá trình tạo ra và gửi các bản tin dữ liệu tới Kafka. Producer gửi các bản tin này tới các topic, mỗi topic là một luồng dữ liệu độc lập.

Broker Broker là các máy chủ trong mạng Kafka, nơi dữ liệu được lưu trữ và xử lý. Kafka thường được triển khai thành một cluster gồm nhiều Broker. Mỗi broker lưu trữ một phần của dữ liệu và tham gia vào việc xử lý và truyền tải các bản tin.

Consumer Consumer là các ứng dụng hoặc quá trình đọc và xử lý các bản tin từ Kafka. Consumer đăng ký với Kafka để đọc dữ liệu từ một hoặc nhiều topic, sau đó xử lý dữ liệu theo nhu cầu của mình.

5.3. Tổ chức và quản lý dữ liệu trong Kafka

5.3.1. Các chủ đề (topic) trong Kafka

Trong hệ thống Apache Kafka, một topic đại diện cho một luồng dữ liệu lớn, là nơi mà các thông điệp (messages) được sản xuất (produced) và tiêu thụ (consumed). Mỗi topic có thể được hiểu như một kênh hoặc một danh mục, trong đó các bản tin dữ liệu được gửi và nhận. Topic giúp tổ chức và quản lý dữ liệu một cách logic và hiệu quả, được xác định bởi một tên duy nhất để đại diện cho một luồng dữ liệu riêng biệt.

Đặc điểm chính của Kafka Topics:

Bền vững Kafka Topics đảm bảo lưu trữ tin nhắn một cách bền vững, chống mất mát dữ liệu ngay cả khi xảy ra sự cố phần cứng hoặc phần mềm. Dữ liệu trong các topic được ghi vào đĩa và có thể được cấu hình để sao lưu (replication) giữa các broker khác nhau nhằm đảm bảo tính toàn vẹn và sẵn sàng cao.

Phân tán Topics có thể được chia thành nhiều phân vùng (partitions), mỗi phân vùng được lưu trữ trên một broker riêng biệt. Phân vùng cho phép Kafka mở rộng khả năng xử lý dữ liệu bằng cách phân chia tải công việc giữa nhiều máy chủ. Điều này giúp tăng

cường khả năng mở rộng và hiệu suất, đồng thời hỗ trợ xử lý song song các luồng dữ liệu lớn.

Hỗ trợ đa người tiêu dùng Nhiều consumer có thể đồng thời nhận và xử lý tin nhắn từ cùng một topic, cho phép phân tán tải và tăng tốc độ xử lý dữ liệu. Mỗi consumer group có thể đọc các phân vùng riêng lẻ, giúp tối ưu hóa việc phân phối và xử lý dữ liệu theo thời gian thực.

5.3.2. Phân vùng (partition) của chủ đề (topic)

Như chúng ta đã biết, Kafka topic là một luồng bản ghi được đặt tên. Vậy Kafka quản lý lưu trữ và truy cập luồng dữ liệu ngày càng tăng lên như thế nào? Câu trả lời nằm ở một khái niệm gọi là phân vùng. Kafka chia mỗi chủ đề thành các phân đoạn nhỏ hơn, dễ quản lý hơn gọi là các partitions. Sau đó, các partitions này được phân phối trên nhiều Kafka brokers. Bằng cách chia dữ liệu thành các partitions, nhiều consumers có thể truy cập và xử lý thông tin đồng thời. Việc xử lý song song này làm tăng đáng kể tốc độ tổng thể của việc thu thập và tiêu thụ dữ liệu. Ngoài ra, có thể dễ dàng thêm nhiều máy chủ hoặc đĩa để chứa thêm partitions. Việc mở rộng theo chiều ngang này đảm bảo Kafka có thể xử lý khối lượng dữ liệu ngày càng tăng mà không ảnh hưởng đến hiệu suất.

Khi các bản ghi sở hữu khóa, Kafka ưu tiên sử dụng khóa đó để xác định phân vùng lưu trữ. Khóa bản ghi hoạt động như một "dấu vân tay" duy nhất, giúp Kafka định hướng lưu trữ các bản ghi có khóa giống nhau vào cùng một phân vùng. Điều này mang lại những lợi ích:

Truy vấn hiệu quả Các consumer quan tâm đến các bản ghi có khóa cụ thể chỉ cần đọc từ một phân vùng duy nhất, cải thiện hiệu suất truy vấn dữ liệu.

Xử lý theo thứ tự Nếu thứ tự xử lý các bản ghi có cùng khóa là quan trọng, việc lưu trữ chúng trên cùng phân vùng đảm bảo thứ tự này được duy trì.

Với các bản ghi không có khóa, Kafka chuyển sang chiến lược phân vùng mặc định - round robin (vòng tròn). Trong round robin, các bản ghi được phân tán tuần tự đến các phân vùng có sẵn.

5.3.3. Nhật ký phân vùng (partition log) và vị trí (offset)

Trong Kafka, thứ tự được đảm bảo cho các bản ghi chỉ bên trong một phân vùng riêng lẻ. Điều này có nghĩa là các bản ghi được thêm vào một phân vùng sẽ được đọc ra bởi từng consumer theo thứ tự chúng được thêm vào. Mỗi phân vùng hoạt động giống như một nhật ký cam kết có cấu trúc (structured commit log), nơi các bản ghi được thêm liên tục vào cuối tệp nhật ký. Mỗi bản ghi trong một phân vùng được gán một số ID tuần tự duy nhất, được

gọi là offset. Offset đóng vai trò như một địa chỉ, giúp xác định vị trí chính xác của một bản ghi cụ thể trong phân vùng. Bằng cách theo dõi offset, các consumer có thể theo dõi tiến trình của họ và tiếp tục với các bản ghi mới chưa được xử lý trước đó.

Tuy nhiên, cần lưu ý là các bản ghi đi vào các partition khác nhau của cùng một topic có thể không theo thứ tự. Điều này là do Kafka chia dữ liệu thành các partition để phân tải và xử lý tải công việc. Ví dụ, nếu bạn gửi các message A, B, và C vào partition 1 của một Kafka topic theo thứ tự A, B, và C, thì khi đọc từ partition 1, bạn sẽ nhận được chúng theo thứ tự đó. Tuy nhiên, nếu bạn gửi các message A, B, và C vào các partition khác nhau của cùng một topic, thì khi đọc từ các partition đó, bạn có thể nhận được chúng theo thứ tự nào tùy thuộc vào cơ chế phân phối và xử lý của Kafka.

5.3.4. Bản sao phân vùng (partition replication) trong Kafka

Một tính năng cốt lõi của Kafka Topics là tổ chức các bản sao phân vùng (partition replication). Đây là cơ chế sao chép dữ liệu của các phân vùng trên nhiều máy chủ, nhằm đảm bảo tính sẵn sàng cao và khả năng chịu lỗi cho hệ thống.

Mỗi phân vùng có nhiều bản sao, lưu trữ trên nhiều broker khác nhau. Trong đó xác định được:

Leader Mỗi phân vùng được chỉ định một máy chủ leader duy nhất. Máy chủ leader đóng vai trò như "trung tâm điều khiển" của phân vùng, chịu trách nhiệm xử lý tất cả các yêu cầu đọc và ghi dữ liệu.

Máy chủ follower Bên cạnh leader, mỗi phân vùng có thể có một hoặc nhiều máy chủ follower. Các follower thụ động sao chép dữ liệu từ leader, đảm bảo chúng có bản sao cập nhật của phân vùng.

Không phải tất cả các follower đều có vai trò ngang bằng. Kafka duy trì một danh sách được gọi là ISR (In-sync Replicas), bao gồm các follower được đồng bộ hóa dữ liệu với leader. Những follower này được coi là có bản sao cập nhật mới nhất (up-to-date replicas) và giống hoàn toàn với bản sao ở leader. Như vậy chúng đủ điều kiện để trở thành leader mới trong trường hợp leader chính bị lỗi.

Tính năng replication đảm bảo khả năng chịu lỗi của Kafka. Nếu máy chủ leader gặp sự cố, Kafka sẽ tự động tiến hành bầu chọn một leader mới từ danh sách ISR. Quá trình này diễn ra nhanh chóng, giúp giảm thiểu thời gian gián đoạn dịch vụ và đảm bảo dữ liệu vẫn có sẵn cho các consumer.

Khi thiết lập cơ chế nhân bản cần quan tâm tới 2 tham số là hệ số sao chép (replication factor) và số ISR tối thiểu. Hệ số sao chép cho biết dữ liệu của phân vùng sẽ được sao chép sang bao nhiêu máy chủ follower. Số lượng bản sao càng cao, khả năng chịu lỗi của hệ thống

càng tốt. Số ISR tối thiểu là số lượng bản sao tối thiểu cần được đồng bộ xong với leader trong mỗi thao tác ghi vào phân vùng để thao tác ghi này được xác định là thành công. Việc thiết lập Minimum ISR cao hơn sẽ cải thiện tính sẵn sàng dữ liệu nhưng có thể ảnh hưởng đến hiệu suất ghi. Ngược lại, Minimum ISR thấp hơn có thể cải thiện hiệu suất nhưng tiềm ẩn rủi ro mất mát dữ liệu. Do đó, cần lựa chọn giá trị Minimum ISR phù hợp dựa trên yêu cầu cụ thể của hệ thống.

5.3.5. Thời gian tồn tại của bản ghi trong Kafka topic

Trong Kafka, cơ chế giữ lại bản ghi (record retention) đóng vai trò then chốt trong việc quản lý lưu trữ dữ liệu của các topics. Mặc dù mặc định Kafka sẽ lưu trữ tất cả các bản ghi được published (xuất bản), chúng ta có thể cấu hình chiến lược giữ lại bản ghi phù hợp để tối ưu hóa việc sử dụng dung lượng lưu trữ và đáp ứng các yêu cầu về vòng đời dữ liệu. Kafka cung cấp ba chiến lược chính để giữ lại bản ghi:

Giữ theo thời gian Chiến lược này cho phép bạn xác định thời gian tối đa mà Kafka sẽ lưu trữ một bản ghi trước khi tự động xóa nó. Bạn có thể thiết lập retention period (thời gian giữ) bằng đơn vị như giây, phút, giờ, ngày hoặc tuần. Ví dụ, bạn có thể thiết lập retention policy (chính sách giữ) là ba ngày, hai tuần hoặc một tháng.

Giữ theo kích thước Chiến lược này cho phép bạn xác định dung lượng tối đa mà Kafka sẽ lưu trữ cho các bản ghi trong một phân vùng trước khi xóa các bản ghi cũ hơn để giải phóng dung lượng. Giá trị này được đặt bằng byte hoặc megabyte.

Chính sách làm gọn nhẹ (Compaction) Làm gọn nhẹ giúp giảm dung lượng lưu trữ bằng cách chỉ giữ lại bản ghi mới nhất có cùng key (khóa) trong một phân vùng. Điều này hữu ích cho các trường hợp bạn chỉ quan tâm đến giá trị mới nhất cho một key cụ thể.

Bất kỳ bản ghi nào được xuất bản đến một Topic đều ở đó cho consumer thực hiện việc tiêu thụ cho đến khi nó bị xóa theo chính sách retention hoặc compaction. Kích thước của topic không ảnh hưởng đến tốc độ tiêu thụ dữ liệu. Consumer vẫn có thể đọc dữ liệu hiệu quả ngay cả khi dung lượng Topic lớn vì các bản ghi được lưu và đọc trong các phân vùng một cách tuần tự.

5.4. Sản xuất và tiêu thụ dữ liệu trong Kafka

5.4.1. Producer: Sản xuất dữ liệu

Producer là thành phần chịu trách nhiệm tạo ra và gửi dữ liệu đến Kafka. Đây là điểm khởi đầu của luồng dữ liệu, nơi các ứng dụng hoặc hệ thống bên ngoài đưa dữ liệu vào Kafka topics.

Các đặc điểm chính của Producer

Định tuyến dữ liệu Producer có thể gửi dữ liệu đến topic cụ thể hoặc để Kafka tự động chọn topic dựa trên cấu hình. Producer cũng có thể chỉ định partition key để đảm bảo các bản ghi có cùng key được gửi đến cùng partition.

Đảm bảo độ tin cậy Producer hỗ trợ ba mức độ đảm bảo ghi dữ liệu:

- **acks=0:** Không chờ xác nhận từ broker (at-most-once)
- **acks=1:** Chờ xác nhận từ leader partition (at-least-once)
- **acks=all:** Chờ xác nhận từ tất cả replica trong ISR (exactly-once)

Ghi theo lô (Batching) Producer có thể gom nhiều bản ghi thành một batch trước khi gửi, giúp tăng hiệu suất truyền tải. Các tham số quan trọng bao gồm:

- `batch.size`: Kích thước tối đa của mỗi batch (bytes)
- `linger.ms`: Thời gian chờ để gom batch (milliseconds)
- `buffer.memory`: Dung lượng bộ đệm cho producer

Xử lý lỗi và thử lại Producer tự động xử lý các lỗi mạng và broker, thực hiện thử lại theo cấu hình. Các tham số quan trọng:

- `retries`: Số lần thử lại tối đa
- `retry.backoff.ms`: Thời gian chờ giữa các lần thử lại
- `delivery.timeout.ms`: Thời gian timeout tổng thể cho việc ghi

Các loại Producer

Sync Producer Gửi dữ liệu đồng bộ, chờ xác nhận trước khi gửi bản ghi tiếp theo. Đảm bảo độ tin cậy cao nhưng hiệu suất thấp hơn.

Async Producer Gửi dữ liệu bất đồng bộ, sử dụng callback để xử lý kết quả. Hiệu suất cao hơn nhưng có thể mất dữ liệu trong trường hợp lỗi.

Idempotent Producer Đảm bảo mỗi bản ghi chỉ được ghi một lần, ngay cả khi có lỗi và thử lại. Sử dụng sequence number để tránh trùng lặp.

Ví dụ cấu hình Producer

Một Producer cho hệ thống ghi log có thể được cấu hình như sau:

- `acks=1`: Chấp nhận rủi ro mất dữ liệu để đạt hiệu suất cao
- `batch.size=16384`: Batch 16KB để tối ưu hiệu suất mạng

- `linger.ms=5`: Chờ tối đa 5ms để gom batch
- `compression.type=snappy`: Nén dữ liệu để tiết kiệm băng thông
- `retries=3`: Thử lại tối đa 3 lần khi gặp lỗi

5.4.2. Consumer: Tiêu thụ dữ liệu

Nhóm tiêu thụ (consumer group) là một tính năng quan trọng của Kafka, giúp tăng cường khả năng mở rộng và tính song song cho việc tiêu thụ dữ liệu từ Kafka topics. Nhóm tiêu thụ hoạt động như một tập hợp các consumer làm việc cùng nhau để lấy ra các bản ghi trong một Topic. Nhóm tiêu thụ có các đặc điểm sau:

Nhận dạng nhóm Mỗi nhóm người tiêu dùng được xác định bởi một ID duy nhất. ID này giúp Kafka phân biệt các nhóm khác nhau và phân phối dữ liệu phù hợp.

Chia sẻ trách nhiệm Các consumer trong cùng một nhóm người tiêu dùng chia sẻ trách nhiệm tiêu thụ dữ liệu từ một Topic. Mỗi consumer sẽ được chỉ định một hoặc nhiều phân vùng (partition) trong Topic để xử lý.

Phân bố song song Kafka không phân phối cùng một bản ghi cho nhiều consumer trong cùng một nhóm người tiêu dùng. Điều này đảm bảo mỗi bản ghi chỉ được xử lý bởi một consumer duy nhất, tránh tình trạng xử lý trùng lặp.

Tự động cân bằng tải Kafka tự động cân bằng tải dữ liệu giữa các consumer trong nhóm. Nếu có consumer mới tham gia nhóm hoặc một consumer cũ rời khỏi nhóm, Kafka sẽ tự động phân bổ lại các phân vùng để đảm bảo mỗi consumer xử lý lượng dữ liệu tương đương.

Tính sẵn sàng cao Nhóm người tiêu dùng đảm bảo tính sẵn sàng cao cho việc tiêu thụ dữ liệu. Nếu một consumer trong nhóm gặp sự cố, các consumer còn lại sẽ tiếp tục xử lý dữ liệu của phân vùng đó.

Tính duy nhất Mỗi consumer trong nhóm người tiêu dùng phải có cùng consumer group id. Một consumer chỉ có thể thuộc về một nhóm người tiêu dùng tại một thời điểm.

Xét mối liên quan giữa số consumer trong 1 nhóm tiêu thụ với số lượng phân vùng của 1 topic. Trường hợp số lượng consumer vượt quá số lượng phân vùng, các consumer thừa sẽ ở trạng thái chờ (idle). Tuy nhiên, chúng vẫn có vai trò quan trọng trong việc xử lý sự cố (failover). Khi một consumer đang hoạt động gặp sự cố, một consumer chờ sẽ thay thế để tiếp tục xử lý dữ liệu. Ngược lại, nếu số lượng phân vùng nhiều hơn số lượng consumer, một số consumer sẽ phải đọc dữ liệu từ nhiều phân vùng. Kafka sẽ tự động phân bổ các phân vùng giữa các consumer để đảm bảo cân bằng tải.

5.4.3. Cơ chế xử lý sự cố của consumer

Trong Kafka, khả năng xử lý sự cố consumer (consumer failover) là yếu tố quan trọng đảm bảo tính sẵn sàng cao và xử lý dữ liệu đáng tin cậy. Khi một consumer thành công xử lý một bản ghi, nó sẽ gửi thông báo xác nhận (commit offset) đến Kafka broker. Thông báo này bao gồm offset, đánh dấu vị trí cuối cùng mà consumer đã đọc dữ liệu trong phân vùng. Kafka lưu trữ offset cho mỗi consumer một cách riêng biệt, thường được đặt tên là `__consumer_offset`.

Nếu một consumer ngừng gửi commit offset trong một khoảng thời gian nhất định (session.timeout.ms), Kafka broker sẽ coi consumer đó đã gặp sự cố. Nó sẽ chọn một consumer khác khỏe mạnh trong cùng nhóm tiêu thụ để tiếp tục xử lý dữ liệu. Consumer mới sẽ bắt đầu xử lý dữ liệu từ offset cuối cùng được commit bởi consumer bị lỗi. Cơ chế failover này đảm bảo mỗi bản ghi được xử lý ít nhất một lần (at least once delivery semantics). Nếu consumer bị lỗi trước khi kịp commit offset cho bản ghi cuối cùng đã xử lý thì consumer mới có nguy cơ xử lý lại bản ghi đó.

5.4.4. Giới hạn về dữ liệu Consumer có thể đọc trong Kafka

Kafka sử dụng hai khái niệm quan trọng liên quan đến vị trí dữ liệu trong một phân vùng:

Offset cuối (Log end offset) Đây là offset đánh dấu vị trí của bản ghi cuối cùng được ghi vào phân vùng. Producer sẽ ghi các bản ghi mới bắt đầu từ vị trí này.

Mức đánh dấu cao (High watermark) Đây là offset đánh dấu vị trí của bản ghi cuối cùng đã được sao chép thành công đến tất cả các follower replica trong phân vùng.

Một điểm quan trọng cần lưu ý là consumer chỉ có thể đọc dữ liệu lên đến high watermark. Điều này đảm bảo tính toàn vẹn dữ liệu. Việc cho phép consumer đọc dữ liệu chưa được sao chép đầy đủ có thể dẫn đến tình trạng consumer ở nhóm tiêu thụ này có thể không đọc được đầy đủ dữ liệu như nhóm tiêu thụ khác do bản ghi chưa được sao chép đã bị mất mát.

5.4.5. Khả năng mở rộng và tốc độ của Kafka

Kafka được thiết kế để xử lý khối lượng dữ liệu lớn với tốc độ cao và khả năng mở rộng linh hoạt. Dưới đây là cách Kafka đạt được điều này khi nhiều producer và consumer cùng đọc/ghi vào cùng một log Topic:

Ghi tuần tự (Sequential writes) trên từng partition Kafka tận dụng khả năng ghi tuần tự của ổ cứng vì ghi trên mỗi partition là ghi tuần tự vào cuối tệp tin log. Điều này cho phép ghi với tốc độ rất nhanh, thường lên đến 700 MB/giây hoặc hơn. So với ghi ngẫu

nhiên, ghi tuần tự giúp giảm thiểu hoạt động đọc/ghi dữ liệu phân tán trên ổ cứng, cải thiện đáng kể hiệu suất ghi.

Ghi song song (Parallel writes) Nhiều producer có thể ghi dữ liệu song song vào các phân vùng khác nhau của cùng một Topic, cải thiện đáng kể tốc độ ghi tổng thể.

Đọc song song (Parallel reads) Nhiều consumer group (nhóm người tiêu dùng) có thể đọc dữ liệu song song từ các phân vùng khác nhau của cùng một Topic, cải thiện tốc độ đọc tổng thể.

Ghép lô (Batching) Thay vì đọc và ghi các bản ghi riêng lẻ, Kafka có cơ chế đọc và ghi theo lô. Việc này giúp giảm chi phí mạng liên quan đến việc thiết lập kết nối, dẫn đến cải thiện hiệu suất đáng kể, đặc biệt đối với các bản ghi nhỏ.

5.5. Tóm tắt chương

Chương này đã giới thiệu về hệ thống truyền thông điệp phân tán, tập trung vào mô hình pub-sub và Apache Kafka. Các điểm chính bao gồm:

- Mô hình pub-sub giúp tách biệt producer và consumer, tăng tính linh hoạt và khả năng mở rộng của hệ thống phân tán
- Apache Kafka là platform truyền thông điệp phân tán với khả năng xử lý luồng dữ liệu lớn, độ tin cậy cao
- Các khái niệm cốt lõi của Kafka: topic, partition, replication, consumer group và cách chúng phối hợp để đảm bảo hiệu năng và độ tin cậy
- Cơ chế đảm bảo độ tin cậy thông qua replication, ISR và cơ chế xử lý sự cố của consumer
- Khả năng mở rộng và tối ưu hiệu năng của Kafka thông qua ghi tuần tự, ghi/đọc song song và cơ chế ghép lô

5.6. Thuật ngữ quan trọng

Publisher/Subscriber Mô hình truyền thông điệp phi đồng bộ cho phép tách biệt giữa nguồn phát và nguồn nhận thông tin thông qua một broker trung gian

Message Broker Thành phần trung gian trong hệ thống pub-sub, chịu trách nhiệm nhận, lưu trữ và phân phối thông điệp giữa publisher và subscriber

Topic Luồng dữ liệu được đặt tên trong Kafka, nơi các thông điệp được tổ chức và lưu trữ theo một chủ đề hoặc mục đích cụ thể

Partition Đơn vị phân tán dữ liệu nhỏ nhất của topic trong Kafka, cho phép phân tán tải và xử lý song song

Consumer Group Nhóm các consumer làm việc cùng nhau để xử lý dữ liệu từ một topic, tự động cân bằng tải và đảm bảo khả năng mở rộng

Replication Cơ chế sao chép dữ liệu giữa các broker trong Kafka để đảm bảo độ tin cậy và khả năng chịu lỗi

ISR (In-Sync Replicas) Tập các replica đồng bộ với leader, đảm bảo tính nhất quán dữ liệu trong Kafka

5.7. Câu hỏi ôn tập

1. So sánh ưu nhược điểm của mô hình pub-sub với mô hình point-to-point messaging? Trong trường hợp nào nên sử dụng mỗi mô hình?
2. Phân tích vai trò của partition trong Kafka. Tại sao Kafka sử dụng cơ chế partition và những thách thức khi thiết kế số lượng partition cho một topic?
3. Giải thích cơ chế đảm bảo tin cậy trong Kafka thông qua replication và ISR. Làm thế nào để cân bằng giữa độ tin cậy và hiệu năng khi cấu hình replication?
4. Một hệ thống thương mại điện tử cần xử lý đơn hàng theo thời gian thực và đồng bộ dữ liệu giữa các service. Hãy thiết kế kiến trúc sử dụng Kafka cho hệ thống này, bao gồm:
 - Xác định các topic cần thiết và chiến lược partition
 - Thiết kế consumer group và cơ chế xử lý message
 - Đề xuất cấu hình về retention, replication để đảm bảo độ tin cậy
 - Phân tích các kịch bản lỗi và cách xử lý
5. Phân tích các yếu tố ảnh hưởng đến hiệu năng của Kafka và đề xuất các giải pháp tối ưu cho:
 - Producer: batch size, linger time, compression
 - Consumer: fetch size, commit strategy
 - Broker: partition allocation, disk I/O

Chương 6

MapReduce – Mô hình lập trình cho xử lý dữ liệu lớn song song & phân tán

6.1. Giới thiệu về MapReduce	195
6.2. Mô hình lập trình và kiến trúc	196
6.3. Tối ưu hóa và xử lý lỗi	199
6.4. Mẫu thiết kế và lập trình	202
6.5. Giới hạn của MapReduce và bức tranh hệ sinh thái	203
6.6. Ví dụ thực tế	204
6.7. Kết luận	207
6.8. Bài tập và câu hỏi ôn tập	207

6.1. Giới thiệu về MapReduce

MapReduce được Google giới thiệu như một mô hình lập trình và một hệ thống thực thi giúp đơn giản hóa xử lý dữ liệu khối lượng rất lớn trên cụm máy hàng nghìn node. Ý tưởng cốt lõi: lập trình viên chỉ viết hai hàm map và reduce trên cặp (*key, value*), phần còn lại (song song hóa, lập lịch, chịu lỗi, giao tiếp mạng, gom nhóm theo khóa) do hệ thống đảm nhiệm [1],[2]. Các triển khai như Hadoop đã biến MapReduce thành nền tảng phổ biến cho nhiều bài toán: lập chỉ mục web, thống kê log, học máy, lọc – biến đổi dữ liệu, sắp xếp phân tán, xây dựng inverted index, v.v. [1],[2].

Sự thành công của MapReduce nương tựa vào hệ lưu trữ phân tán HDFS (lấy cảm hứng từ GFS) với khối lớn và nhân bản đa bản, cho phép “đẩy tính toán đến dữ liệu” nhằm giảm chi phí truyền thông [4],[1].

6.2. Mô hình lập trình và kiến trúc

6.2.1. Mô hình lập trình MapReduce

Khái niệm cơ bản về Job và Task

Một chương trình MapReduce được gọi là một **job**, được phân rã thành nhiều **task** độc lập và phân tán trên các node khác nhau của cụm để thực thi. Mỗi task được thực thi độc lập với các task khác, giúp đạt được tính khả mở (scalability) và giảm thiểu truyền thông giữa các node máy chủ. Điều này cũng tránh phải thực hiện cơ chế đồng bộ phức tạp giữa các task.

Trong môi trường Hadoop, MapReduce thường làm việc với dữ liệu có sẵn trên HDFS. Khi thực thi, mã chương trình MapReduce được gửi tới các node đã có dữ liệu tương ứng, tuân theo nguyên tắc "đẩy tính toán đến dữ liệu" để tối ưu hiệu suất.

Kiểu dữ liệu và hàm người dùng

MapReduce hoạt động trên kiểu dữ liệu cơ bản là danh sách các cặp (*key, value*). Trong chương trình MapReduce, dữ liệu được nhìn nhận như là các cặp khóa-giá trị, và các hàm Map và Reduce nhận đầu vào và trả về đầu ra đều là các cặp (*key, value*).

Hàm **Map** nhận đầu vào là một cặp (*k1, v1*) và trả về danh sách các cặp (*k2, v2*). Hàm này có thể biến đổi hoặc chiết tách dữ liệu, tạo ra từ 0 đến N cặp đầu ra. Hàm **Reduce** nhận đầu vào là một khóa *k2* cùng với danh sách các giá trị [*v2*] và trả về danh sách các cặp (*k3, v3*). Khung MapReduce đảm bảo rằng mọi giá trị có cùng khóa sẽ được gom về cùng một tác vụ reduce, nhờ cơ chế shuffle và partition.

Ví dụ minh họa: Tính doanh số theo nhân viên

Để minh họa cách hoạt động của MapReduce, xét bài toán tính tổng doanh số bán hàng theo từng nhân viên. **Đầu vào** là tệp văn bản chứa thông tin về order ID, employee name, và sale amount. **Đầu ra** là doanh số bán (sales) theo từng nhân viên (employee).

Bước Map: Dữ liệu đầu vào được xử lý bởi nhiều tác vụ Mapping độc lập. Số lượng các tác vụ Mapping được xác định theo lượng dữ liệu đầu vào (tương ứng với số chunks). Mỗi tác vụ Mapping xử lý một phần dữ liệu (chunk) của khối dữ liệu ban đầu. Với mỗi tác vụ Mapping, Mapper xử lý lần lượt từng bản ghi đầu vào. Với mỗi bản ghi đầu vào (key-value), Mapper đưa ra 0 hoặc nhiều bản ghi đầu ra (key-value trung gian). Trong ví dụ này, tác vụ Mapping đơn giản đọc từng dòng văn bản và đưa ra tên nhân viên và doanh số tương ứng.

Bước Shuffle & Sort: Hadoop tự động sắp xếp và gộp đầu ra của các Mappers theo các partitions. Mỗi partition là đầu vào cho một Reducer.

Bước Reduce: Reducer nhận dữ liệu đầu vào từ bước shuffle & sort. Tất cả các bản ghi key-value tương ứng với một key được xử lý bởi một Reducer duy nhất. Giống bước Map, Reducer xử lý lần lượt từng key, mỗi lần với toàn bộ các values tương ứng. Trong ví dụ, hàm reduce đơn giản là tính tổng doanh số cho từng nhân viên, đầu ra là các cặp key-value tương ứng với tên nhân viên - doanh số tổng.

Lưu ý: Ví dụ chi tiết hơn với dữ liệu cụ thể và hình ảnh minh họa sẽ được trình bày ở phần "Ví dụ thực tế" cuối chương này.

Các thành phần mở rộng

Partitioner ánh xạ khóa đến R vùng (thường sử dụng $\text{hash}(k) \bmod R$). Thành phần này có thể được tùy biến để kiểm soát cân bằng tải và tính địa phương dữ liệu.

Combiner thực hiện tổng hợp cục bộ sau giai đoạn map, giảm kích thước dữ liệu trong quá trình shuffle. Combiner chỉ hoạt động hiệu quả khi phép gộp có tính kết hợp và giao hoán.

Comparator và **Grouping comparator** quyết định thứ tự và cách nhóm khóa trong quá trình sort và merge.

InputFormat/RecordReader và **OutputFormat/RecordWriter** chuyển đổi giữa bytes và các cặp (k, v) , đồng thời quản lý splits và commit đầu ra.

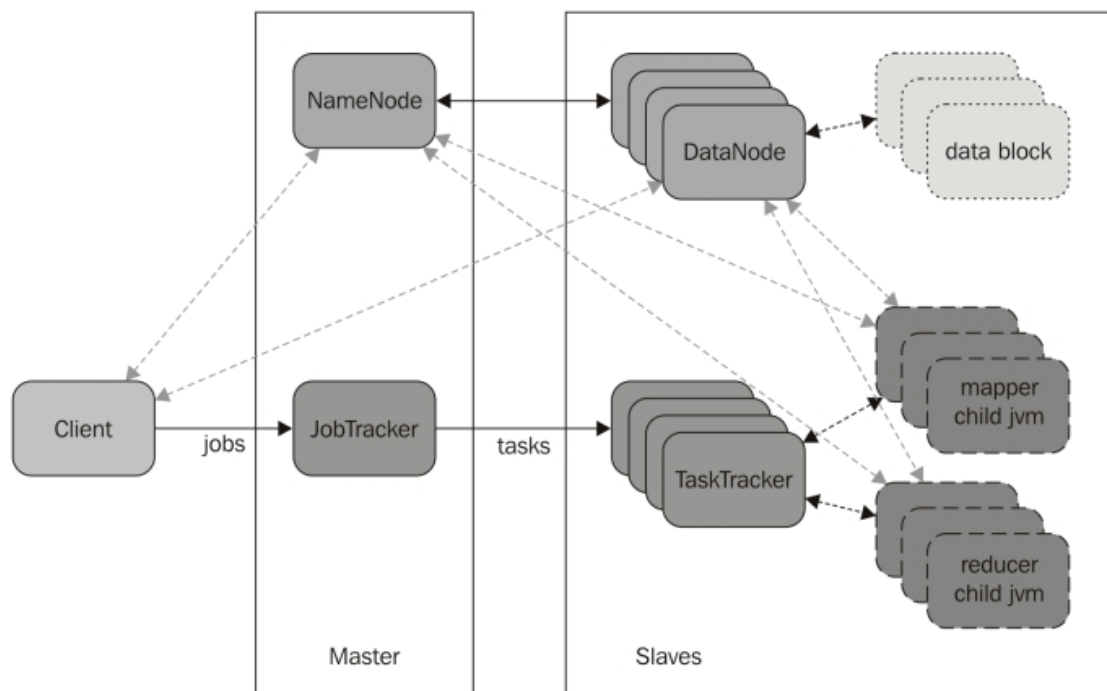
6.2.2. Kiến trúc thực thi và vòng đời job

Hadoop MRv1 (Kiến trúc cổ điển)

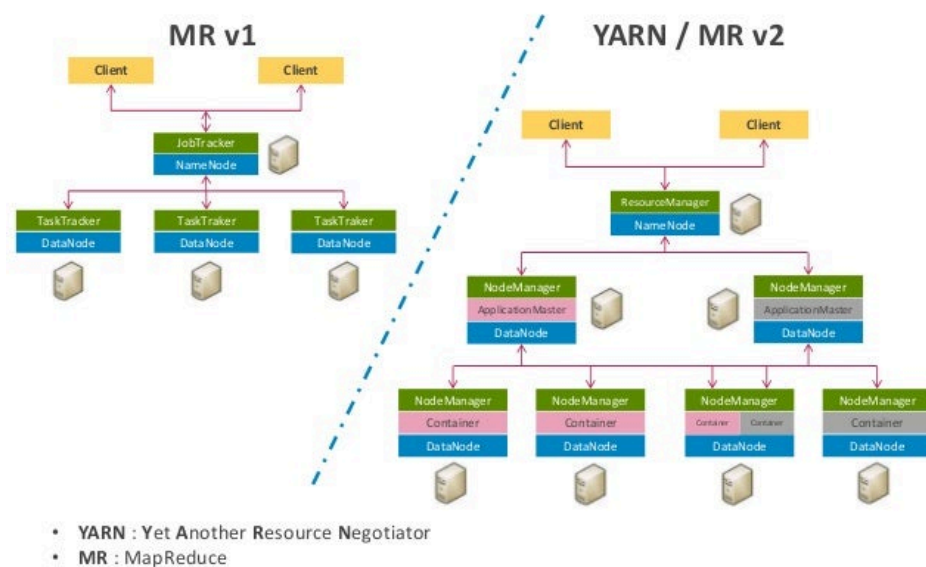
Trong kiến trúc MRv1, *JobTracker* chịu trách nhiệm lập lịch và theo dõi các tác vụ, trong khi *TaskTracker* thực thi các tác vụ map và reduce. Mô hình này đơn giản nhưng có tính "đơn khối", hạn chế khả năng mở rộng và dễ gây nghẽn cổ chai tại JobTracker.

Hadoop MRv2 trên YARN

Kiến trúc MRv2 trên YARN tách biệt quản lý tài nguyên (ResourceManager/NodeManager) khỏi mô hình tính toán. Mỗi job có ApplicationMaster riêng để tự điều phối vòng đời tác vụ. MapReduce trong kiến trúc này chỉ là một framework chạy trên YARN, cạnh tranh tài nguyên với các framework khác như Spark, Tez.

Hình 1: Kiến trúc MRv1 *JobTracker* và *TaskTracker*

MR VS. YARN ARCHITECTURE



Hình 2: Kiến trúc MRv2

Input splitting và record reading

Tập tin đầu vào được lưu trữ trên HDFS và được chia thành các splits, thường có kích thước tương đương với block HDFS. Các splits này được phân phối cho các map task để xử lý song song. RecordReader có nhiệm vụ chuyển đổi dữ liệu từ định dạng bytes thành các cặp (k, v) . Hệ thống ưu tiên lập lịch các map task gần với dữ liệu (data locality) để giảm thiểu lưu lượng mạng.

Giai đoạn Map: spill, sort, partition và combine

Đầu ra từ các map task được buffer trong bộ nhớ và spill ra đĩa khi đạt ngưỡng nhất định. Dữ liệu được sắp xếp theo khóa và phân chia thành R phần để chuẩn bị cho giai đoạn shuffle. Nếu được cấu hình, combiner sẽ thực hiện tổng hợp cục bộ để giảm kích thước dữ liệu.

Shuffle và sort phía reduce

Các reduce task thu thập các phân vùng tương ứng từ tất cả map task thông qua giao thức HTTP. Sau đó, chúng thực hiện merge-sort theo khóa và tạo các nhóm $(k, [v])$ để chuẩn bị cho việc gọi hàm reduce. Quá trình shuffle và sort diễn ra đồng thời (pipelining) để tối ưu hiệu suất. Hadoop hiện tại hỗ trợ pluggable shuffle/sort để thay đổi giao thức (như RDMA) hoặc thuật toán sắp xếp.

Output commit và tính nguyên tử

Reduce task ghi kết quả cuối cùng ra HDFS, sử dụng cơ chế "rename nguyên tử" để đảm bảo tính nhất quán và tránh trùng lặp khi có retry.

6.3. Tối ưu hóa và xử lý lỗi

6.3.1. Chịu lỗi và fault tolerance

MapReduce có khả năng phát hiện lỗi node, thực hiện lại các task thất bại và quản lý tiến độ thông qua cơ chế heartbeat. Vấn đề stragglers (các tác vụ chạy bất thường chậm) được giảm thiểu thông qua backup tasks hay speculative execution. Khi job gần hoàn thành, hệ thống sẽ chạy bản sao suy đoán của các task chậm, và task nào hoàn thành trước sẽ được chấp nhận.

Trên Hadoop, speculative execution cho map và reduce có thể được bật/tắt và điều chỉnh bằng các thông số cụ thể. Ngoài ra, nhiều nghiên cứu đã tập trung vào xử lý skew và straggler,

như SkewTune tự động phát hiện và chia nhỏ lại các phần việc ”nóng” trong reduce để cân bằng tải.

6.3.2. HDFS và tính cục bộ dữ liệu

HDFS lưu trữ các tệp thành các block lớn (thường từ 128MB trở lên) và thực hiện nhân bản (thường là 3 bản sao). Bộ lập lịch ưu tiên chạy các map task trên cùng node (node-local) hoặc cùng rack (rack-local) nơi có bản sao của block, nhằm giảm thiểu shuffle distance.

HDFS được thiết kế để tối ưu cho thông lượng cao và khối lượng dữ liệu lớn, đặc biệt phù hợp với các tác vụ ghi tuần tự và đọc tuần tự - điều này hoàn toàn phù hợp với pattern của MapReduce.

6.3.3. Giảm chi phí shuffle

Giai đoạn shuffle thường là bottleneck chính trong MapReduce vì nó liên quan đến việc truyền dữ liệu qua mạng giữa các node. Có nhiều kỹ thuật để tối ưu hóa quá trình này. **Combiner và in-mapper combining** là hai phương pháp hiệu quả để giảm kích thước dữ liệu trung gian trước khi shuffle. Combiner thực hiện tổng hợp cục bộ tại mapper, trong khi in-mapper combining tích hợp logic tổng hợp trực tiếp vào hàm map để tránh overhead của combiner.

Nén dữ liệu là một kỹ thuật quan trọng khác. Việc nén dữ liệu đầu ra của map và trên HDFS giúp tiết kiệm đáng kể băng thông mạng và dung lượng đĩa. Tuy nhiên, cần cân đối giữa lợi ích tiết kiệm băng thông với chi phí CPU để nén/giải nén. Các thuật toán nén như Snappy hoặc LZ4 thường được ưa chuộng vì có tốc độ cao.

Tối ưu số reducer là một yếu tố quan trọng khác. Số reducer quá ít (under-partition) có thể dẫn đến tải không đều và giảm hiệu suất song song. Ngược lại, số reducer quá nhiều (over-partition) tạo ra nhiều file nhỏ và tăng overhead quản lý. Quy tắc chung là đặt số reducer bằng khoảng 0.95-1.75 lần số slot reduce có sẵn trong cluster.

Custom partitioner cho phép kiểm soát cách phân phối dữ liệu đến các reducer. Một partitioner tốt sẽ cân bằng tải và gom các khóa liên quan vào cùng partition để giảm giao tiếp chéo. Cuối cùng, việc sử dụng **định dạng file nhị phân** như SequenceFile, Avro, hoặc Parquet cùng với batch size nhỏ và IO buffer hợp lý có thể cải thiện đáng kể hiệu suất I/O.

6.3.4. Xử lý data skew

Data skew là một vấn đề phổ biến trong MapReduce, xảy ra khi một số khóa xuất hiện với tần suất cực cao so với các khóa khác. Điều này dẫn đến tình trạng một số reducer bị quá tải trong khi các reducer khác hoàn thành sớm và phải chờ đợi. Ví dụ, trong phân tích log web, một số URL có thể được truy cập hàng triệu lần trong khi các URL khác chỉ có vài

lần truy cập.

Lấy mẫu (sampling) là kỹ thuật đầu tiên để giải quyết vấn đề này. Bằng cách phân tích một mẫu nhỏ của dữ liệu trước khi chạy job chính, ta có thể hiểu được phân phối khóa và thiết kế partitioner phù hợp. Partitioner thông minh sẽ chia dữ liệu sao cho mỗi reducer nhận được khối lượng công việc tương đương.

Tách khóa nóng (key-salting) là một kỹ thuật tinh vi hơn. Thay vì để một khóa nóng được xử lý bởi một reducer duy nhất, ta chia nó thành nhiều khóa con bằng cách thêm suffix ngẫu nhiên. Ví dụ, khóa "popular_url" có thể được chia thành "popular_url#0", "popular_url#1", "popular_url#2". Sau đó, ta chạy một job MapReduce thứ hai để hợp nhất kết quả từ các khóa con này.

Kiến trúc hai tầng reduce (reduce-reduce) là giải pháp cho các trường hợp phức tạp hơn. Tầng đầu tiên xử lý dữ liệu thô và tạo ra các nhóm nhỏ hơn. Tầng thứ hai thực hiện phép tổng hợp cuối cùng. Phương pháp này đặc biệt hiệu quả khi xử lý các thuật toán phức tạp như PageRank hoặc các thuật toán đồ thị khác.

Cuối cùng, **tự động tái cân bằng** sử dụng các hệ thống như SkewTune để theo dõi hiệu suất job trong thời gian thực và tự động điều chỉnh phân phối tải. Các hệ thống này có thể phát hiện straggler (các task chạy chậm bất thường) và tự động chia nhỏ chúng thành các phần nhỏ hơn để xử lý song song.

6.3.5. Speculative execution và cấu hình

Speculative execution là một cơ chế quan trọng trong MapReduce để xử lý vấn đề straggler - những task chạy chậm bất thường so với các task khác trong cùng job. Cơ chế này hoạt động bằng cách chạy bản sao dự phòng (backup tasks) của các task chậm trên các node khác. Task nào hoàn thành trước sẽ được chấp nhận, còn task còn lại sẽ bị hủy bỏ.

Tuy nhiên, việc sử dụng speculative execution cần được cân nhắc cẩn thận. Khi cluster có tải nặng hoặc dữ liệu nóng (được truy cập thường xuyên), việc chạy thêm backup tasks có thể làm tăng áp lực lên hệ thống. Hơn nữa, nếu job có tác dụng phụ (side effects) như ghi file hoặc gửi email, speculative execution có thể dẫn đến việc thực hiện các tác dụng phụ này nhiều lần.

Trên Hadoop, có nhiều thông số cấu hình để kiểm soát speculative execution, bao gồm thời gian chờ trước khi bắt đầu backup task, tỷ lệ task chậm để kích hoạt speculative execution, và các điều kiện khác. Việc điều chỉnh các thông số này cần dựa trên đặc điểm của cluster và workload cụ thể để đảm bảo hiệu quả tối ưu mà không lãng phí tài nguyên.

6.4. Mẫu thiết kế và lập trình

6.4.1. Biến đổi và tổng hợp

Các mẫu thiết kế cơ bản trong MapReduce thường xoay quanh việc biến đổi và tổng hợp dữ liệu. **WordCount** và **Histogram** là những ví dụ kinh điển của pattern này. Trong WordCount, Map task phát thải các cặp (từ, 1) cho mỗi từ xuất hiện trong tài liệu. Sau đó, Combiner và Reduce thực hiện cộng dồn để đếm tần suất xuất hiện của mỗi từ. Pattern này có thể được mở rộng cho nhiều bài toán khác như đếm số lần truy cập URL, thống kê theo địa lý, hoặc phân tích tần suất sự kiện.

Inverted Index là một pattern quan trọng khác, đặc biệt hữu ích cho các hệ thống tìm kiếm. Map task phát thải các cặp (từ, docID), trong khi Reduce task hợp nhất và sắp xếp các vị trí xuất hiện của từ trong các tài liệu. Điều này tạo ra một cấu trúc dữ liệu cho phép tìm kiếm nhanh các tài liệu chứa từ khóa cụ thể.

Filter, Top-K, và Distinct là các pattern xử lý dữ liệu phổ biến khác. Map task thực hiện lọc dữ liệu dựa trên các điều kiện cụ thể, trong khi Reduce task có thể sử dụng các cấu trúc dữ liệu nội bộ như heap để tìm top-K phần tử hoặc loại bỏ các bản ghi trùng lặp. Việc sử dụng partition hợp lý có thể tối ưu hiệu suất cho các bài toán này.

Một điểm quan trọng là các phép tổng hợp có tính kết hợp và giao hoán có thể khai thác combiner hoặc in-mapper combining để tiết kiệm đáng kể chi phí shuffle. Điều này đặc biệt quan trọng khi xử lý dữ liệu lớn, vì nó có thể giảm băng thông mạng và tăng hiệu suất tổng thể của job.

6.4.2. Join trên dữ liệu lớn

Join là một trong những thao tác phổ biến nhất trong xử lý dữ liệu, nhưng việc thực hiện join trên dữ liệu lớn trong MapReduce đòi hỏi các kỹ thuật đặc biệt. **Reduce-side join** là phương pháp phổ biến nhất, hoạt động bằng cách để Map task gắn nhãn nguồn dữ liệu và phát thải các bản ghi với cùng khóa join. Reduce task sau đó gom nhóm và thực hiện phép nối. Phương pháp này phù hợp với dữ liệu lớn vì không yêu cầu sắp xếp hoặc đồng phân mảnh trước, nhưng có nhược điểm là tốn chi phí shuffle do phải truyền tất cả dữ liệu qua mạng.

Map-side join và Composite join là các phương pháp tối ưu hơn, nhưng yêu cầu các tiền điều kiện nghiêm ngặt. Dữ liệu phải được đồng sắp xếp và đồng phân mảnh trước. CompositeInputFormat gộp dữ liệu trước khi đưa vào map task, giảm thiểu chi phí shuffle nhưng chỉ áp dụng được khi dữ liệu đã được chuẩn bị sẵn. Phương pháp này đặc biệt hiệu quả cho các bài toán như star schema join, nơi có một bảng dimension nhỏ và một bảng fact lớn.

6.4.3. Secondary sort

Secondary sort là một kỹ thuật quan trọng khi cần sắp xếp dữ liệu theo khóa phụ trong mỗi nhóm khóa chính. Ví dụ, khi phân tích log web, ta có thể muốn sắp xếp các sự kiện theo thời gian trong mỗi nhóm user ID. Để thực hiện điều này, ta sử dụng composite key (kết hợp khóa chính và khóa phụ) cùng với comparator tùy biến và grouping comparator. Comparator tùy biến quyết định thứ tự sắp xếp tổng thể, trong khi grouping comparator chỉ so sánh khóa chính để nhóm dữ liệu. Kết quả là reducer sẽ nhận được dữ liệu đã được sắp xếp theo thứ tự mong muốn, rất hữu ích cho các thuật toán cần xử lý dữ liệu theo thứ tự thời gian hoặc thứ tự khác.

6.4.4. Đồ thị và thuật toán lặp

Các thuật toán đồ thị và học máy như PageRank, Connected Components, K-means, và EM thường cần nhiều vòng lặp để hội tụ đến kết quả cuối cùng. Trong MapReduce, các thuật toán này được thực hiện thông qua nhiều vòng MapReduce với driver program kiểm soát sự hội tụ. Driver sẽ kiểm tra điều kiện dừng sau mỗi vòng và quyết định có tiếp tục hay không. Tuy nhiên, MapReduce không hiệu quả bằng các framework giữ trạng thái trong bộ nhớ như Spark hoặc Flink, vì mỗi vòng lặp đều phải ghi/đọc dữ liệu trung gian từ đĩa. Điều này dẫn đến overhead đáng kể và làm chậm quá trình hội tụ của các thuật toán lặp.

6.5. Giới hạn của MapReduce và bức tranh hệ sinh thái

MapReduce rất mạnh cho xử lý batch tuần tự với throughput cao, nhưng kém hiệu quả cho các bài toán lặp, tương tác, và streaming vì mọi vòng lặp đều phải ghi/đọc dữ liệu trung gian từ đĩa (barrier giữa map và reduce). Do đó, các hệ thống DAG tổng quát và bộ nhớ phân tán đã ra đời:

Dryad (Microsoft): Sử dụng đồ thị đỉnh-cạnh tổng quát với các ống dữ liệu file/pipe/FIFO, linh hoạt hơn MapReduce.

Spark (RDDs): Sử dụng bộ nhớ phân tán RDD với lineage để khôi phục, hỗ trợ lặp và khảo sát tương tác, thường nhanh hơn MapReduce cho thuật toán lặp do xử lý in-memory.

Flink: Thực thi dòng trạng thái (stateful streaming) nhất quán với snapshot phân tán, thống nhất batch và stream trong một động cơ.

Trong Hadoop, YARN biến MapReduce thành "một trong nhiều" framework chạy chung trên cụm, cho phép lựa chọn công cụ phù hợp cho từng lớp workload (MR, Spark, Tez, Flink).

6.5.1. Các chủ đề nâng cao

Ngoài các kỹ thuật cơ bản, MapReduce còn hỗ trợ nhiều tính năng nâng cao để xử lý các bài toán phức tạp. **Total order sort** là một trong những tính năng quan trọng, cho phép đảm bảo thứ tự toàn cục của dữ liệu đầu ra. Điều này đạt được thông qua việc sử dụng range partitioning sau bước lấy mẫu để xác định ranh giới phân vùng, đảm bảo rằng tất cả các bản ghi trong file part-r-00000 đều nhỏ hơn các bản ghi trong part-r-00001, và cứ thế tiếp tục.

Secondary sort là một kỹ thuật mạnh mẽ khác, cho phép sắp xếp dữ liệu theo khóa phụ trong mỗi nhóm khóa chính. Điều này được thực hiện thông qua việc sử dụng composite key (kết hợp khóa chính và khóa phụ) cùng với custom comparator và grouping comparator. Kết quả là reducer sẽ nhận được luồng giá trị đã được sắp xếp theo thứ tự mong muốn, rất hữu ích cho các thuật toán như PageRank hoặc các bài toán cần xử lý dữ liệu theo thứ tự thời gian.

Pluggable shuffle/sort cho phép thay đổi giao thức hoặc thuật toán shuffle/sort để tối ưu hóa cho các môi trường cụ thể. Ví dụ, sử dụng RDMA (Remote Direct Memory Access) có thể giảm đáng kể độ trễ và tăng băng thông cho các cluster có hỗ trợ phần cứng phù hợp. Điều này đặc biệt quan trọng trong các môi trường high-performance computing.

Bảo mật là một khía cạnh quan trọng khác, bao gồm xác thực Kerberos, quản lý quyền HDFS, và cách ly tài nguyên YARN. Các cơ chế này đảm bảo rằng chỉ những người dùng được ủy quyền mới có thể truy cập dữ liệu và tài nguyên cluster, đồng thời ngăn chặn các job của người dùng này can thiệp vào job của người dùng khác.

Cuối cùng, **kiến trúc lưu trữ** đóng vai trò quan trọng trong hiệu suất xử lý. Việc lựa chọn định dạng file phù hợp như Parquet hoặc ORC (các định dạng cột) có thể cải thiện đáng kể hiệu suất cho các bài toán phân tích, đặc biệt khi chỉ cần truy cập một số cột cụ thể thay vì toàn bộ bản ghi.

6.6. Ví dụ thực tế

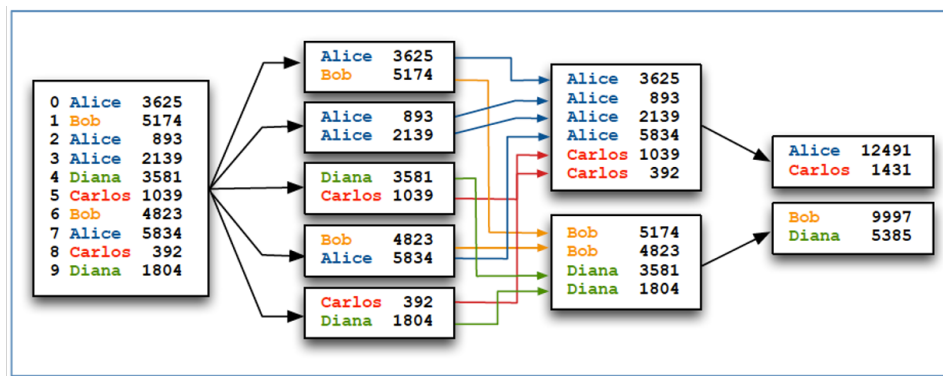
Tính tổng doanh số bán hàng theo nhân viên (E-commerce)

Xét bài toán thực tế trong thương mại điện tử: tính tổng doanh số bán hàng theo từng nhân viên từ tập dữ liệu đơn hàng lớn. Dữ liệu đầu vào là tệp văn bản lưu trên HDFS với cấu trúc <OrderID>, <EmployeeName>, <SaleAmount>. Mục tiêu là tính tổng SaleAmount cho mỗi EmployeeName.

Với dữ liệu mẫu gồm 5 đơn hàng từ 3 nhân viên (Alice, Bob, Charlie), quá trình MapReduce diễn ra như sau. Trong giai đoạn Map, mỗi split được xử lý bởi một mapper riêng biệt, tạo ra các cặp (employee, amount). Ví dụ, split 1 tạo ra (Alice,250), (Bob,180), (Alice,300), trong khi split 2 tạo ra (Charlie,120), (Bob,200).

Giai đoạn Combiner thực hiện tổng hợp cục bộ tại mỗi mapper để giảm dữ liệu trung gian. Mapper 1 cộng dồn Alice: $250+300=550$, giữ nguyên Bob: 180. Mapper 2 giữ nguyên Charlie: 120 và Bob: 200. Điều này giảm từ 5 bản ghi xuống còn 4 bản ghi, tiết kiệm băng thông mạng trong giai đoạn shuffle.

Hệ thống partitioner phân chia dữ liệu đến các reducer dựa trên hash của khóa. Với 2 reducer, Alice và Charlie được gửi đến R0, Bob đến R1. Mỗi reducer nhận dữ liệu đã được sắp xếp và nhóm theo khóa, sau đó thực hiện phép tổng hợp cuối cùng. Kết quả là Alice: 550, Charlie: 120, Bob: 380.



Hình 6.1 Luồng hoạt động của MapReduce trong bài toán tính tổng doanh số bán hàng theo nhân viên

Ví dụ này minh họa cách MapReduce xử lý hiệu quả các bài toán tổng hợp dữ liệu trên quy mô lớn, với khả năng song song hóa tự động và tối ưu hóa băng thông mạng thông qua combiner.

Xây dựng Inverted Index (Tìm kiếm văn bản)

Inverted index là cấu trúc dữ liệu cốt lõi của các hệ thống tìm kiếm, lưu trữ ánh xạ từ mỗi từ khóa đến danh sách các tài liệu chứa từ khóa đó cùng với vị trí xuất hiện. MapReduce cung cấp một cách hiệu quả để xây dựng inverted index cho hàng tỷ trang web.

Xét ví dụ với 3 tài liệu văn bản: doc1 chứa "to be or not to be", doc2 chứa "to be or not", và doc3 chứa "be or not to be". Trong giai đoạn Map, mỗi mapper xử lý một tài liệu, tách từ và phát ra các cặp (từ, docID@vị_trí). Doc1 tạo ra (to, doc1@0), (be, doc1@1), (or, doc1@2), (not, doc1@3), (to, doc1@4), (be, doc1@5). Tương tự, doc2 và doc3 cũng được xử lý để tạo ra các cặp tương ứng.

Hệ thống shuffle và sort tự động nhóm tất cả các cặp theo từ khóa. Trong giai đoạn Reduce, mỗi reducer xử lý một từ cụ thể, gom tất cả vị trí xuất hiện và tạo ra inverted index. Từ "to" xuất hiện ở doc1@0,4, doc2@0, doc3@3; từ "be" ở doc1@1,5, doc2@1, doc3@0,4; từ "or" ở doc1@2, doc2@2, doc3@1; và từ "not" ở doc1@3, doc2@3, doc3@2.

Kết quả cuối cùng là một inverted index cho phép tìm kiếm nhanh chóng. Khi người dùng tìm từ "be", hệ thống ngay lập tức biết từ này xuất hiện ở doc1@1,5, doc2@1, và doc3@0,4. Đây chính là nền tảng của các công cụ tìm kiếm như Google, cho phép xử lý hiệu quả hàng tỷ trang web và trả về kết quả trong thời gian thực.

Reduce-side Join (nối dữ liệu lớn)

Join là một trong những thao tác phổ biến nhất trong xử lý dữ liệu, nhưng việc thực hiện join trên dữ liệu lớn đòi hỏi các kỹ thuật đặc biệt. Xét bài toán nối bảng orders(order_id, customer_id, amount) với customers(customer_id, name) theo customer_id để tạo ra danh sách đơn hàng kèm tên khách hàng.

Với dữ liệu mẫu gồm 3 đơn hàng (o1,o2 thuộc c1; o3 thuộc c2) và 2 khách hàng (c1=Alice, c2=Bob), quá trình MapReduce join diễn ra như sau. Trong giai đoạn Map, mỗi mapper xử lý một bảng, chuẩn hóa dữ liệu về cặp (customer_id, value) và gắn nhãn nguồn để phân biệt. Từ bảng orders, mapper phát ra (c1, O|o1,100), (c1, O|o2,250), (c2, O|o3,300). Từ bảng customers, mapper phát ra (c1, C|Alice), (c2, C|Bob).

Hệ thống partitioner đảm bảo tất cả bản ghi có cùng customer_id được gửi đến cùng một reducer. Trong giai đoạn Reduce, mỗi reducer nhận được tất cả dữ liệu liên quan đến một customer_id cụ thể, sau đó tách thành hai "giỏ": giỏ C chứa thông tin khách hàng, giỏ O chứa thông tin đơn hàng. Reducer thực hiện tích Descartes có điều kiện giữa hai giỏ này.

Kết quả là c1 tạo ra (o1,Alice,100) và (o2,Alice,250), c2 tạo ra (o3,Bob,300). Reduce-side join có ưu điểm là dễ triển khai và chịu được dữ liệu lớn, nhưng nhược điểm là tốn chi phí shuffle do phải truyền tất cả dữ liệu qua mạng. Khi bảng customers nhỏ, có thể sử dụng replicated map-side join để giảm chi phí mạng.

Secondary Sort (sắp xếp theo khóa phụ trong từng nhóm)

Secondary sort là kỹ thuật quan trọng khi cần sắp xếp dữ liệu theo khóa phụ trong mỗi nhóm khóa chính. Xét bài toán phân tích hành vi người dùng với dữ liệu sự kiện (user_id, timestamp, action), cần sắp xếp các sự kiện theo thời gian trong mỗi nhóm user_id để phân tích chuỗi hành động.

Với dữ liệu mẫu gồm các sự kiện từ user u1 (timestamp 100, 090, 110) và u2 (timestamp 050), kỹ thuật secondary sort sử dụng composite key kết hợp user_id và timestamp. Comparator tổng thể sắp xếp theo (user_id asc, timestamp asc), trong khi grouping comparator chỉ so sánh user_id để nhóm dữ liệu. Kết quả là reducer cho u1 nhận được các sự kiện đã được sắp xếp theo thứ tự thời gian 090→100→110 mà không cần sort lại trong hàm reduce.

Kỹ thuật này đặc biệt hữu ích cho các thuật toán cần xử lý dữ liệu theo thứ tự thời gian như phân tích session, phát hiện pattern hành vi, hoặc các thuật toán đồ thị cần thứ tự cạnh.

Replicated (Map-side) Join (tối ưu mạng)

Khi một bảng có kích thước nhỏ (ví dụ bảng customers vài MB), có thể sử dụng replicated map-side join để tối ưu hiệu suất. Hệ thống phát tán bảng nhỏ đến tất cả mapper thông qua `Job.addCacheFile`, mỗi mapper nạp dữ liệu vào `HashMap` trong hàm `setup()`. Khi xử lý bảng lớn, mapper chỉ cần tra cứu `customer_id` trong `HashMap` để tạo ra bản ghi đã join.

Phương pháp này loại bỏ hoàn toàn giai đoạn shuffle, giảm đáng kể chi phí mạng và tăng hiệu suất. Tuy nhiên, nó chỉ áp dụng được khi bảng nhỏ có thể fit trong bộ nhớ của mỗi mapper và phù hợp với mô hình star schema nơi có bảng dimension nhỏ và bảng fact lớn.

6.7. Kết luận

MapReduce đã cách mạng hóa việc xử lý dữ liệu lớn bằng cách đưa ra mô thức lập trình đơn giản mà mạnh mẽ. Thay vì phải lo lắng về các vấn đề phức tạp như song song hóa, lập lịch, chịu lỗi, và giao tiếp mạng, lập trình viên chỉ cần thiết kế hai hàm map và reduce. Hệ thống sẽ tự động xử lý tất cả các vấn đề còn lại.

Sự thành công của MapReduce nằm ở khả năng xử lý hiệu quả các bài toán batch với throughput cao, đặc biệt phù hợp với các tác vụ như ETL, sắp xếp, tổng hợp dữ liệu, và xây dựng chỉ mục. Tuy nhiên, mô hình này có những hạn chế rõ ràng với các bài toán lập, tương tác, và streaming.

Mặc dù các framework mới như Spark và Flink đã giải quyết được nhiều hạn chế của MapReduce, nhưng những nguyên lý cốt lõi của MapReduce - phân chia dữ liệu, xử lý song song, và tổng hợp kết quả - vẫn là nền tảng của hầu hết các hệ thống Big Data hiện đại. Việc hiểu rõ MapReduce tạo nền tảng vững chắc để tiếp cận các công nghệ mới trong tương lai.

6.8. Bài tập và câu hỏi ôn tập

Bài tập thực hành

1. **WordCount với lọc stopwords:** Chuẩn hóa văn bản, loại bỏ từ dừng, đếm tần suất từ. So sánh hiệu suất với/không có Combiner.
2. **Distinct URLs:** Đếm số URL duy nhất từ log web (ts, uid, url, rc).
3. **Reduce-side join:** Nối `purchases(uid, item, price)` với `users(uid, age, city)` để tạo `(city, total_amount)`.
4. **Secondary sort:** Với dữ liệu `sensor(sensor_id, ts, value)`, sắp xếp value theo thứ tự tăng ts trong mỗi nhóm `sensor_id`.

5. **Skew handling:** Xử lý khóa cực nóng bằng key salting và reduce tăng 2.

Câu hỏi ôn tập

1. Vì sao combiner không phải lúc nào cũng đúng? Nêu điều kiện toán học cần thỏa.
2. Phân tích trade-off khi chọn số reducers.
3. So sánh reduce-side join và map-side join: tiền điều kiện, chi phí mạng, độ phức tạp triển khai.
4. Giải thích cơ chế speculative execution và tác động tới thông lượng cụm.
5. Khi nào ưu tiên Spark/Flink thay vì MapReduce?

Ghi chú triển khai

Để triển khai MapReduce hiệu quả, nên bắt đầu với dữ liệu mẫu nhỏ để kiểm chứng logic, sau đó chạy trên cụm với dữ liệu lớn để quan sát hành vi shuffle. Sử dụng JobHistory Server để theo dõi các metric quan trọng như MAP_OUTPUT_BYTES, REDUCE_SHUFFLE_BYTES, và SPILLED_RECORDS. Thêm Counters để đếm bản ghi lỗi và log mẫu để chẩn đoán phân phối khóa.

Chương 7

Các kỹ thuật xử lý luồng dữ liệu lớn với Spark Streaming

7.1. Dữ liệu luồng	209
7.2. Spark Streaming	210
7.3. Structured Streaming	218
7.4. Kết chương	224

7.1. Dữ liệu luồng

Dữ liệu luồng là dữ liệu được tạo ra liên tục từ một hay nhiều nguồn khác nhau theo thời gian thực. Loại dữ liệu này đến dưới dạng các chuỗi sự kiện hoặc bản ghi nhỏ, thường có thứ tự và không có điểm kết thúc rõ ràng. Khác với dữ liệu tĩnh (được lưu trữ trong các tệp hoặc cơ sở dữ liệu), dữ liệu luồng cần được xử lý ngay khi nó đến.

Đặc điểm chính của dữ liệu luồng

- **Tốc độ (Velocity):** Dữ liệu được tạo ra với tốc độ rất nhanh, đòi hỏi các hệ thống phải xử lý ngay lập tức để trích xuất giá trị.
- **Khối lượng (Volume):** Khối lượng dữ liệu có thể rất lớn, thường lên tới terabyte hoặc petabyte mỗi ngày.
- **Liên tục:** Dữ liệu không có điểm kết thúc, liên tục được sinh ra và gửi đến hệ thống.
- **Có thứ tự:** Các sự kiện thường có một trình tự thời gian, và việc duy trì thứ tự này rất quan trọng cho việc phân tích.

Các ví dụ điển hình về dữ liệu luồng

- **IoT và cảm biến:** Dữ liệu từ các thiết bị thông minh, cảm biến trên xe cộ, máy móc công nghiệp, hoặc thiết bị đeo tay.

- Giao dịch tài chính: Dữ liệu về các giao dịch thẻ tín dụng, giao dịch chứng khoán, hoặc chuyển khoản ngân hàng.
- Truyền thông xã hội: Các bài đăng, bình luận, và lượt thích trên các nền tảng như X (Twitter) hoặc Facebook.
- Phân tích clickstream: Dữ liệu về hành vi người dùng trên các trang web, bao gồm lượt click, lượt xem trang và thời gian ở lại.
- Hệ thống giám sát: Dữ liệu log từ máy chủ, ứng dụng và mạng để theo dõi hiệu suất và phát hiện sự cố.

Với tốc độ phát sinh cao, khối lượng lớn, và thường có những yêu cầu khắt khe về độ trễ phân tích, việc xử lý dữ liệu luồng thường đòi hỏi hạ tầng tính toán lớn cũng như những framework mạnh mẽ và linh hoạt như Spark Streaming, Apache Flint, Apache Storm,...

7.2. Spark Streaming

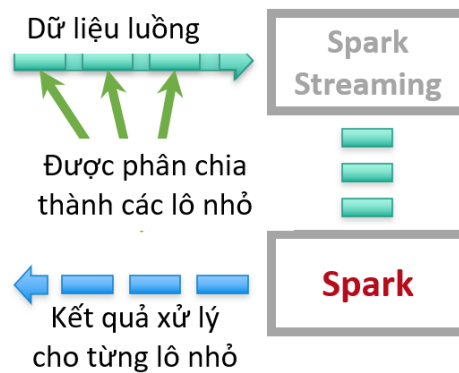
Spark Streaming là một thành phần mở rộng của Apache Spark được xây dựng trên nền Spark Core nhằm xử lý dữ liệu luồng theo thời gian thực. Spark Streaming không xử lý từng sự kiện phát sinh riêng lẻ mà hoạt động dựa trên mô hình xử lý lô nhỏ (micro-batching) trong đó luồng dữ liệu liên tục được chia theo các khoảng thời gian nhỏ (lô nhỏ - micro batch). Mỗi lô nhỏ này được xử lý giống như các tập dữ liệu tĩnh thông thường. Mô hình xử lý dữ liệu luồng của Spark Streaming được mô tả như sau (hình 7.1).

- Luồng dữ liệu thời gian thực được phân chia thành các lô nhỏ theo từng khoảng thời gian X giây. Người lập trình có thể thiết lập giá trị cho tham số X này.
- Mỗi lô dữ liệu được Spark biểu diễn như một RDD và được xử lý thông qua các phép toán RDD.
- Kết quả trả về là luồng kết quả của các lô nhỏ. Kết quả xử lý của các lô nhỏ trong quá khứ cũng có thể được lưu trữ lại và được sử dụng cho việc tổng hợp kết quả của các bước tính toán trong tương lai.

7.2.1. Các khái niệm và thao tác cơ bản

DStream.

DStream, thuật ngữ viết tắt của Discretized Stream - luồng dữ liệu rời rạc, được hiểu là dữ liệu luồng được phân chia rời rạc thành chuỗi của các RDD.



Hình 7.1 Mô hình xử lý dữ liệu luồng của Spark Streaming.

Đọc dữ liệu đầu vào. Để bắt đầu quá trình xử lý, Dữ liệu luồng từ nguồn đầu vào như các bản tin của Twitter, các bản ghi trên tệp HDFS, các bản ghi từ hàng đợi Kafka, chuỗi dữ liệu nhận từ cổng TCP socket,... được đọc vào và hình thành các DStream. Ví dụ sau đây minh họa việc đọc vào DStream từ các bản tin trên Twitter.

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
```

Biến đổi - Transformation.

DStream có thể được biến đổi thành các DStream khác qua các phép biến đổi. Có hai loại phép biến đổi:

- Sử dụng các phép biến đổi RDD như: `map`, `join`, `reduce`,... Bản chất của DStream là chuỗi của các RDD, do đó có thể áp dụng các phép biến đổi RDD cho từng phần tử của chuỗi này.
- Các phép biến đổi có trạng thái (stateful operations) như: `window`, `countByValueAndWindow`,... Biến đổi RDD chỉ có hiệu lực trên từng RDD đơn lẻ. Các phép biến đổi có trạng thái cho phép nhớ và ghép kết quả của các RDD trên DStream để tạo ra RDD mới trong mỗi bước tính toán. Các phép biến đổi này rất cần thiết cho việc tổng hợp và phân tích thông tin của cả hiện tại và quá khứ.

Ví dụ dưới đây đếm các hashtag trong các bản tin Twitter trong 10 phút gần nhất và cho kết quả 1 giây một lần.

```
val hashTags = tweets.flatMap(status => getTags(status))
val tagCounts = hashTags.window(Minutes(10), Seconds(1)).countByValue()
```

Trong ví dụ này tweets là DStream của các bản tin trên Twitter. Các hashtags của các bản tin trong mỗi bước thời gian được trích xuất (getTags) và ghép lại (flatMap) để hình thành RDD cho DStream hashTags. Các hashTags này được lưu trữ và tổng hợp theo khung thời 10 phút với tần suất 1 giây một lần (window(Minutes(10), Seconds(1))), sau đó được đếm theo từng tag (countByValue()). Có thể kết hợp việc tổng hợp theo khung thời gian với thao tác đếm trong cùng một thao tác như sau:

```
val hashTags = tweets.flatMap (status => getTags(status))
val tagCounts = hashTags.countByValueAndWindow(Minutes(10), Seconds(1))
```

Trong ví dụ này, cũng có thể thực hiện việc đếm hashtag một cách linh hoạt và thông minh hơn với reduceByKeyAndWindow() như sau:

```
val tagCounts = hashTags.reduceByKeyAndWindow
    (_ + _, _ - _, Minutes(10), Seconds(1))
```

Thông thường với RDD, reduceByKey() thực hiện gom nhóm các phần tử trên RDD và thực hiện tổng hợp cho từng nhóm bằng một toán tử có tính giao hoán và kết hợp (ví dụ toán tử cộng). reduceByKeyAndWindow cũng có tác dụng tương tự nhưng được áp dụng cho từng cửa sổ. Để việc tổng hợp được hiệu quả trong chuỗi cửa sổ liên tục, tránh việc tính toán lại, reduceByKeyAndWindow thực hiện việc bổ sung số đếm (bằng toán tử +) mỗi khi có một lô đầu vào đi vào cửa sổ và loại trừ số đếm cho những lô đã đi ra khỏi cửa sổ (bằng toán tử -). Cấu trúc tổng quát cho reduceByKeyAndWindow được mô tả như sau:

```
reduceByKeyAndWindow(func,
    invFunc,
    windowDuration,
    slideDuration=None,
    numPartitions=None,
    filterFunc=None)
```

trong đó:

- func: hàm tổng hợp được thực hiện để tổng hợp các kết quả từ lô mới đi vào cửa sổ.
- invFunc: hàm loại trừ được thực hiện để loại trừ các kết quả của các lô đã đi ra cửa sổ.
- windowDuration: giá trị thời gian cho mỗi khung cửa sổ. Giá trị này phải là bội số của thời gian cho mỗi lô của DStream đầu vào.

- `slideDuration`: giá trị thời gian cho mỗi lô kết quả. Giá trị này phải là bội số của thời gian cho mỗi lô của DStream đầu vào.
- `numPartition`: số phân đoạn của mỗi RDD của DStream đầu ra.
- `filterFunc`: hàm lọc các giá trị trên RDD được đưa vào xử lý.

Kết xuất kết quả.

Dữ liệu từ các DStream có thể được kết xuất ra các hệ thống bên ngoài như ghi ra màn hình, lưu trữ trên các tệp, lưu trữ trên CSDL,... SparkStreaming hỗ trợ các phép toán sau để thực hiện việc kết xuất dữ liệu này:

- `print()`: In 10 phần tử đầu của mỗi RDD trên DStream ra console.
- `saveAsTextFiles(prefix)`: Lưu từng RDD của DStream ra tệp cục bộ.
- `saveAsHadoopFiles(...)`: Lưu từng RDD của DStream ra tệp HDFS.
- `foreach(func)`: Kết xuất dữ liệu linh hoạt. mỗi RDD trên DStream được gửi đến hàm tham số `func`. Người lập trình có thể quyết định cách thức kết xuất dữ liệu cho từng RDD này, ví dụ lưu ra CSDL.

7.2.2. Caching và persistence

Trong Spark Streaming, caching và persistence là các kỹ thuật tối ưu hóa hiệu suất quan trọng cho DStream. Về cơ bản, chúng cho phép lưu trữ các RDD tạo ra bởi một DStream trong bộ nhớ hoặc trên đĩa để tránh phải tính toán lại chúng nhiều lần.

Caching là một cách nhanh chóng để lưu trữ các RDD của một DStream vào bộ nhớ (RAM). Khi gọi `dstream.cache()`, Spark sẽ đánh dấu DStream đó để các RDD tạo ra từ nó được lưu vào bộ nhớ. Lần đầu tiên một hành động trên DStream đã được cache, các RDD tương ứng sẽ được tính toán và lưu vào RAM. Các phép toán tiếp theo trên cùng DStream sẽ truy cập dữ liệu đã lưu trong bộ nhớ thay vì tính toán lại từ đầu, giúp tăng tốc đáng kể. Có thể lựa chọn lưu trữ thay thế cho bộ nhớ với `dstream.persist(storageLevel)`. Trong đó tham số `storageLevel` có thể nhận các giá trị:

- `MEMORY_ONLY`. Lưu trữ RDD dưới dạng đối tượng trong RAM. Đây là phương thức cho tốc độ nhanh nhất nhưng tiêu tốn nhiều bộ nhớ. Nếu không đủ RAM, Spark sẽ phải tính toán lại các phân vùng bị thiếu.
- `MEMORY_AND_DISK`. Lưu trữ trong RAM, nhưng nếu không đủ, các phân vùng còn lại sẽ được lưu vào đĩa.

- `DISK_ONLY`. Chỉ lưu trữ RDD trên đĩa. Tốc độ chậm hơn nhưng phù hợp với các tập dữ liệu lớn không vừa RAM.
- Ngoài ra còn có các mức lưu trữ có tính sao chép (ví dụ `MEMORY_ONLY_2`), giúp tăng khả năng chịu lỗi bằng cách tạo bản sao của dữ liệu trên các node khác nhau.

7.2.3. Biến tích lũy (accumulator) và biến quảng bá (broadcast variable)

Như đã biết trong Spark, RDD được lưu trữ và xử lý một cách phân tán trên nhiều nút tính toán. Việc tính toán xử lý được thực hiện một cách cục bộ trên từng nút tham gia sau đó được tổng hợp lại. Phương thức này dẫn đến một trở ngại khi một số bài toán đòi hỏi phải duy trì một trạng thái toàn cục để mỗi tiến trình phân tán có thể truy cập. Ví dụ khi cần tính toán một giá trị tích lũy từ nhiều phân đoạn của RDD, mỗi tiến trình tham gia có thể thực hiện tính toán cục bộ cho giá trị tích lũy riêng phần trên phân đoạn của mình sau đó cập nhật giá trị tích lũy này lên một biến toàn cục đại diện cho giá trị tích lũy toàn phần. Các lập trình viên thiếu kinh nghiệm thường gặp một sai lầm khi định nghĩa biến toàn cục này như một biến thông thường trong chương trình (do vậy chỉ có hiệu lực trên từng tiến trình phân tán). Đoạn code sau minh họa cho một sai lầm như vậy.

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd = sc.parallelize(data)

even_count = 0

def count_even(number):
    if number % 2 == 0:
        even_count += 1

rdd.foreach(count_even)

print(f"Total even numbers: {even_count}")
```

Trong ví dụ này biến `even_count` thực chất là một biến cục bộ trên tiến trình driver. Khi thực thi hàm `count_even` trên các tiến trình tham gia tính toán, biến này được nhân bản (với giá trị khởi tạo = 0) và các bản sao này chỉ được cập nhật cục bộ trên từng tiến trình mà không được đồng bộ với nhau và đồng bộ với biến trên tiến trình driver. Vì vậy, giá trị in ra sẽ không thay đổi so với giá trị khởi tạo trên tiến trình driver.

Để khắc phục vấn đề về biến toàn cục này, Spark cung cấp hai cơ chế là biến tích lũy (accumulator) và biến quảng bá (broadcast variable).

Biến tích lũy - Accumulator

Biến tích lũy là biến được chia sẻ chỉ có thể được thêm (tích lũy) vào, và không thể đọc được bởi các tiến trình trong quá trình thực thi phân tán. Chúng được sử dụng để thực hiện các phép toán tổng hợp một cách an toàn và hiệu quả, chẳng hạn như đếm số lượng phần tử hoặc tính tổng các giá trị. Chúng được tích lũy từ các tiến trình tính toán trên các worker node và chỉ có thể lấy kết quả về từ tiến trình driver.

Ví dụ bên trên có thể được viết lại một cách đúng đắn với biến tích lũy như sau:

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd = sc.parallelize(data)

even_count = sc.accumulator(0)

def count_even(number):
    global even_count
    if number % 2 == 0:
        even_count += 1

rdd.foreach(count_even)

print(f"Total even numbers: {even_count.value}")
```

Spark hiện tại hỗ trợ hai loại biến tích lũy:

- **Biến tích lũy số (Numeric Accumulator):** Dùng để tích lũy các giá trị số (ví dụ: Int, Long, Double). Có thể tạo một biến tích lũy số bằng cách gọi `sc.longAccumulator()` hoặc `sc.doubleAccumulator()`.
- **Biến tích lũy tự tạo (Custom Accumulator):** Cho phép định nghĩa các phép toán tích lũy tùy chỉnh trên các kiểu dữ liệu khác, chẳng hạn như List, Set hoặc Map. Để làm điều này, cần kế thừa lớp `AccumulatorV2` và định nghĩa các phương thức `add`, `merge`, `reset` và `isZero`.

Trong xử lý dữ liệu luồng với Spark Streaming, biến tích lũy cũng có thể được sử dụng để tích lũy giá trị tính toán trong quá trình xử lý của nhiều lô nhỏ. Tuy nhiên, kỹ thuật này không được khuyến khích do các lý do sau:

- Các luồng dữ liệu thực tế thường liên tục và kéo dài, đòi hỏi việc xử lý chúng cũng phải được duy trì liên tục. Trong quá trình xử lý, các sự cố hoặc lỗi có thể xảy ra.

Spark hỗ trợ các cơ chế tính toán lại khi có lỗi, tuy nhiên việc tính toán lại này có thể dẫn đến việc biến tích lũy bị cập nhật lại nhiều lần, đặc biệt trong các phép biến đổi như map hay filter.

- Cơ chế checkpoint của Spark Streaming cho phép lưu trữ lại các RDD của các lô nhỏ để đề phòng lỗi. Tuy nhiên checkpoint không lưu dữ liệu của các biến tích lũy nên nếu thực hiện lại việc xử lý luồng từ các checkpoint có thể khiến biến tích lũy không nhất quán với dữ liệu trên checkpoint.
- Do kỹ thuật tính toán trì hoãn của Spark (thao tác tính toán chỉ thực sự được thực hiện tại thời điểm kết xuất dữ liệu của các lô nhỏ), giá trị của các biến tích lũy cũng chỉ được thay đổi tại những thời điểm này chứ không thực sự phản ánh theo thời gian thực.

Để tổng hợp hoặc duy trì trạng thái trong Spark Streaming, nên sử dụng các API được thiết kế riêng cho mục đích này như `updateStateByKey` hay `mapWithState` hoặc lưu trữ các trạng thái ra những hệ thống bên ngoài như CSDL Cassandra, Redis hay HDFS.

Biến quảng bá - Broadcast variable

Khác với biến tích lũy là các biến chỉ cập nhật trên các tiến trình tính toán, biến quảng bá là các biến chỉ đọc trên các tiến trình tính toán. Mục đích của biến quảng bá là truyền các tập dữ liệu lớn từ driver đến các worker một cách hiệu quả.

Quá trình quảng bá được thực hiện như sau:

- Giá trị biến quảng bá được khởi tạo trên driver.
- Biến quảng bá được gửi một lần duy nhất đến các worker cần sử dụng chúng.
- Bản sao của biến quảng bá được lưu trữ trên các worker và được tham chiếu cục bộ khi cần.

Lưu ý rằng biến quảng bá khác với các biến thông thường khác ở chỗ nó chỉ được gửi một lần duy nhất tới các worker. Ví dụ sau minh họa cách sử dụng của biến quảng bá:

```
val states = Map(("NY","New York"),("CA","California"),("FL","Florida"))
val countries = Map(("USA","United States of America"),("IN","India"))

val broadcastStates = spark.sparkContext.broadcast(states)
val broadcastCountries = spark.sparkContext.broadcast(countries)

val data = Seq(("James","Smith","USA","CA"),
```

```
( "Michael", "Rose", "USA", "NY" ),
( "Robert", "Williams", "USA", "CA" ),
( "Maria", "Jones", "USA", "FL" )
)

val rdd = spark.sparkContext.parallelize(data)

val rdd2 = rdd.map(f=>{
    val country = f._3
    val state = f._4
    val fullCountry = broadcastCountries.value.get(country).get
    val fullState = broadcastStates.value.get(state).get
    (f._1, f._2, fullCountry, fullState)
})

println(rdd2.collect().mkString("\n"))
```

Trong ví dụ trên, các biến quảng bá `broadcastCountries` và `broadcastStates` được khởi tạo trên driver, truyền đến các worker và được sử dụng trong thao tác biến đổi map.

7.2.4. Cơ chế chịu lỗi của Spark Streaming

Spark Streaming được xây dựng trên nền Spark Core, do vậy thừa hưởng khả năng chịu lỗi của Spark Core. Bên cạnh đó, do việc xử lý dữ liệu luồng là một quá trình lâu dài và liên tục, Spark Streaming bổ sung thêm các cơ chế chịu lỗi để đảm bảo các sự cố lỗi không ảnh hưởng đến kết quả tính toán. Cụ thể là:

- Trên các worker
 - Dữ liệu nguồn được lưu trữ và nhân bản trên bộ nhớ của nhiều worker node. Spark sử dụng cơ chế xác nhận (acknowledgement). Sau khi dữ liệu đã được nhận và sao chép vào bộ nhớ của Spark, receiver sẽ gửi xác nhận trở lại nguồn dữ liệu. Điều này đảm bảo rằng dữ liệu không bị mất nếu một receiver bị lỗi trước khi dữ liệu được xử lý.
 - Spark ghi nhớ các thao tác tính toán biến đổi RDD trong lineage graph.
 - Các tác vụ bị lỗi sẽ được khởi động lại trên các worker khác dựa trên lineage để đảm bảo kết quả chính xác mà không mất mát dữ liệu.
- Trên driver

- Tính chịu lỗi của driver được đảm bảo bằng cơ chế Checkpointing. Checkpointing cho phép lưu trữ thông tin về trạng thái của ứng dụng vào một hệ thống tệp tin đáng tin cậy như HDFS hoặc S3.
- Thông tin được lưu trữ bao gồm:
 - * Metadata Checkpointing: Lưu trữ định nghĩa của DStream (các phép biến đổi đã áp dụng), các phép biến đổi đang chờ xử lý và các lô dữ liệu đã được đưa vào hàng đợi. Điều này giúp Spark biết cách xây dựng lại luồng xử lý nếu driver bị lỗi.
 - * Data Checkpointing: Áp dụng cho các phép toán yêu cầu trạng thái như `updateStateByKey` và `mapWithState`. Dữ liệu trạng thái này được định kỳ lưu vào checkpoint, cho phép ứng dụng khôi phục trạng thái từ thời điểm checkpoint cuối cùng.

7.3. Structured Streaming

Mặc dù kỹ thuật xử lý dữ liệu luồng với DStream mang lại một phương án xử lý dữ liệu luồng đặc biệt quen thuộc với các lập trình viên RDD, tuy nhiên kỹ thuật này cũng còn tồn tại nhiều hạn chế:

1. Xử lý với thời gian sự kiện (Event-Time) và dữ liệu đến muộn (Late Data) Điểm yếu lớn nhất của DStream là việc xử lý các sự kiện dựa trên thời gian xử lý (processing-time), thay vì thời gian sự kiện (event-time) thực tế.
 - API của DStream được xây dựng xung quanh các RDD được tạo ra trong các lô dữ liệu theo một khoảng thời gian cố định. Mọi phép biến đổi và tính toán đều được thực hiện dựa trên thời điểm lô đó được xử lý, chứ không phải dựa trên thời điểm sự kiện thực sự xảy ra.
 - Việc xử lý dữ liệu đến muộn (late data) là một thách thức lớn. DStream không có cơ chế tích hợp để xác định liệu một sự kiện có đến muộn hay không và xử lý nó một cách chính xác. Do đó, các phép tổng hợp dựa trên thời gian sự kiện sẽ không chính xác.

Ví dụ: Nếu một sự kiện xảy ra lúc 10:00:00 nhưng đến hệ thống xử lý lúc 10:00:15, và kích thước lô là 10 giây, DStream sẽ xử lý sự kiện này trong lô 10:00:10 - 10:00:20, thay vì lô 10:00:00 - 10:00:10. Điều này làm cho việc phân tích dữ liệu luồng trở nên thiếu chính xác.

2. Xử lý luồng và lô không thống nhất Mặc dù Spark được thiết kế để xử lý cả luồng và batch, DStream lại tạo ra một sự tách biệt không cần thiết giữa hai mô hình này.

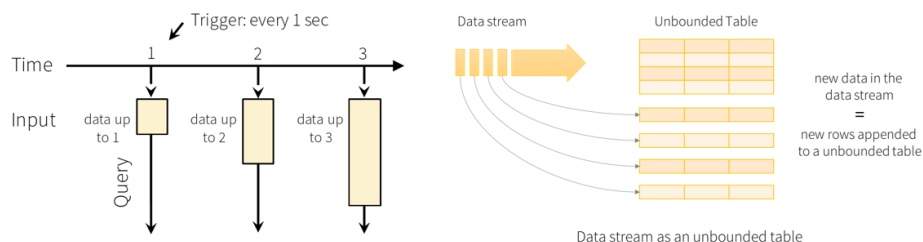
- DStream được xây dựng trên RDD, vì vậy các API của chúng có vẻ tương tự. Tuy nhiên, chúng lại không hoàn toàn giống nhau. Điều này buộc các nhà phát triển phải duy trì hai tập hợp code riêng biệt cho cùng một logic xử lý, một cho xử lý lô (dùng RDD/DataFrame) và một cho xử lý luồng (dùng DStream).
- Việc kết hợp xử lý luồng DStream với xử lý lô hoặc xử lý tương tác (interactive) là phức tạp và không hiệu quả. Điều này gây khó khăn trong việc xây dựng các kiến trúc dữ liệu hiện đại, nơi mà xử lý luồng, lô và truy vấn tương tác thường xuyên được kết hợp với nhau.

3. Đảm bảo tính toàn vẹn từ đầu đến cuối (End-to-End Guarantees)

- DStream không tự động đảm bảo tính toàn vẹn dữ liệu ở cấp độ đầu ra. Mặc dù nó đảm bảo mỗi lô dữ liệu sẽ được xử lý ít nhất một lần (at-least-once), việc đảm bảo exactly-once (chính xác một lần) đòi hỏi lập trình viên phải xây dựng các sink đầu ra có khả năng xử lý lỗi.
- Nếu một sink đầu ra không có tính chất bất biến (idempotent), dữ liệu có thể bị ghi trùng lặp nếu một lô bị xử lý lại. Điều này đòi hỏi các lập trình viên phải áp dụng các chiến lược như sử dụng transaction hoặc các lock để ngăn chặn việc ghi dữ liệu trùng lặp.

Để khắc phục những hạn chế này, bắt đầu từ phiên bản 2.0, Spark bổ sung thêm thành phần xử lý dữ liệu luồng Structured Streaming. Structured Streaming được xây dựng trên nền tảng Spark SQL và sử dụng các API quen thuộc của DataFrame/Dataset, giúp làm cho việc xử lý dữ liệu luồng trở nên dễ dàng và hiệu quả hơn.

Structured Streaming tiếp cận dữ liệu luồng như thể đó là dữ liệu tĩnh. Nó coi dữ liệu đầu vào là một bảng đang được thêm hàng liên tục (append-only table). Mỗi truy vấn trên luồng dữ liệu sẽ tạo ra một bảng kết quả (result table), được cập nhật liên tục khi có dữ liệu mới đến. Cơ chế xử lý của Structured Streaming được mô tả như trong hình 7.2.



Hình 7.2 Mô hình xử lý dữ liệu luồng của Structured Streaming.

Ví dụ sau minh họa một ứng dụng đếm từ trên luồng dữ liệu truyền qua cổng TCP socket.

```
# 1. Định nghĩa DataFrame từ luồng dữ liệu đầu vào TCP socket, cổng 9999
lines = spark.readStream. \
    format("socket"). \
    option("host", "localhost"). \
    option("port", 9999). \
    load()

# 2. Định nghĩa các phép biến đổi
words = lines.select(explode(split(lines.value, " ")).alias("word"))
wordCounts = words.groupBy("word").count()

# 3. Định nghĩa truy vấn
query = wordCounts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

# 4. Bắt đầu và đợi cho đến khi truy vấn kết thúc
query.awaitTermination()
```

Từ ví dụ này ta thấy các bước lập trình cho việc thực hiện truy vấn trên dữ liệu luồng bao gồm:

1. Định nghĩa dữ liệu đầu vào
2. Định nghĩa các phép biến đổi
3. Định nghĩa truy vấn với việc chỉ định đích đến của truy vấn, thiết lập chế độ đầu ra và cách thức kích hoạt việc thực thi truy vấn (trigger).
4. Bắt đầu thực hiện truy vấn

Cần lưu ý là truy vấn với Structured Streaming không phải là một truy vấn một lần như trong Spark SQL thông thường. Thay vào đó, đây là một chuỗi các hoạt động được thực thi lặp đi lặp lại để xử lý các lô dữ liệu đến. Mỗi khi trigger được kích hoạt, một vi lô mới sẽ được tạo ra và truy vấn sẽ được thực thi trên vi lô này.

7.3.1. Dữ liệu đầu vào

Structured Streaming hỗ trợ nhiều nguồn dữ liệu đầu vào khác nhau. Một số nguồn dữ liệu được tích hợp sẵn trong Spark, trong khi một số khác cần sử dụng các gói thư viện bên

ngoài. Một số nguồn dữ liệu phổ biến bao gồm:

- **File Source:** Đọc dữ liệu từ các tệp mới được thêm vào một thư mục được giám sát. Nguồn này hỗ trợ nhiều định dạng tệp khác nhau như Parquet, ORC, JSON, CSV và Text. Spark SS liên tục kiểm tra thư mục để tìm các tệp mới và xử lý chúng.
- **Socket Source:** Lắng nghe dữ liệu văn bản từ một socket TCP. Nguồn này rất hữu ích để thử nghiệm các tính năng của Structured Streaming, vì nó cho phép bạn gửi dữ liệu thủ công từ một máy khách.
- **Kafka Source:** Đây là nguồn dữ liệu phổ biến nhất cho Structured Streaming. Spark đọc dữ liệu từ một hoặc nhiều topic Kafka. Nguồn Kafka của Spark có khả năng chịu lỗi và đảm bảo tính nhất quán "exactly-once" bằng cách quản lý offset một cách hiệu quả.
- **Rate Source:** Nguồn này tạo ra dữ liệu theo một tốc độ định trước, hữu ích cho việc kiểm tra hiệu suất và benchmark. Nó có thể tạo ra các hàng với tốc độ cố định, tăng dần hoặc ngẫu nhiên.
- **Kinesis Source:** Đọc dữ liệu từ Amazon Kinesis. Nguồn này yêu cầu gói thư viện bên ngoài.

Ví dụ sau tạo một luồng từ một thư mục chứa các tệp JSON.

```
df = spark.readStream.json("path/to/folder")
```

Ví dụ sau tạo một luồng từ một topic trên hàng đợi Kafka:

```
df = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
    .option("subscribe", "topic1") \
    .load()
```

Các luồng này được xem như các dataframe (để phân biệt, chúng ta gọi chúng là các dataframe luồng). có thể áp dụng các phép biến đổi với dataframe cho trên các luồng này.

7.3.2. Biến đổi

Như giới thiệu bên trên, có thể áp dụng hầu hết các phép biến đổi của dataframe cho các dataframe luồng. Điều này đơn giản hóa việc lập trình luồng cho các lập trình viên đã quen thuộc với Spark SQL. Bên cạnh đó có một số phép biến đổi nâng cao đặc thù cho xử lý luồng.

Các phép biến đổi cơ bản.

Các phép biến đổi này hoạt động trên dữ liệu luồng giống hệt như trên dữ liệu tĩnh:

- Chiếu (Projection) và Chọn (Selection): Sử dụng `select()`, `selectExpr()`, và `withColumn()` để chọn, đổi tên, hoặc tạo các cột mới từ dữ liệu luồng. Ví dụ, có thể trích xuất các trường cụ thể từ một luồng JSON.
- Lọc(Filtering): Dùng `filter()` hoặc `where()` để loại bỏ các hàng không mong muốn dựa trên một điều kiện. Chẳng hạn, bạn có thể lọc các giao dịch có giá trị âm.
- Gom nhóm (Grouping) và Tổng hợp (Aggregation): Sử dụng `groupBy()` kết hợp với các hàm tổng hợp như `count()`, `sum()`, `avg()`, hoặc `max()`. Đây là các phép biến đổi có trạng thái (stateful), vì Spark phải duy trì trạng thái của các nhóm qua các khoảng thời gian.

Các phép biến đổi nâng cao.

Các phép biến đổi này được thiết kế đặc biệt để giải quyết các thách thức của xử lý luồng:

- Tạo cửa sổ (Windowing): Cho phép thực hiện các phép tổng hợp trên các cửa sổ thời gian cố định. Ví dụ, có thể đếm số lượng nhấp chuột trong mỗi khoảng 5 phút.
- Đóng dấu thời gian (Watermarking): Đây là một cơ chế quan trọng để xử lý dữ liệu đến trễ. Khi áp dụng `withWatermark()`, chúng ta định nghĩa một ngưỡng "độ trễ" mà Spark nên chấp nhận trước khi loại bỏ các trạng thái cũ. Điều này giúp ngăn chặn các phép biến đổi có trạng thái (như `groupBy` hoặc `join`) lưu trữ trạng thái vô hạn, vốn có thể dẫn đến lỗi hết bộ nhớ.
- Kết nối (Joins): Có thể kết nối một dataframe luồng với một dataframe, hoặc thậm chí là kết nối hai dataframe luồng với nhau. Khi kết nối hai dataframe luồng, thường phải sử dụng watermark để xử lý các bản ghi đến trễ và loại bỏ trạng thái cũ.
- Loại bỏ trùng lặp (Deduplication): Dùng `dropDuplicates()` để loại bỏ các bản ghi trùng lặp trong một dataframe luồng dữ liệu. Điều này rất hữu ích khi nguồn dữ liệu có thể gửi cùng một bản ghi nhiều lần.

7.3.3. Đầu ra

Trong Structured Streaming, đầu ra (output) là cách xác định nơi và cách dữ liệu đã được xử lý sẽ được ghi ra. Spark cung cấp các chế độ đầu ra và các loại sink (đích đến) khác nhau để phù hợp với nhiều trường hợp sử dụng.

Đích đến (Sink).

Sink là nơi dữ liệu luồng kết quả được ghi ra. Sink được định nghĩa bằng df.writeStream đi cùng với các tùy chọn xác định loại sink mong muốn. Spark hỗ trợ nhiều loại sink khác nhau:

- **Console Sink:** Dùng để in dữ liệu ra màn hình console để gỡ lỗi và kiểm tra. Đây là một sink đơn giản và rất hữu ích trong quá trình phát triển.
- **File Sink:** Ghi dữ liệu luồng ra các tệp tin trong một thư mục, hỗ trợ nhiều định dạng như Parquet, JSON, ORC và CSV. Có thể sử dụng tính năng "trigger once" để xử lý toàn bộ dữ liệu hiện có và sau đó dừng lại.
- **Kafka Sink:** Ghi dữ liệu vào một chủ đề (topic) của Kafka. Đây là một trong những sink quan trọng nhất cho các kiến trúc xử lý luồng, cho phép dữ liệu đã được xử lý được đưa vào một luồng khác để xử lý tiếp hoặc để tiêu thụ bởi các ứng dụng khác.
- **Foreach Sink:** Cung cấp một cách linh hoạt để xử lý từng bản ghi trong luồng dữ liệu bằng cách sử dụng một hàm tùy chỉnh. Điều này hữu ích khi cần tích hợp với các hệ thống không được hỗ trợ bởi các sink có sẵn của Spark, chẳng hạn như ghi dữ liệu vào một cơ sở dữ liệu NoSQL.

Chế độ đầu ra (Output Modes) Có thể xác định cách các bản ghi mới được ghi vào sink bằng cách sử dụng phương thức `.outputMode()`. Có ba chế độ chính:

- **Append Mode:** Đây là chế độ mặc định và được sử dụng phổ biến nhất. Spark chỉ ghi những bản ghi mới đã được thêm vào "bảng" dataframe luồng kể từ lần kích hoạt (trigger) cuối cùng. Chế độ này phù hợp với các phép biến đổi đơn giản như `select()`, `filter()`, và không có các phép tổng hợp.
- **Complete Mode:** Chế độ này ghi toàn bộ kết quả của "bảng" dataframe luồng vào sink sau mỗi lần kích hoạt. Nó hữu ích cho các phép tổng hợp toàn cục, nơi mà kết quả cuối cùng của từng nhóm thay đổi sau mỗi lần dữ liệu mới đến.
- **Update Mode:** Chế độ này chỉ ghi những bản ghi đã được cập nhật kể từ lần kích hoạt cuối cùng. Nó được thiết kế đặc biệt cho các phép tổng hợp, nơi bạn chỉ muốn ghi lại những giá trị tổng hợp đã thay đổi, thay vì toàn bộ bảng.

7.3.4. Trigger

Trigger kiểm soát cách thức và tần suất các truy vấn luồng được thực thi. Có thể coi trigger như bộ đếm thời gian hoặc trình kích hoạt để xử lý một lô dữ liệu mới.

Có bốn loại trigger chính trong Structured Streaming:

1. Trigger mặc định. Khi bạn không chỉ định trigger, Spark sẽ sử dụng một trigger mặc định. Nó sẽ chạy một micro-batch ngay lập tức sau khi micro-batch trước đó hoàn thành. Điều này đảm bảo độ trễ thấp nhất có thể. Tốc độ xử lý phụ thuộc vào tốc độ của hệ thống (tức là khi nào một micro-batch hoàn thành).
2. Fixed-interval Trigger. Chạy một micro-batch sau một khoảng thời gian cố định. Ví dụ `df.writeStream.trigger(processingTime='5 seconds')` sẽ kích hoạt một micro-batch sau mỗi 5 giây, bất kể micro-batch trước đó đã hoàn thành hay chưa. Nếu micro-batch trước đó mất nhiều thời gian hơn 5 giây, micro-batch tiếp theo sẽ được bắt đầu ngay lập tức khi micro-batch cũ kết thúc. Loại trigger này cho phép dễ dàng kiểm soát độ trễ tối đa của ứng dụng.
3. One-time Trigger. Chạy một micro-batch duy nhất, xử lý tất cả dữ liệu có sẵn tại thời điểm đó, và sau đó dừng ứng dụng. Ví dụ `df.writeStream.trigger(once=True)`. Loại trigger này hữu ích cho các tác vụ xử lý hàng loạt ngắn hạn (ví dụ: chuyển đổi dữ liệu luồng thành định dạng Parquet) hoặc khi bạn cần đảm bảo không có thêm dữ liệu nào được xử lý sau lần chạy đầu tiên.
4. Continuous Trigger (Thử nghiệm): Đây là chế độ xử lý luồng thực sự (continuous processing) và không sử dụng micro-batching. Nó xử lý từng bản ghi khi chúng đến. Chế độ này vẫn đang trong giai đoạn thử nghiệm và không hỗ trợ tất cả các phép biến đổi của DataFrame.

7.4. Kết chương

Nội dung chương đã giới thiệu sơ lược về các kỹ thuật xử lý dữ liệu luồng với Apache Spark Streaming. Hai kỹ thuật chính đã được giới thiệu bao gồm:

- Xử lý dữ liệu luồng với DStream
- Xử lý dữ liệu luồng với Structured Streaming

Chương 8

Kiến trúc Lambda & Kappa

8.1. Kiến trúc Lambda	225
8.2. Kiến trúc Kappa	230

8.1. Kiến trúc Lambda

Kiến trúc Lambda là một kiến trúc xử lý dữ liệu được thiết kế để xử lý khối lượng dữ liệu lớn bằng cách tận dụng cả hai phương pháp xử lý theo lô (batch) và xử lý luồng (stream). Nó nhằm mục đích kết hợp ưu điểm của cả hai mô hình xử lý để cung cấp một giải pháp toàn diện cho dữ liệu lớn.

Trong thực tiễn, các doanh nghiệp cần khả năng xử lý dữ liệu ngay khi nó được tạo ra (real-time processing) nhằm đưa ra những quyết định kịp thời, đồng thời cần khả năng xử lý khối lượng dữ liệu lớn một cách hiệu quả (batch processing) để có cái nhìn tổng quan và sâu sắc hơn về hoạt động của mình. Được giới thiệu bởi Nathan Marz và James Warren trong cuốn sách "Big Data: Principles and Best Practices of Scalable Real-Time Data Systems," kiến trúc Lambda đã trở thành một mô hình phổ biến trong việc xây dựng các hệ thống dữ liệu lớn.

8.1.1. Khái niệm cốt lõi: Xử lý theo lô và xử lý luồng

Kiến trúc Lambda tận dụng hai cách tiếp cận xử lý dữ liệu khác biệt:

Xử lý theo lô (Batch Processing) Liên quan đến việc thu thập dữ liệu trong một khoảng thời gian và xử lý nó theo các lô lớn, có kích thước cố định. Xử lý các tập dữ liệu hữu hạn, tĩnh, cung cấp thông lượng cao nhưng có độ trễ cao hơn. Phù hợp với phân tích phức tạp và chuyển đổi dữ liệu quy mô lớn.

Xử lý luồng (Stream Processing) Liên quan đến việc xử lý dữ liệu theo thời gian thực khi nó đến, thường là một bản ghi tại một thời điểm hoặc theo micro-batch. Xử lý các luồng dữ liệu liên tục, không giới hạn, cung cấp độ trễ thấp và kết quả theo thời gian

thực hoặc gần thời gian thực. Phù hợp với thông tin chi tiết ngay lập tức và phản ứng nhanh chóng.

8.1.2. Các thành phần chính của Kiến trúc Lambda

Kiến trúc Lambda về cơ bản bao gồm ba tầng riêng biệt:

Tầng lưu trữ và xử lý theo lô (Batch layer)

Mục đích: Tầng này quản lý tập dữ liệu chính và tính toán trước các batch view. Nó đóng vai trò là nguồn sự thật cho toàn bộ tập dữ liệu.

Thành phần chính: Thường sử dụng hệ thống tập tin phân tán (ví dụ: HDFS) để lưu trữ dữ liệu và hệ thống xử lý batch (ví dụ: Apache Hadoop, Apache Spark) để xử lý.

Chức năng: Chức năng chính bao gồm lưu trữ dữ liệu, xử lý batch và tạo ra các batch view.

Tính toàn vẹn dữ liệu: Dữ liệu thô được lưu trữ không thay đổi (immutable data storage) trong tầng này, đảm bảo tính toàn vẹn dữ liệu ngay cả khi xử lý thất bại.

Tầng Batch trong kiến trúc Lambda chịu trách nhiệm xử lý và lưu trữ dữ liệu lịch sử. Đây là nền tảng của việc xử lý dữ liệu theo lô (batch processing), nơi mà dữ liệu được thu thập và xử lý trong các khoảng thời gian định kỳ. Lớp này thường sử dụng các hệ thống lưu trữ dữ liệu lớn như Hadoop hoặc Amazon S3 và các công cụ xử lý batch như Apache Hadoop hoặc Apache Spark.

Chức năng chính của tầng Batch:

Xử lý dữ liệu lịch sử Lớp Batch xử lý toàn bộ dữ liệu thô được thu thập trong khoảng thời gian dài, tạo ra những bộ dữ liệu tổng hợp và chi tiết nhất. Dữ liệu phục vụ mục tiêu phân tích nên được lưu trữ dưới dạng bất biến, không thay đổi để đảm bảo tính toàn vẹn và khả năng truy vết nguồn gốc dữ liệu.

Sử dụng các nền tảng xử lý dữ liệu theo lô Các nền tảng như Apache Hadoop Mapreduce và Apache Spark được sử dụng để thực hiện các tác vụ xử lý phức tạp trên dữ liệu thô.

Tầng Batch lưu trữ dữ liệu thô ở định dạng nguyên bản nhất, không thực hiện bất kỳ loại bỏ hay chuyển đổi nào. Dữ liệu trong lớp Batch là bất biến, có nghĩa là không được phép cập nhật hoặc xóa. Thay vào đó, dữ liệu luôn được thêm vào với dấu thời gian để có thể truy vấn các phiên bản mới nhất khi cần. Điều này đảm bảo rằng dữ liệu luôn có sẵn cho mọi phân tích trong tương lai mà không bị mất mát hoặc thay đổi.

Việc truy vấn trực tiếp trên dữ liệu thô có thể tốn nhiều thời gian xử lý. Để khắc phục vấn đề này, các batch view được tạo và lưu trữ định kỳ. Các batch view này là các phiên bản

đã qua xử lý và tổng hợp của dữ liệu thô, giúp tăng tốc độ truy vấn và phân tích. Mỗi khi một batch view mới được tạo, batch view cũ sẽ bị loại bỏ, đảm bảo rằng hệ thống luôn cung cấp thông tin mới nhất và chính xác nhất.

Tầng Batch yêu cầu một hệ thống lưu trữ bền vững, có khả năng hỗ trợ truy vấn ngẫu nhiên ở quy mô lớn. Hệ thống này không cần phải hỗ trợ ghi ngẫu nhiên do dữ liệu được nạp vào theo lô ở tần suất định kỳ. Sự bền vững và khả năng mở rộng của hệ thống lưu trữ là yếu tố then chốt để đảm bảo tầng Batch hoạt động hiệu quả.

Tầng xử lý luồng (Speed layer)

Mục đích: Tầng này bù đắp cho độ trễ xử lý của tầng batch. Nó xử lý dữ liệu mới nhất theo thời gian thực để cung cấp thông tin chi tiết cập nhật mà tầng batch không thể cung cấp ngay lập tức.

Thành phần chính: Sử dụng các hệ thống xử lý luồng (ví dụ: Apache Storm, Apache Flink).

Chức năng: Chức năng bao gồm xử lý dữ liệu thời gian thực và cập nhật tăng dần các real-time view.

Tầng speed, còn được gọi là tầng xử lý luồng, chịu trách nhiệm xử lý dữ liệu thời gian thực được truyền theo luồng. Điều này vốn không thể thực hiện ở tầng Batch do tính chất thực hiện định kỳ của nó (ví dụ hàng ngày). Kết quả phân tích ở tầng Batch luôn có độ trễ so với hiện tại.

Chức năng chính của lớp Streaming:

Xử lý dữ liệu thời gian thực Xử lý dữ liệu ngay khi nó được truyền vào hệ thống, đảm bảo khả năng phản hồi nhanh chóng cho các yêu cầu phân tích.

Thu thập dữ liệu từ các nguồn streaming Tiếp nhận dữ liệu liên tục từ các nguồn phát dữ liệu, như cảm biến, thiết bị IoT, và ứng dụng web.

Sử dụng các công cụ xử lý streaming Áp dụng các công cụ và framework như Apache Kafka, Apache Flink, hoặc Apache Spark Streaming để thực hiện xử lý và phân tích dữ liệu theo thời gian thực.

Tầng speed được thiết kế để xử lý dữ liệu với độ trễ thấp, đảm bảo rằng dữ liệu có sẵn cho phân tích ngay sau khi được thu thập. Tầng speed tạo ra các góc nhìn (view) cho dữ liệu thời gian thực. Các view của lớp Speed sẽ được hợp nhất với các batch view để cung cấp một bức tranh toàn diện và cập nhật.

Tầng phục vụ (Serving layer)

Mục đích: Tầng này phản hồi các truy vấn bằng cách hợp nhất batch view và real-time view. Nó làm cho dữ liệu đã xử lý từ cả Tầng Batch và Speed có sẵn để sử dụng.

Thành phần chính: Thường sử dụng các cơ sở dữ liệu có độ trễ thấp, được tối ưu hóa cho việc đọc (ví dụ: Apache HBase, Cassandra).

Chức năng: Trách nhiệm bao gồm lập chỉ mục batch view, tạo điều kiện cho việc đọc nhanh và hợp nhất batch view và real-time view.

Tầng phục vụ đóng vai trò trung tâm trong việc cung cấp một bức tranh toàn diện và cập nhật nhất về phân tích dữ liệu, được tạo bởi việc hợp nhất các kết quả xử lý ở lớp Batch và lớp Speed để các hệ thống khác hoặc người dùng khác truy vấn.

8.1.3. Mô hình nhất quán cuối cùng (Eventual Consistency Model)

Kiến trúc Lambda hoạt động theo mô hình nhất quán cuối cùng. Điều này có nghĩa là hệ thống sẽ trở nên nhất quán theo thời gian, giả định rằng nó ngừng nhận các cập nhật mới.

Tầng Batch Cung cấp một view nhất quán mạnh mẽ nhưng có thể lỗi thời.

Tầng Speed Cung cấp một view có thể không nhất quán nhưng cập nhật.

Tầng Serving Hợp nhất cả hai view, cuối cùng đạt được sự nhất quán cuối cùng.

8.1.4. Ưu điểm của kiến trúc Lambda

Thiết kế của kiến trúc Lambda mang lại một số lợi ích:

Khả năng chịu lỗi (Fault-tolerance) Dữ liệu thô được lưu trữ không thay đổi trong Tầng Batch, đảm bảo tính toàn vẹn dữ liệu ngay cả khi xử lý thất bại. Cả Tầng Batch và Speed đều sử dụng hệ thống phân tán, có nghĩa là nếu một node thất bại, công việc có thể được phân phối lại. Dữ liệu được xử lý ở cả hai tầng, cung cấp backup nếu một tầng thất bại.

Giảm thiểu lỗi con người trong lập trình Nếu tìm thấy lỗi, mã có thể được sửa và toàn bộ tập dữ liệu có thể được chạy lại mà không mất dữ liệu. Tầng Batch và Speed tách biệt, có nghĩa là lỗi ở một tầng không nhất thiết ảnh hưởng đến tầng khác.

Thuận lợi cho việc xử lý lại dữ liệu Dữ liệu gốc luôn có sẵn để xử lý lại, không có rủi ro về sự hỏng dữ liệu trong quá trình cập nhật. Tầng Batch được tối ưu hóa để xử lý hiệu quả toàn bộ tập dữ liệu, cho phép tính toán lại dễ dàng khi cần thiết.

Kiến trúc Lambda mang lại nhiều lợi thế khiến nó trở thành lựa chọn phù hợp để xây dựng Data Lake cho doanh nghiệp. Một số ưu điểm chính bao gồm:

Phân tích toàn diện và kịp thời Kiến trúc Lambda kết hợp cả xử lý dữ liệu theo lô (batch processing) và xử lý dữ liệu thời gian thực (stream processing), cho phép hệ thống đáp ứng các nhu cầu xử lý khác nhau một cách linh hoạt. Khả năng kết hợp dữ liệu lịch sử từ lớp Batch và dữ liệu thời gian thực từ lớp Streaming giúp cung cấp một cái nhìn toàn diện và cập nhật về dữ liệu, hỗ trợ quyết định kinh doanh chính xác và kịp thời.

Khả năng mở rộng cao, linh hoạt Kiến trúc Lambda tách biệt các trách nhiệm khác nhau thành các tầng được xác định rõ ràng và có ranh giới. Nhờ đó, các tầng này có thể sử dụng các công nghệ khác nhau và các kỹ thuật tối ưu hóa khác nhau để hoạt động tốt hơn mà không ảnh hưởng đến các tầng khác. Kiến trúc dễ dàng để tích hợp các nền tảng lưu trữ và xử lý dữ liệu phổ biến như Hadoop, Apache Spark, và Apache Kafka.

Lưu trữ dữ liệu thô Dữ liệu được lưu trữ ở định dạng gốc. Nhờ đó, bất cứ lúc nào, các thuật toán, phân tích mới hoặc các trường hợp sử dụng kinh doanh mới cũng có thể được áp dụng cho Data Lake chỉ đơn giản bằng cách tạo các view batch và speed mới. Đây là một trong những lợi thế lớn nhất so với các kho dữ liệu truyền thống, nơi dữ liệu được lọc và lưu trữ. Do đó, các trường hợp sử dụng mới sẽ cần thay đổi lược đồ dữ liệu, thường tốn thời gian và công sức.

8.1.5. Thách thức và hạn chế

Mặc dù có nhiều ưu điểm, kiến trúc Lambda cũng đặt ra một số thách thức và hạn chế:

Độ phức tạp của việc duy trì hệ thống kép Yêu cầu quản lý hai hệ thống xử lý riêng biệt (batch và speed), dẫn đến sự phức tạp trong việc phối hợp cập nhật, đảm bảo đồng bộ hóa và tăng nỗ lực giám sát và khắc phục sự cố.

Mã dư thừa giữa Tầng Batch và Speed Cùng một tính toán thường cần được thực hiện hai lần, làm tăng rủi ro về sự không nhất quán và lỗi.

Khả năng không nhất quán dữ liệu Có thể có độ trễ giữa kết quả xử lý batch và thời gian thực, đặt ra thách thức trong việc hợp nhất chính xác batch view và real-time view.

Tài nguyên tiêu tốn Chạy cả hai tầng thường yêu cầu tài nguyên tính toán đáng kể.

Chi phí phát triển và vận hành cao hơn Do độ phức tạp của nó, có thể phát sinh chi phí phát triển và vận hành cao hơn so với các kiến trúc đơn giản hơn.

Mặc dù mang lại nhiều ưu điểm, việc lựa chọn Kiến trúc Lambda để xây dựng Data Lake cho doanh nghiệp cũng đi kèm với một số nhược điểm nhất định. Kiến trúc Lambda đòi hỏi việc thiết kế và duy trì các lớp khác nhau, dẫn đến độ phức tạp cao trong việc triển khai và quản lý. Điều này có thể tăng chi phí và yêu cầu kỹ thuật cao từ đội ngũ phát triển. Việc

duy trì cả hai hệ thống xử lý theo lô và thời gian thực có thể tăng chi phí hạ tầng và quản lý. Kiến trúc Lambda đôi khi bị chỉ trích vì thực hiện cùng một tác vụ hai lần ở lớp batch và lớp speed.

8.2. Kiến trúc Kappa

Kiến trúc Kappa là một mô hình kiến trúc phần mềm cho xử lý dữ liệu thời gian thực sử dụng một hệ thống duy nhất, thống nhất cho cả xử lý luồng và xử lý theo lô. Nó xuất hiện từ nhu cầu xử lý dữ liệu thời gian thực trong các ứng dụng hiện đại, nhằm giải quyết những thách thức liên quan đến việc duy trì các hệ thống xử lý batch và stream riêng biệt, đồng thời mong muốn có những kiến trúc dữ liệu lớn đơn giản hơn và dễ bảo trì hơn.

8.2.1. Kappa như một sự đơn giản hóa của kiến trúc Lambda

Kiến trúc Kappa được phát triển như một sự đơn giản hóa, chủ yếu để khắc phục những hạn chế của kiến trúc Lambda. Những hạn chế chính mà Kappa giải quyết bao gồm độ phức tạp của việc duy trì hai hệ thống riêng biệt (tầng batch và speed), khả năng không nhất quán dữ liệu giữa tầng batch và speed, cũng như chi phí phát triển và vận hành cao hơn.

Để giải quyết những vấn đề này, Kappa cung cấp một đường xử lý duy nhất cho cả dữ liệu thời gian thực và dữ liệu lịch sử, loại bỏ sự cần thiết của một hệ thống xử lý batch riêng biệt. Thiết kế đơn giản hóa này giúp giảm độ phức tạp và khả năng xảy ra lỗi, đồng thời tạo ra một kiến trúc dễ hiểu và bảo trì hơn so với Lambda.

8.2.2. Nguyên tắc cốt lõi của kiến trúc Kappa

Kiến trúc Kappa được xây dựng dựa trên một số nguyên tắc cốt lõi:

Dữ liệu như một luồng sự kiện Tất cả dữ liệu được coi như một chuỗi các sự kiện bất biến.

Các sự kiện được sắp xếp theo thời gian xảy ra. Điều này tạo thành một log chỉ ghi thêm các sự kiện (ví dụ: Kafka topics).

Khả năng xử lý lại Cho phép khả năng xử lý lại toàn bộ luồng sự kiện từ đầu. Khả năng này cho phép sửa lỗi, cập nhật thuật toán và phân tích lịch sử.

Pipeline duy nhất Có một đường xử lý cho cả workload thời gian thực và giống batch. Điều này có nghĩa là một codebase thống nhất cho tất cả các tác vụ xử lý dữ liệu.

8.2.3. Các thành phần chính

Kiến trúc Kappa thường bao gồm ba thành phần chính. Thành phần đầu tiên là nền tảng luồng sự kiện (Event Streaming Platform), có mục đích cung cấp lưu trữ bền vững cho tất

cả các sự kiện đầu vào. Apache Kafka là một ví dụ tiêu biểu của nền tảng này, với các tính năng chính bao gồm thông lượng cao, khả năng chịu lỗi và khả năng mở rộng.

Thành phần thứ hai là hệ thống xử lý luồng (Stream Processing System), chịu trách nhiệm xử lý thời gian thực các luồng sự kiện. Các ví dụ phổ biến bao gồm Apache Flink, Kafka Streams, và Apache Spark Streaming. Những hệ thống này cung cấp các tính năng chính như độ trễ thấp, xử lý có trạng thái và ngữ nghĩa chính xác một lần, đảm bảo tính toàn vẹn dữ liệu trong quá trình xử lý.

Thành phần cuối cùng là tầng phục vụ (Serving Layer), có mục đích lưu trữ và phục vụ kết quả đã xử lý. Các ví dụ tiêu biểu bao gồm Apache Cassandra, Elasticsearch, và Apache Druid. Tầng này được thiết kế với các tính năng chính là đọc nhanh và hỗ trợ cập nhật thời gian thực, đảm bảo khả năng phản hồi nhanh chóng cho các yêu cầu truy vấn từ người dùng và ứng dụng.

8.2.4. Luồng dữ liệu trong kiến trúc Kappa

Luồng dữ liệu trong kiến trúc Kappa tuân theo một đường liên tục, bao gồm các giai đoạn chính sau:

Thu thập dữ liệu Các sự kiện được thu thập từ nhiều nguồn khác nhau như tương tác người dùng, thiết bị IoT, cảm biến, và các hệ thống khác. Những sự kiện này sau đó được xuất bản lên nền tảng luồng sự kiện, thường là các Kafka topics, để đảm bảo tính bền vững và khả năng xử lý đồng thời.

Xử lý luồng Hệ thống tiêu thụ các sự kiện từ nền tảng streaming một cách liên tục, áp dụng logic kinh doanh, thực hiện các phép chuyển đổi và tổng hợp dữ liệu. Quá trình này thường liên quan đến xử lý có trạng thái sử dụng local state stores để duy trì thông tin trạng thái cần thiết cho việc xử lý các sự kiện tiếp theo.

Cập nhật trạng thái Kết quả đã xử lý được ghi vào tầng phục vụ, nơi các cập nhật tăng dần được thực hiện để duy trì các view thời gian thực. Điều này đảm bảo rằng dữ liệu luôn được cập nhật và sẵn sàng cho việc truy vấn.

Phục vụ truy vấn Các ứng dụng và người dùng có thể đọc dữ liệu đã qua xử lý từ tầng phục vụ, nơi hỗ trợ cả truy vấn thời gian thực và truy vấn lịch sử, cung cấp một cái nhìn toàn diện về dữ liệu.

8.2.5. Xử lý lại dữ liệu trong kiến trúc Kappa

Xử lý lại là một khả năng cốt lõi trong kiến trúc Kappa, giải quyết các tình huống khác nhau. Các tình huống yêu cầu xử lý lại bao gồm sửa lỗi trong logic xử lý, cải tiến thuật toán, thêm tính năng mới hoặc trường dữ liệu mới, và phân tích lịch sử với logic đã cập nhật.

Về cơ chế xử lý lại, một ứng dụng xử lý luồng đã cập nhật được triển khai, consumer offsets được reset về đầu event log, toàn bộ luồng sự kiện sau đó được xử lý lại với logic mới, và kết quả sau đó được ghi đè hoặc hợp nhất trong tầng phục vụ.

So sánh với kiến trúc Lambda, Kappa cung cấp một codebase duy nhất để bảo trì và xử lý lại đơn giản hơn, trong khi Lambda yêu cầu một hệ thống batch riêng biệt cho việc xử lý lại, có thể dẫn đến sự không nhất quán tiềm ẩn.

8.2.6. Ưu điểm của kiến trúc Kappa

Kiến trúc Kappa mang lại một số ưu điểm:

Kiến trúc hệ thống đơn giản hóa Một pipeline xử lý duy nhất giảm độ phức tạp, dẫn đến một codebase thống nhất dễ hiểu và bảo trì hơn.

Giảm độ trễ Cho phép xử lý thời gian thực tất cả dữ liệu, không có độ trễ chờ đợi các job xử lý batch.

Cải thiện tính nhất quán Duy trì một nguồn sự thật duy nhất (event log), ngăn chặn sự khác biệt giữa batch view và real-time view.

Khả năng mở rộng và linh hoạt Nền tảng luồng sự kiện được thiết kế cho khả năng mở rộng cao, giúp dễ dàng thêm consumer mới hoặc sửa đổi logic xử lý hiện có.

Hiệu quả về chi phí Dẫn đến giảm nhu cầu hạ tầng so với kiến trúc Lambda và đơn giản hóa hoạt động và bảo trì.

8.2.7. Thách thức và hạn chế

Mặc dù có nhiều ưu điểm, kiến trúc Kappa cũng đặt ra một số thách thức:

Yêu cầu lưu trữ dài hạn Nhu cầu giữ lại toàn bộ lịch sử sự kiện để xử lý lại có thể dẫn đến chi phí lưu trữ đáng kể cho các hệ thống có khối lượng cao.

Độ phức tạp trong thiết kế xử lý luồng Các thách thức bao gồm xử lý các sự kiện không theo thứ tự và dữ liệu muộn, đảm bảo ngữ nghĩa xử lý chính xác một lần, và quản lý trạng thái trong các stream processor phân tán.

Hạn chế trong xử lý giống batch Một số hoạt động khó khăn hơn trong mô hình streaming (ví dụ: joins trên các tập dữ liệu lớn), và có thể có vấn đề hiệu suất tiềm ẩn cho việc xử lý lại quy mô lớn.

Đường cong học tập Yêu cầu sự thay đổi trong tư duy từ xử lý batch truyền thống và quen thuộc với các khái niệm và công cụ xử lý luồng.

Khả năng áp dụng trường hợp sử dụng Có thể không tối ưu cho tất cả các tình huống, đặc biệt là những tình huống có sự phân tách rõ ràng giữa nhu cầu dữ liệu lịch sử và thời gian thực.

Bằng cách loại bỏ tầng xử lý batch, kiến trúc Kappa giúp việc xây dựng và duy trì các hệ thống xử lý dữ liệu lớn trở nên dễ dàng hơn. Điều này có nghĩa là tất cả dữ liệu, bao gồm cả dữ liệu lịch sử, đều được xử lý theo thời gian thực và được lưu trữ trong tầng xử lý luồng. Dữ liệu lịch sử đơn giản được coi như một luồng dữ liệu khác và được xử lý theo cách giống như dữ liệu thời gian thực. Nhìn chung, Kappa là lựa chọn tốt cho các tổ chức cần xử lý dữ liệu theo thời gian thực và không cần sự linh hoạt của kiến trúc Lambda.

Chương 9

Phân tích dữ liệu lớn với SparkML

9.1. Giới thiệu chung	234
9.2. Spark ML	235
9.3. Kết chương	240

9.1. Giới thiệu chung

Các bài toán phân tích dữ liệu nói chung và dữ liệu lớn nói riêng thường áp dụng các kỹ thuật thống kê, phân lớp, phân cụm, hồi quy, dự báo, lọc cộng tác,... trên các tập dữ liệu đầu vào. Trong khi Spark hỗ trợ rất tốt cho các tác vụ thống kê thông qua các API của Spark Core (RDD), Spark SQL, hay Spark Streaming, thì việc thực hiện các tác vụ phân lớp, phân cụm, dự báo, hồi quy,... lại là một thách thức lớn do nhu cầu cài đặt của các giải thuật học máy phức tạp.

Để vượt qua thách thức này, Spark cung cấp hai thành phần chính:

- Spark MLlib. Đây là thành phần cung cấp các API hỗ trợ việc xây dựng và sử dụng các giải thuật học máy trên nền Spark Core. MLlib sử dụng RDD để lưu trữ và xử lý dữ liệu. Các RDD này được phân tán trên các cluster, cho phép các thuật toán MLlib hoạt động hiệu quả trên các tập dữ liệu lớn. Ngoài ra Spark MLlib cũng hỗ trợ cho việc biểu diễn các bộ dữ liệu dưới dạng vector, bao gồm vector dày (dense vector) và vector thưa (sparse vector).
- Spark ML. Đây là thành phần cung cấp các tính năng học máy tương tự như MLlib, nhưng được xây dựng trên nền của Spark SQL. Do vậy nó cho phép xử lý dữ liệu một cách linh hoạt hơn và tối ưu hơn. Hiện tại Spark ML được khuyến khích sử dụng thay vì Spark MLlib.

Spark MLlib và Spark ML được xây dựng với một khung cấu trúc tổng quát phù hợp với hầu hết bài toán học máy khác nhau. Lập trình viên có thể tùy biến phương pháp học máy của mình theo cấu trúc này. Để giúp lập trình viên có thể cài đặt các phương pháp học

máy được nhanh chóng, Spark MLlib và Spark ML cung cấp sẵn một số giải thuật học máy phổ biến bao gồm:

- Phân loại (Classification): Hồi quy logistic, SVM, Naive Bayes.
- Hồi quy (Regression): Hồi quy tuyến tính, Hồi quy cây quyết định.
- Phân cụm (Clustering): K-Means, Gaussian Mixture.
- Lọc cộng tác (Collaborative Filtering): ALS (Alternating Least Squares).
- Tiền xử lý dữ liệu (Feature Extraction): TF-IDF, Word2Vec, chuẩn hóa (normalization).

Các nội dung tiếp theo của chương tập trung giới thiệu về Spark ML, các khái niệm trong Spark ML, cũng như các giải thuật học máy được cung cấp sẵn của Spark ML.

9.2. Spark ML

Học máy được hiểu là việc xây dựng một mô hình phản ánh các quy luật tiềm ẩn trong một bộ dữ liệu mẫu (thường là dữ liệu thu thập được từ quá khứ) và việc sử dụng mô hình đó trên các dữ liệu phát sinh thực tế. Spark ML hỗ trợ hiệu quả cho việc xây dựng và sử dụng các mô hình học máy như vậy. Với Spark ML, một quy trình học máy cơ bản bao gồm các bước sau:

1. Chuẩn bị bộ dữ liệu mẫu. Đây là bước thu thập dữ liệu mẫu từ các nguồn phát sinh, hoặc từ các kho lưu trữ. Dữ liệu từ các nguồn phát sinh hoặc kho dữ liệu này có thể được tải về và lưu trữ trên các Dataframe của Spark thông qua các API của Spark SQL (hoặc Structured Streaming).
2. Tiền xử lý dữ liệu. Là bước làm sạch và biến đổi các dữ liệu mẫu về khuôn dạng phù hợp cho việc huấn luyện. Spark ML cung cấp các lớp Transformer cho công việc này. Transformer thực chất là lớp biểu diễn cho một giải thuật biến đổi từ một dataframe này thành một dataframe khác. Spark ML cung cấp một số lớp Transformer có sẵn như HashingTF, LogisticRegressionModel,...
3. Thiết kế mô hình. Transformer cũng có thể coi như một mô hình học máy với khả năng diễn giải từ các bộ dữ liệu đầu vào thành các bộ dữ liệu đầu ra (các lớp trong bài toán phân lớp hoặc các giá trị hồi quy trong bài toán hồi quy, hay các cụm trong bài toán phân cụm). Trong học máy, mô hình được huấn luyện từ các bộ dữ liệu mẫu bằng các giải thuật học máy. Để biểu diễn cho các giải thuật học máy này,

Spark ML cung cấp các lớp Estimator. Chúng có thể nhận đầu vào là các bộ dữ liệu mẫu (dữ liệu huấn luyện) dưới dạng dataframe và cho kết quả là các mô hình dưới dạng Transformer. Nhiều Estimator được cài đặt sẵn trong Spark ML như LogisticRegression, DecisionTreeClassifier, KMeans,...

Bên cạnh đó, Spark ML cũng cung cấp một cách thức linh hoạt để thiết kế các giải thuật học máy phức tạp với lớp Pipeline. Một Pipeline có thể xem như là sự ghép nối liên tiếp của nhiều Transformer và Estimator lại với nhau. Ví dụ chúng ta có thể ghép một Transformer với một Transformer khác qua Pipeline để tạo thành một Transformer mới, hoặc ghép một Transformer với một Estimator để tạo nên một Estimator mới.

4. Huấn luyện mô hình. Là bước xác định các tham số của mô hình để nó phản ánh quy luật mong muốn trên tập dữ liệu mẫu. Trong Spark ML, việc huấn luyện có thể được thực hiện bằng phương thức `fit(df)` của một Estimator với bộ dữ liệu huấn luyện `df`. Kết quả của việc huấn luyện này là một Transformer đại diện cho mô hình học máy mong muốn.
5. Sử dụng mô hình và đánh giá. Là việc sử dụng mô hình cho các dữ liệu mới phát sinh và việc đánh giá chất lượng của việc sử dụng mô hình này. Với mô hình là một Transformer, việc sử dụng mô hình đơn giản chỉ là việc gọi phương thức `transform(df)` với bộ dữ liệu đầu vào `df`.

9.2.1. Transformer

Như giới thiệu ở trên, một Transformer là một thuật toán cho phép biến đổi từ một DataFrame này thành một DataFrame khác. Nó có phương thức `.transform()` nhận một DataFrame đầu vào và trả về một DataFrame mới với một hoặc nhiều cột được thêm vào. Transformer có thể đại diện cho một kỹ thuật tiền xử lý dữ liệu hoặc một mô hình học máy.

Ví dụ sau minh họa việc tách các câu trên DataFrame đầu vào thành danh sách từ bằng cách sử dụng Transformer Tokenizer.

```
sentenceDataFrame = spark.createDataFrame([
    (0, "Hi I heard about Spark"),
    (1, "I wish Java could use case classes"),
    (2, "Logistic, regression, models, are, neat")
], ["id", "sentence"])
tokenizer = Tokenizer(inputCol="sentence", outputCol="words")
tokenized = tokenizer.transform(sentenceDataFrame)
```

Các Transformer của Spark ML có thể được phân loại thành nhiều nhóm dựa trên chức năng của chúng:

- Transformer Xử lý Đặc trưng (Feature Transformers). Nhóm này bao gồm các Transformer dùng để tạo, biến đổi, hoặc chọn các đặc trưng từ dữ liệu thô:
 - Tokenizer: Tách văn bản thành một mảng các từ (tokens).
 - VectorAssembler: Kết hợp nhiều cột đặc trưng (numeric, boolean, vector) thành một cột vector duy nhất. Đây là bước quan trọng trước khi đưa dữ liệu vào các mô hình học máy.
 - CountVectorizer: Biến đổi một tập hợp tài liệu văn bản thành các vector đếm số lần xuất hiện của từ.
 - TF-IDF: Tính toán tần suất từ (Term Frequency-Inverse Document Frequency), giúp đánh giá mức độ quan trọng của một từ trong một tài liệu so với toàn bộ kho tài liệu.
 - Word2Vec: Học vector từ (word embeddings) từ một tập hợp tài liệu.
- Transformer Chuẩn hóa và Biến đổi (Scaling and Normalization Transformers). Các Transformer này dùng để chuẩn hóa dữ liệu, giúp các thuật toán học máy hoạt động hiệu quả hơn:
 - StandardScaler: Chuẩn hóa các đặc trưng bằng cách loại bỏ giá trị trung bình và chia cho độ lệch chuẩn.
 - MinMaxScaler: Biến đổi các đặc trưng để chúng nằm trong một khoảng nhất định (thường là $[0, 1]$).
 - Normalizer: Chuẩn hóa vector đầu vào theo chuẩn L1 hoặc L2.
- Transformer Mã hóa (Encoding Transformers). Nhóm này dùng để chuyển đổi dữ liệu phân loại hoặc dữ liệu văn bản thành dạng số mà các mô hình có thể xử lý:
 - StringIndexer: Chuyển đổi các chuỗi phân loại thành các chỉ số số nguyên (từ 0 đến $N-1$).
 - OneHotEncoder: Biến đổi một cột chỉ số số nguyên thành một vector nhị phân (one-hot vector). Đây là bước tiếp theo sau StringIndexer.
- Transformer Lựa chọn Đặc trưng (Feature Selection Transformers) Các Transformer này giúp giảm số lượng đặc trưng bằng cách loại bỏ các đặc trưng không cần thiết hoặc trùng lặp:

- VectorSlicer: Cắt ra một phần của một vector đặc trưng.
- RFormula: Tạo một vector đặc trưng dựa trên công thức R, tự động xử lý các đặc trưng phân loại.

Transformer có thể được tạo ra bằng cách ghép nối nhiều Transformer khác sử dụng Pipeline. Ví dụ sau minh họa cho việc ghép nối này.

```
tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashing_tf = HashingTF(inputCol="words", outputCol="raw_features", numFeatures=1000)
idf = IDF(inputCol="raw_features", outputCol="features")

pipeline = Pipeline(stages=[tokenizer, hashing_tf, idf])
```

Các Transformer cũng có thể được tạo ra bằng cách huấn luyện trên các giải thuật học máy (Estimator).

9.2.2. Estimator

Estimator là một thuật toán hoặc quy trình học từ dữ liệu. Nó đóng vai trò là "người huấn luyện" trong quy trình học máy. Estimator nhận một DataFrame làm đầu vào, học từ dữ liệu trong đó, và sau đó tạo ra một Transformer (chính là mô hình đã được huấn luyện).

- Phương thức chính: Estimator có một phương thức duy nhất là fit().
- Đầu vào: Một DataFrame có các trường tính năng (features) và nhãn (labels).
- Đầu ra: Một Transformer đã được huấn luyện.

Ví dụ sau minh họa việc huấn luyện mô hình phân lớp với giải thuật LinearRegression.

```
data = [(Vectors.dense([0.0, 1.0]), 1.0),
        (Vectors.dense([1.0, 0.0]), 2.0),
        (Vectors.dense([2.0, 1.0]), 3.0),
        (Vectors.dense([3.0, 2.0]), 4.0)]
df = spark.createDataFrame(data, ["features", "label"])
lr_estimator = LinearRegression(featuresCol="features", labelCol="label")
lr_model = lr_estimator.fit(df)
predictions = lr_model.transform(df)
predictions.show()
```

Spark ML cung cấp nhiều Estimator cho các tác vụ học máy khác nhau:

- Phân loại (Classification):
 - LogisticRegression: Huấn luyện mô hình hồi quy logistic.
 - DecisionTreeClassifier: Xây dựng cây quyết định để phân loại.
 - RandomForestClassifier: Huấn luyện mô hình rừng ngẫu nhiên.
- Hồi quy (Regression):
 - LinearRegression: Huấn luyện mô hình hồi quy tuyến tính.
 - GBTRegressor: Huấn luyện mô hình Gradient-Boosted Tree để hồi quy.
- Phân cụm (Clustering):
 - KMeans: Tìm các cụm trong dữ liệu bằng thuật toán K-Means.
- Phân tích dữ liệu (Feature Analysis):
 - VectorAssembler: Dù không phải là một thuật toán học máy, nó là một Estimator/Transformer quan trọng để kết hợp nhiều cột thành một cột vector duy nhất.
- Các Estimator khác:
 - StringIndexer: Ánh xạ các giá trị chuỗi thành các chỉ số số.
 - StandardScaler: Chuẩn hóa các tính năng bằng cách loại bỏ giá trị trung bình và chia cho độ lệch chuẩn.

9.2.3. Pipeline

Pipeline (đường ống) là một công cụ mạnh mẽ giúp kết hợp nhiều bước trong một quy trình học máy thành một thực thể duy nhất. Nó giúp chuẩn hóa, tự động hóa, và tái sử dụng toàn bộ quy trình từ tiền xử lý dữ liệu đến huấn luyện mô hình.

Một Pipeline được tạo thành từ một chuỗi các Stages (bước). Mỗi Stage là một Estimator hoặc một Transformer.

Có hai cách chính để tạo nên Pipeline. Cách thứ nhất là ghép các Transformer để tạo thành một Transformer mới. Ví dụ sau minh họa việc ghép hai Transformer Tokenizer và HashingTF.

```
# Định nghĩa các Transformer
tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(inputCol="words", outputCol="features")
```

```
# Xây dựng Pipeline
pipeline = Pipeline(stages=[tokenizer, hashingTF])
```

Cách thứ hai là ghép nối một chuỗi Transformer với một Estimator để tạo thành một Estimator mới. Ví dụ sau minh họa cho cách ghép nối này.

```
tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(inputCol="words", outputCol="features")
lr = LogisticRegression(maxIter=10, regParam=0.001)

pipeline = Pipeline(stages=[tokenizer, hashingTF, lr])
model = pipeline.fit(trainingData)
```

9.3. Kết chương

Nội dung chương đã trình bày về các kỹ thuật học máy trên dữ liệu lớn với Spark ML. Những khái niệm như Transformer, Estimator, hay Pipeline và cách thức sử dụng chúng cũng được giới thiệu chi tiết trong nội dung chương.

Tài liệu tham khảo

- [1] H. Rademacher, *Higher Mathematics from an Elementary point of view*, Birkhauser, 1983.
- [2] Martin Aigner and Gunter M. Ziegler, *Proofs from the book*, Springer, 1999.
- [3] T. Andreescu, R. Gelca, *Mathematical olympiad challenges*, Birkhauser, 2002.
- [4] Loren C. Larson, *Problem-Solving through problems*, Springer-Verlag, 1983.
- [5] L. E. Dickson, *New first course in the theory of equations* John Wiley & Sons, 1946.
- [6] Judita Cofman, *What to solve? Problems and Suggestions for Young Mathematicians*, Clarendon Press-Oxford, 1989.
- [7] Nguyễn Hữu Điển, *Phương pháp Dirichle và ứng dụng*, NXBKHK, 1999.
- [8] Nguyễn Hữu Điển, *Phương pháp Quy nạp toán học*, NXBGD, 2000.
- [9] Nguyễn Hữu Điển, *Những phương pháp điển hình trong giải toán phổ thông*, NXBGD, 2001.
- [10] Nguyễn Hữu Điển, *Những phương pháp giải bài toán cực trị trong hình học*, NXBKHK, 2001.
- [11] Nguyễn Hữu Điển, *Sáng tạo trong giải toán phổ thông*, NXBGD, 2002.
- [12] Nguyễn Hữu Điển, *Đa thức và ứng dụng*, NXBGD, 2003.
- [13] Nguyễn Hữu Điển, *Giải phương trình vô định nghiệm nguyên*, NXBĐHQ, 2004.
- [14] Nguyễn Hữu Điển, *Giải toán bằng phương pháp đại lượng bất biến*, NXBGD, 2004.